# Sistemas Operativos I

## Introducción y fundamentos

# Contenido


**1 Créditos Editoriales** **1**

**2 Acerca de este volumen** **3**

**3 Presentación de la clase** **5**
3.1 Objetivos de aprendizaje . . . . . 5

**4 ¿Qué es un sistema operativo?** **7**
4.1 Una comparación sencilla . . . . . 8

**5 El kernel: núcleo del sistema operativo** **9**
5.1 Modo usuario y modo privilegiado . . . . . 9

**6 Funciones principales de un sistema operativo** **11**
6.1 Gestión de procesos . . . . . 11
6.2 Gestión de memoria . . . . . 11
6.3 Gestión de archivos y almacenamiento . . . . . 11
6.4 Gestión de dispositivos . . . . . 12
6.5 Seguridad y protección . . . . . 12
6.6 Redes y comunicación . . . . . 12

**7 Breve evolución de los sistemas operativos** **13**
7.1 Primeras computadoras . . . . . 13
7.2 Monitor residente . . . . . 13
7.3 Procesamiento por lotes . . . . . 13
7.4 Multiprogramación y tiempo compartido . . . . . 13
7.5 Computadoras personales e interfaces gráficas . . . . . 14
7.6 Internet, dispositivos móviles, nube y virtualización . . . . . 14

**8 Panorama actual de sistemas operativos** **15**
8.1 Windows y Windows Server . . . . . 15
8.2 GNU/Linux . . . . . 16
8.3 macOS . . . . . 16
8.4 Android e iOS . . . . . 17

**9 Interfaces de usuario** **19**
9.1 Interfaz gráfica de usuario . . . . . 19
9.2 Interfaz de línea de comandos . . . . . 19

9.3 No son enemigas . . . 20

**10 Clasificación de los sistemas operativos** **21**
10.1 Monotarea y multitarea . . . 21
10.2 Monousuario y multiusuario . . . 21
10.3 Procesamiento por lotes . . . 21
10.4 Tiempo compartido . . . 21
10.5 Sistemas de red y distribuidos . . . 22
10.6 Sistemas de tiempo real . . . 22

**11 Llamadas al sistema** **23**
11.1 Bibliotecas y API . . . 23

**12 Relación con la arquitectura del computador** **25**
12.1 CPU . . . 25
12.2 Memoria principal . . . 25
12.3 Almacenamiento secundario . . . 25
12.4 Unidad de control y ALU . . . 25
12.5 Periféricos y entrada/salida . . . 26

**13 Virtualización y contenedores: una mirada actual** **27**

**14 Laboratorio 1: conozca su sistema operativo** **29**
14.1 Propósito . . . 29
14.2 Actividad A: información general . . . 29
14.3 Actividad B: Windows . . . 29
14.4 Actividad C: macOS . . . 30
14.5 Actividad D: GNU/Linux . . . 30
14.6 Actividad E: comparación . . . 31

**15 Caso de análisis** **33**

**16 Resumen de la clase** **35**

**17 Tarea sugerida** **37**

**18 Fuentes oficiales para actualización** **39**

**19 Administración del sistema de archivos desde la línea de comandos** **41**

**20 Presentación de la clase** **43**
20.1 Objetivos de aprendizaje . . . 43
20.2 Ruta de trabajo de la sesión . . . 43

**21 ¿Dónde está realmente un archivo?** **45**

**22 El sistema de archivos** **47**
22.1 Unidades de almacenamiento . . . 47
22.2 Directorios y carpetas . . . 48
22.3 Archivos y extensiones . . . 48

**23 Rutas: la dirección de la información** **49**
23.1 Ruta absoluta . . . 49
23.2 Ruta relativa . . . 50
23.3 Separadores y espacios . . . 50

**24 De la interfaz gráfica a la línea de comandos** **51**

**25 ¿Qué es un comando?** **53**

**26 CMD en Windows** **55**
26.1 Cómo abrirlo . . . 55

**27 Anatomía del prompt** **57**

**28 Laboratorio narrativo: construyendo el proyecto SOI** **59**
28.1 Situación . . . 59
28.2 Paso 1. Saber dónde estamos . . . 59
28.3 Paso 2. Ver el contenido . . . 59
28.3.1 Variantes útiles . . . 60
28.4 Paso 3. Ir a Documentos . . . 60
28.5 Paso 4. Crear el directorio principal . . . 60
28.6 Paso 5. Entrar al proyecto . . . 61
28.7 Paso 6. Crear la estructura del semestre . . . 61
28.8 Paso 7. Visualizar el árbol . . . 61

**29 Navegación con `cd`** **63**
29.1 Entrar a un directorio . . . 63
29.2 Subir un nivel . . . 63
29.3 Ir a la raíz de la unidad . . . 63
29.4 Cambiar directamente mediante una ruta . . . 63
29.5 Volver al directorio anterior . . . 63

**30 Crear archivos de práctica** **65**

**31 Copiar archivos con `copy`** **67**
31.1 Copiar en el mismo directorio . . . 67
31.2 Copiar hacia otro directorio . . . 67
31.3 Evitar sobrescrituras accidentales . . . 67

**32 Renombrar con `ren`** **69**

**33 Mover con `move`** **71**

**34 Eliminar archivos con `del`** **73**
34.1 Riesgo de los comodines . . . 73

**35 Eliminar directorios con `rmdir`** **75**
35.1 Directorio vacío . . . 75
35.2 Directorio con contenido . . . 75

**36 Limpiar la pantalla con `cls`** **77**

**37 Resumen de comandos** **79**

**38 Reto de aplicación** **81**
38.1 Escenario . . . 81
38.2 Reglas . . . 81
38.3 Evidencia sugerida . . . 81

**39 Diagnóstico de errores frecuentes** **83**
39.1 “El sistema no puede encontrar la ruta especificada” . . . 83
39.2 “El sistema no puede encontrar el archivo especificado” . . . 83
39.3 “Acceso denegado” . . . 83
39.4 El comando se ejecutó en el lugar equivocado . . . 84

**40 Buenas prácticas** **85**

**41 Autoevaluación** **87**
41.1 Selección breve . . . 87
41.2 Preguntas de reflexión . . . 88

**42 Resumen de la clase** **89**

**43 Tarea sugerida** **91**

**44 Cierre** **93**

**45 Clase 3. Sistemas Operativos de Red** **95**

**46 Presentación de la clase** **97**

**47 Objetivos de aprendizaje** **99**

**48 Ruta de aprendizaje** **101**

**49 ¿Por qué existen los Sistemas Operativos de Red?** **103**
49.1 ¿Qué es un Sistema Operativo de Red? . . . 103
49.2 Sistema operativo de escritorio y Sistema Operativo de Red . . . 104

**50 Evolución de los Sistemas Operativos de Red** **105**
50.1 Los primeros sistemas multiusuario . . . 105
50.2 TCP/IP y la expansión de las redes . . . 106
50.3 Redes locales y Novell NetWare . . . 106
50.4 Windows NT y Windows Server . . . 106
50.5 El crecimiento de Linux . . . 107
50.6 Virtualización, contenedores y nube . . . 107

**51 Arquitecturas de red** **109**

**52 Arquitectura Punto a Punto** **111**
52.1 Ejemplo . . . 111
52.2 Ventajas . . . 111
52.3 Limitaciones . . . 112

52.4 ¿Cuándo puede utilizarse? . . . 112

**53 Arquitectura Cliente/Servidor 113**
53.1 Ejemplo universitario . . . 114
53.2 Ventajas . . . 114
53.3 Limitaciones . . . 114
53.4 Comparación general . . . 114

**54 Servicios que ofrece un Sistema Operativo de Red 117**

**55 Servicio de autenticación 119**
55.1 Autenticación y autorización . . . 119

**56 Servicio DNS 121**
56.1 Ejemplo de funcionamiento . . . 121
56.2 ¿Qué ocurriría sin DNS? . . . 122

**57 Servicio DHCP 123**
57.1 Beneficios . . . 123
57.2 Concesión o *lease* . . . 124

**58 Servicio de archivos 125**
58.1 Ventajas . . . 125
58.2 Ejemplo de estructura . . . 125
58.3 Riesgos que deben administrarse . . . 125

**59 Servicio de impresión 127**

**60 Servicio de directorio 129**

**61 Administración remota 131**

**62 ¿Qué ocurre detrás de escena al iniciar sesión? 133**

**63 Usuarios, grupos y permisos 135**

**64 Usuarios 137**
64.1 ¿Por qué evitar cuentas compartidas? . . . 137

**65 Grupos 139**
65.1 Ventaja administrativa . . . 139

**66 Permisos 141**
66.1 Caso: carpeta de calificaciones . . . 141

**67 Principio del mínimo privilegio 143**
67.1 Ejemplos . . . 143

**68 Plataformas de Sistemas Operativos de Red 145**

**69 Windows Server** **147**
69.1 Fortalezas . . . 147
69.2 Consideraciones . . . 147

**70 Linux Server** **149**
70.1 Fortalezas . . . 149
70.2 Consideraciones . . . 149

**71 UNIX** **151**

**72 Infraestructuras híbridas** **153**

**73 Roles de un servidor** **155**

**74 Servidor de archivos** **157**

**75 Servidor de impresión** **159**

**76 Servidor web** **161**

**77 Servidor de bases de datos** **163**

**78 Servidor de correo** **165**

**79 Servidor DNS** **167**

**80 Servidor DHCP** **169**

**81 Controlador de dominio o servidor de identidad** **171**

**82 Servidor de aplicaciones** **173**

**83 Servidor de respaldo** **175**

**84 Servidor de monitoreo** **177**

**85 ¿Un servidor puede tener varios roles?** **179**

**86 Infraestructura simplificada de una universidad** **181**

**87 Seguridad en los Sistemas Operativos de Red** **183**

**88 Triada de la seguridad** **185**
88.1 Confidencialidad . . . 185
88.2 Integridad . . . 185
88.3 Disponibilidad . . . 186

**89 Contraseñas y autenticación multifactor** **187**

**90 Actualizaciones** **189**

**91 Copias de seguridad** **191**
91.1 Regla 3-2-1 . . . 191
91.2 Un respaldo debe probarse . . . 191

**92 Registros y auditoría** **193**

**93 Seguridad física** **195**

**94 Continuidad operativa** **197**

**95 Ingeniería en la vida real** **199**

**96 Verifica tu comprensión** **201**
96.1 Pregunta 1 . . . 201
96.2 Pregunta 2 . . . 201
96.3 Pregunta 3 . . . 201
96.4 Pregunta 4 . . . 202
96.5 Pregunta 5 . . . 202
96.6 Pregunta 6 . . . 202

**97 Laboratorio narrativo: diseñando la red de una academia** **205**
97.1 Situación . . . 205
97.2 Misión . . . 205
97.3 Parte 1. Arquitectura . . . 205
97.4 Parte 2. Roles . . . 206
97.5 Parte 3. Usuarios y grupos . . . 206
97.6 Parte 4. Permisos . . . 206
97.7 Parte 5. Seguridad . . . 207
97.8 Parte 6. Diagrama . . . 207
97.9 Producto esperado . . . 207

**98 Reto del Ingeniero** **209**

**99 Resumen de la clase** **211**

**100 Cierre de la clase** **213**

**101 Referencias** **215**

**102 Clase 4. Administración de Procesos** **217**

**103 Presentación** **219**

**104 ¿Qué es un proceso?** **221**
104.1 Analogía . . . 221

**105 Modelo de Procesos** **223**

**106 Programa vs. Proceso** **225**

**107 Espacio de Direcciones** **227**

**108 Bloque de Control del Proceso (PCB) 229**

**109 Tabla de Procesos 231**
109.0.1 Ejemplo de una Tabla de Procesos . . . 232

**110 Módelos de Estados de un Proceso 233**
110.1 Evolución de los modelos de estados . . . 233
110.2 Modelo de cinco estados . . . 233
110.2.1 Descripción de los estados . . . 234
110.3 Modelo con procesos suspendidos (Swap) . . . 235
110.3.1 Nuevos estados . . . 236

**111 Resumen 239**

**112 Scheduler y Dispatcher 241**

**113 Cambio de contexto (Context Switch) 243**

**114 Ejemplo completo: Microsoft Word 245**
114.1 Autoevaluación . . . 247

**115 Clase 5. Control de Procesos y Concurrencia 249**

**116 Clase 5. Control de Procesos y Concurrencia 251**
116.1 Presentación de la clase . . . 251
116.2 Objetivos de aprendizaje . . . 251
116.3 Ruta de aprendizaje . . . 252

**117 ¿Qué es el control de procesos? 253**

**118 Proceso e hilo 255**
118.1 Proceso . . . 255
118.2 Hilo . . . 255
118.3 Comparación . . . 256

**119 Concurrencia y paralelismo 259**
119.1 Concurrencia . . . 259
119.2 Paralelismo . . . 259

**120 Quantum de tiempo 261**
120.1 ¿Qué ocurre cuando termina el quantum? . . . 261
120.2 Efecto del tamaño del quantum . . . 262

**121 Planificación Round Robin 263**
121.0.1 Interpretación . . . 263

**122 PCB y control del proceso 265**
122.1 Componentes principales . . . 265

**123 Cambio de contexto** **267**
123.1 Del mundo real al Sistema Operativo . . . 267
123.1.1 Relación entre el restaurante y el Sistema Operativo . . . 268
123.2 ¿Qué sucede realmente? . . . 268

**124 Recursos compartidos** **271**
124.1 Ejemplo: contador compartido . . . 271

**125 Condición de carrera** **273**

**126 Sección crítica** **275**

**127 Mecanismos de sincronización** **277**

**128 Algoritmo de Dekker** **279**
128.1 Idea general . . . 279
128.2 Pseudocódigo conceptual . . . 279
128.3 Explicación . . . 280

**129 Interbloqueo, inanición y livelock** **281**
129.1 Ejemplo de interbloqueo . . . 281

**130 Inteligencia artificial y control de procesos** **283**
130.1 Áreas de aplicación . . . 283
130.1.1 Predicción de carga . . . 283
130.1.2 Planificación adaptativa . . . 283
130.1.3 Detección de anomalías . . . 284
130.1.4 Administración energética . . . 284

**131 IA, PCB y Scheduler: relación correcta** **285**
131.1 Estado actual: planificadores extensibles . . . 286

**132 Comparación: planificación clásica y adaptativa** **287**

**133 Caso integrador: navegador durante una videoclase** **289**
133.1 Análisis . . . 289

**134 Laboratorio: observando procesos e hilos** **291**
134.1 Propósito . . . 291
134.2 Parte A. Windows . . . 291
134.3 Parte B. macOS . . . 291
134.4 Parte C. GNU/Linux . . . 292
134.5 Preguntas de análisis . . . 292

**135 Actividad en clase: simulación de Round Robin** **293**
135.1 Materiales . . . 293
135.2 Datos . . . 293
135.3 Instrucciones . . . 293
135.4 Tabla de resultados . . . 294

**136 Reto del Ingeniero** **295**

**137 Autoevaluación** **297**
137.1 Selección múltiple . . . . . 297
137.2 Preguntas de reflexión . . . . . 298

**138 Resumen de la clase** **299**

**139 Tarea sugerida** **301**

**140 Referencias y actualización** **303**

**141 Capítulo 6. ¿Cómo decide el Sistema Operativo quién utiliza el procesador?** **305**

**142 Capítulo 6. Algoritmos de planificación del CPU** **307**
142.1 Relación con la clase anterior . . . . . 307

**143 Objetivos de aprendizaje** **309**

**144 Motivación** **311**

**145 ¿Por qué es necesaria la planificación?** **313**

**146 ¿Qué aprenderemos en este capítulo?** **315**

**147 Preparándonos para la siguiente sección** **317**

**148 El Scheduler, el Dispatcher y el Kernel** **319**

**149 El Scheduler** **321**

**150 El Dispatcher** **323**

**151 ¿Qué hace realmente el Kernel?** **325**

**152 ¿Cómo trabajan juntos?** **327**

**153 Preparándonos para el primer algoritmo** **329**

**154 Algoritmo FCFS (FIFO)** **331**

**155 Un ejemplo cotidiano** **333**

**156 ¿Cómo trabaja FCFS?** **335**

**157 Ejemplo** **337**

**158 Diagrama de Gantt** **339**

**159 Ventajas** **341**

**160 Desventajas** **343**

**161 ¿Cuándo utilizar FCFS?** **345**

**162 Preparándonos para el siguiente algoritmo** **347**

**163 Algoritmo SJF (Shortest Job First)** **349**

**164 Un ejemplo cotidiano** **351**

**165 ¿Cómo trabaja SJF?** **353**

**166 Ejemplo** **355**

**167 Ventajas** **357**

**168 Desventajas** **359**

**169 ¿Cuándo utilizar SJF?** **361**

**170 FCFS vs. SJF** **363**
170.0.1 Ejemplo . . . . . 363

**171 Preparándonos para el siguiente algoritmo** **365**

**172 Algoritmo de Planificación por Prioridades** **367**

**173 Un ejemplo cotidiano** **369**

**174 ¿Cómo trabaja?** **371**

**175 Ejemplo** **373**

**176 Diagrama de Gantt** **375**
176.0.1 ¿Quién asigna la prioridad? . . . . . 375

**177 Ventajas** **377**

**178 Desventajas** **379**

**179 ¿Cómo evitar la inanición?** **381**

**180 Comparación** **383**

**181 Preparándonos para el siguiente algoritmo** **385**

**182 Algoritmo Round Robin (RR)** **387**

**183 Un ejemplo cotidiano** **389**

**184 ¿Qué es el Quantum?** **391**

**185 Ejemplo** **393**
185.0.1 Primera ronda . . . . . 393
185.0.2 Segunda ronda . . . . . 393
185.0.3 Tercera ronda . . . . . 393

**186 Diagrama de Gantt** **395**

**187 Ventajas** **397**

**188 Desventajas** **399**
188.1 ¿Cómo elegir un buen Quantum? . . . . . 399

**189 Comparación general** **401**

**190 Preparándonos para el cierre** **403**

**191 Comparación general de los algoritmos de planificación** **405**

**192 Caso integrador** **407**
192.1 Analiza . . . . . 407

**193 Autoevaluación** **409**

**194 Resumen del capítulo** **411**

**195 Conexión con el siguiente capítulo** **413**

**196 Capítulo 7. Del hardware al usuario: comprendiendo las capas del Sistema Operativo** **415**

**197 Capítulo 7. Capas del Sistema Operativo** **417**
197.1 Relación con la clase anterior . . . . . 417

**198 Objetivos de aprendizaje** **419**

**199 Motivación** **421**

**200 Un viaje por el Sistema Operativo** **423**

**201 ¿Por qué organizar el Sistema Operativo por capas?** **425**

**202 ¿Qué aprenderemos en este capítulo?** **427**

**203 Preparándonos para la siguiente sección** **429**

**204 Conociendo las capas del Sistema Operativo** **431**
204.1 Vista general . . . . . 432

**205 Capa 1. Usuario** **433**

**206 Capa 2. Aplicaciones** **435**

**207 Capa 3. Interfaz gráfica (GUI)** **437**

**208 Capa 4. Llamadas al Sistema (System Calls)** **439**

**209 ¿Qué observamos hasta ahora?** **441**

**210 El Kernel: el corazón del Sistema Operativo** **443**

**211 ¿Qué es el Kernel? 445**

**212 ¿Qué responsabilidades tiene el Kernel? 447**

**213 ¿Qué ocurriría si el Kernel no existiera? 449**

**214 Relación con los temas estudiados 451**

**215 Preparándonos para la siguiente sección 453**

**216 Tipos de Kernel 455**

**217 Kernel Monolítico 457**
217.1 Ventajas . . . 457
217.2 Desventajas . . . 457

**218 Microkernel 459**
218.1 Ventajas . . . 459
218.2 Desventajas . . . 459

**219 Kernel Híbrido 461**
219.1 Ventajas . . . 461
219.2 Desventajas . . . 461

**220 Comparación 463**

**221 Preparándonos para el cierre 465**

**222 Caso integrador: ¿Qué ocurre cuando abrimos una aplicación? 467**
222.1 Paso 1. El usuario solicita abrir la aplicación . . . 467
222.2 Paso 2. La aplicación solicita servicios . . . 467
222.3 Paso 3. El Kernel entra en acción . . . 467
222.4 Paso 4. Los Drivers controlan el hardware . . . 468
222.5 Paso 5. La aplicación aparece en pantalla . . . 468

**223 Autoevaluación 471**

**224 Resumen del capítulo 473**

**225 Conexión con el siguiente capítulo 475**

**226 Capítulo 8. Cuando nadie puede avanzar: introducción a los interbloqueos 477**

**227 Capítulo 8. Interbloqueos (Deadlocks) 479**
227.1 Relación con la clase anterior . . . 479

**228 Objetivos de aprendizaje 481**

**229 Motivación 483**

**230 ¿Qué es un interbloqueo? 485**

**231 Un ejemplo mínimo con dos procesos** **487**

**232 Las cuatro condiciones necesarias** **489**
232.1 1. Exclusión mutua . . . 489

**233 2. Retención y espera** **491**

**234 3. No expropiación** **493**

**235 4. Espera circular** **495**

**236 Idea clave: deben cumplirse las cuatro** **497**

**237 Tipos de recursos** **499**
237.1 Recursos exclusivos . . . 499
237.2 Recursos compartidos . . . 499

**238 ¿Qué componentes del Sistema Operativo están implicados?** **501**

**239 Ejemplo real: cruce vehicular** **503**

**240 Ejemplo clásico: filósofos comensales** **505**

**241 Analizando a los filósofos con las cuatro condiciones** **507**

**242 Ejemplo adicional: impresoras en red** **509**
242.1 ¿Cómo podría evitarse? . . . 509

**243 Ejemplo adicional: software de oficina** **511**

**244 Ejemplo adicional: cajero automático** **513**

**245 Actividad en clase: identifiquemos el interbloqueo** **515**

**246 Actividades por grupos** **517**
246.1 Grupo 1. Cruce vehicular . . . 517
246.2 Grupo 2. Filósofos comensales . . . 517
246.3 Grupo 3. Las cuatro condiciones . . . 517
246.4 Grupo 4. Clasificación de recursos . . . 518
246.5 Grupo 5. Una historia de interbloqueo . . . 518
246.6 Grupo 6. Semáforo lógico . . . 518

**247 Soluciones para discusión** **519**
247.1 Grupo 1 . . . 519
247.2 Grupo 2 . . . 519
247.3 Grupo 3 . . . 519
247.4 Grupo 4 . . . 519
247.5 Grupo 5 . . . 520
247.6 Grupo 6 . . . 520

**248 ¿Cómo evitar un interbloqueo?** **521**

**249 Caso integrador** **523**

**250 Autoevaluación** **525**

**251 Resumen del capítulo** **527**

**252 Conexión con el siguiente capítulo** **529**

**253 Bibliografía base** **531**

**254 Capítulo 9. Prevención, detección y recuperación de interbloqueos** **533**
254.1 Relación con la clase anterior . . . 533

**255 Objetivos de aprendizaje** **535**

**256 1. Recordemos antes de avanzar** **537**

**257 2. ¿Por qué prevenir interbloqueos?** **539**

**258 3. Prevención: atacar las condiciones necesarias** **541**
258.1 3.1 Exclusión mutua . . . 541
258.1.1 Mini ejercicio . . . 541

**259 4. Prevención de retención y espera** **543**
259.1 ¿Qué ocurre? . . . 543
259.2 ¿Cómo podemos prevenirlo? . . . 543
259.2.1 Ventaja . . . 543
259.2.2 Desventaja . . . 543

**260 5. Prevención mediante expropiación** **545**
260.1 El problema . . . 545

**261 6. Prevención de espera circular** **547**
261.0.1 Ejercicio rápido . . . 548

**262 7. Prevención, evitación y detección no son lo mismo** **549**

**263 8. Detección de interbloqueos** **551**

**264 9. Grafo de asignación de recursos** **553**
264.1 Practiquemos . . . 553

**265 10. El algoritmo del banquero** **555**

**266 11. ¿Qué es un estado seguro?** **557**

**267 12. Ejemplo básico del banquero** **559**

**268 13. Actividad por pares: ¿cómo lo resolverías?** **561**

**269 14. Recuperación después del interbloqueo** **563**
269.1 Finalizar procesos . . . 563

269.2 Retirar recursos selectivamente . . . 563

**270 15. Caso integrador 565**
270.1 Nivel 1. Diagnóstico . . . 565
270.2 Nivel 2. Prevención . . . 565
270.3 Nivel 3. Detección . . . 565
270.4 Nivel 4. Recuperación . . . 565
270.4.1 Nivel 1. Diagnóstico . . . 566
270.4.2 Nivel 2. Prevención . . . 566
270.4.3 Nivel 3. Detección . . . 566
270.4.4 Nivel 4. Recuperación . . . 566

**271 16. Actividad individual 567**

**272 17. Autoevaluación 569**

**273 18. Resumen del capítulo 571**

**274 Conexión con la siguiente clase 573**

**275 Bibliografía base 575**

**276 Actividad - Diagnóstico de un Sistema Operativo 577**

**277 Universidad Mariano Gálvez de Guatemala 579**
277.1 Escenario . . . 580

**278 Misión 1. Diagnóstico inicial — 5 puntos 581**

**279 Misión 2. ¿Qué ocurre dentro del Sistema Operativo? — 5 puntos 583**

**280 Misión 3. Sigan un proceso — 5 puntos 585**

**281 Misión 4. Ustedes son el Sistema Operativo — 5 puntos 587**

**282 Misión 5. Diagnóstico final — 5 puntos 589**
282.1 A. Recorrido de una solicitud . . . 589
282.2 B. Decisión final del grupo . . . 589

**283 Criterios de evaluación 591**

**284 Actividad - Diagnóstico de un Sistema Operativo 593**

**285 Antes de revisar las respuestas 595**

**286 Recordemos el escenario 597**

**287 Misión 1. Diagnóstico inicial — 5 puntos 599**
287.1 1. ¿Cuál parece ser el principal problema? . . . 599
287.2 2. ¿Qué otros factores pueden contribuir? . . . 599
287.3 3. ¿La CPU está dañada? . . . 600

288 Misión 2. ¿Qué ocurre dentro del Sistema Operativo? — 5 puntos 601

289 Misión 3. Sigan el proceso de impresión — 5 puntos 603
289.1 Una secuencia posible . . . 603
289.2 a. ¿Por qué aparece `Waiting`? . . . 603
289.3 b. ¿Qué debe conservar el Sistema Operativo? . . . 603
289.4 c. ¿Qué ocurre cuando responde la impresora? . . . 604

290 Misión 4. Ustedes son el Sistema Operativo — 5 puntos 605
290.1 1. ¿Qué criterio utilizar? . . . 605
290.2 2. ¿Todos deben tratarse igual? . . . 606
290.3 3. ¿Qué pasa si uno conserva demasiado la CPU? . . . 606

291 Misión 5. Diagnóstico final — 5 puntos 607
291.1 A. ¿Qué ocurre cuando presionamos Imprimir? . . . 607
291.2 B. ¿Qué acciones podrían realizarse? . . . 607
291.2.1 ¿Qué observar primero? . . . 608

292 Uniendo todo 609

293 ¿Cómo podía obtenerse el punteo? 611

294 Para cerrar 613

295 Capítulo 12. Virtualización de Sistemas Operativos 615

296 Capítulo 12. Virtualización de Sistemas Operativos 617
296.1 De una computadora… a varias computadoras . . . 617

297 Objetivos de aprendizaje 621

298 1. ¿Qué es la virtualización? 623

299 2. Cuatro conceptos que no debemos confundir 625
299.1 Ejemplo . . . 625

300 3. ¿De dónde salen los recursos de la VM? 627

301 4. Una VM no es simplemente otra ventana 629

302 5. ¿Para qué virtualizamos? 631

303 6. ¿Qué ocurre si algo sale mal? 633
303.1 Snapshot: el punto de guardado . . . 633

304 7. ISO, disco virtual y Sistema Operativo 635
304.1 Imagen ISO . . . 635
304.2 Disco virtual . . . 635
304.3 La relación . . . 636

**305 8. Antes de descargar: x86-64 y ARM64** **637**
305.1 x86-64 . . . 637
305.2 ARM64 . . . 637

**306 9. Virtualización y emulación no son lo mismo** **639**
306.1 Virtualización . . . 639
306.2 Emulación . . . 639

**307 10. ¿Qué programa podemos utilizar en 2026?** **641**

**308 11. Descargas oficiales** **643**
308.1 Hipervisores . . . 643
308.2 Sistemas Operativos . . . 643

**309 12. ¿Qué debería elegir un estudiante?** **645**
309.1 Caso A: PC Windows Intel/AMD . . . 645
309.2 Caso B: Mac Intel . . . 645
309.3 Caso C: Mac Apple Silicon . . . 645

**310 13. ¿Cuánta RAM, CPU y disco asignamos?** **647**
310.1 Ejemplo didáctico . . . 647

**311 14. Flujo para crear una máquina virtual** **649**

**312 15. Actividad guiada: diseñemos una VM sin instalar nada** **651**

**313 16. Mini reto: tú eres el administrador** **653**
313.0.1 Equipo A . . . 653
313.0.2 Equipo B . . . 653
313.0.3 Equipo C . . . 653

**314 17. Errores comunes al comenzar** **655**
314.1 “Le asignaré toda la RAM a la VM” . . . 655
314.2 “Descargué cualquier ISO de Internet” . . . 655
314.3 “Si tengo Apple Silicon puedo ejecutar cualquier x86 como si fuera nativo” . . . 655
314.4 “Snapshot significa que ya tengo backup” . . . 655
314.5 “VirtualBox es la única opción” . . . 655

**315 18. Autoevaluación** **657**

**316 19. Para recordar** **659**

**317 20. Síntesis visual del capítulo** **661**

**318 Conexión con la siguiente clase** **665**

**319 Referencias y actualización** **667**

**320 Capítulo 13. Manejo de memoria: particiones y paginación** **669**

**321 Capítulo 13. Manejo de memoria: particiones y paginación 671**
321.1 ¿Cómo caben tantos programas en la memoria? . . . . . 671

**322 Objetivos de aprendizaje 673**

**323 1. La memoria RAM como recurso crítico 675**
323.1 ¿Por qué importa? . . . . . 675

**324 2. Dirección lógica y dirección física 677**

**325 3. Primera estrategia: dividir la memoria en particiones 679**

**326 4. Particiones fijas 681**
326.1 Ventaja principal . . . . . 682
326.2 Problema principal . . . . . 682

**327 5. Particiones dinámicas 683**

**328 6. Fragmentación interna y externa 685**
328.1 Fragmentación interna . . . . . 685
328.2 Fragmentación externa . . . . . 685

**329 7. ¿Qué hueco debemos utilizar? 687**

**330 8. First Fit: el primer espacio suficiente 689**

**331 9. Best Fit: el espacio más ajustado 691**

**332 10. Worst Fit: el espacio más grande 693**

**333 11. Comparando los tres métodos 695**
333.1 Actividad rápida . . . . . 695

**334 12. El problema de exigir memoria contigua 697**

**335 13. Introducción a la paginación 699**

**336 14. La gran ventaja: memoria no contigua 701**

**337 15. Ejemplo numérico de paginación 703**

**338 16. ¿Cómo sabe el Sistema Operativo dónde está cada página? 705**

**339 17. Dirección lógica = página + desplazamiento 707**

**340 18. Calculando una dirección física 709**
340.1 Practiquemos . . . . . 710

**341 19. ¿La paginación elimina toda fragmentación? 711**

**342 20. Ventajas y costos de la paginación 713**
342.1 Ventajas . . . . . 713

342.2 Costos o desventajas . . . 713

**343 21. De la paginación a la memoria virtual 715**

**344 22. Caso práctico integrador 717**
344.1 Escenario A: particiones dinámicas . . . 717
344.2 Escenario B: paginación de 2 KB . . . 718

**345 23. Laboratorio en clase: tú administras la RAM 719**
345.1 Parte A — Particiones dinámicas . . . 719
345.2 Parte B — Paginación . . . 719
345.3 Parte C — Reflexión . . . 720
345.3.1 First Fit . . . 720
345.3.2 Best Fit . . . 720
345.3.3 Worst Fit . . . 721
345.3.4 Paginación de 2 KB . . . 721

**346 24. Errores comunes 723**
346.1 "Fragmentación interna y externa son lo mismo" . . . 723
346.2 "Best Fit siempre es la mejor estrategia" . . . 723
346.3 "Las páginas deben quedar juntas en RAM" . . . 723
346.4 "Página y marco son sinónimos" . . . 723
346.5 "La paginación elimina toda fragmentación" . . . 723

**347 25. Autoevaluación 725**

**348 26. Reto final 727**
348.0.1 1. Número de páginas . . . 727
348.0.2 2. Número de marcos . . . 727
348.0.3 3. Memoria física asignada . . . 727
348.0.4 4. Fragmentación interna . . . 727
348.0.5 5. Dirección física . . . 728
348.0.6 6. ¿Deben ser consecutivas? . . . 728

**349 27. Para recordar 729**

**350 Conexión con la siguiente clase 731**

**351 Referencias 733**

**352 Capítulo 14. Administración de memoria en ejecución 735**

**353 Capítulo 14. Administración de memoria en ejecución 737**
353.1 Del mapa de memoria a las decisiones del Sistema Operativo . . . 737

**354 Objetivos de aprendizaje 739**

**355 1. ¿Qué administra realmente el Sistema Operativo? 741**

**356 2. La jerarquía de memoria 743**
356.1 Una analogía sencilla . . . 744

**357 3. ¿Por qué existe la caché? 745**

**358 4. Dirección lógica y dirección física: retomemos el concepto 747**

**359 5. Protección de memoria 749**

**360 6. Registro base y límite 751**
360.1 ¿Y si solicita 2300? . . . . . 752

**361 7. Mapas de memoria: la fotografía de la RAM 753**

**362 8. Liberación: la memoria debe regresar al sistema 757**

**363 9. Compactación: reunir espacios libres 759**

**364 10. ¿Qué ocurre cuando la RAM no alcanza? 761**

**365 11. Swapping y paginación: no son exactamente lo mismo 763**

**366 12. Administración durante la ejecución 765**

**367 13. Cuando necesitamos una página que no está en RAM 767**

**368 14. Introducción a los algoritmos de reemplazo 769**

**369 15. Caso integrador: 1024 KB de RAM 771**
369.1 Paso 1. Memoria utilizada . . . . . 771
369.2 Paso 2. Memoria libre . . . . . 771

**370 16. Simulemos un mapa de memoria 773**

**371 17. Sistemas reales: una precisión importante 775**

**372 18. Caso de análisis final 777**

**373 19. Autoevaluación 779**

**374 20. Para recordar 781**

**375 Conexión con la siguiente clase 783**

**376 Referencias 785**

**377 Cierre de la clase 787**
377.1 Próximo paso . . . . . 787

# 1 Créditos Editoriales

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas

SISTEMAS OPERATIVOS I

Introducción y Fundamentos

Editorial Académica

Ingeniería en Sistemas

Ing. Jorge Luis García Chacón

Segundo Semestre

Edición 2026

Diseñar una vez.

Enseñar toda la vida.

# 2 Acerca de este volumen

Este libro forma parte de la colección **Editorial Académica de Ingeniería en Sistemas**.

Su propósito es proporcionar material de estudio moderno, visual y práctico para apoyar el proceso de enseñanza y aprendizaje.

Cada capítulo ha sido diseñado para combinar teoría, ejemplos, diagramas, laboratorios y ejercicios que permitan al estudiante comprender los conceptos y aplicarlos en situaciones reales.

# 3 Presentación de la clase

Bienvenidos al curso de **Sistemas Operativos I**. En esta primera clase estudiaremos el software que coordina prácticamente todo lo que ocurre en una computadora, un teléfono, un servidor o un dispositivo inteligente.

Imagine que enciende su computadora y, en pocos segundos, puede abrir un navegador, reproducir música, guardar archivos, conectarse a Internet y utilizar varios programas al mismo tiempo. Mientras usted observa una pantalla aparentemente sencilla, el equipo está detectando dispositivos, reservando memoria, iniciando servicios, creando procesos, aplicando controles de seguridad y organizando el acceso al almacenamiento.

La pieza que coordina esas actividades es el **sistema operativo**.

> **Idea central de la clase**
>
> El sistema operativo administra recursos, ofrece servicios a los programas y crea un entorno seguro y utilizable para las personas y las aplicaciones.

## 3.1 Objetivos de aprendizaje

Al finalizar esta clase, el estudiante será capaz de:

- Explicar qué es un sistema operativo y por qué es indispensable.
- Identificar la relación entre usuario, aplicaciones, sistema operativo y hardware.
- Reconocer las funciones principales del sistema operativo.
- Diferenciar una interfaz gráfica de una interfaz de línea de comandos.
- Clasificar sistemas operativos según su propósito y forma de trabajo.
- Explicar, de manera introductoria, qué son el kernel, los procesos y las llamadas al sistema.
- Identificar el sistema operativo, la versión y las características básicas de su propio equipo.

> **El libro que nos hubiera gustado tener**
>
> En este curso no memorizaremos listas aisladas. Relacionaremos cada concepto con situaciones que un ingeniero encontrará al administrar computadoras, servidores, redes y aplicaciones.

# 4 ¿Qué es un sistema operativo?

Un **sistema operativo** es el software fundamental que administra los recursos de un sistema informático y proporciona servicios comunes para que las aplicaciones puedan ejecutarse.

Actúa como intermediario entre:

- las personas que utilizan el equipo;
- las aplicaciones;
- el hardware;
- y los dispositivos conectados.

Sin un sistema operativo, cada programa tendría que conocer directamente cómo funciona cada modelo de procesador, disco, teclado, tarjeta de red o pantalla. El sistema operativo reduce esa complejidad mediante servicios y abstracciones.

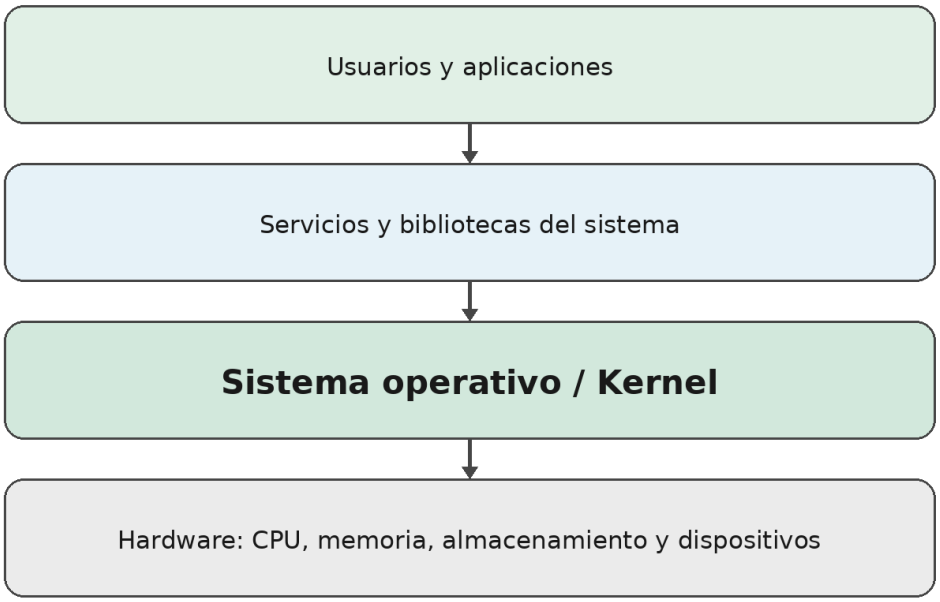



Figura 4.1: Capas principales de un sistema informático

## 4.1 Una comparación sencilla

Pensemos en un hotel:

- Los **huéspedes** representan a los usuarios y aplicaciones.
- Las **habitaciones, energía, ascensores y servicios** representan los recursos de hardware.
- La **administración del hotel** representa al sistema operativo.

Cada huésped utiliza servicios sin controlar directamente toda la infraestructura. La administración asigna recursos, organiza prioridades, establece reglas y evita conflictos.

> **!** Importante
>
> El sistema operativo no es únicamente la apariencia del escritorio. La interfaz gráfica es solo una de sus capas. Debajo de ella existen procesos, controladores, servicios, mecanismos de seguridad y un kernel que trabaja continuamente.

# 5 El kernel: núcleo del sistema operativo

El **kernel** es la parte central del sistema operativo. Se ejecuta con privilegios elevados y controla el acceso a los recursos esenciales del computador.

Entre sus responsabilidades se encuentran:

- planificar el uso del procesador;
- administrar la memoria;
- controlar dispositivos mediante controladores;
- coordinar operaciones de entrada y salida;
- aplicar mecanismos de protección;
- ofrecer servicios a los programas mediante llamadas al sistema.

El kernel no debe confundirse con todo el sistema operativo. Un sistema completo también incluye bibliotecas, utilidades, servicios, herramientas administrativas e interfaces de usuario.

## 5.1 Modo usuario y modo privilegiado

Los procesadores modernos permiten separar al menos dos niveles de ejecución:

| Nivel | Característica principal | Ejemplo |
|---|---|---|
| Modo usuario | Acceso limitado; las aplicaciones no controlan directamente todo el hardware | Navegador, editor de texto, reproductor |
| Modo kernel o privilegiado | Acceso a operaciones críticas y recursos protegidos | Kernel, controladores y componentes esenciales |

Esta separación evita que un error común en una aplicación tenga acceso irrestricto a toda la memoria o a los dispositivos del sistema.

> ⚠ Error común
>
> Decir que una aplicación “habla directamente con el hardware” suele ser una simplificación. En la mayoría de los casos, la aplicación solicita servicios al sistema operativo, y este coordina

la operación con el hardware.

# 6 Funciones principales de un sistema operativo

## 6.1 Gestión de procesos

Un **proceso** es un programa en ejecución. El sistema operativo debe decidir:

- qué proceso utiliza el procesador;
- durante cuánto tiempo;
- cuándo se pausa;
- cuándo continúa;
- y cómo se comunica con otros procesos.

Cuando una persona abre un navegador, un editor y una videollamada, el equipo parece ejecutar todo simultáneamente. En realidad, el sistema operativo distribuye el tiempo del procesador entre muchas actividades, aprovechando además los múltiples núcleos de la CPU.

## 6.2 Gestión de memoria

El sistema operativo asigna memoria a cada proceso y procura que una aplicación no modifique sin autorización la memoria de otra.

También puede utilizar **memoria virtual**, una técnica que permite organizar el espacio de direcciones de los procesos y, cuando es necesario, apoyar temporalmente la memoria física con almacenamiento secundario.

## 6.3 Gestión de archivos y almacenamiento

El sistema operativo organiza la información mediante archivos, carpetas, permisos y sistemas de archivos.

Entre sus tareas se encuentran:

- crear, leer, modificar y eliminar archivos;
- organizar directorios;
- controlar permisos;

- registrar ubicaciones y metadatos;
- coordinar el uso de discos, memorias USB y otras unidades.

## 6.4 Gestión de dispositivos

Los dispositivos poseen características distintas. Para utilizarlos, el sistema operativo emplea **controladores** o *drivers*.

Un controlador traduce solicitudes generales del sistema a operaciones específicas del dispositivo. Gracias a esta capa, una aplicación puede imprimir sin conocer todos los detalles internos de cada impresora.

## 6.5 Seguridad y protección

El sistema operativo administra usuarios, sesiones, permisos, credenciales y aislamiento entre procesos.

La seguridad no consiste únicamente en utilizar una contraseña. También incluye:

- controlar quién puede acceder a un archivo;
- limitar privilegios;
- proteger la memoria;
- aplicar actualizaciones;
- registrar eventos;
- bloquear operaciones no autorizadas.

## 6.6 Redes y comunicación

Los sistemas operativos modernos incluyen una pila de red que permite utilizar protocolos, interfaces, puertos y servicios de comunicación.

Cuando un navegador accede a una página web, el sistema operativo participa en la resolución de nombres, el envío de paquetes, la recepción de información y la entrega de datos a la aplicación correspondiente.

> En la práctica profesional
>
> Cuando un servidor deja de responder, un administrador no debería comenzar reinstalando el sistema operativo. Primero revisa el uso de CPU, memoria, disco, red, procesos y registros para identificar la causa del problema.

# 7 Breve evolución de los sistemas operativos

La evolución de los sistemas operativos está estrechamente relacionada con la evolución del hardware y las necesidades de los usuarios.

## 7.1 Primeras computadoras

En las primeras generaciones, los programas se cargaban manualmente y el operador controlaba gran parte del proceso. El tiempo de la computadora era costoso y se perdía mucho tiempo preparando cada trabajo.

## 7.2 Monitor residente

El **monitor residente** permanecía en memoria y automatizaba parte de la carga y ejecución de programas. Fue un paso inicial hacia la coordinación automática de trabajos.

## 7.3 Procesamiento por lotes

El procesamiento por lotes agrupó varios trabajos para ejecutarlos secuencialmente con poca interacción humana. Esta modalidad mejoró la utilización del equipo, aunque el usuario debía esperar para recibir resultados.

## 7.4 Multiprogramación y tiempo compartido

La multiprogramación permitió mantener varios trabajos en memoria y utilizar el procesador mientras otro esperaba una operación de entrada o salida.

El tiempo compartido extendió esta idea para atender a varios usuarios de forma interactiva, ofreciendo la impresión de que cada uno disponía del sistema.

## 7.5 Computadoras personales e interfaces gráficas

Las computadoras personales acercaron la informática a hogares y oficinas. Las interfaces gráficas facilitaron el uso mediante ventanas, iconos, menús y dispositivos apuntadores.

## 7.6 Internet, dispositivos móviles, nube y virtualización

Los sistemas actuales ya no se limitan a una computadora de escritorio. Existen sistemas para:

- servidores físicos y virtuales;
- teléfonos y tabletas;
- dispositivos integrados;
- infraestructura en la nube;
- vehículos y equipos industriales;
- plataformas de contenedores.

> **Perspectiva histórica**
>
> Los sistemas antiguos como MS-DOS, Windows XP o las primeras versiones de Mac OS siguen siendo importantes para comprender la evolución, pero no deben presentarse como el panorama tecnológico actual.

# 8 Panorama actual de sistemas operativos

La siguiente tabla presenta familias relevantes para un estudiante de Ingeniería en Sistemas.

| Familia | Uso frecuente | Característica general |
|---|---|---|
| Windows | Equipos personales, organizaciones y estaciones de trabajo | Amplio ecosistema comercial y compatibilidad de aplicaciones |
| Windows Server | Servidores empresariales | Servicios de red, identidades, virtualización y administración centralizada |
| GNU/Linux | Servidores, nube, desarrollo, escritorio, supercomputación y sistemas integrados | Código abierto, variedad de distribuciones y gran capacidad de automatización |
| macOS | Computadoras Mac | Integración estrecha entre hardware y software de Apple |
| Android | Teléfonos, tabletas, televisores y dispositivos | Plataforma móvil basada en el kernel Linux |
| iOS/iPadOS | iPhone y iPad | Integración con el ecosistema y hardware de Apple |
| Sistemas de tiempo real | Industria, vehículos, telecomunicaciones y control | Respuesta predecible dentro de límites temporales |

## 8.1 Windows y Windows Server

A julio de 2026, Windows 11 continúa como la familia principal de escritorio de Microsoft. Microsoft mantiene información oficial de versiones y canales de actualización en su portal de salud de versiones. Windows Server 2025 es la versión LTSC vigente de la plataforma de servidores y se orienta a seguridad, virtualización, almacenamiento e integración híbrida.

La línea de tiempo incluida en el material original muestra parte de la evolución de Windows. Se conserva como referencia histórica, pero debe leerse junto con la información actualizada de esta sección.

Figura 8.1: Evolución de Windows en el material original de la clase

## 8.2 GNU/Linux

Linux es el kernel iniciado por Linus Torvalds. Cuando se combina con herramientas, bibliotecas, gestores de paquetes y entornos de usuario, forma sistemas completos distribuidos en variantes como Ubuntu, Debian, Fedora, Rocky Linux, AlmaLinux o Arch Linux.

Ubuntu 24.04 LTS continúa siendo una versión de soporte prolongado ampliamente utilizada. Las versiones LTS priorizan estabilidad y mantenimiento extendido, una característica valiosa en organizaciones y servidores.

> **!** Precisión terminológica
>
> Linux es técnicamente el kernel. En el uso cotidiano se habla de “sistema operativo Linux”, pero una expresión más precisa para muchas distribuciones es GNU/Linux o simplemente el nombre de la distribución: Ubuntu, Debian, Fedora, etc.

## 8.3 macOS

macOS es el sistema operativo de las computadoras Mac. A julio de 2026, macOS Tahoe 26 es la versión estable con actualizaciones de seguridad publicadas por Apple; macOS 27 se encuentra anunciado o en ciclo previo a su disponibilidad general según el momento de consulta.

Para un libro reutilizable es preferible enseñar la familia, su arquitectura general y sus herramientas, en lugar de depender únicamente del número de versión.

## 8.4 Android e iOS

Android domina una gran variedad de dispositivos móviles y utiliza el kernel Linux como base, aunque su entorno de aplicaciones y servicios es diferente al de una distribución GNU/Linux de escritorio.

Android 17 fue publicado en junio de 2026. En el ecosistema Apple, iOS 26 continúa como versión estable con actualizaciones de seguridad, mientras iOS 27 fue anunciado para una etapa posterior.

> **Consejo del Ingeniero**
>
> Las versiones cambian con rapidez. En la práctica profesional, siempre consulte la documentación oficial antes de planificar una instalación, una actualización o una migración.

# 9 Interfaces de usuario

Una interfaz permite que una persona interactúe con el sistema operativo.

## 9.1 Interfaz gráfica de usuario

La **GUI** (*Graphical User Interface*) utiliza ventanas, iconos, menús, botones y otros elementos visuales.

Sus ventajas incluyen:

- facilidad de aprendizaje;
- descubrimiento visual de opciones;
- interacción mediante mouse, pantalla táctil o teclado;
- buena experiencia para tareas generales.

## 9.2 Interfaz de línea de comandos

La **CLI** (*Command-Line Interface*) permite escribir instrucciones en una terminal o intérprete de comandos.

Entre sus ventajas se encuentran:

- automatización mediante scripts;
- administración remota;
- precisión y repetibilidad;
- menor consumo de recursos en muchos escenarios;
- acceso rápido a herramientas avanzadas.

Figura 9.1: Ejemplos de interfaces conservados del material original

## 9.3 No son enemigas

GUI y CLI no deben verse como opciones excluyentes. Un profesional utiliza ambas según la tarea.

| Situación | Interfaz conveniente |
|---|---|
| Configurar una opción ocasional | GUI |
| Repetir la misma tarea en 100 equipos | CLI y automatización |
| Explorar visualmente archivos | GUI |
| Administrar un servidor remoto | CLI |
| Analizar procesos rápidamente | GUI o CLI, según el contexto |

**En la práctica profesional**

PowerShell, Bash y otras shells permiten convertir procedimientos manuales en scripts repetibles. Esa capacidad reduce errores y facilita administrar muchos equipos.

# 10 Clasificación de los sistemas operativos

Un sistema operativo puede pertenecer a varias categorías al mismo tiempo.

## 10.1 Monotarea y multitarea

- **Monotarea:** ejecuta una tarea principal a la vez.
- **Multitarea:** administra varios procesos concurrentes y distribuye recursos entre ellos.

Los sistemas generales actuales son multitarea.

## 10.2 Monousuario y multiusuario

- **Monousuario:** está diseñado principalmente para una sesión o usuario a la vez.
- **Multiusuario:** permite atender varios usuarios y separar recursos, permisos y sesiones.

Un servidor Linux o Windows Server puede atender múltiples usuarios y servicios simultáneamente.

## 10.3 Procesamiento por lotes

Ejecuta conjuntos de trabajos con poca interacción durante el procesamiento. Sigue siendo útil en tareas como generación nocturna de reportes, respaldo de información o procesamiento masivo de datos.

## 10.4 Tiempo compartido

Distribuye el tiempo de CPU entre múltiples usuarios o procesos interactivos, proporcionando respuestas suficientemente rápidas para mantener la sensación de atención individual.

## 10.5 Sistemas de red y distribuidos

Un sistema operativo de red facilita compartir servicios y recursos entre equipos conectados. Un sistema distribuido coordina componentes ubicados en diferentes nodos para ofrecer un servicio integrado.

## 10.6 Sistemas de tiempo real

Un sistema de tiempo real debe responder dentro de límites temporales definidos.

- **Tiempo real duro:** incumplir el plazo puede producir una falla crítica.
- **Tiempo real blando:** el retraso reduce la calidad del servicio, pero no necesariamente causa una falla catastrófica.

Ejemplos:

- control industrial;
- sistemas automotrices;
- telecomunicaciones;
- robótica;
- instrumentación médica;
- control de energía.

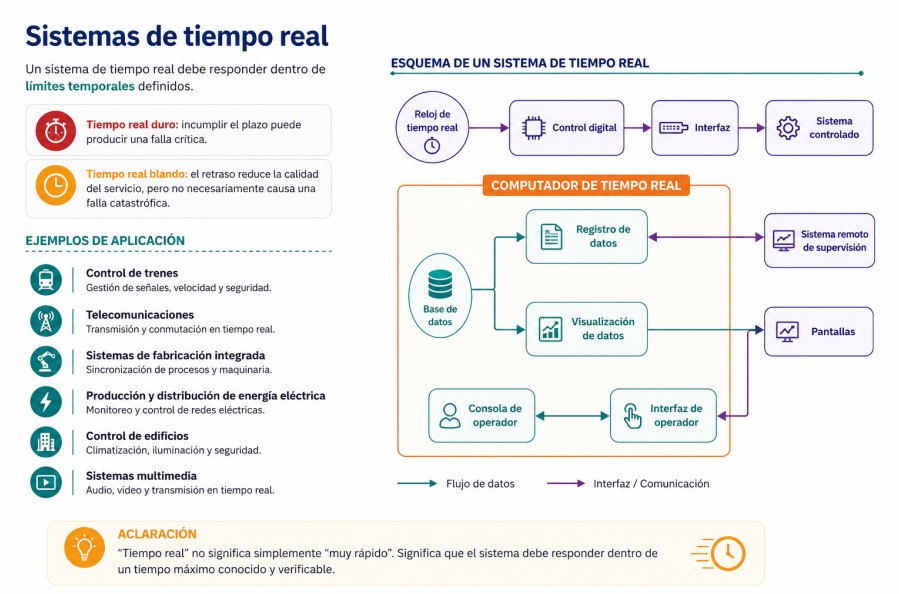



Figura 10.1: Esquema de sistema de tiempo real tomado del material original

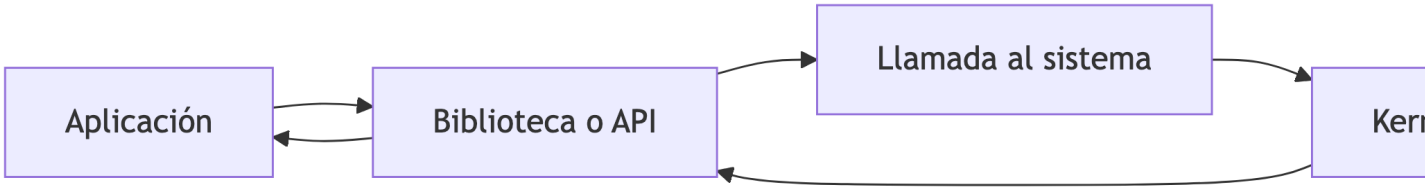
> **Aclaración**
>
> "Tiempo real" no significa simplemente "muy rápido". Significa que el sistema debe responder dentro de un tiempo máximo conocido y verificable.

# 11 Llamadas al sistema

Las aplicaciones necesitan solicitar servicios al kernel. Para hacerlo utilizan **llamadas al sistema**.

Algunos servicios típicos son:

- abrir o cerrar un archivo;
- crear un proceso;
- reservar memoria;
- comunicarse por red;
- leer datos de un dispositivo;
- consultar la hora del sistema.




## 11.1 Bibliotecas y API

Los lenguajes de alto nivel suelen utilizar bibliotecas que presentan funciones más sencillas. La biblioteca prepara los parámetros y realiza la transición necesaria para solicitar el servicio al sistema operativo.

Por ejemplo, una aplicación puede llamar una función para abrir un archivo. Esa función termina utilizando servicios del sistema operativo para validar permisos, localizar el archivo y coordinar el acceso al almacenamiento.

> **Observe**
>
> Las llamadas al sistema forman una frontera controlada entre las aplicaciones y el kernel. Esta frontera permite ofrecer servicios sin entregar a cada programa control total sobre el equipo.

# 12 Relación con la arquitectura del computador

Para comprender un sistema operativo también necesitamos recordar los componentes básicos del computador.

## 12.1 CPU

La **unidad central de procesamiento** ejecuta instrucciones y realiza operaciones aritméticas, lógicas y de control.

En sistemas modernos, una CPU puede incluir varios núcleos, memorias caché y mecanismos especializados. El sistema operativo planifica procesos e hilos para aprovechar esos recursos.

## 12.2 Memoria principal

La memoria RAM almacena temporalmente instrucciones y datos necesarios durante la ejecución.

Es rápida, pero normalmente pierde su contenido al apagar el equipo. El sistema operativo administra qué regiones utiliza cada proceso y coordina la memoria virtual.

## 12.3 Almacenamiento secundario

SSD, discos duros y otros medios conservan información de forma persistente. Son más lentos que la RAM, pero poseen mayor capacidad.

## 12.4 Unidad de control y ALU

La unidad de control coordina la ejecución de instrucciones dentro del procesador. La unidad aritmética y lógica realiza operaciones matemáticas y comparaciones.

En cursos modernos estos componentes se estudian como partes de la arquitectura del procesador, mientras el sistema operativo se analiza como el software que administra su utilización.

## 12.5 Periféricos y entrada/salida

Teclado, mouse, pantalla, almacenamiento, impresoras y adaptadores de red intercambian información con el sistema mediante controladores y mecanismos de entrada/salida.

> **Consejo del Ingeniero**
>
> Cuando una computadora está lenta, no asuma inmediatamente que “la CPU está mala”. El cuello de botella puede encontrarse en memoria, almacenamiento, red, temperatura, controladores o en un proceso específico.

# 13 Virtualización y contenedores: una mirada actual

La virtualización permite ejecutar máquinas virtuales con su propio sistema operativo sobre un hipervisor.

Una máquina virtual normalmente incluye:

- un sistema operativo invitado;
- su propio kernel;
- memoria virtual asignada;
- dispositivos virtuales.

Los contenedores aíslan procesos y sus dependencias, pero comparten el kernel del sistema anfitrión. Por eso suelen ser más ligeros que una máquina virtual completa.

| Característica | Máquina virtual | Contenedor |
|---|---|---|
| Kernel propio | Sí | Generalmente no; comparte el kernel anfitrión |
| Aislamiento | Alto | A nivel de procesos y recursos |
| Tamaño | Mayor | Menor |
| Inicio | Más lento | Habitualmente rápido |
| Uso típico | Sistemas completos, pruebas de SO, aislamiento fuerte | Empaquetado y despliegue de aplicaciones |

> **Primera aproximación**
>
> Virtualización y contenedores se estudiarán con mayor profundidad más adelante. En esta clase basta comprender que ambos dependen de funciones esenciales del sistema operativo.

# 14 Laboratorio 1: conozca su sistema operativo

## 14.1 Propósito

Identificar el sistema operativo del equipo y relacionar los conceptos estudiados con información real.

## 14.2 Actividad A: información general

Registre los siguientes datos:

| Dato | Resultado de su equipo |
|---|---|
| Sistema operativo | |
| Edición o distribución | |
| Versión | |
| Arquitectura | |
| Procesador | |
| Memoria RAM | |
| Nombre del equipo | |
| Usuario actual | |
| Espacio disponible | |

## 14.3 Actividad B: Windows

Abra **Terminal**, **PowerShell** o **Símbolo del sistema** y ejecute:

```
systeminfo
```

También puede utilizar:

```
Get-ComputerInfo
```

Comandos básicos seguros para explorar:

```
hostname
whoami
Get-Location
Get-ChildItem
```

Responda:

1. ¿Qué versión y compilación de Windows utiliza?
2. ¿Cuánta memoria física aparece instalada?
3. ¿Cuál es el nombre del equipo?
4. ¿Qué diferencia observa entre `Get-ChildItem` y la vista del Explorador de archivos?

## 14.4 Actividad C: macOS

Abra **Terminal** y ejecute:

```
sw_vers
uname -a
whoami
hostname
pwd
ls
system_profiler SPHardwareDataType
```

Responda:

1. ¿Qué versión de macOS utiliza?
2. ¿Qué arquitectura reporta el equipo?
3. ¿Qué información muestra `pwd`?
4. ¿Qué relación encuentra entre Finder y `ls`?

## 14.5 Actividad D: GNU/Linux

Abra una terminal y ejecute:

```
cat /etc/os-release
uname -r
hostnamectl
whoami
pwd
ls
```

Responda:

1. ¿Cuál es la distribución?
2. ¿Qué versión del kernel utiliza?
3. ¿Qué arquitectura tiene el equipo?
4. ¿Qué información proporciona `/etc/os-release`?

## 14.6 Actividad E: comparación

Complete la tabla:

| Acción | Windows | macOS/Linux |
|---|---|---|
| Mostrar usuario | `whoami` | `whoami` |
| Mostrar nombre del equipo | `hostname` | `hostname` |
| Mostrar carpeta actual | `Get-Location` | `pwd` |
| Listar archivos | `Get-ChildItem` | `ls` |
| Información del sistema | `systeminfo` | `sw_vers`, `hostnamectl` o `/etc/os-release` |

> **Regla del laboratorio**
>
> Ejecute únicamente los comandos indicados y lea su salida. No modifique usuarios, permisos, particiones, fecha, hora ni configuraciones del sistema durante esta práctica.

# 15 Caso de análisis

Una empresa posee:

- 40 computadoras de oficina;
- un servidor de archivos;
- una aplicación web;
- teléfonos utilizados por personal de campo;
- un sistema industrial que debe responder en milisegundos.

Analice qué tipo de sistema operativo podría utilizarse en cada contexto y justifique su respuesta considerando:

- propósito;
- seguridad;
- administración;
- interfaz;
- tiempo de respuesta;
- compatibilidad de aplicaciones.

# 16 Resumen de la clase

En esta clase aprendimos que:

- El sistema operativo administra hardware y ofrece servicios a las aplicaciones.
- El kernel es el núcleo privilegiado del sistema.
- Los procesos, la memoria, los archivos, los dispositivos, la seguridad y la red son recursos administrados por el sistema operativo.
- GUI y CLI son formas complementarias de interacción.
- Los sistemas operativos pueden clasificarse por su número de tareas, usuarios, forma de procesamiento y requisitos temporales.
- Las llamadas al sistema permiten a las aplicaciones solicitar servicios al kernel.
- Windows, GNU/Linux, macOS, Android, iOS y los sistemas de tiempo real responden a necesidades distintas.
- La virtualización y los contenedores dependen de capacidades fundamentales del sistema operativo.

> Consejo final del Ingeniero
>
> Un buen administrador no memoriza únicamente comandos. Comprende qué recurso está observando, qué problema intenta resolver y qué efecto tendrá cada acción.

# 17 Tarea sugerida

1. Elabore una comparación entre el sistema operativo de su computadora y el de su teléfono.
2. Identifique tres procesos activos en su equipo y explique, con sus propias palabras, para qué sirven.
3. Explique la diferencia entre kernel, sistema operativo e interfaz gráfica.
4. Investigue un ejemplo de sistema de tiempo real y explique por qué el cumplimiento del tiempo es importante.
5. Describa una situación en la que utilizaría una máquina virtual y otra en la que utilizaría un contenedor.
6. Incluya capturas de pantalla de la información obtenida en el laboratorio, ocultando datos personales que no desee compartir.

# 18 Fuentes oficiales para actualización

- Microsoft. *Windows 11 release information*: https://learn.microsoft.com/windows/release-health/windows11-release-information
- Microsoft. *Windows Server release information*: https://learn.microsoft.com/windows/release-health/windows-server-release-info
- Apple. *Apple security releases*: https://support.apple.com/100100
- Canonical. *Ubuntu release cycle*: https://ubuntu.com/about/release-cycle
- Linux Kernel Organization. *The Linux Kernel Archives*: https://www.kernel.org/
- Android Developers. *Android platform updates*: https://developer.android.com/latest-updates
- Docker. *What is a container?*: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

> **Nota editorial**
>
> La información de versiones fue revisada en julio de 2026. Debido a que los sistemas operativos se actualizan continuamente, las versiones exactas deben verificarse en la documentación oficial al impartir nuevamente la clase.

# 19 Administración del sistema de archivos desde la línea de comandos

Clase 2

# 20 Presentación de la clase

En la clase anterior estudiamos qué es un sistema operativo y por qué constituye la base de todo sistema informático. En esta sesión nos concentraremos en una de sus funciones más visibles: **organizar y administrar la información almacenada**.

Cada documento, fotografía, programa o proyecto necesita un lugar dentro del sistema. Cuando esa información está bien organizada, puede localizarse, copiarse, trasladarse o eliminarse con rapidez. Cuando no lo está, incluso un archivo que sigue existiendo puede parecer perdido.

La administración de archivos puede realizarse mediante ventanas e iconos, pero también a través de instrucciones escritas. Por ello, en esta clase utilizaremos el **Símbolo del sistema de Windows (CMD)** para comprender cómo piensa el sistema operativo cuando trabaja con unidades, directorios, rutas y archivos.

> **i** Idea central
>
> Los comandos no son palabras aisladas para memorizar. Son instrucciones que expresan una necesidad concreta: ver, entrar, crear, copiar, mover, renombrar o eliminar.

## 20.1 Objetivos de aprendizaje

Al finalizar esta clase, el estudiante será capaz de:

- Explicar cómo organiza el sistema operativo la información.
- Diferenciar unidades, directorios, subdirectorios y archivos.
- Interpretar rutas absolutas y relativas.
- Reconocer la función del intérprete de comandos de Windows.
- Utilizar comandos básicos para navegar y organizar información.
- Aplicar buenas prácticas para evitar errores durante la administración de archivos.

## 20.2 Ruta de trabajo de la sesión

| Momento | Actividad | Tiempo aproximado |
|---|---|---|
| Inicio | Situación problemática y conceptos base | 15 min |
| Desarrollo | Directorios, archivos, rutas y CMD | 20 min |

| Momento | Actividad | Tiempo aproximado |
|---|---|---|
| Laboratorio | Construcción del proyecto `SOI` | 35 min |
| Aplicación | Reto guiado y análisis de errores | 15 min |
| Cierre | Síntesis y autoevaluación | 5 min |

# 21 ¿Dónde está realmente un archivo?

Imagine que debe entregar un proyecto al finalizar la clase. Está seguro de haberlo guardado, pero no recuerda si quedó en **Documentos**, **Descargas**, **Escritorio** o una memoria USB.

El archivo no necesariamente se perdió. El problema puede ser que desconoce su ubicación.

Los sistemas operativos resuelven esta situación mediante una organización jerárquica. En lugar de almacenar toda la información en un único espacio, la distribuyen en unidades, directorios y subdirectorios.

Figura 21.1: Organización jerárquica del sistema de archivos

En esta estructura:

- La **unidad** representa un dispositivo o volumen de almacenamiento.
- El **directorio** agrupa información relacionada.
- El **subdirectorio** es un directorio ubicado dentro de otro.
- El **archivo** contiene los datos que utiliza el usuario o una aplicación.

Consejo del Ingeniero

Un archivo bien nombrado y guardado en una estructura lógica puede ahorrar más tiempo que cualquier comando avanzado.

# 22 El sistema de archivos

Un **sistema de archivos** es el conjunto de reglas y estructuras que utiliza el sistema operativo para registrar, organizar, localizar y proteger la información dentro de un dispositivo de almacenamiento.

El usuario observa carpetas y nombres. Internamente, el sistema operativo también mantiene datos como:

- ubicación lógica;
- tamaño;
- fecha de creación y modificación;
- permisos;
- atributos;
- relación entre directorios y archivos.

La organización lógica evita que el usuario deba conocer el lugar físico exacto donde se encuentran los datos dentro del dispositivo.

## 22.1 Unidades de almacenamiento

En Windows, las unidades suelen identificarse mediante una letra seguida de dos puntos:

```
C:
D:
E:
```

La unidad `C:` suele contener el sistema operativo y los perfiles de usuario. Otras letras pueden representar particiones, memorias USB, discos externos, unidades ópticas o recursos conectados.

> ⚠ No confundir
>
> La letra no indica por sí sola el tipo de dispositivo. Una memoria USB puede aparecer como `D:`, `E:` o cualquier otra letra disponible.

## 22.2 Directorios y carpetas

En Windows se utiliza con frecuencia la palabra **carpeta**. En el lenguaje técnico y en la línea de comandos se emplea el término **directorio**.

Ambos describen una estructura que permite agrupar archivos y otros directorios.

```
C:\
└── Usuarios
    └── Estudiante
        ├── Documentos
        ├── Descargas
        └── Escritorio
```

## 22.3 Archivos y extensiones

Un archivo es una unidad de información identificada mediante un nombre.

```
informe.docx
presupuesto.xlsx
programa.cpp
manual.pdf
imagen.png
```

La parte ubicada después del último punto se denomina **extensión**. Esta ayuda al sistema operativo y a las aplicaciones a reconocer el formato del archivo.

> **!** Idea clave
>
> Cambiar `fotografia.jpg` por `fotografia.pdf` no transforma la imagen en un PDF. Solo modifica el nombre.

# 23 Rutas: la dirección de la información

Una **ruta** indica la ubicación de un archivo o directorio dentro de la jerarquía del sistema.

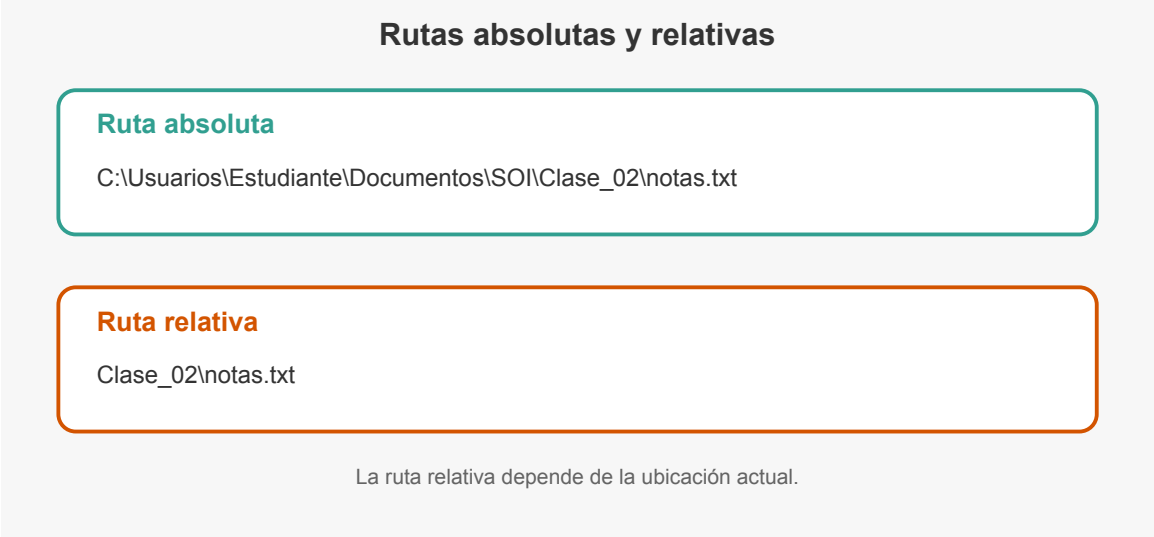



Figura 23.1: Ruta absoluta y ruta relativa

## 23.1 Ruta absoluta

Una ruta absoluta comienza desde la unidad y describe el recorrido completo.

```
C:\Usuarios\Estudiante\Documentos\SOI\Clase_02\notas.txt
```

Esta ruta puede interpretarse de izquierda a derecha:

1. Unidad `C:`.
2. Directorio `Usuarios`.
3. Perfil `Estudiante`.
4. Directorio `Documentos`.
5. Proyecto `SOI`.

6. Directorio `Clase_02`.
7. Archivo `notas.txt`.

## 23.2 Ruta relativa

Una ruta relativa se interpreta desde la ubicación actual.

Si la ubicación actual es:

```
C:\Usuarios\Estudiante\Documentos\SOI
```

la ruta relativa hacia el archivo anterior sería:

```
Clase_02\notas.txt
```

El símbolo `..` representa el directorio superior.

```
cd ..
```

significa:

> Subir un nivel dentro de la estructura.

## 23.3 Separadores y espacios

Windows utiliza la barra invertida `\` para separar los componentes de una ruta.

Cuando una ruta contiene espacios, es recomendable escribirla entre comillas:

```
cd "C:\Usuarios\Estudiante\Mis documentos"
```

# 24 De la interfaz gráfica a la línea de comandos

La interfaz gráfica permite administrar información mediante ventanas, iconos, menús y el puntero. La interfaz de línea de comandos permite hacerlo escribiendo instrucciones.

| Interfaz gráfica | Línea de comandos |
|---|---|
| Utiliza ventanas e iconos | Utiliza instrucciones escritas |
| Es visual e intuitiva | Es precisa y reproducible |
| Facilita tareas ocasionales | Facilita automatización y diagnóstico |
| Requiere navegación manual | Puede ejecutar secuencias rápidamente |

Ninguna reemplaza completamente a la otra. Un profesional utiliza la herramienta más adecuada para cada situación.

# 25 ¿Qué es un comando?

Un **comando** es una instrucción que el usuario entrega a un intérprete para solicitar una acción.

Ejemplos:

```
dir
```

Solicita mostrar el contenido del directorio actual.

```
mkdir Clase_02
```

Solicita crear un directorio llamado `Clase_02`.

```
copy notas.txt respaldo.txt
```

Solicita crear una copia del archivo.

Los comandos pueden aceptar:

- **argumentos**, que indican sobre qué elemento trabajar;
- **opciones**, que modifican la forma en que se ejecuta la instrucción.

Ejemplo:

```
dir /w
```

- `dir` es el comando.
- `/w` es una opción que muestra una vista resumida.

# 26 CMD en Windows

El **Símbolo del sistema**, también conocido como **CMD**, es un intérprete de comandos incluido en Windows.

No debe confundirse con MS-DOS. Aunque conserva numerosos comandos históricos, CMD funciona dentro de Windows y utiliza los servicios del sistema operativo actual.

## 26.1 Cómo abrirlo

Puede abrirse de distintas maneras:

1. Presionar la tecla Windows.
2. Escribir `cmd`.
3. Seleccionar **Símbolo del sistema**.

También puede utilizarse la ventana **Ejecutar**:

```
Windows + R
```

Luego escribir:

```
cmd
```

y presionar Enter.

> ⚠ Permisos administrativos
>
> Para esta práctica no es necesario abrir CMD como administrador. Trabajar con privilegios elevados aumenta el impacto de un error.

# 27 Anatomía del prompt

Al abrir CMD aparece una línea similar a:

```
C:\Users\Estudiante>
```

Figura 27.1: Anatomía del prompt de CMD

El prompt muestra la ubicación actual y termina con el símbolo > para indicar que el intérprete espera una instrucción.

```
C:                          unidad
\Users\Estudiante           ruta actual
>                           espera un comando
```

> **Observe**
>
> Cada vez que cambia de directorio, el prompt también cambia. Por eso funciona como una brújula dentro del sistema de archivos.

# 28 Laboratorio narrativo: construyendo el proyecto SOI

## 28.1 Situación

Durante el semestre se generarán apuntes, prácticas, recursos y entregables. Para evitar que todo quede mezclado, construiremos una estructura permanente llamada `SOI`.

El laboratorio se realizará dentro de **Documentos**. No trabaje en directorios del sistema como `C:\Windows` o `C:\Program Files`.

## 28.2 Paso 1. Saber dónde estamos

Escriba:

```
cd
```

CMD mostrará la ruta actual.

También puede observarla directamente en el prompt.

## 28.3 Paso 2. Ver el contenido

Ejecute:

```
dir
```

El comando `dir` muestra directorios y archivos de la ubicación actual.

En la salida encontrará datos como:

- fecha y hora;
- indicador `<DIR>` para directorios;
- tamaño de archivos;
- nombre de cada elemento.

### 28.3.1 Variantes útiles

```
dir /w
```

Muestra una vista resumida.

```
dir /p
```

Pausa la salida cuando ocupa más de una pantalla.

```
dir *.txt
```

Muestra únicamente archivos con extensión `.txt`.

> **Lectura antes de acción**
>
> Antes de crear, mover o borrar algo, use `dir`. Confirmar la ubicación reduce errores.

## 28.4 Paso 3. Ir a Documentos

Según el idioma y la configuración de Windows, el nombre visible puede variar. Una forma práctica es utilizar la variable del perfil:

```
cd /d "%USERPROFILE%\Documents"
```

Si la carpeta se muestra en español pero la ruta interna utiliza `Documents`, el comando seguirá funcionando.

La opción `/d` permite cambiar de unidad y directorio en una sola instrucción.

## 28.5 Paso 4. Crear el directorio principal

```
mkdir SOI
```

También puede utilizarse la forma abreviada:

```
md SOI
```

Ambos comandos crean el directorio.

Compruebe el resultado:

```
dir
```

## 28.6 Paso 5. Entrar al proyecto

```
cd SOI
```

El prompt debería terminar de forma similar a:

```
...\Documents\SOI>
```

## 28.7 Paso 6. Crear la estructura del semestre

Ejecute:

```
mkdir Clase_01 Clase_02 Clase_03 Laboratorios Recursos Proyecto_Final
```

Luego verifique:

```
dir
```

La estructura esperada es:

```
SOI
├── Clase_01
├── Clase_02
├── Clase_03
├── Laboratorios
├── Recursos
└── Proyecto_Final
```

## 28.8 Paso 7. Visualizar el árbol

Ejecute:

```
tree
```

Para incluir también archivos:

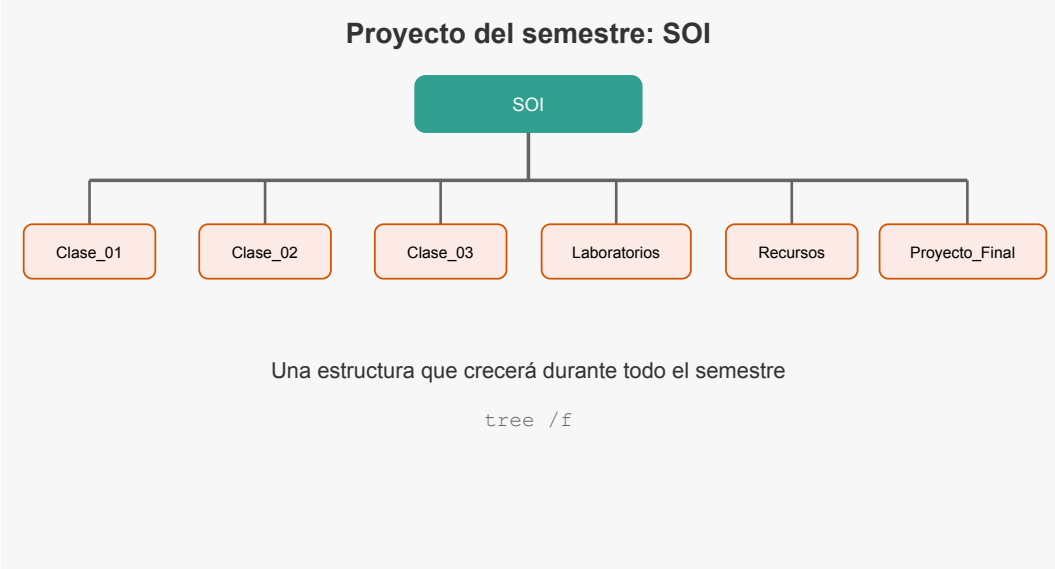



Figura 28.1: Estructura inicial del proyecto SOI

```
tree /f
```

`tree` permite comprobar visualmente la jerarquía construida.

# 29 Navegación con cd

El comando `cd` significa **change directory**, es decir, cambiar de directorio.

## 29.1 Entrar a un directorio

```
cd Clase_02
```

## 29.2 Subir un nivel

```
cd ..
```

## 29.3 Ir a la raíz de la unidad

```
cd \
```

## 29.4 Cambiar directamente mediante una ruta

```
cd /d "%USERPROFILE%\Documents\SOI\Clase_02"
```

## 29.5 Volver al directorio anterior

CMD no ofrece exactamente el mismo comportamiento que algunos intérpretes modernos, por lo que conviene leer el prompt y utilizar rutas relativas o absolutas de manera consciente.

**Error común**

`cd...` no es la forma correcta de subir un nivel. Debe existir un espacio:

```
cd ..
```

# 30 Crear archivos de práctica

Para continuar necesitamos archivos sencillos.

Ubíquese dentro de `Clase_02`:

```
cd /d "%USERPROFILE%\Documents\SOI\Clase_02"
```

Cree un archivo de texto:

```
echo Apuntes de la Clase 2 > apuntes.txt
```

Agregue una segunda línea:

```
echo Practica de comandos de Windows >> apuntes.txt
```

Los operadores utilizados son:

- `>` crea o reemplaza el contenido.
- `>>` agrega contenido al final.

Compruebe el archivo:

```
dir
```

Muestre su contenido:

```
type apuntes.txt
```

> **!** Precaución
>
> El operador `>` reemplaza el contenido existente. Antes de usarlo sobre un archivo importante, compruebe el nombre y la ruta.

# 31 Copiar archivos con copy

El comando `copy` crea una copia de uno o varios archivos.

## 31.1 Copiar en el mismo directorio

```
copy apuntes.txt respaldo_apuntes.txt
```

## 31.2 Copiar hacia otro directorio

```
copy apuntes.txt ..\Recursos\
```

La ruta `..\Recursos\` significa:

1. subir desde `Clase_02` hacia `SOI`;
2. entrar en `Recursos`.

Compruebe el resultado:

```
dir ..\Recursos
```

## 31.3 Evitar sobrescrituras accidentales

Puede utilizar:

```
copy /-y apuntes.txt respaldo_apuntes.txt
```

La opción `/-y` solicita confirmación antes de reemplazar un archivo existente.

# 32 Renombrar con `ren`

El comando `ren` cambia el nombre de un archivo o directorio.

```
ren respaldo_apuntes.txt apuntes_copia.txt
```

Compruebe:

```
dir
```

La instrucción cambia el nombre, pero no traslada el archivo a otra ubicación.

> **Diferencia importante**
>
> - `ren` cambia el nombre.
> - `move` cambia la ubicación y también puede cambiar el nombre.

# 33 Mover con `move`

Para trasladar un archivo:

```
move apuntes_copia.txt ..\Laboratorios\
```

Compruebe el directorio de destino:

```
dir ..\Laboratorios
```

También puede mover y renombrar en una sola instrucción:

```
move apuntes.txt ..\Laboratorios\bitacora_clase_02.txt
```

# 34 Eliminar archivos con `del`

El comando `del` elimina archivos.

Primero cree un archivo temporal:

```
echo Archivo temporal > temporal.txt
```

Compruebe:

```
dir
```

Luego elimínelo solicitando confirmación:

```
del /p temporal.txt
```

> **Advertencia**
>
> Los archivos eliminados desde CMD normalmente no pasan por la Papelera de reciclaje. Verifique siempre la ruta, el nombre y los comodines antes de confirmar.

## 34.1 Riesgo de los comodines

```
del *.txt
```

elimina todos los archivos `.txt` del directorio actual.

Este tipo de instrucción no debe ejecutarse sin revisar previamente:

```
dir *.txt
```

La secuencia profesional es:

```
1. Mostrar
2. Verificar
3. Ejecutar
```

# 35 Eliminar directorios con `rmdir`

`rmdir` o `rd` elimina directorios.

## 35.1 Directorio vacío

```
mkdir Prueba_Vacia
rmdir Prueba_Vacia
```

## 35.2 Directorio con contenido

```
rmdir /s Carpeta
```

La opción `/s` incluye todo su contenido.

Para solicitar confirmación:

```
rmdir /s Carpeta
```

Para omitirla existe `/q`, pero no debe utilizarse durante el aprendizaje:

```
rmdir /s /q Carpeta
```

`rmdir /s /q` puede eliminar una estructura completa sin preguntar. No lo utilice fuera de carpetas de práctica.

# 36 Limpiar la pantalla con `cls`

```
cls
```

Este comando limpia la pantalla, pero no elimina archivos ni modifica la ubicación actual.

Después de ejecutarlo, el prompt continúa en el mismo directorio.

# 37 Resumen de comandos

| Necesidad | Comando principal | Ejemplo |
|---|---|---|
| Mostrar ubicación | `cd` | `cd` |
| Listar contenido | `dir` | `dir /w` |
| Cambiar directorio | `cd` | `cd Clase_02` |
| Crear directorio | `mkdir` o `md` | `mkdir Recursos` |
| Mostrar árbol | `tree` | `tree /f` |
| Mostrar texto | `type` | `type apuntes.txt` |
| Copiar archivo | `copy` | `copy a.txt b.txt` |
| Renombrar | `ren` | `ren a.txt b.txt` |
| Mover | `move` | `move a.txt ..\Recursos\` |
| Eliminar archivo | `del` | `del /p a.txt` |
| Eliminar directorio | `rmdir` o `rd` | `rmdir Carpeta` |
| Limpiar pantalla | `cls` | `cls` |

# 38 Reto de aplicación

## 38.1 Escenario

El directorio `SOI` debe quedar organizado de la siguiente manera:

```
SOI
├── Clase_01
├── Clase_02
├── Clase_03
├── Laboratorios
│   └── bitacora_clase_02.txt
├── Recursos
│   └── apuntes.txt
└── Proyecto_Final
```

## 38.2 Reglas

Realice el reto únicamente con comandos.

1. Compruebe la ubicación actual.
2. Navegue hasta `SOI`.
3. Use `tree /f` para revisar la estructura.
4. Asegúrese de que `apuntes.txt` esté en `Recursos`.
5. Asegúrese de que `bitacora_clase_02.txt` esté en `Laboratorios`.
6. Elimine únicamente archivos temporales creados durante la práctica.
7. Muestre el árbol final.

## 38.3 Evidencia sugerida

El estudiante puede entregar:

- captura del comando `tree /f`;
- lista de comandos utilizados;
- explicación breve de un error cometido y cómo lo corrigió.

# 39 Diagnóstico de errores frecuentes

## 39.1 “El sistema no puede encontrar la ruta especificada”

Posibles causas:

- el directorio no existe;
- el nombre fue escrito incorrectamente;
- la ruta contiene espacios y no se utilizaron comillas;
- la ubicación actual no es la esperada.

Acciones:

```
cd
dir
```

Luego revise la ruta paso a paso.

## 39.2 “El sistema no puede encontrar el archivo especificado”

Compruebe:

```
dir
dir *.txt
```

Revise el nombre y la extensión real.

## 39.3 “Acceso denegado”

Puede ocurrir cuando:

- el archivo está protegido;
- otro programa lo utiliza;
- el usuario no posee permisos;
- se trabaja en un directorio del sistema.

No resuelva automáticamente el problema ejecutando todo como administrador. Primero determine la causa.

## 39.4 El comando se ejecutó en el lugar equivocado

Compruebe siempre el prompt. Una instrucción correcta ejecutada en un directorio incorrecto puede producir un resultado no deseado.

**Método de recuperación**

Cuando se sienta perdido:

```
cd
dir
```

Primero determine dónde está. Luego decida a dónde necesita ir.

# 40 Buenas prácticas

1. Utilice nombres descriptivos.
2. Evite crear todos los archivos en el Escritorio.
3. Compruebe la ubicación antes de modificar información.
4. Use `dir` antes de aplicar comandos destructivos.
5. Practique con archivos temporales.
6. No trabaje como administrador sin necesidad.
7. Mantenga respaldos de los archivos importantes.
8. Lea los mensajes del sistema antes de repetir una instrucción.

# 41 Autoevaluación

## 41.1 Selección breve

1. ¿Qué representa `C:`?

    - a) Un archivo
    - b) Una unidad
    - c) Un comando
    - d) Una extensión

2. ¿Qué comando muestra el contenido del directorio actual?

    - a) `show`
    - b) `list`
    - c) `dir`
    - d) `type`

3. ¿Qué significa `..` dentro de una ruta?

    - a) Directorio actual
    - b) Directorio superior
    - c) Unidad principal
    - d) Archivo oculto

4. ¿Cuál comando crea un directorio?

    - a) `mkdir`
    - b) `copy`
    - c) `type`
    - d) `cls`

5. ¿Cuál acción requiere mayor precaución?

    - a) `dir`
    - b) `cd`
    - c) `del`
    - d) `cls`

## 41.2 Preguntas de reflexión

1. ¿Por qué una ruta absoluta funciona aunque el usuario se encuentre en otro directorio?
2. ¿Qué ventaja ofrece una ruta relativa?
3. ¿Por qué conviene ejecutar `dir` antes de `del`?
4. ¿Qué diferencia existe entre copiar y mover?
5. ¿Por qué CMD no debe presentarse simplemente como MS-DOS?

# 42 Resumen de la clase

En esta clase aprendimos que:

- El sistema operativo organiza la información mediante un sistema de archivos.
- Las unidades contienen directorios, subdirectorios y archivos.
- Una ruta expresa la ubicación de un elemento.
- Las rutas pueden ser absolutas o relativas.
- CMD permite comunicarse con Windows mediante comandos.
- `dir`, `cd`, `mkdir` y `tree` permiten explorar y construir estructuras.
- `copy`, `ren` y `move` permiten reorganizar archivos.
- `del` y `rmdir` requieren especial cuidado.
- Una práctica profesional comienza por verificar la ubicación y el contenido.

> Consejo final del Ingeniero
>
> La línea de comandos se domina practicando. La velocidad llegará después; primero deben desarrollarse precisión, lectura y criterio.

# 43 Tarea sugerida

Construya desde CMD una estructura para organizar sus cursos del semestre:

```
Universidad
├── Sistemas_Operativos_I
├── Algoritmos
├── Programacion
└── Proyecto_Semestral
```

Dentro de cada curso cree:

```
Apuntes
Laboratorios
Tareas
Recursos
```

Genere un archivo `descripcion.txt` en cada curso y escriba una línea que describa su propósito. Entregue el resultado de:

```
tree /f
```

# 44 Cierre

En la próxima clase estudiaremos herramientas de administración de Windows que permiten inspeccionar configuración, servicios, inicio del sistema y otros componentes. El objetivo no será abrir utilidades por curiosidad, sino comprender qué problema resuelve cada una y qué riesgos implica modificarla.

# 45 Clase 3. Sistemas Operativos de Red

Administración centralizada, servicios y seguridad

# 46 Presentación de la clase

Cuando una computadora trabaja de forma aislada, su sistema operativo administra los recursos del equipo: procesador, memoria, archivos, dispositivos y aplicaciones. Sin embargo, en cuanto varias computadoras deben compartir información, impresoras, usuarios o aplicaciones, aparece una necesidad diferente: **administrar la red como un sistema organizado**.

Un laboratorio universitario, una clínica, una empresa o una institución pública no puede depender de que cada usuario configure manualmente su equipo, guarde los archivos donde quiera y decida por su cuenta qué información comparte. Conforme aumenta el número de computadoras, también crecen los riesgos, los errores y el esfuerzo de administración.

Los **Sistemas Operativos de Red** surgieron para resolver este problema. Permiten centralizar usuarios, permisos, servicios y recursos, de manera que una organización pueda trabajar con mayor seguridad, orden y continuidad.

> **Pregunta detonadora**
>
> Cuando un estudiante enciende una computadora del laboratorio e inicia sesión con su cuenta institucional, ¿quién valida su contraseña, le asigna una dirección IP, localiza los servidores y determina qué carpetas puede consultar?

La respuesta no depende de un único programa. Detrás de esa acción participan varios servicios coordinados por uno o más servidores. Esta clase explica cómo funciona esa infraestructura y por qué constituye una pieza esencial de cualquier organización moderna.

# 47 Objetivos de aprendizaje

Al finalizar esta clase, el estudiante será capaz de:

1. Explicar qué es un Sistema Operativo de Red y por qué es necesario.
2. Describir la evolución de los Sistemas Operativos de Red.
3. Diferenciar las arquitecturas Punto a Punto y Cliente/Servidor.
4. Identificar los principales servicios que operan dentro de una red.
5. Explicar la relación entre usuarios, grupos y permisos.
6. Reconocer los roles más comunes de un servidor.
7. Comparar, de manera general, Windows Server, Linux y UNIX.
8. Aplicar principios básicos de seguridad y continuidad operativa.
9. Diseñar una propuesta elemental de infraestructura para un caso real.

# 48 Ruta de aprendizaje

La clase seguirá la siguiente secuencia:

1. Comprenderemos por qué surgieron los Sistemas Operativos de Red.
2. Revisaremos su evolución histórica.
3. Compararemos las arquitecturas Punto a Punto y Cliente/Servidor.
4. Estudiaremos servicios como DNS, DHCP, autenticación y archivos.
5. Analizaremos usuarios, grupos y permisos.
6. Identificaremos plataformas y roles de servidor.
7. Relacionaremos la infraestructura con la seguridad.
8. Resolveremos un laboratorio basado en una organización real.

# 49 ¿Por qué existen los Sistemas Operativos de Red?

Imaginemos una oficina pequeña con tres computadoras. Cada usuario guarda sus documentos localmente y comparte algunos archivos mediante una carpeta. Esta solución puede funcionar al inicio.

Ahora imaginemos que la oficina crece hasta tener cincuenta empleados, varias sedes, impresoras compartidas, una base de datos y aplicaciones institucionales. Administrar cada computadora por separado se vuelve lento, inseguro y difícil de controlar.

Sin una administración centralizada pueden aparecer problemas como los siguientes:

- Usuarios con contraseñas débiles o compartidas.
- Archivos duplicados en distintas computadoras.
- Información importante sin respaldo.
- Permisos asignados de manera inconsistente.
- Direcciones IP repetidas.
- Equipos sin actualizaciones.
- Dificultad para identificar quién realizó una acción.
- Interrupciones prolongadas cuando falla un equipo.

Un Sistema Operativo de Red proporciona los mecanismos necesarios para organizar estos recursos y servicios.

## 49.1 ¿Qué es un Sistema Operativo de Red?

Un **Sistema Operativo de Red**, también llamado *Network Operating System* o NOS, es un sistema diseñado para administrar los recursos y servicios compartidos dentro de una red de computadoras.

Entre sus responsabilidades se encuentran:

- Administrar usuarios y equipos.
- Controlar el acceso a los recursos.
- Compartir archivos e impresoras.
- Proporcionar servicios de comunicación.
- Registrar eventos y actividades.
- Aplicar políticas de seguridad.

- Facilitar la administración remota.
- Mantener disponibles las aplicaciones institucionales.

**Idea central**

Un Sistema Operativo de Red no se limita a conectar computadoras. Su función principal es **organizar, proteger y administrar los recursos compartidos de una organización**.

## 49.2 Sistema operativo de escritorio y Sistema Operativo de Red

Un sistema operativo de escritorio está orientado principalmente a la experiencia de una persona frente a un equipo. Un Sistema Operativo de Red está orientado a ofrecer servicios a muchos usuarios y dispositivos.

| Aspecto | Sistema de escritorio | Sistema Operativo de Red |
|---|---|---|
| Enfoque principal | Uso individual | Servicios compartidos |
| Número de usuarios | Uno o pocos | Decenas, cientos o miles |
| Administración | Local | Centralizada |
| Disponibilidad esperada | Variable | Alta |
| Seguridad | Centrada en el equipo | Centrada en usuarios, servicios y datos |
| Acceso remoto | Complementario | Fundamental |
| Escalabilidad | Limitada | Diseñada para crecer |

En la práctica, ambos pueden compartir tecnologías. Un sistema moderno de escritorio puede compartir archivos y un servidor puede incluir interfaz gráfica. La diferencia principal está en el **propósito**, la **configuración** y el nivel de administración requerido.

# 50 Evolución de los Sistemas Operativos de Red

Los Sistemas Operativos de Red no aparecieron de forma repentina. Son el resultado de décadas de evolución en la forma de compartir recursos y procesar información.

Figura 50.1: Evolución de los Sistemas Operativos de Red

## 50.1 Los primeros sistemas multiusuario

Durante las décadas de 1960 y 1970, las computadoras eran equipos costosos y de gran tamaño. Para aprovecharlas, varios usuarios se conectaban mediante terminales. En este contexto, UNIX introdujo conceptos fundamentales:

- Multiusuario.
- Multitarea.
- Sistemas de archivos jerárquicos.
- Permisos.

- Administración remota.
- Herramientas pequeñas que podían combinarse.

Estas ideas influyeron en numerosos sistemas posteriores.

## 50.2 TCP/IP y la expansión de las redes

En la década de 1980, las implementaciones derivadas de BSD incorporaron herramientas de comunicación basadas en TCP/IP y sockets. Esto facilitó que diferentes equipos pudieran intercambiar información utilizando estándares comunes.

TCP/IP terminó convirtiéndose en la base de Internet y de la mayoría de redes actuales.

## 50.3 Redes locales y Novell NetWare

Con la expansión de las redes locales, empresas y universidades comenzaron a compartir archivos e impresoras. Novell NetWare se convirtió en una plataforma muy importante para estas tareas.

Su éxito se debió a que ofrecía:

- Buen rendimiento como servidor de archivos.
- Administración de usuarios.
- Servicios de impresión.
- Compatibilidad con diferentes clientes.
- Capacidad para operar en redes empresariales.

Aunque perdió presencia con el crecimiento de Windows Server y Linux, NetWare representa una etapa clave en la historia de las redes.

## 50.4 Windows NT y Windows Server

Microsoft desarrolló Windows NT como una plataforma orientada a entornos empresariales. Posteriormente evolucionó hacia la familia Windows Server.

Estas plataformas incorporaron servicios como:

- Administración de dominios.
- Active Directory.
- Políticas de grupo.
- Servicios de archivos e impresión.
- DNS y DHCP.
- Virtualización.
- Administración remota.

Su integración con equipos Windows facilitó su adopción en muchas organizaciones.

## 50.5 El crecimiento de Linux

Linux heredó la filosofía de UNIX y se desarrolló mediante una comunidad internacional. Su estabilidad, flexibilidad y capacidad de adaptación lo convirtieron en una opción ampliamente utilizada para servidores.

Hoy es común encontrar Linux en:

- Servidores web.
- Bases de datos.
- Centros de datos.
- Plataformas de nube.
- Dispositivos de red.
- Supercomputadoras.
- Sistemas embebidos.

## 50.6 Virtualización, contenedores y nube

Durante muchos años se instalaba un servidor físico para cada servicio. Esto provocaba altos costos de energía, espacio y mantenimiento.

La virtualización permitió ejecutar varios servidores virtuales dentro de un mismo equipo físico. Posteriormente, los contenedores facilitaron empaquetar aplicaciones y dependencias de forma más ligera.

La computación en la nube añadió otra posibilidad: utilizar infraestructura administrada por proveedores externos y pagar según la capacidad consumida.

> **Ingeniería en la vida real**
>
> Una universidad que antes necesitaba comprar un servidor físico para cada aplicación puede ejecutar varios servidores virtuales en un clúster local, mantener respaldos fuera del campus y utilizar servicios en la nube para atender períodos de alta demanda.

# 51 Arquitecturas de red

Una arquitectura de red describe cómo se organizan los equipos y cómo solicitan o proporcionan servicios. Dos modelos fundamentales son:

- Punto a Punto.
- Cliente/Servidor.

# 52 Arquitectura Punto a Punto

En una red **Punto a Punto**, también llamada *Peer to Peer* o P2P, los equipos poseen un nivel semejante. Cada computadora puede solicitar recursos y también compartir los propios.

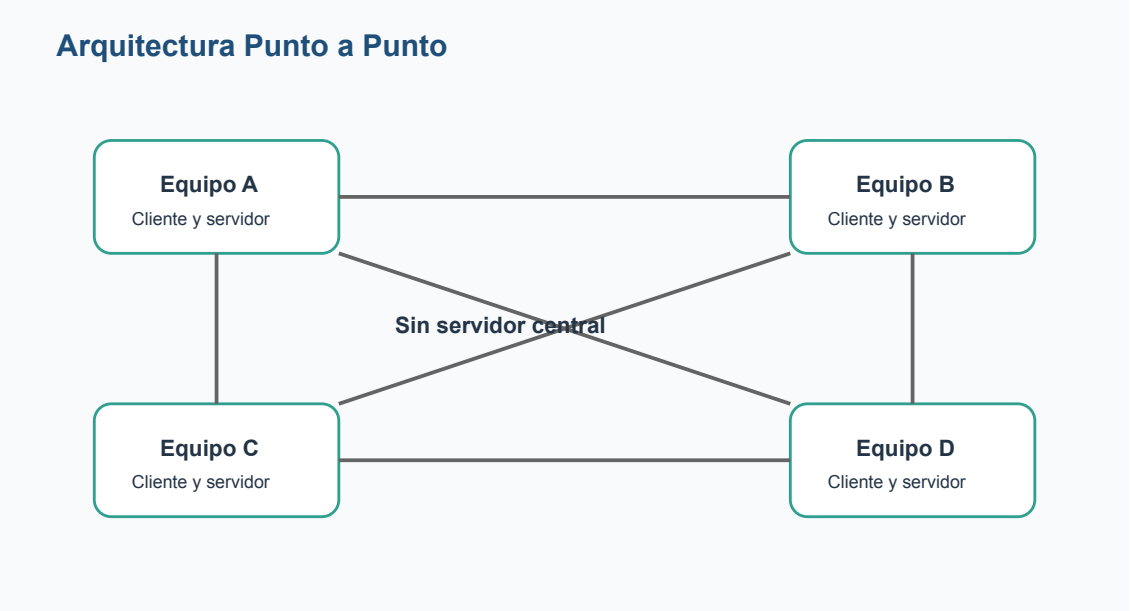



Figura 52.1: Arquitectura Punto a Punto

## 52.1 Ejemplo

Un pequeño despacho posee cuatro computadoras. Una comparte la impresora, otra contiene una carpeta de proyectos y las demás permiten consultar ciertos documentos.

No existe un servidor central. Cada usuario decide qué comparte y con quién.

## 52.2 Ventajas

- Bajo costo inicial.
- Configuración relativamente sencilla.
- No requiere un servidor dedicado.

- Puede ser suficiente para pocos usuarios.
- Permite compartir recursos rápidamente.

## 52.3 Limitaciones

- La administración depende de cada equipo.
- Es difícil mantener permisos consistentes.
- Los respaldos quedan dispersos.
- La seguridad es limitada.
- Un recurso deja de estar disponible si se apaga el equipo que lo comparte.
- El crecimiento aumenta rápidamente la complejidad.

## 52.4 ¿Cuándo puede utilizarse?

Puede ser apropiada cuando:

- Hay pocos equipos.
- La información no es crítica.
- El presupuesto es muy reducido.
- No se requiere administración centralizada.
- La red es temporal o doméstica.

> **Cuidado**
>
> Que una red Punto a Punto sea fácil de instalar no significa que sea adecuada para cualquier organización. Cuando existen datos sensibles, muchos usuarios o necesidad de auditoría, este modelo suele quedarse corto.

# 53 Arquitectura Cliente/Servidor

En una arquitectura **Cliente/Servidor**, uno o varios servidores proporcionan servicios especializados y los equipos cliente los solicitan.

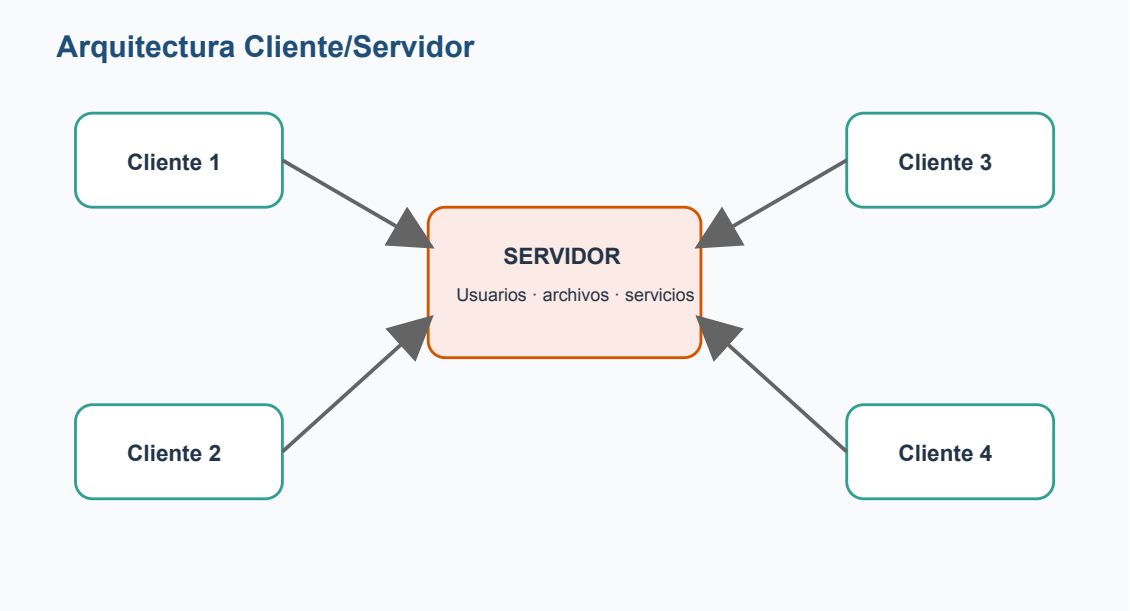



Figura 53.1: Arquitectura Cliente/Servidor

Un cliente puede ser:

- Una computadora de escritorio.
- Una computadora portátil.
- Un teléfono.
- Una tableta.
- Una aplicación.
- Otro servidor.

El servidor procesa las solicitudes y responde de acuerdo con las reglas, permisos y recursos disponibles.

## 53.1 Ejemplo universitario

En un laboratorio, los estudiantes inician sesión desde diferentes computadoras. Un servidor valida las credenciales, otro aloja el Campus Virtual y otro contiene las bases de datos.

El estudiante observa una experiencia sencilla, pero la infraestructura se encuentra distribuida y administrada.

## 53.2 Ventajas

- Administración centralizada.
- Mayor control de usuarios.
- Permisos consistentes.
- Respaldos más sencillos.
- Mejor capacidad de crecimiento.
- Posibilidad de alta disponibilidad.
- Auditoría y monitoreo centralizados.

## 53.3 Limitaciones

- Mayor inversión inicial.
- Requiere personal capacitado.
- Los servidores son componentes críticos.
- Una mala configuración puede afectar a muchos usuarios.
- Se necesita planificar respaldo, redundancia y recuperación.

## 53.4 Comparación general

| Característica | Punto a Punto | Cliente/Servidor |
|---|---|---|
| Servidor dedicado | No es necesario | Generalmente sí |
| Administración | Distribuida | Centralizada |
| Seguridad | Básica | Mayor control |
| Escalabilidad | Baja | Alta |
| Respaldos | Dispersos | Centralizados |
| Costo inicial | Bajo | Medio o alto |
| Disponibilidad | Depende de cada equipo | Puede diseñarse con redundancia |
| Uso típico | Hogares y oficinas pequeñas | Organizaciones medianas y grandes |

**Criterio de ingeniería**

La arquitectura adecuada no es la más costosa ni la más moderna. Es la que responde a los requerimientos de seguridad, capacidad, disponibilidad, presupuesto y crecimiento de la organización.

# 54 Servicios que ofrece un Sistema Operativo de Red

Los servidores proporcionan **servicios**. Cada servicio resuelve una necesidad específica de la red.

Entre los más importantes se encuentran:

- Autenticación.
- DNS.
- DHCP.
- Compartición de archivos.
- Impresión.
- Directorio.
- Bases de datos.
- Aplicaciones web.
- Correo electrónico.
- Administración remota.

# 55 Servicio de autenticación

La autenticación verifica la identidad de una persona o sistema.

El proceso básico es:

1. El usuario proporciona una identidad.
2. Presenta una credencial.
3. El servidor valida la información.
4. Autoriza o rechaza el acceso.
5. Registra el evento.

La contraseña es una credencial común, pero no es la única. También pueden utilizarse:

- Códigos temporales.
- Aplicaciones de autenticación.
- Certificados.
- Llaves físicas.
- Datos biométricos.

## 55.1 Autenticación y autorización

Estos conceptos no significan lo mismo.

- **Autenticación:** confirma quién es el usuario.
- **Autorización:** determina qué puede hacer.

Una persona puede autenticarse correctamente y aun así no tener permiso para abrir una carpeta financiera.

# 56 Servicio DNS

Las computadoras se comunican mediante direcciones IP, pero las personas recuerdan mejor los nombres.

El **Domain Name System** traduce nombres como:

```
campus.umg.edu.gt
```

en una dirección IP asociada al servidor.

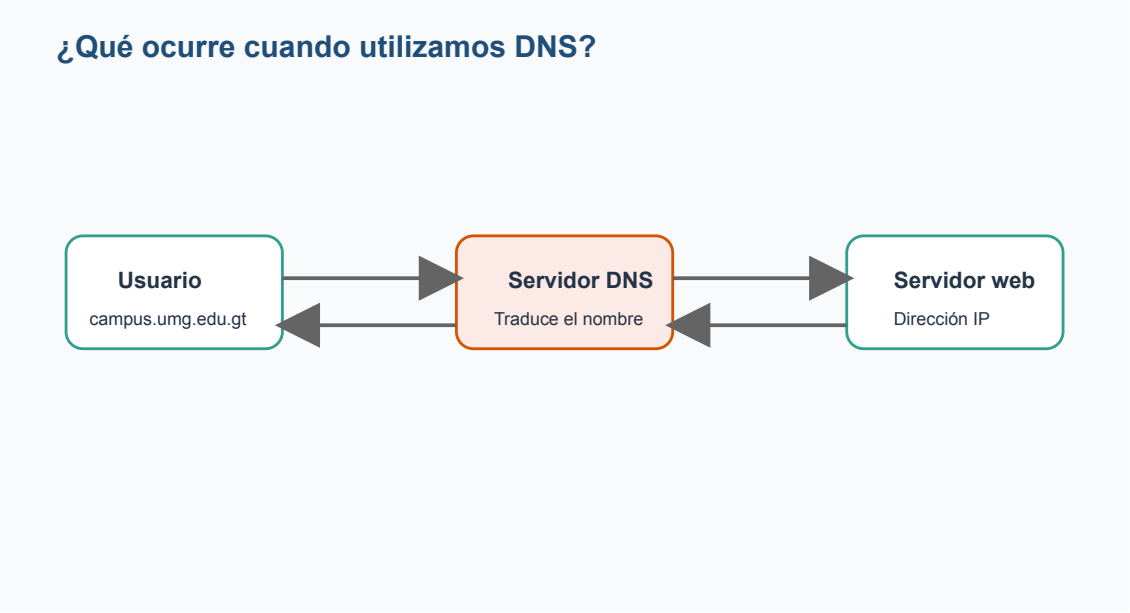



Figura 56.1: Proceso simplificado de resolución DNS

## 56.1 Ejemplo de funcionamiento

1. El usuario escribe una dirección en el navegador.
2. El equipo consulta al servidor DNS.
3. El servidor DNS busca la dirección IP.
4. La respuesta regresa al equipo.
5. El navegador se comunica con el servidor web.

**Idea central**

DNS responde a la pregunta: **¿qué dirección IP corresponde a este nombre?**

## 56.2 ¿Qué ocurriría sin DNS?

Los usuarios tendrían que memorizar direcciones IP. Además, cualquier cambio de dirección obligaría a comunicar la nueva configuración a todas las personas y aplicaciones.

DNS desacopla el nombre lógico de la dirección física utilizada en un momento determinado.

# 57 Servicio DHCP

Cada dispositivo necesita una configuración de red. Entre otros datos:

- Dirección IP.
- Máscara de subred.
- Puerta de enlace.
- Servidor DNS.

El **Dynamic Host Configuration Protocol** entrega estos valores automáticamente.

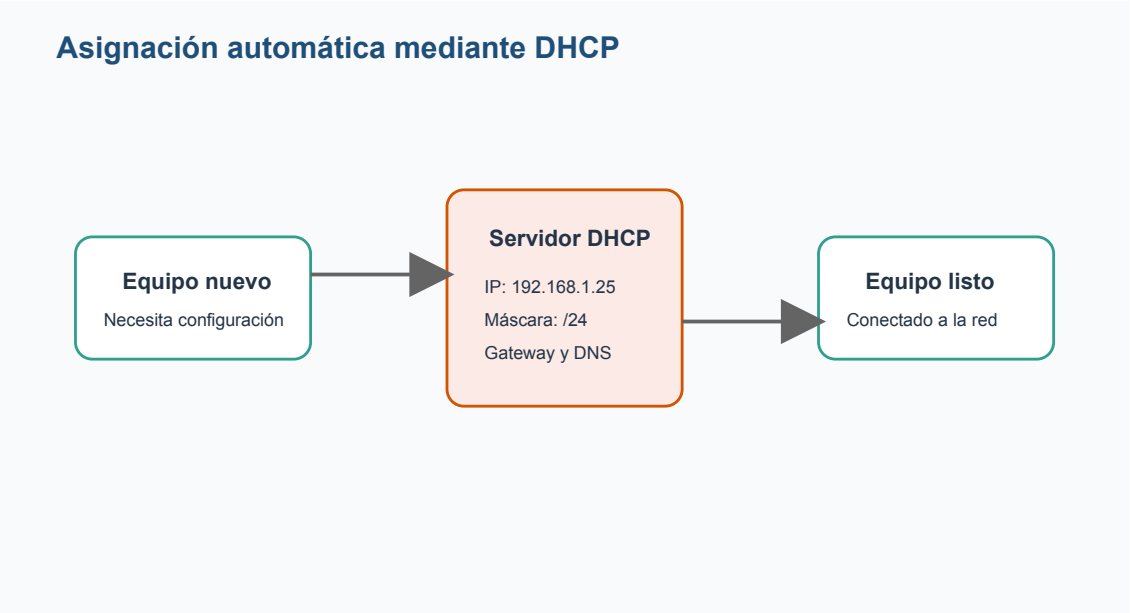



Figura 57.1: Asignación automática mediante DHCP

## 57.1 Beneficios

- Reduce errores manuales.
- Evita direcciones duplicadas.
- Facilita la incorporación de equipos.
- Permite cambiar la configuración de forma centralizada.
- Ahorra tiempo al personal técnico.

## 57.2 Concesión o *lease*

DHCP normalmente asigna una dirección durante un período determinado. El equipo puede renovarla mientras permanece conectado.

Esto permite reutilizar las direcciones disponibles cuando los dispositivos abandonan la red.

> **DNS y DHCP trabajan juntos**
>
> DHCP puede entregar al equipo la dirección del servidor DNS. DNS permite localizar servicios por nombre. Uno configura la comunicación y el otro facilita encontrar los recursos.

# 58 Servicio de archivos

Un servidor de archivos almacena información de forma centralizada y la comparte según los permisos definidos.

## 58.1 Ventajas

- Facilita los respaldos.
- Reduce la dispersión de documentos.
- Permite trabajo colaborativo.
- Mejora el control de acceso.
- Facilita auditoría y recuperación.
- Evita depender de un equipo personal.

## 58.2 Ejemplo de estructura

```
/Institución
├── Dirección
├── Administración
├── Docentes
├── Estudiantes
├── Biblioteca
└── Soporte
```

Cada carpeta puede tener permisos distintos.

## 58.3 Riesgos que deben administrarse

- Eliminación accidental.
- Modificaciones no autorizadas.
- Uso excesivo del almacenamiento.
- Archivos maliciosos.

- Acceso desde dispositivos no autorizados.
- Falta de versiones o respaldos.

# 59 Servicio de impresión

El servidor de impresión administra impresoras compartidas y las colas de trabajos.

Puede:

- Publicar impresoras.
- Controlar quién puede imprimir.
- Priorizar trabajos.
- Registrar consumo.
- Instalar controladores.
- Pausar o cancelar documentos.

En una organización grande, esta administración evita instalar y configurar cada impresora de forma independiente en todos los equipos.

# 60 Servicio de directorio

Un servicio de directorio almacena y organiza objetos de la red.

Entre ellos:

- Usuarios.
- Grupos.
- Equipos.
- Impresoras.
- Unidades organizativas.
- Políticas.
- Recursos compartidos.

Active Directory es una implementación conocida en entornos Microsoft. En otros contextos pueden utilizarse LDAP y herramientas compatibles.

Un directorio no es únicamente una lista de nombres. Permite consultar relaciones, aplicar políticas y administrar identidades a gran escala.

# 61 Administración remota

Los servidores suelen encontrarse en centros de datos, cuartos técnicos o servicios en la nube. Por eso deben poder administrarse sin estar físicamente frente a ellos.

La administración remota permite:

- Revisar el estado del sistema.
- Instalar actualizaciones.
- Consultar registros.
- Reiniciar servicios.
- Modificar configuraciones.
- Automatizar tareas.
- Atender incidentes.

Debe utilizar canales seguros, cuentas individuales y registros de auditoría.

# 62 ¿Qué ocurre detrás de escena al iniciar sesión?

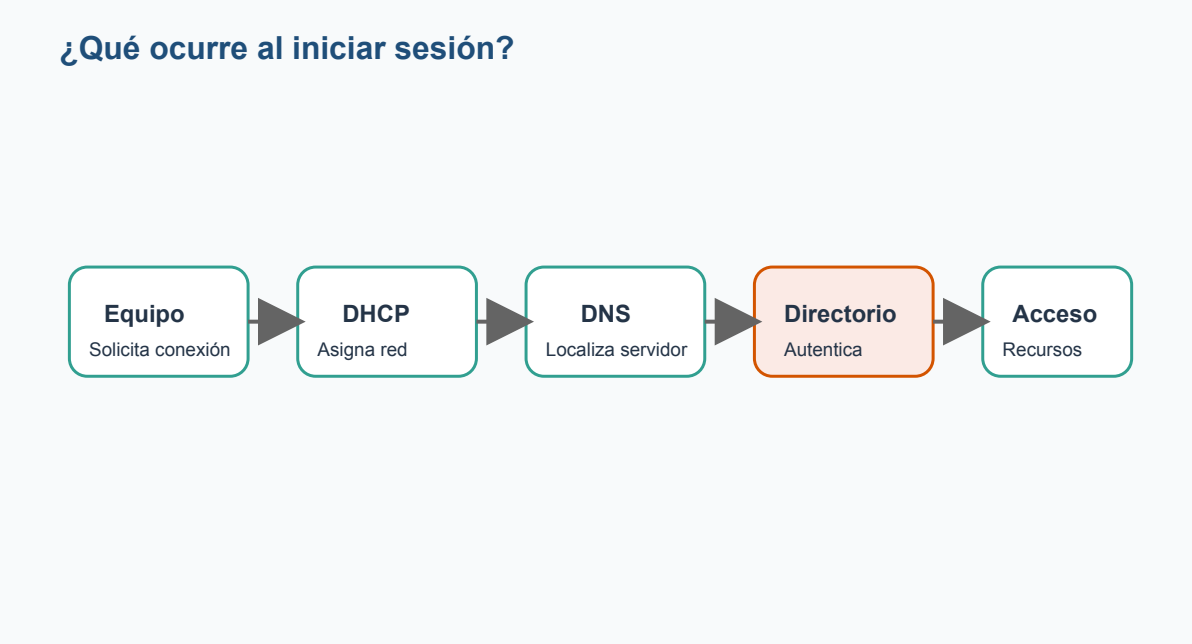



Figura 62.1: Flujo simplificado de inicio de sesión

Un inicio de sesión puede involucrar los siguientes pasos:

1. El equipo se conecta a la red.
2. DHCP proporciona la configuración necesaria.
3. DNS localiza el servidor de autenticación.
4. El usuario presenta sus credenciales.
5. El servicio de directorio valida la identidad.
6. El sistema consulta grupos y permisos.
7. Se aplican políticas.
8. Se habilitan carpetas, impresoras y aplicaciones autorizadas.
9. El evento queda registrado.

Para el usuario, el proceso dura unos segundos. Para el administrador, representa una cadena de servicios que debe funcionar correctamente.

# 63 Usuarios, grupos y permisos

La seguridad de una red se apoya en tres componentes relacionados:

- Usuarios.
- Grupos.
- Permisos.

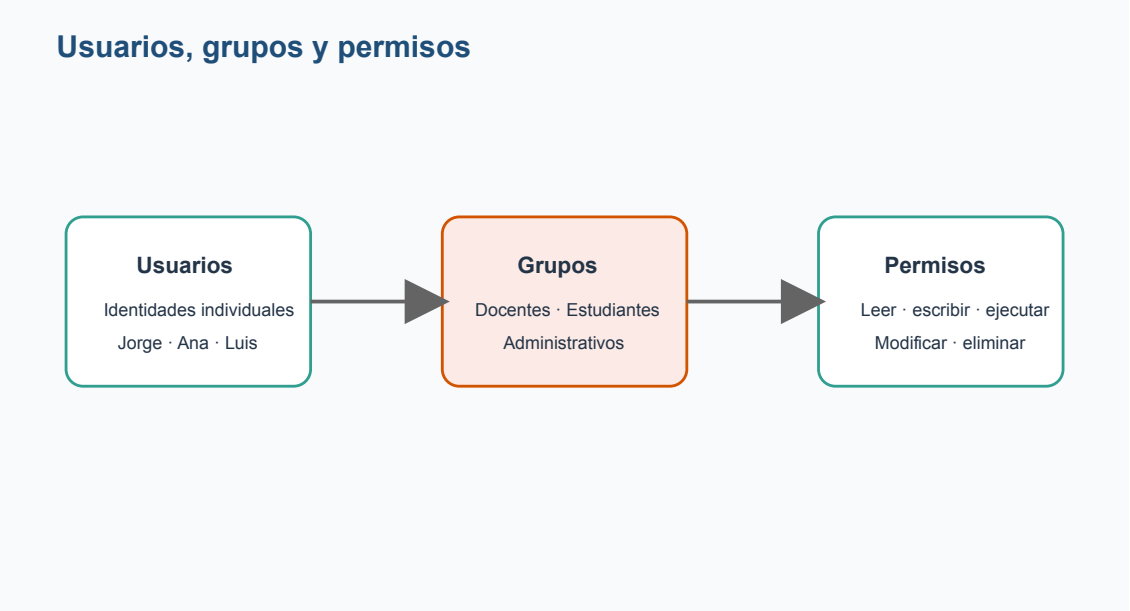



Figura 63.1: Relación entre usuarios, grupos y permisos

# 64 Usuarios

Un usuario representa una identidad digital. Puede corresponder a:

- Una persona.
- Un servicio.
- Una aplicación.
- Un equipo.
- Una cuenta temporal.

Una cuenta debería ser individual cuando se necesita responsabilidad y auditoría.

## 64.1 ¿Por qué evitar cuentas compartidas?

Si varias personas utilizan la misma cuenta:

- No se sabe quién realizó una acción.
- Es difícil revocar el acceso de una sola persona.
- La contraseña se comparte ampliamente.
- La auditoría pierde valor.
- Aumenta el riesgo de uso indebido.

# 65 Grupos

Un grupo reúne usuarios con funciones semejantes.

Ejemplos:

- Docentes.
- Estudiantes.
- Tesorería.
- Recursos Humanos.
- Soporte Técnico.
- Administradores.

En lugar de asignar permisos persona por persona, se asignan al grupo y luego se incorporan los usuarios correspondientes.

## 65.1 Ventaja administrativa

Si ingresan diez docentes nuevos, basta con agregarlos al grupo **Docentes** para que reciban los permisos establecidos.

Si una persona cambia de puesto, se modifican sus grupos en lugar de revisar recurso por recurso.

# 66 Permisos

Los permisos determinan las acciones autorizadas sobre un recurso.

| Permiso | Descripción |
|---|---|
| Lectura | Consultar información |
| Escritura | Modificar contenido |
| Crear | Agregar nuevos elementos |
| Ejecutar | Iniciar un programa o proceso |
| Eliminar | Borrar información |
| Administrar | Cambiar permisos o configuración |

## 66.1 Caso: carpeta de calificaciones

| Rol | Leer | Escribir | Eliminar |
|---|---|---|---|
| Coordinación | Sí | Sí | Sí |
| Docente | Sí | Sí | Según política |
| Estudiante | Solo sus resultados | No | No |
| Visitante | No | No | No |

# 67 Principio del mínimo privilegio

Este principio indica que una persona debe recibir únicamente los permisos necesarios para realizar su trabajo.

## 67.1 Ejemplos

- Un estudiante no necesita administrar servidores.
- Una recepcionista no necesita modificar respaldos.
- Un técnico de soporte no necesita consultar salarios.
- Una aplicación web no debería conectarse a la base de datos como administrador.

> ! **Idea central**
>
> Dar permisos “por si acaso” facilita el trabajo al inicio, pero aumenta el riesgo. El acceso debe justificarse por una necesidad real.

# 68 Plataformas de Sistemas Operativos de Red

No existe una plataforma universalmente superior. La selección depende de requisitos, experiencia del equipo, aplicaciones, presupuesto y soporte.

# 69 Windows Server

Windows Server está orientado a entornos empresariales y se integra estrechamente con estaciones de trabajo Windows.

Entre sus capacidades se encuentran:

- Active Directory.
- DNS y DHCP.
- Políticas de grupo.
- Servicios de archivos.
- Servicios de impresión.
- Virtualización.
- Administración gráfica y mediante línea de comandos.
- Integración con servicios Microsoft.

## 69.1 Fortalezas

- Administración centralizada.
- Amplia adopción empresarial.
- Integración con escritorios Windows.
- Herramientas gráficas.
- Ecosistema de soporte.

## 69.2 Consideraciones

- Licenciamiento.
- Requisitos de hardware.
- Necesidad de planificar actualizaciones.
- Dependencia de herramientas y conocimientos especializados.

# 70 Linux Server

Linux está disponible mediante diferentes distribuciones.

Algunas están orientadas a estabilidad empresarial, otras a facilidad de uso o flexibilidad.

Es común utilizar Linux para:

- Servidores web.
- Bases de datos.
- DNS y DHCP.
- Almacenamiento.
- Contenedores.
- Automatización.
- Infraestructura en la nube.

## 70.1 Fortalezas

- Flexibilidad.
- Estabilidad.
- Automatización.
- Amplia comunidad.
- Posibilidad de utilizar herramientas abiertas.
- Buen desempeño en servidores.

## 70.2 Consideraciones

- La administración puede requerir experiencia en línea de comandos.
- Las distribuciones poseen ciclos de soporte diferentes.
- Debe seleccionarse una versión adecuada para producción.
- La flexibilidad también exige disciplina de configuración.

# 71 UNIX

UNIX representa una familia y una filosofía de sistemas que influyó profundamente en la informática moderna.

Sus características históricas incluyen:

- Multiusuario.
- Multitarea.
- Portabilidad.
- Permisos robustos.
- Herramientas modulares.
- Administración mediante texto y comandos.

Algunas variantes comerciales continúan utilizándose en ambientes especializados y sistemas críticos.

# 72 Infraestructuras híbridas

En la práctica, muchas organizaciones combinan plataformas.

Ejemplo:

- Windows Server para usuarios y políticas.
- Linux para aplicaciones web.
- Un dispositivo especializado para firewall.
- Servicios de respaldo en la nube.
- Bases de datos alojadas en otra plataforma.

**Criterio profesional**

El trabajo del ingeniero no consiste en defender una marca. Consiste en justificar una decisión técnica con base en necesidades, riesgos, costos y capacidades.

# 73 Roles de un servidor

Un **rol** describe la función que cumple un servidor dentro de la infraestructura.

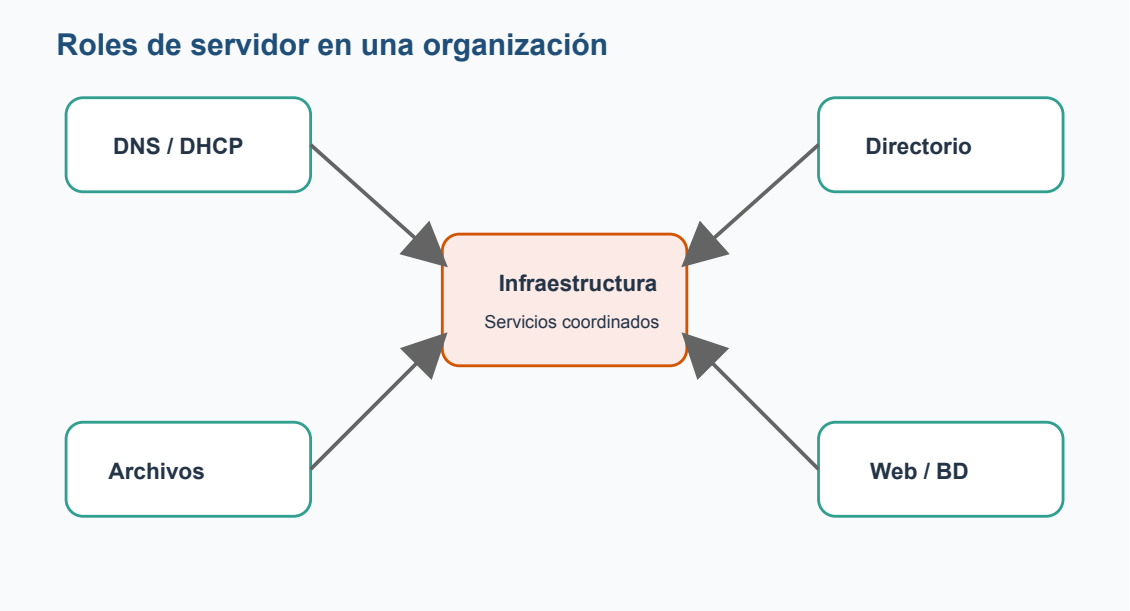



Figura 73.1: Roles de servidor

# 74 Servidor de archivos

Centraliza documentos y los comparte según permisos.

# 75 Servidor de impresión

Administra impresoras, colas y control de acceso.

# 76 Servidor web

Publica sitios y aplicaciones mediante protocolos web.

Programas conocidos incluyen Apache, Nginx e IIS.

# 77 Servidor de bases de datos

Almacena información estructurada y procesa consultas.

Puede utilizar motores como:

- PostgreSQL.
- MySQL.
- Microsoft SQL Server.
- Oracle Database.

# 78 Servidor de correo

Gestiona envío, recepción, buzones y autenticación del correo institucional.

# 79 Servidor DNS

Traduce nombres en direcciones IP.

# 80 Servidor DHCP

Entrega automáticamente la configuración de red.

# 81 Controlador de dominio o servidor de identidad

Administra usuarios, grupos, equipos y políticas de autenticación.

# 82 Servidor de aplicaciones

Ejecuta lógica de negocio utilizada por los clientes.

# 83 Servidor de respaldo

Recibe y conserva copias de seguridad para recuperación.

# 84 Servidor de monitoreo

Supervisa disponibilidad, uso de recursos, eventos y alertas.

# 85 ¿Un servidor puede tener varios roles?

Sí. En una organización pequeña, un servidor puede proporcionar DNS, DHCP, archivos y autenticación.

Sin embargo, concentrar demasiados roles puede generar riesgos:

- Una falla afecta múltiples servicios.
- El mantenimiento requiere detener varias funciones.
- Aumenta la superficie de ataque.
- Se dificulta identificar problemas.
- Los recursos pueden competir entre sí.

En organizaciones grandes se distribuyen los roles y se utilizan mecanismos de redundancia.

# 86 Infraestructura simplificada de una universidad

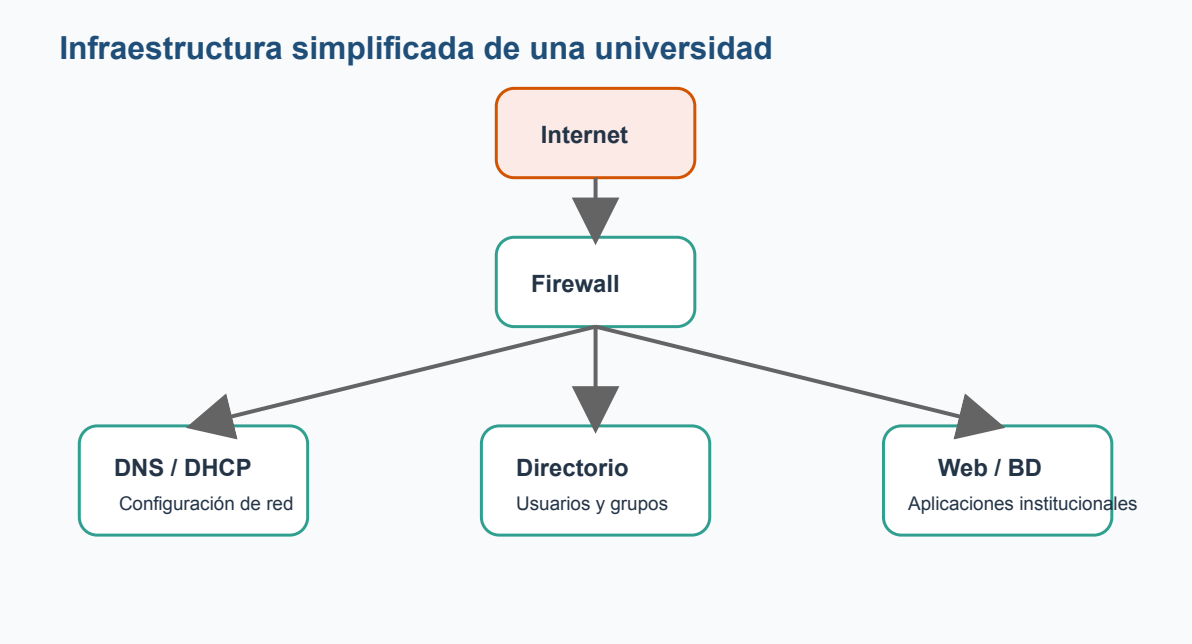



Figura 86.1: Infraestructura universitaria simplificada

Una infraestructura universitaria puede incluir:

- Conexión a Internet.
- Firewall.
- DNS y DHCP.
- Servicio de identidad.
- Servidor de archivos.
- Aplicaciones web.
- Bases de datos.
- Respaldos.
- Monitoreo.

La arquitectura real dependerá del tamaño y las necesidades institucionales.

# 87 Seguridad en los Sistemas Operativos de Red

Centralizar recursos mejora la administración, pero también concentra riesgos. Por eso, la seguridad debe formar parte del diseño desde el inicio.

# 88 Triada de la seguridad

Tres objetivos fundamentales son:

- Confidencialidad.
- Integridad.
- Disponibilidad.

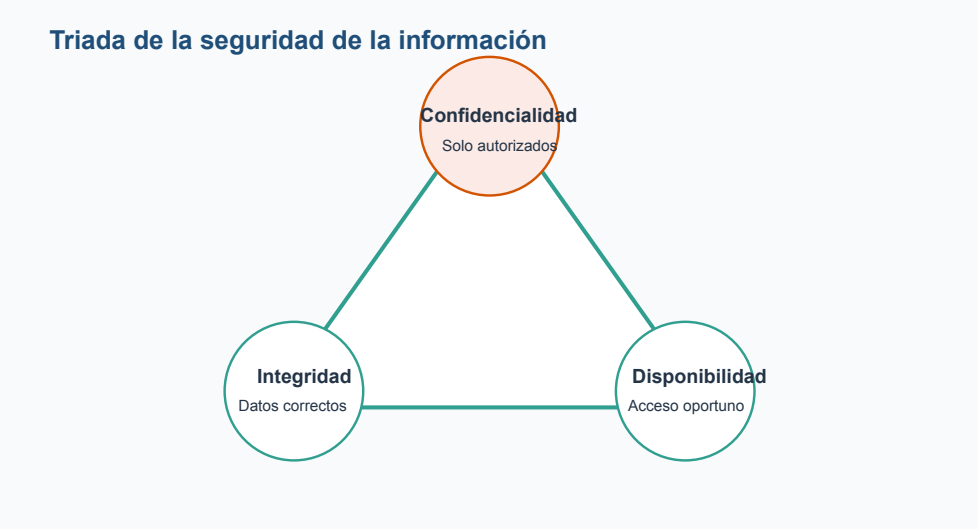



Figura 88.1: Triada de seguridad

## 88.1 Confidencialidad

La información solo debe ser accesible para personas autorizadas.

Ejemplo: un estudiante consulta sus propias calificaciones, no las de todo el grupo.

## 88.2 Integridad

La información debe mantenerse correcta y solo modificarse mediante procesos autorizados.

Ejemplo: el docente registra una nota y el estudiante únicamente la consulta.

## 88.3 Disponibilidad

Los servicios deben estar accesibles cuando se necesitan.

Ejemplo: la plataforma de inscripción debe funcionar durante el período de asignación de cursos.

# 89 Contraseñas y autenticación multi-factor

Una contraseña segura debe ser difícil de adivinar y no reutilizarse en múltiples servicios.

Buenas prácticas:

- Utilizar frases largas.
- Evitar información personal.
- No compartir credenciales.
- Utilizar un gestor de contraseñas cuando sea permitido.
- Activar autenticación multifactor.
- Bloquear intentos repetidos.
- Deshabilitar cuentas que ya no se utilizan.

# 90 Actualizaciones

Los fabricantes publican correcciones para errores y vulnerabilidades.

Un proceso responsable incluye:

1. Identificar actualizaciones.
2. Revisar su impacto.
3. Probarlas.
4. Programar una ventana de mantenimiento.
5. Aplicarlas.
6. Verificar los servicios.
7. Documentar el resultado.

# 91 Copias de seguridad

Un respaldo es una copia diseñada para recuperar información después de una falla, error o incidente.

## 91.1 Regla 3-2-1

- Mantener tres copias de la información.
- Utilizar dos medios diferentes.
- Conservar una copia fuera del sitio principal.

## 91.2 Un respaldo debe probarse

No basta con que el sistema indique “copia finalizada”. Es necesario verificar que los archivos puedan recuperarse.

> **Advertencia**
>
> Un respaldo que nunca ha sido restaurado es una promesa, no una garantía.

# 92 Registros y auditoría

Los servidores generan registros o *logs*.

Pueden indicar:

- Inicios de sesión.
- Intentos fallidos.
- Cambios de configuración.
- Errores.
- Accesos a recursos.
- Inicio y detención de servicios.

La auditoría permite investigar incidentes y demostrar el cumplimiento de políticas.

# 93 Seguridad física

También deben protegerse:

- El acceso al cuarto de servidores.
- La energía eléctrica.
- La temperatura.
- La humedad.
- El riesgo de incendio.
- La conectividad.
- Los dispositivos de almacenamiento.

Un servidor protegido con contraseñas no está seguro si cualquier persona puede desconectarlo o retirar sus discos.

# 94 Continuidad operativa

La continuidad busca que la organización pueda seguir funcionando o recuperarse en un tiempo aceptable.

Incluye:

- Redundancia.
- Respaldos.
- Planes de recuperación.
- Documentación.
- Contactos de emergencia.
- Pruebas.
- Responsables claramente definidos.

# 95 Ingeniería en la vida real

Supongamos que una universidad tiene un único servidor que proporciona autenticación, DNS, DHCP, archivos y el Campus Virtual.

Si ese servidor falla:

- Los usuarios no pueden iniciar sesión.
- Los equipos nuevos no reciben IP.
- Los nombres no se resuelven.
- Las carpetas desaparecen.
- La plataforma deja de funcionar.

El problema no es únicamente técnico. Puede detener clases, evaluaciones y procesos administrativos.

Un ingeniero debe anticipar:

- ¿Qué servicios son críticos?
- ¿Cuánto tiempo pueden estar fuera de línea?
- ¿Qué datos deben recuperarse?
- ¿Existe un servidor alterno?
- ¿Quién tomará decisiones durante el incidente?

# 96 Verifica tu comprensión

## 96.1 Pregunta 1

¿Cuál es la diferencia principal entre una arquitectura Punto a Punto y una Cliente/Servidor?

a. El tipo de cable utilizado.

b. La existencia de administración y servicios centralizados.

c. La marca de las computadoras.

d. La velocidad de Internet.

## 96.2 Pregunta 2

¿Qué servicio traduce nombres de dominio en direcciones IP?

a. DHCP.

b. DNS.

c. SMB.

d. SSH.

## 96.3 Pregunta 3

¿Qué servicio asigna automáticamente direcciones IP?

a. DNS.

b. HTTP.

c. DHCP.

d. LDAP.

## 96.4 Pregunta 4

¿Qué elemento facilita asignar permisos a muchas personas con funciones semejantes?

a. Un grupo.

b. Una impresora.

c. Una dirección IP.

d. Un navegador.

## 96.5 Pregunta 5

¿Cuál es un ejemplo del principio del mínimo privilegio?

a. Todos son administradores.

b. Cada usuario recibe únicamente el acceso necesario.

c. Se comparte una sola cuenta.

d. Se desactiva la auditoría.

## 96.6 Pregunta 6

¿Qué objetivo de seguridad se relaciona con mantener un servicio accesible?

a. Confidencialidad.

b. Integridad.

c. Disponibilidad.

d. Portabilidad.

**Respuestas**

1. b
2. b
3. c
4. a
5. b
6. c

# 97 Laboratorio narrativo: diseñando la red de una academia

## 97.1 Situación

La Academia Horizonte abrirá una nueva sede.

Contará con:

- 28 computadoras para estudiantes.
- 4 computadoras administrativas.
- 3 computadoras para docentes.
- 2 impresoras.
- Una conexión a Internet.
- Una plataforma académica.
- Documentos financieros.
- Materiales de clase compartidos.

Actualmente, cada computadora utiliza cuentas locales y los archivos se transfieren mediante memorias USB.

La dirección solicita una propuesta básica de red.

## 97.2 Misión

Actúe como integrante del equipo de infraestructura y diseñe una solución inicial.

## 97.3 Parte 1. Arquitectura

Responda:

1. ¿Implementaría Punto a Punto o Cliente/Servidor?
2. Justifique la decisión con al menos tres criterios.
3. ¿Qué riesgos tendría mantener cuentas locales independientes?

## 97.4 Parte 2. Roles

Seleccione los roles necesarios.

| Rol | ¿Se necesita? | Justificación |
|---|---|---|
| DNS | | |
| DHCP | | |
| Directorio | | |
| Archivos | | |
| Impresión | | |
| Web | | |
| Base de datos | | |
| Respaldo | | |
| Monitoreo | | |

## 97.5 Parte 3. Usuarios y grupos

Proponga grupos.

Ejemplo:

- Administrativos.
- Docentes.
- Estudiantes.
- Soporte Técnico.

Indique qué recursos debe utilizar cada grupo.

## 97.6 Parte 4. Permisos

Diseñe una matriz.

| Recurso | Administrativos | Docentes | Estudiantes | Soporte |
|---|---|---|---|---|
| Finanzas | | | | |
| Materiales docentes | | | | |
| Entregas estudiantiles | | | | |
| Configuración de servidores | | | | |
| Impresoras | | | | |

Utilice valores como:

- Sin acceso.
- Lectura.
- Lectura y escritura.
- Administración.

## 97.7 Parte 5. Seguridad

Proponga al menos una medida para cada categoría:

- Contraseñas.
- Respaldos.
- Actualizaciones.
- Auditoría.
- Seguridad física.
- Continuidad.

## 97.8 Parte 6. Diagrama

Dibuje la infraestructura propuesta.

Debe incluir:

- Internet.
- Firewall o router.
- Servidor o servidores.
- Switch.
- Computadoras.
- Impresoras.
- Respaldos.

## 97.9 Producto esperado

Entregue:

1. Una justificación de la arquitectura.
2. La lista de roles.
3. Los grupos.
4. La matriz de permisos.
5. Las medidas de seguridad.
6. El diagrama.

# 98 Reto del Ingeniero

Una clínica pequeña posee diez computadoras. El expediente de pacientes se almacena en una carpeta compartida desde la computadora de recepción.

La recepcionista apaga su equipo al finalizar la jornada y, durante ese tiempo, nadie puede consultar los expedientes. Todos conocen la misma contraseña y no existen respaldos.

Responda:

1. Identifique cinco riesgos.
2. Proponga una arquitectura mejor.
3. Defina tres grupos de usuarios.
4. Asigne permisos generales.
5. Indique qué servicios implementaría.
6. Diseñe una estrategia básica de respaldo.
7. Explique cómo mejoraría la disponibilidad.

# 99 Resumen de la clase

En esta clase aprendimos que:

- Un Sistema Operativo de Red administra recursos compartidos.
- La arquitectura Punto a Punto es sencilla, pero posee limitaciones.
- Cliente/Servidor permite centralizar servicios y crecer.
- DNS traduce nombres en direcciones IP.
- DHCP entrega automáticamente la configuración de red.
- Los usuarios representan identidades.
- Los grupos facilitan la administración.
- Los permisos controlan acciones.
- Un servidor puede desempeñar diferentes roles.
- Windows Server, Linux y UNIX ofrecen soluciones distintas.
- La seguridad debe proteger confidencialidad, integridad y disponibilidad.
- Los respaldos, actualizaciones, registros y controles físicos son esenciales.
- Una infraestructura debe diseñarse según necesidades reales.

**Lo esencial que debes recordar**

Si puedes explicar con tus propias palabras cómo un usuario obtiene una dirección IP, localiza un servidor, inicia sesión y accede únicamente a los recursos autorizados, habrás comprendido la idea central de esta clase.

# 100 Cierre de la clase

Un Sistema Operativo de Red convierte un conjunto de computadoras en una infraestructura organizada.

La tecnología visible para el usuario puede ser una pantalla de inicio de sesión o una carpeta compartida. Sin embargo, detrás de esa simplicidad existen decisiones de arquitectura, servicios, seguridad y continuidad.

El trabajo del ingeniero consiste en lograr que todos esos componentes funcionen como un sistema confiable.

# 101 Referencias


- Kurose, J. F., & Ross, K. W. *Computer Networking: A Top-Down Approach*. Pearson.
- Nemeth, E., Snyder, G., Hein, T. R., Whaley, B., & Mackin, D. *UNIX and Linux System Administration Handbook*. Addison-Wesley.
- Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*. Wiley.
- Stallings, W. *Operating Systems: Internals and Design Principles*. Pearson.
- Tanenbaum, A. S., & Bos, H. *Modern Operating Systems*. Pearson.
- Documentación técnica de Microsoft Learn sobre Windows Server y servicios de red.
- Documentación técnica de Red Hat sobre administración de sistemas Linux.

# 102 Clase 4. Administración de Procesos

# 103 Presentación

Cada vez que abrimos un navegador, reproducimos música o editamos un documento, el Sistema Operativo crea y administra procesos. Aunque parezca que todas las aplicaciones trabajan al mismo tiempo, en realidad el procesador distribuye su tiempo entre ellas de forma extremadamente rápida.

Objetivos

- Comprender qué es un proceso.
- Diferenciar programa y proceso.
- Explicar el modelo de procesos.
- Comprender los estados de un proceso.
- Interpretar el funcionamiento del PCB.

# 104 ¿Qué es un proceso?

Un **proceso** es un programa que se encuentra en ejecución junto con todos los recursos que necesita para funcionar: memoria, registros, archivos abiertos y tiempo de CPU.

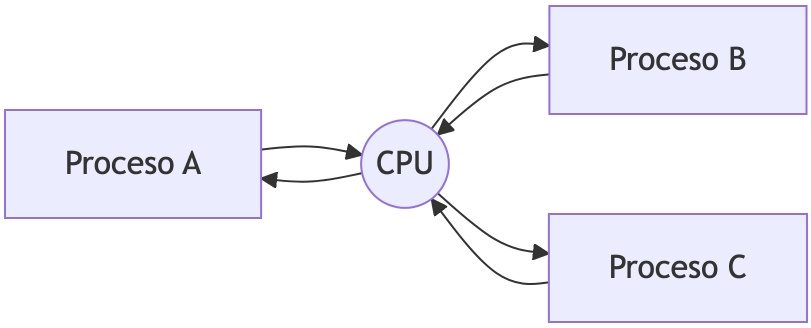
| Programa | Proceso |
|---|---|
| Archivo almacenado | Programa en ejecución |
| Pasivo | Dinámico |
| No consume recursos | Consume CPU, memoria y E/S |

## 104.1 Analogía

Una receta de cocina representa un **programa**. Preparar la receta utilizando ingredientes, tiempo y utensilios representa el **proceso**.

# 105 Modelo de Procesos

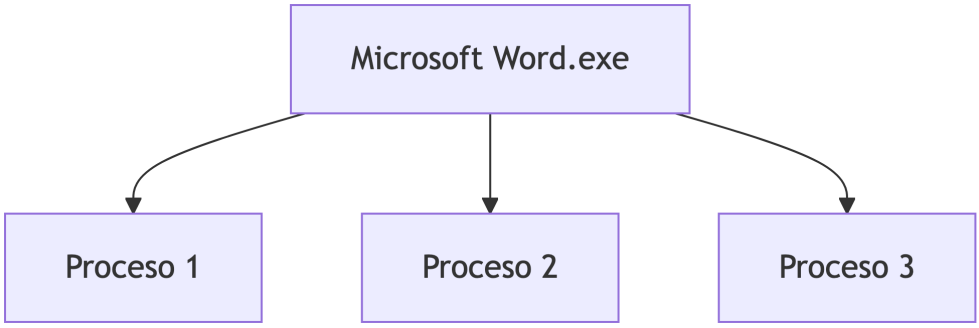
El Sistema Operativo crea la ilusión de que cada aplicación posee su propio procesador. En realidad, el **Scheduler** distribuye pequeños intervalos de tiempo entre todos los procesos.




> **Idea clave**
>
> La multitarea es posible porque el Sistema Operativo cambia rápidamente entre procesos mediante el **Cambio de Contexto (Context Switch)**.

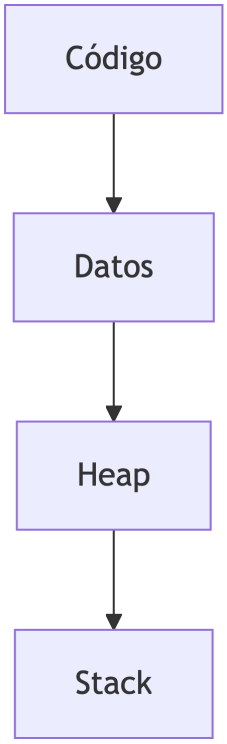
# 106 Programa vs. Proceso

Un único programa puede generar múltiples procesos independientes.



# 107 Espacio de Direcciones

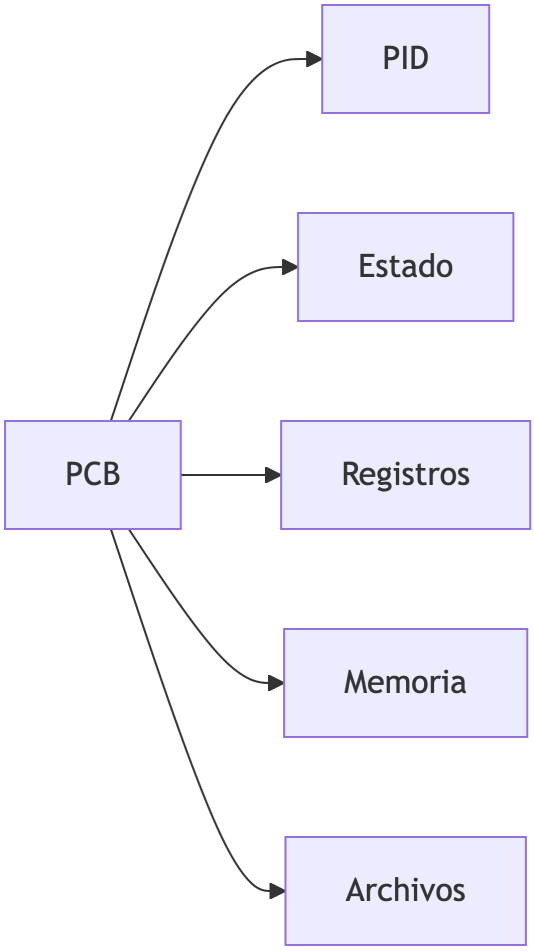
Cada proceso dispone de un espacio de memoria independiente.



# 108 Bloque de Control del Proceso (PCB)

El PCB es el expediente administrativo del proceso.

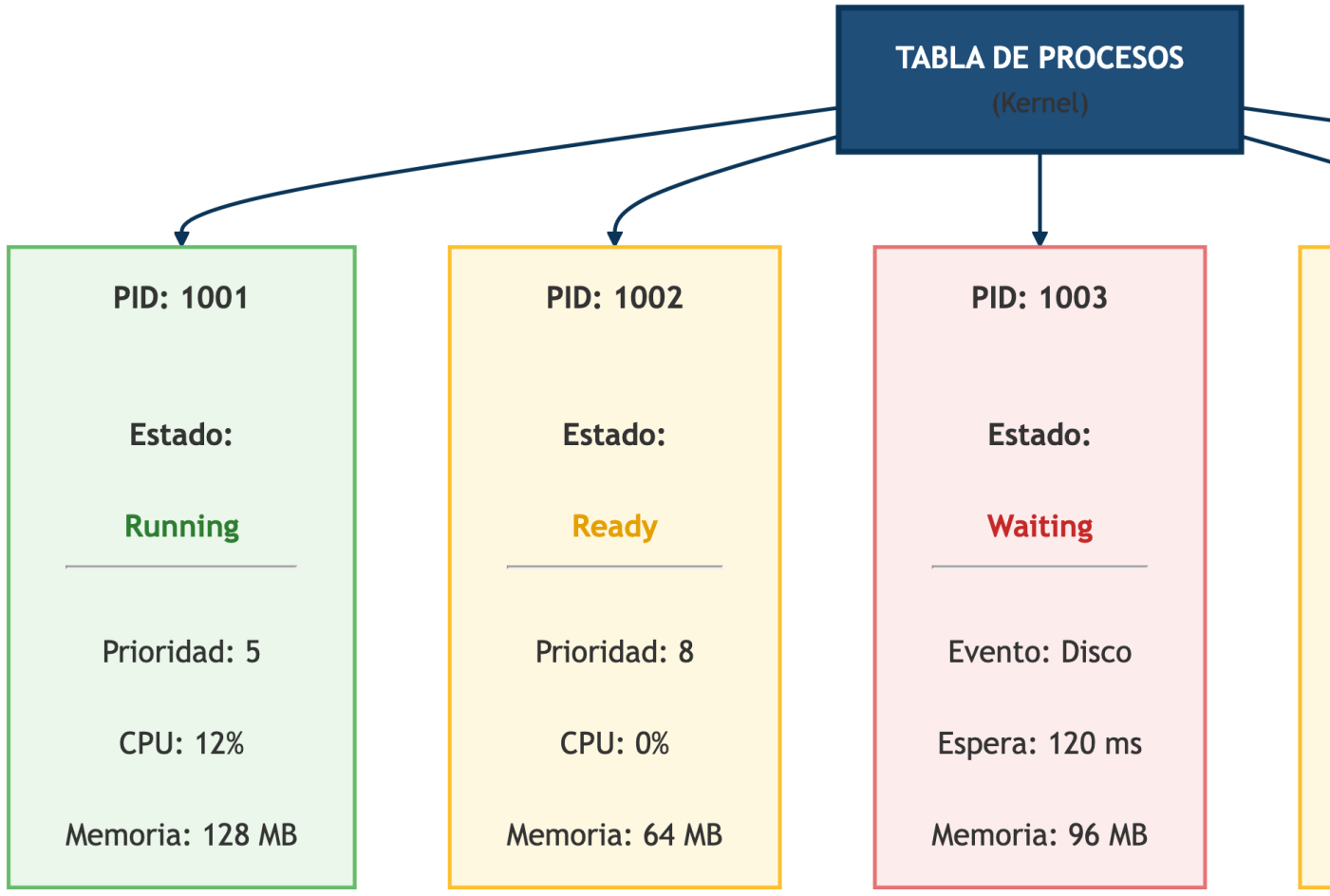
| Campo | Descripción |
|---|---|
| PID | Identificador único |
| Estado | Estado actual |
| Program Counter | Próxima instrucción |
| Registros | Contexto de CPU |
| Memoria | Espacio asignado |
| Archivos | Recursos abiertos |


PID
Estado
PCB
Registros
Memoria
Archivos

# 109 Tabla de Procesos

El kernel mantiene una tabla donde registra todos los PCB existentes.

Estructura simplificada de una Tabla de Procesos




La tabla de procesos es una estructura interna mantenida por el **Kernel**, donde se almacena el **PCB (Process Control Block)** de todos los procesos existentes en el sistema. Cada entrada contiene la información necesaria para que el sistema operativo pueda administrar correctamente cada proceso.

### 109.0.1 Ejemplo de una Tabla de Procesos

| PID | Nombre del proceso | Estado | Prioridad | Memoria (MB) | CPU ( %) | Archivos abiertos |
|---|---|---|---|---|---|---|
| 1001 | Chrome.exe | Running | Alta | 245 | 18 | 27 |
| 1002 | Word.exe | Ready | Media | 132 | 0 | 12 |
| 1003 | Spotify.exe | Waiting | Baja | 95 | 0 | 8 |
| 1004 | Teams.exe | Ready | Alta | 318 | 0 | 34 |
| 1005 | Explorer.exe | Running | Alta | 76 | 5 | 19 |

**Interpretación**

- **PID:** Identificador único asignado por el sistema operativo.
- **Nombre del proceso:** Aplicación o servicio que se encuentra en ejecución.
- **Estado:** Situación actual del proceso (Running, Ready o Waiting).
- **Prioridad:** Nivel de importancia utilizado por el Scheduler.
- **Memoria:** Cantidad de memoria RAM reservada por el proceso.
- **CPU:** Porcentaje de utilización del procesador en ese instante.
- **Archivos abiertos:** Número de archivos o recursos que el proceso mantiene abiertos.

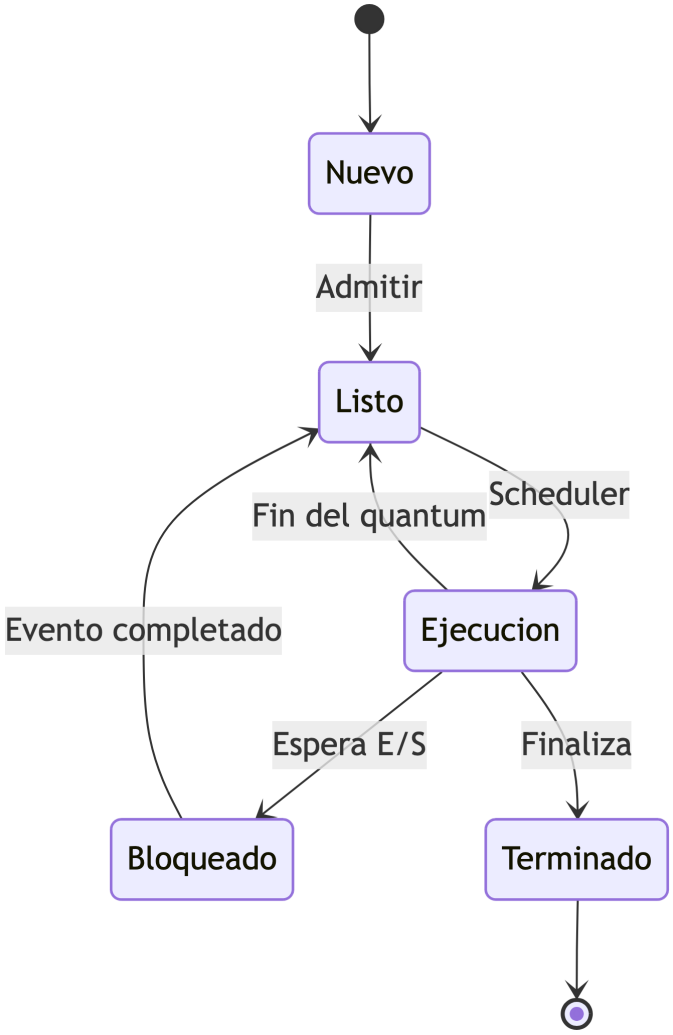
# 110 Módelos de Estados de un Proceso

## 110.1 Evolución de los modelos de estados

| Modelo | Estados | ¿Dónde se utiliza? |
|---|---|---|
| Dos estados | Listo y Ejecución | Sistemas muy simples o con fines didácticos. |
| Cinco estados | Nuevo, Listo, Ejecución, Bloqueado y Terminado | Es el modelo más utilizado para explicar la administración de procesos. |
| Siete estados | Cinco estados + Listo Suspendido + Bloqueado Suspendido | Sistemas operativos modernos que utilizan memoria virtual y Swap. |

## 110.2 Modelo de cinco estados

El modelo de cinco estados es uno de los más utilizados para explicar el ciclo de vida de un proceso en un sistema operativo. En este modelo, un proceso puede encontrarse en uno de cinco estados bien definidos, y el sistema operativo controla las transiciones entre ellos.




### 110.2.1 Descripción de los estados

| Estado | Descripción |
|---|---|
| **Nuevo** | El proceso acaba de ser creado y aún no está listo para ejecutarse. |
| **Listo (Ready)** | El proceso posee todos los recursos necesarios excepto la CPU. Espera su turno para ejecutarse. |
| **Ejecución (Running)** | El proceso está utilizando el procesador y ejecutando instrucciones. |
| **Bloqueado (Waiting)** | El proceso espera la finalización de una operación de entrada/salida o algún evento externo. |
| **Terminado (Exit)** | El proceso concluyó su ejecución y el sistema operativo libera los recursos utilizados. |

Idea clave

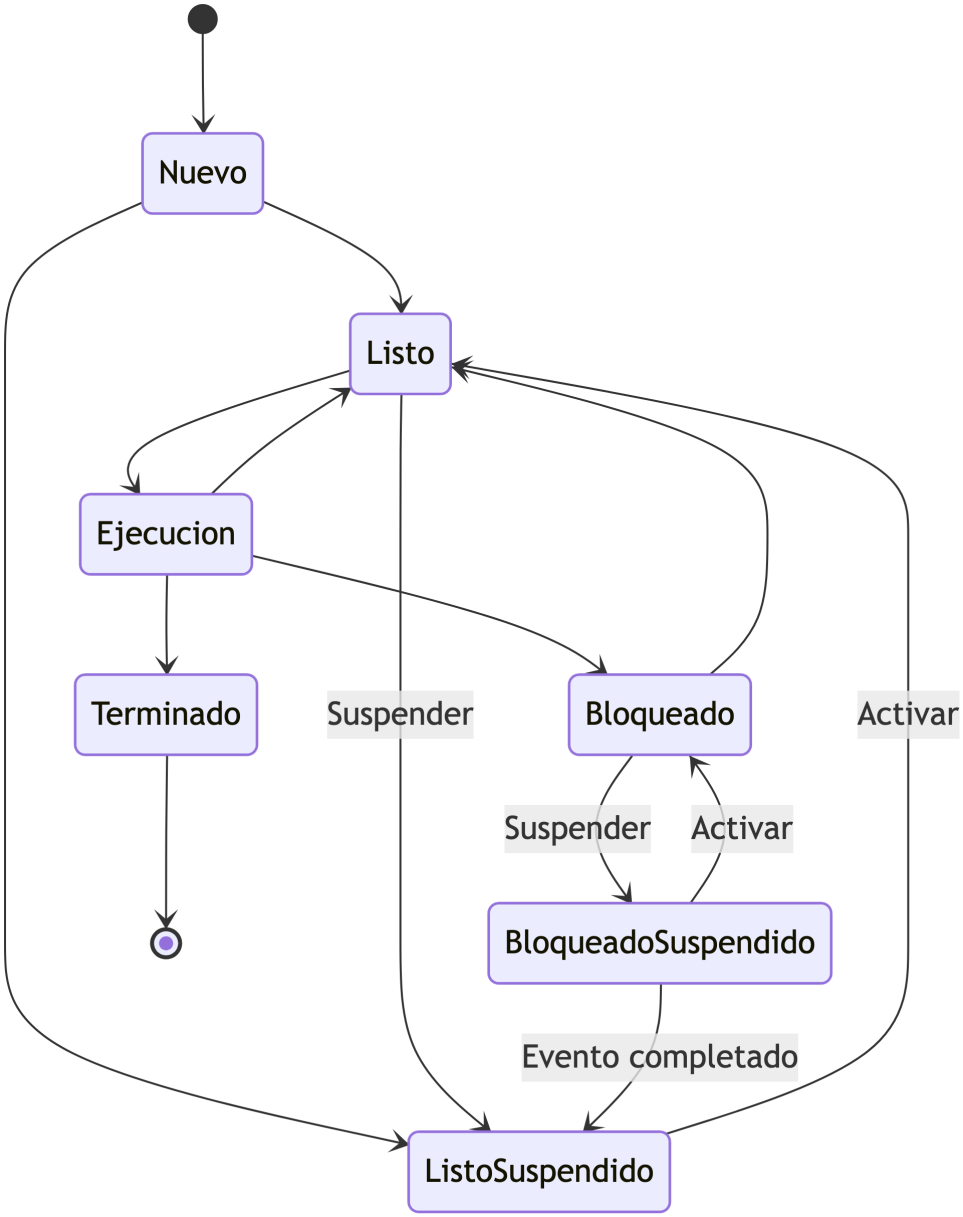
Todo proceso pasa necesariamente por los estados **Nuevo** y **Terminado**, mientras que los estados **Listo**, **Ejecución** y **Bloqueado** pueden repetirse varias veces durante su ciclo de vida.

## 110.3 Modelo con procesos suspendidos (Swap)

En sistemas multitarea es posible que la memoria principal no sea suficiente para mantener todos los procesos cargados simultáneamente. En estos casos, el sistema operativo puede mover temporalmente algunos procesos al almacenamiento secundario mediante una técnica denominada **Swap** o **Intercambio de memoria**.

Cuando esto ocurre aparecen dos nuevos estados:

- **Listo y Suspendido**
- **Bloqueado y Suspendido**




### 110.3.1 Nuevos estados

| Estado | Descripción |
|---|---|
| **Listo y Suspendido** | El proceso está preparado para ejecutarse, pero fue trasladado al disco porque no existe memoria RAM suficiente. |
| **Bloqueado y Suspendido** | El proceso permanece esperando un evento y, además, fue movido temporalmente al almacenamiento secundario. |

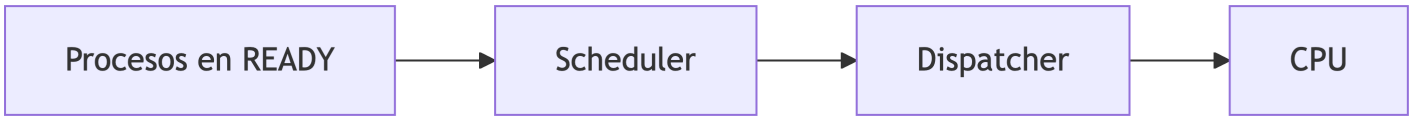
> ¿Qué es el Swap?
>
> El **Swap** consiste en mover procesos completos o parte de su memoria desde la RAM hacia un área especial del disco duro o del SSD para liberar memoria física y permitir que otros procesos continúen ejecutándose.
> Aunque esta técnica incrementa la capacidad de ejecución del sistema, también reduce el rendimiento, ya que acceder al disco es mucho más lento que acceder a la memoria RAM.

# 111 Resumen

- Un programa se convierte en proceso al ejecutarse.
- Cada proceso posee memoria y recursos propios.
- El PCB almacena toda la información necesaria para administrarlo.
- El Scheduler comparte el procesador entre todos los procesos.
- En la siguiente sección estudiaremos el ciclo de vida y los estados de un proceso.

# 112 Scheduler y Dispatcher

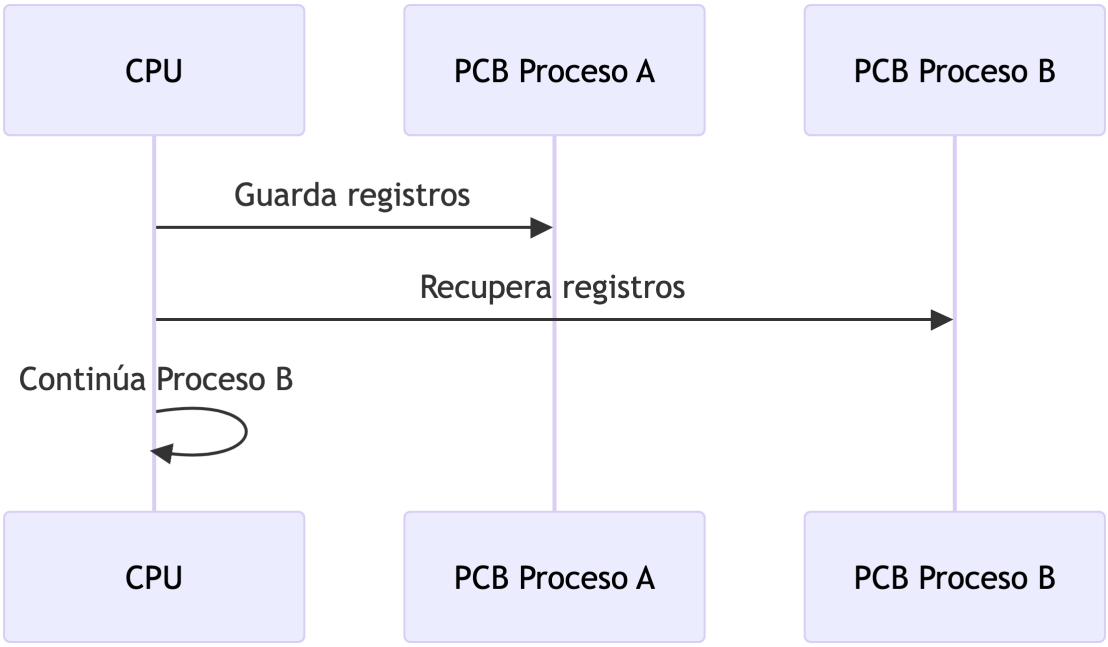
El **Scheduler** decide qué proceso obtendrá el procesador cuando existen varios procesos listos para ejecutarse. Para tomar esta decisión puede considerar la prioridad, el tiempo de espera o el algoritmo de planificación configurado por el sistema operativo.




El **Dispatcher** es el componente encargado de entregar realmente la CPU al proceso seleccionado y realizar el cambio de contexto.

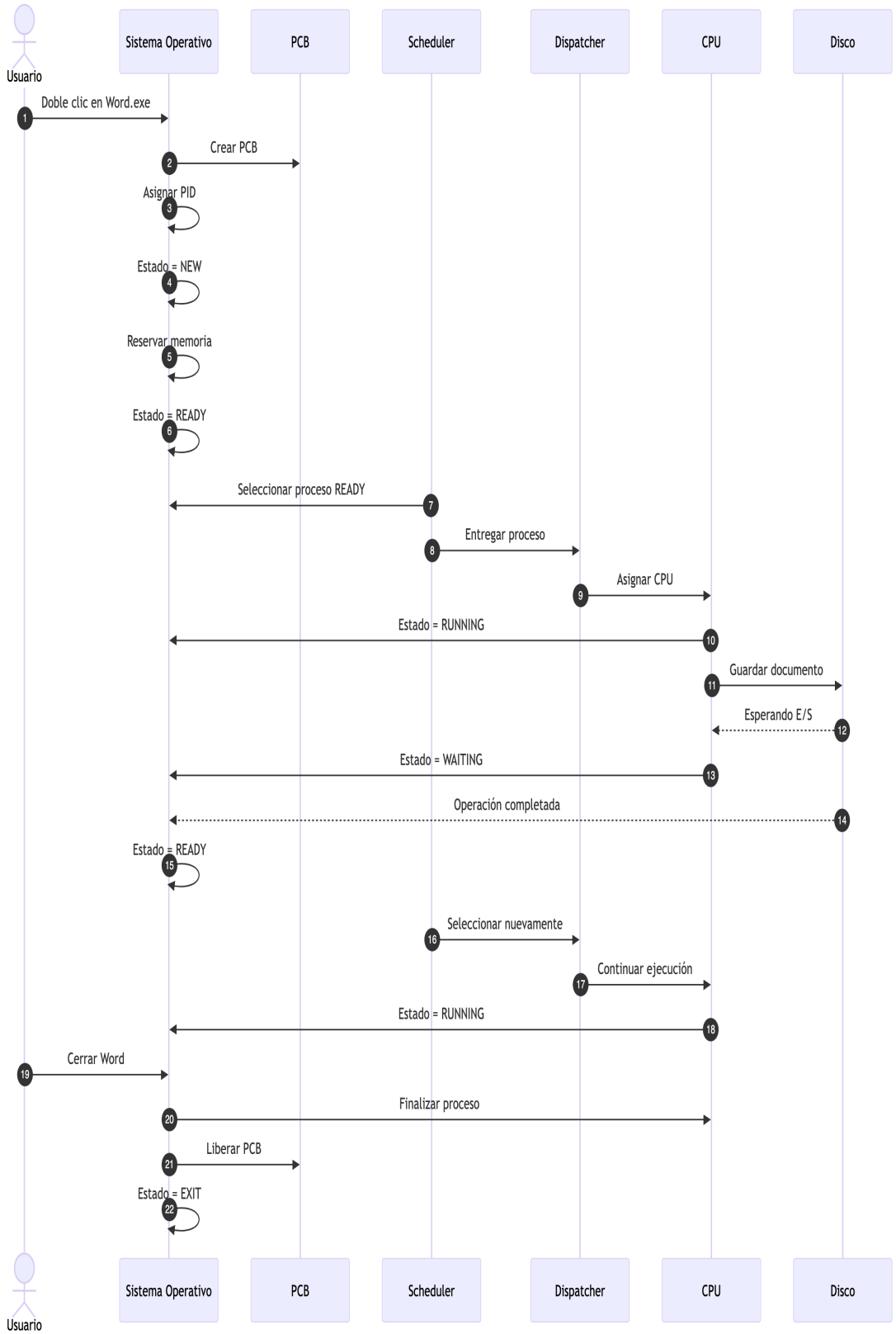
# 113 Cambio de contexto (Context Switch)

Un cambio de contexto ocurre cuando el sistema operativo detiene temporalmente un proceso, guarda su estado en el PCB y restaura el estado de otro proceso.



# 114 Ejemplo completo: Microsoft Word

1. El usuario hace doble clic sobre **Word.exe**.
2. El Kernel crea el proceso y asigna un PID.
3. El proceso entra al estado **NEW**.
4. Se reserva memoria y se crea el PCB.
5. El proceso pasa a **READY**.
6. El Scheduler selecciona el proceso.
7. El Dispatcher entrega la CPU.
8. El proceso entra en **RUNNING**.
9. Al guardar un archivo, espera la respuesta del disco (**WAITING**).
10. Finalizada la operación de E/S vuelve a **READY**.
11. El Scheduler le asigna nuevamente la CPU.
12. Al cerrar Word, el proceso entra en **EXIT** y el sistema libera memoria, archivos y demás recursos.


Usuario
Sistema Operativo
PCB
Scheduler
Dispatcher
CPU
Disco
1 Doble clic en Word.exe
2 Crear PCB
3 Asignar PID
4 Estado = NEW
5 Reservar memoria
6 Estado = READY
7 Seleccionar proceso READY
8 Entregar proceso
9 Asignar CPU
10 Estado = RUNNING
11 Guardar documento
12 Esperando E/S
13 Estado = WAITING
14 Operación completada
15 Estado = READY
16 Seleccionar nuevamente
17 Continuar ejecución
18 Estado = RUNNING
19 Cerrar Word
20 Finalizar proceso
21 Liberar PCB
22 Estado = EXIT
Usuario
Sistema Operativo
PCB
Scheduler
Dispatcher
CPU
Disco

## 114.1 Autoevaluación

1. ¿Cuál es la diferencia entre un programa y un proceso?
2. ¿Qué información almacena el PCB?
3. ¿Qué función cumple el Scheduler?
4. ¿Cuándo ocurre un cambio de contexto?
5. Explique el recorrido NEW → READY → RUNNING → WAITING → READY → EXIT con sus propias palabras.

# 115 Clase 5. Control de Procesos y Concurrencia

# 116 Clase 5. Control de Procesos y Concurrencia

## 116.1 Presentación de la clase

En la clase anterior estudiamos cómo el sistema operativo crea, registra y administra los procesos mediante estructuras como el **PCB**, la tabla de procesos, el **Scheduler**, el **Dispatcher** y los cambios de contexto.

En esta clase ampliaremos esa base para comprender cómo el sistema operativo controla varios procesos e hilos que avanzan de forma concurrente, cómo distribuye el tiempo del procesador y qué problemas aparecen cuando varias unidades de ejecución comparten datos o recursos.

También analizaremos el papel real que puede desempeñar la inteligencia artificial en la administración de recursos. La IA no sustituye al kernel ni modifica por sí misma el PCB; puede emplearse como apoyo para construir políticas de planificación, predicción y detección de anomalías.

> **Pregunta detonadora**
>
> Cuando un navegador reproduce música, descarga un archivo, muestra varias pestañas y responde al teclado casi al mismo tiempo, ¿cómo evita el sistema operativo que todas esas actividades interfieran entre sí?

## 116.2 Objetivos de aprendizaje

Al finalizar la clase, el estudiante será capaz de:

1. Explicar qué significa controlar un proceso.
2. Diferenciar procesos e hilos.
3. Comprender la relación entre concurrencia y paralelismo.
4. Explicar el propósito del quantum de tiempo.
5. Relacionar el quantum con la planificación Round Robin.
6. Identificar los componentes principales del PCB.
7. Reconocer una sección crítica y una condición de carrera.
8. Explicar la finalidad de la exclusión mutua.
9. Interpretar el algoritmo de Dekker como una solución histórica para dos procesos.

10. Analizar, con criterio técnico, posibles aplicaciones de inteligencia artificial en la administración de recursos.

## 116.3 Ruta de aprendizaje

La clase seguirá esta secuencia:

1. Control del ciclo de vida de los procesos.
2. Diferencia entre proceso e hilo.
3. Concurrencia y paralelismo.
4. Quantum y planificación Round Robin.
5. PCB y cambio de contexto.
6. Recursos compartidos y condiciones de carrera.
7. Sección crítica y exclusión mutua.
8. Algoritmo de Dekker.
9. IA aplicada a planificación y observabilidad.
10. Caso integrador, laboratorio y autoevaluación.

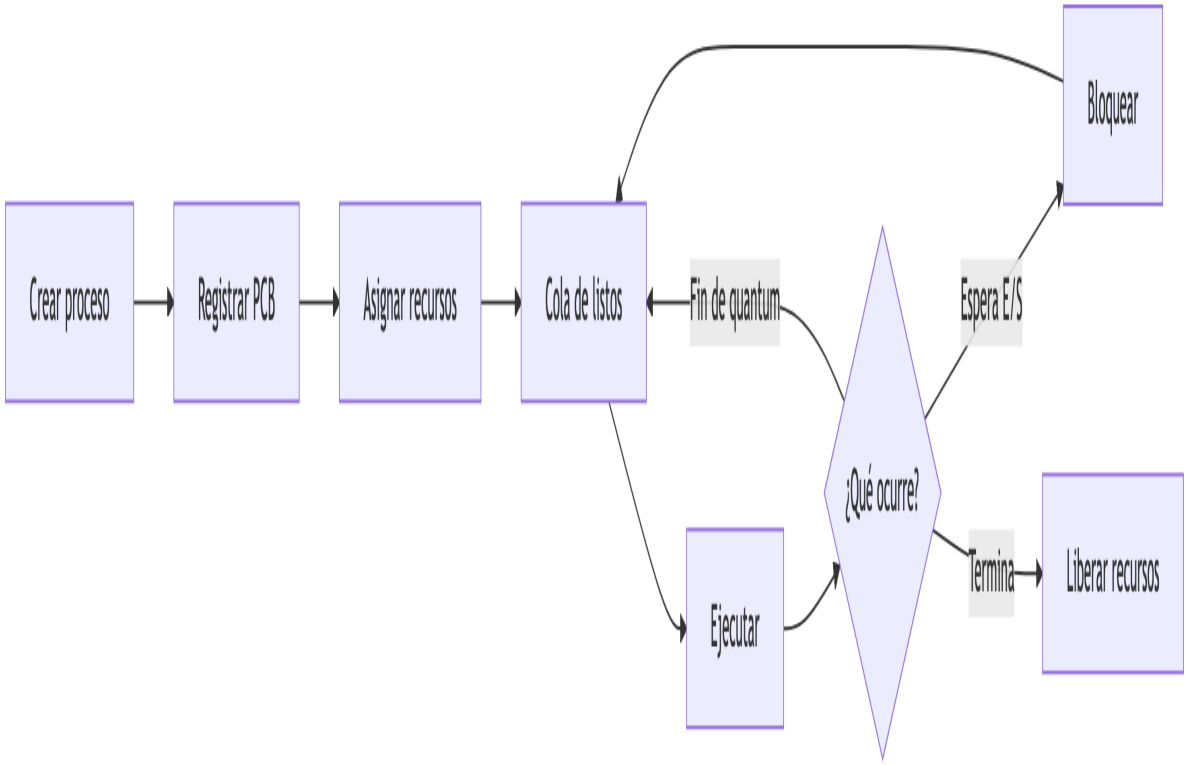
# 117 ¿Qué es el control de procesos?

El **control de procesos** es el conjunto de mecanismos utilizados por el sistema operativo para crear, registrar, planificar, pausar, reanudar, sincronizar y finalizar procesos.

El sistema operativo no se limita a “abrir programas”. Debe mantener una visión organizada de todo lo que se encuentra en ejecución.

Entre sus responsabilidades están:

- crear el proceso;
- asignarle un identificador;
- reservar memoria;
- crear y actualizar su PCB;
- ubicarlo en una cola de planificación;
- asignarle tiempo de CPU;
- atender solicitudes de entrada y salida;
- proteger sus recursos;
- coordinar su comunicación con otros procesos;
- finalizarlo y liberar los recursos utilizados.




**Idea central**

Controlar un proceso significa conservar suficiente información para detenerlo, reanudarlo y finalizarlo sin perder la coherencia de su ejecución.

# 118 Proceso e hilo

## 118.1 Proceso

Un **proceso** es una instancia de un programa en ejecución. Posee su propio espacio de direcciones y los recursos que el sistema operativo le ha asignado.

Un proceso normalmente incluye:

- código;
- datos;
- heap;
- stack;
- archivos abiertos;
- credenciales y permisos;
- uno o más hilos;
- un PCB o estructura equivalente mantenida por el kernel.

## 118.2 Hilo

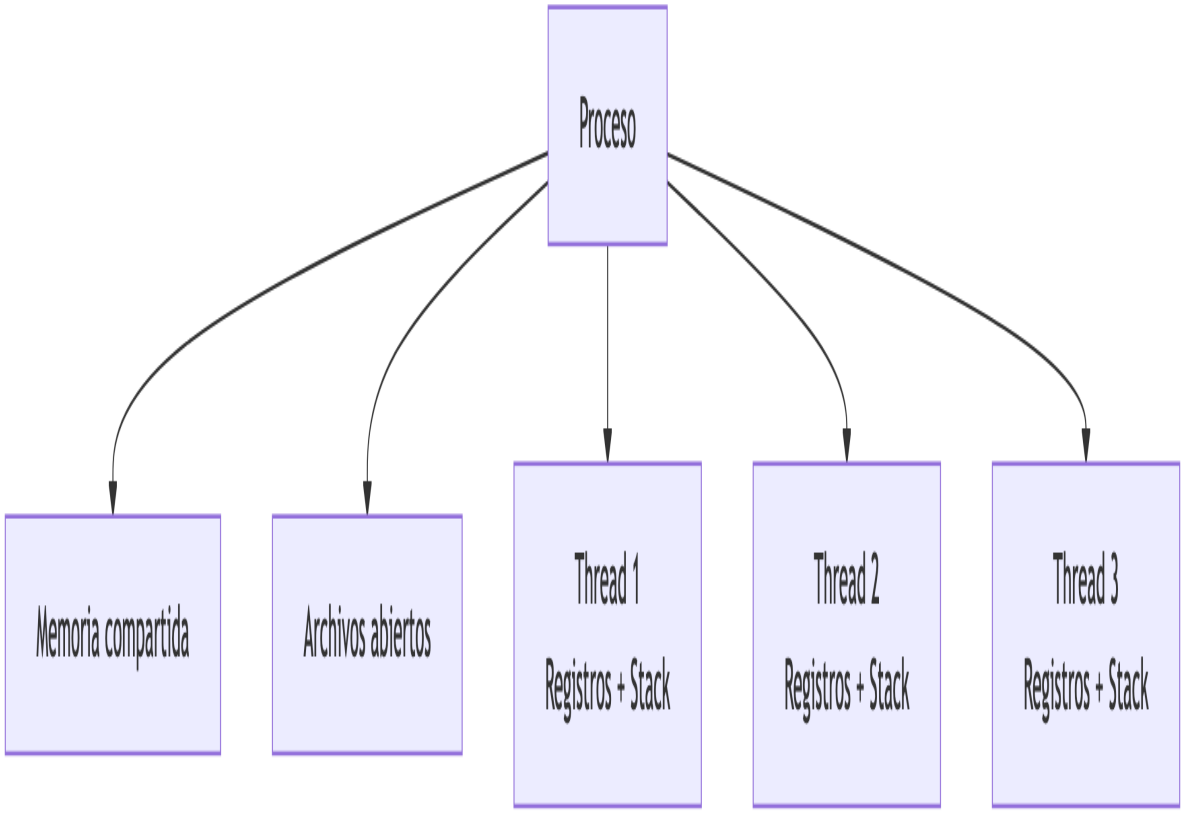
Un **hilo** o **thread** es una unidad de ejecución dentro de un proceso.

Los hilos de un mismo proceso suelen compartir:

- código;
- datos globales;
- heap;
- archivos abiertos;
- espacio de direcciones.

Cada hilo conserva elementos propios, como:

- contador de programa;
- registros;
- stack;
- estado de ejecución;
- identificador del hilo.




## 118.3 Comparación

| Característica | Proceso | Hilo |
|---|---|---|
| Espacio de memoria | Generalmente independiente | Comparte el espacio del proceso |
| Costo de creación | Mayor | Menor |
| Comunicación | Requiere mecanismos de IPC | Puede compartir variables y estructuras |
| Aislamiento | Mayor | Menor |
| Fallo | Puede quedar aislado | Puede afectar al proceso completo |
| Uso típico | Aplicaciones o servicios separados | Tareas relacionadas dentro de una aplicación |

**Precisión importante**

No debe afirmarse que cada pestaña de un navegador siempre corresponde a un hilo. Los navegadores modernos pueden utilizar una arquitectura multiproceso y, dentro de cada proceso, varios hilos. La implementación exacta depende del navegador y de su modelo de

aislamiento.

# 119 Concurrencia y paralelismo

Estos conceptos están relacionados, pero no significan lo mismo.

## 119.1 Concurrencia

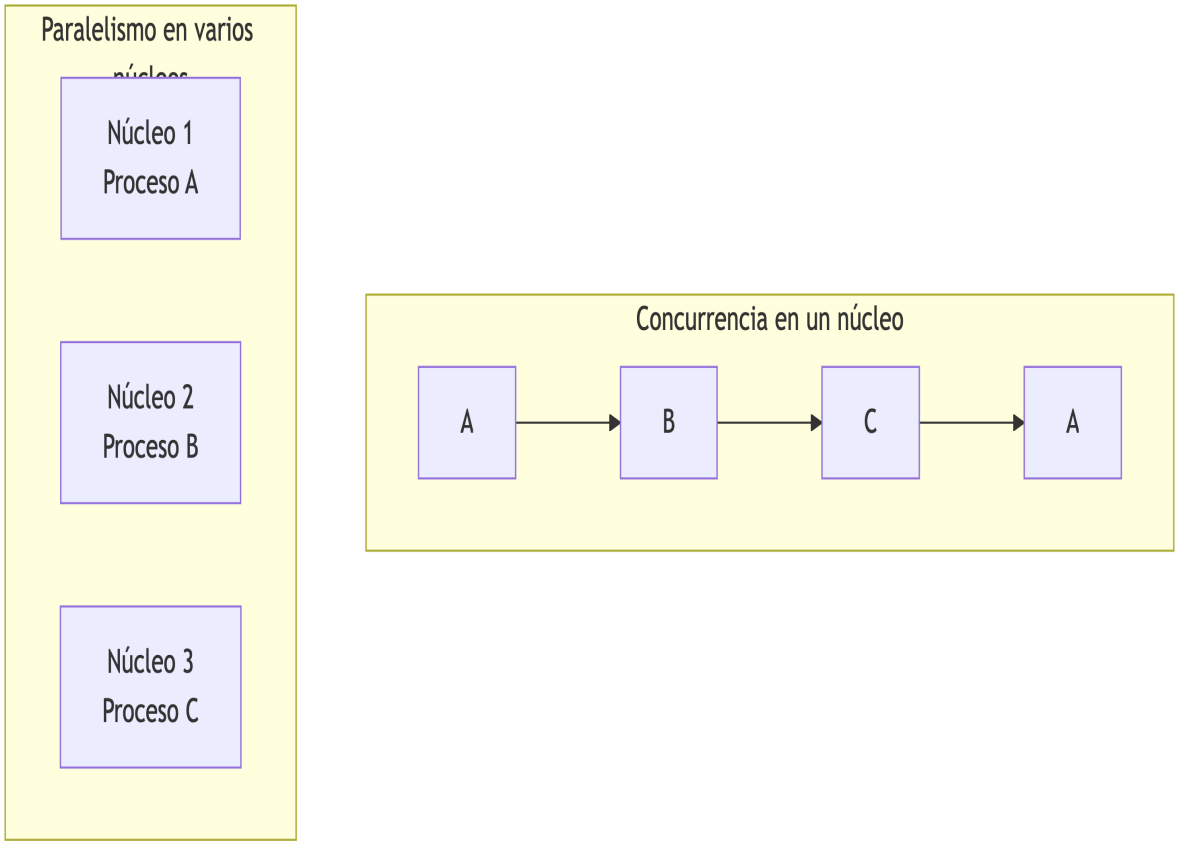
Existe **concurrencia** cuando varias tareas progresan durante un mismo período, aunque no necesariamente se ejecuten exactamente al mismo tiempo.

En una CPU de un solo núcleo, el sistema puede intercalar rápidamente la ejecución:

```
Proceso A → Proceso B → Proceso C → Proceso A → ...
```

## 119.2 Paralelismo

Existe **paralelismo** cuando dos o más tareas se ejecutan físicamente al mismo tiempo, por ejemplo, en núcleos diferentes.




| Concepto | Pregunta clave |
|---|---|
| Concurrencia | ¿Varias tareas avanzan durante el mismo período? |
| Paralelismo | ¿Varias tareas se ejecutan simultáneamente en hardware distinto? |

**Ejemplo**

Una persona puede atender concurrentemente varias conversaciones alternando entre ellas. Dos personas pueden atender conversaciones en paralelo.

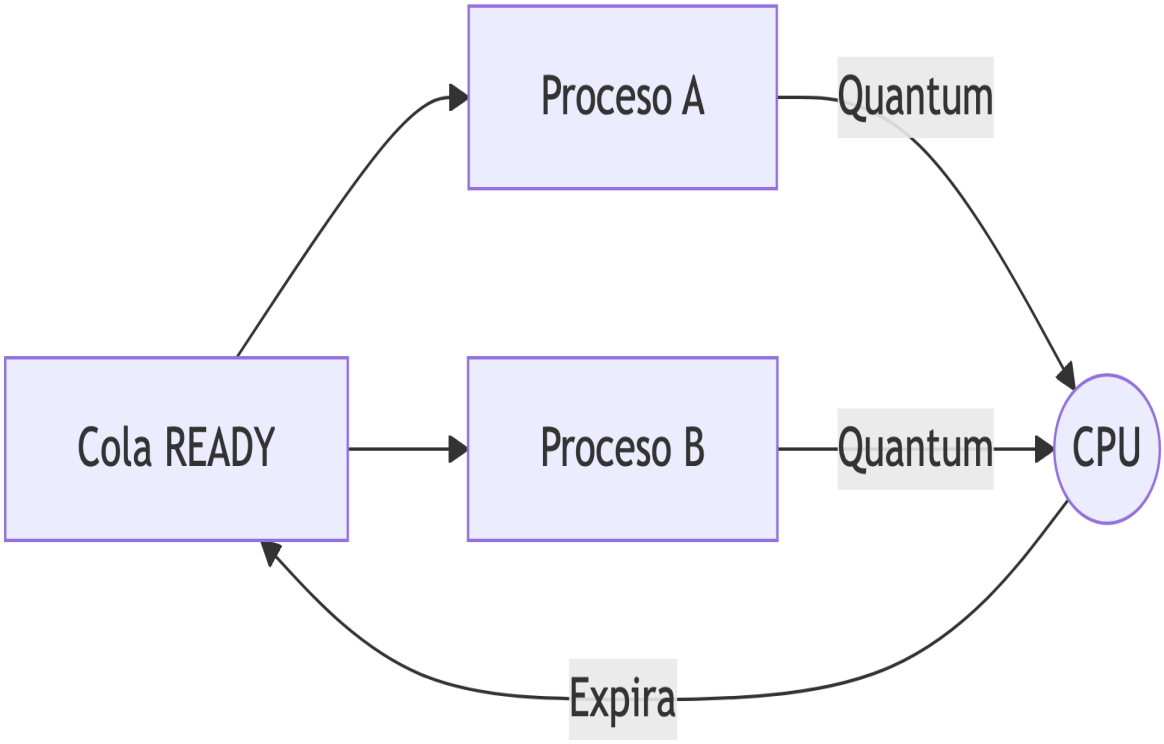
# 120 Quantum de tiempo

El **quantum**, también llamado **time slice**, es el intervalo máximo durante el cual un proceso puede conservar la CPU antes de ser interrumpido por el planificador, siempre que no se bloquee o termine antes.

El quantum es especialmente importante en algoritmos de planificación apropiativos como **Round Robin**.

## 120.1 ¿Qué ocurre cuando termina el quantum?

1. El reloj genera una interrupción.
2. El kernel recupera el control.
3. Se guarda el contexto del proceso.
4. El proceso pasa de Ejecución a Listo.
5. El Scheduler selecciona otro proceso.
6. El Dispatcher restaura su contexto.
7. El nuevo proceso continúa ejecutándose.




## 120.2 Efecto del tamaño del quantum

| Quantum | Ventaja | Riesgo |
|---|---|---|
| Muy pequeño | Mayor sensación de respuesta | Más cambios de contexto |
| Moderado | Equilibrio entre respuesta y costo | Debe adaptarse al sistema |
| Muy grande | Menos cambios de contexto | Se aproxima a FCFS y reduce interactividad |

> **!** No existe un quantum universal
>
> El valor apropiado depende de la carga, el hardware, los objetivos de respuesta y la política del sistema operativo.

# 121 Planificación Round Robin

**Round Robin** organiza los procesos listos en una cola y asigna a cada uno un quantum de CPU.

Supongamos:

```
Quantum = 4 unidades
```

y tres procesos:

| Proceso | Ráfaga de CPU |
|---|---|
| P1 | 9 |
| P2 | 5 |
| P3 | 3 |

La ejecución inicial sería:

```
0        4        8       11       15       16       17
|   P1   |   P2   | P3  |   P1   |   P2   | P1 |
```

### 121.0.1 Interpretación

- P1 utiliza 4 unidades y necesita 5 más.
- P2 utiliza 4 unidades y necesita 1 más.
- P3 necesita solo 3 y termina en su primer turno.
- P1 utiliza otras 4 y necesita 1.
- P2 termina.
- P1 termina.

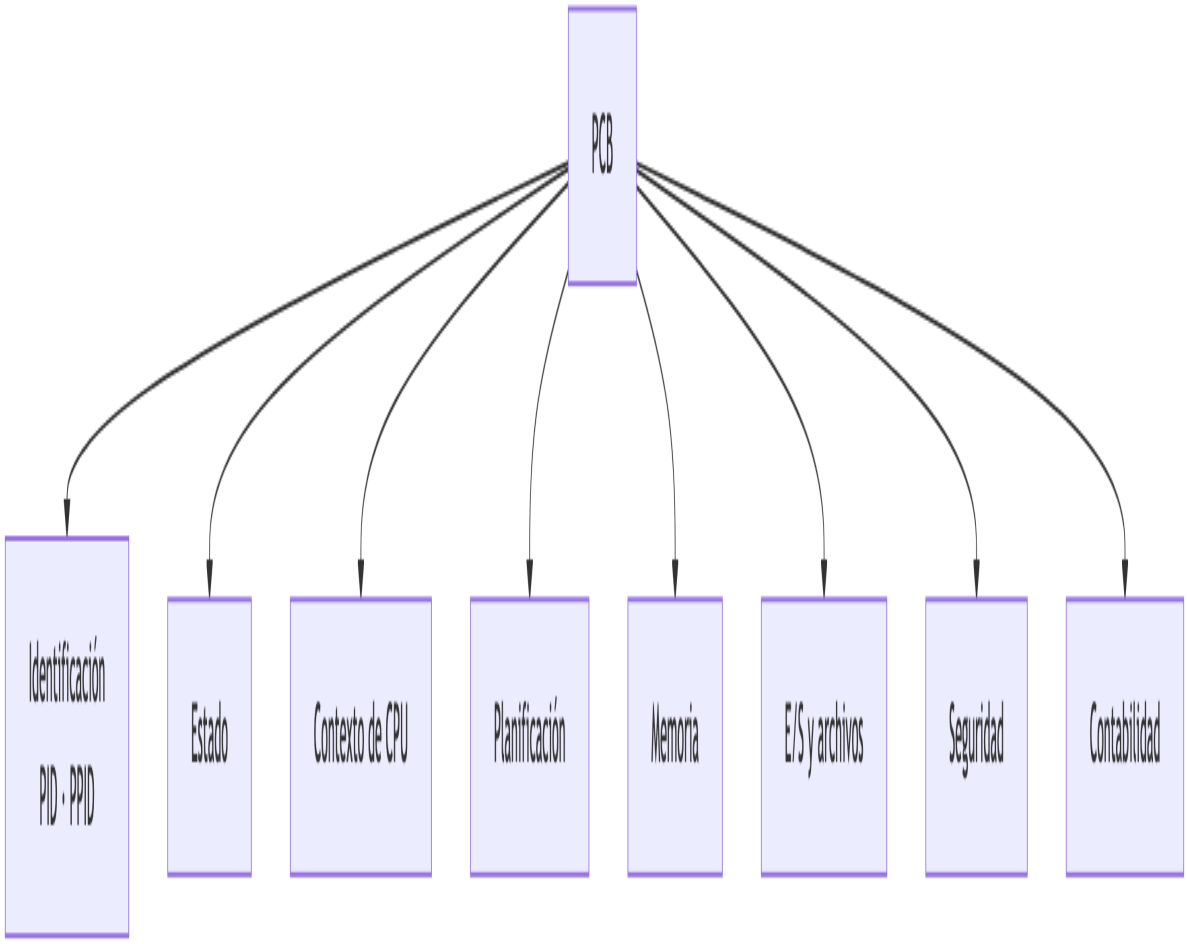
> **Ventaja de Round Robin**
>
> Todos los procesos listos reciben oportunidades periódicas de ejecución, lo que favorece la interactividad y evita que uno monopolice indefinidamente la CPU.

# 122 PCB y control del proceso

El **Process Control Block** es la estructura mediante la cual el sistema operativo conserva la información necesaria para administrar un proceso.

## 122.1 Componentes principales

| Categoría | Información |
|---|---|
| Identificación | PID, PPID, usuario y grupo |
| Estado | Nuevo, listo, ejecución, bloqueado o terminado |
| Contexto de CPU | Registros, contador de programa y puntero de pila |
| Planificación | Prioridad, política, tiempo utilizado y afinidad |
| Memoria | Espacio de direcciones y estructuras de administración |
| Entrada/salida | Dispositivos, operaciones pendientes y archivos abiertos |
| Seguridad | Credenciales, privilegios y permisos |
| Contabilidad | Tiempo de CPU, fecha de creación y estadísticas |




**Modelo conceptual**

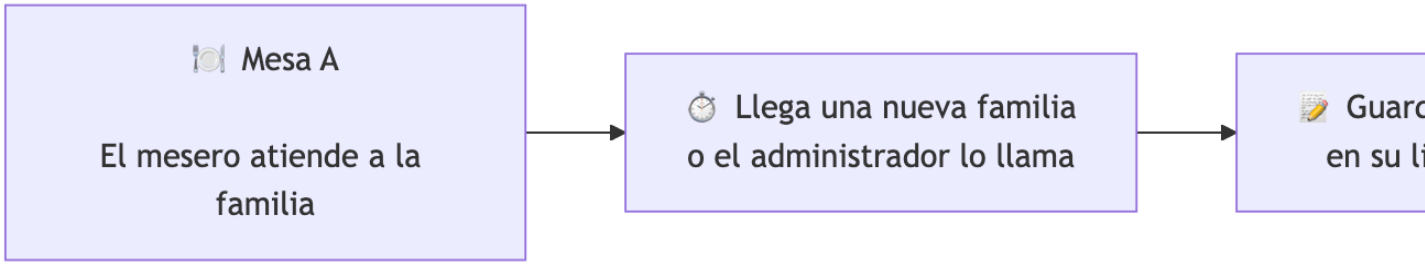
Los nombres y campos exactos varían entre sistemas operativos. “PCB” es una forma didáctica de agrupar la información; un kernel real puede distribuirla entre varias estructuras internas.

# 123 Cambio de contexto

Un **cambio de contexto** ocurre cuando la CPU deja de ejecutar una unidad de trabajo y comienza otra.

## 123.1 Del mundo real al Sistema Operativo

> **Analogía: El mesero del restaurante**
>
> Imagina un restaurante donde **solo hay un mesero** atendiendo varias mesas.
> En la **Mesa A** una familia está haciendo su pedido, pero mientras el mesero la atiende, llega una nueva familia a la **Mesa B**.
> El administrador le pide que primero reciba a los nuevos clientes.
> Antes de irse, el mesero **anota exactamente en qué punto quedó atendiendo la Mesa A** para no olvidar ningún detalle.
> Después atiende la Mesa B y, cuando termina, regresa a la Mesa A. Gracias a las anotaciones de su libreta puede continuar exactamente donde había quedado.
> La CPU realiza un proceso muy similar cuando cambia de un proceso a otro.




*Flujo*

```
1. El mesero atiende a la **Mesa A**.
2. Ocurre una **interrupción**.
3. Guarda notas en su libreta.
4. Atiende la **Mesa B**.
5. Recupera sus notas.
6. Continúa atendiendo la **Mesa A**.
```

### 123.1.1 Relación entre el restaurante y el Sistema Operativo

| En el restaurante | En el Sistema Operativo |
|---|---|
| Mesero | CPU |
| Mesa A | Proceso A |
| Mesa B | Proceso B |
| Libreta del mesero | PCB (Process Control Block) |
| Administrador | Kernel |
| Cambiar de una mesa a otra | Cambio de contexto |

## 123.2 ¿Qué sucede realmente?

Cuando la CPU está ejecutando un proceso y ocurre una interrupción o finaliza su **quantum**, el sistema operativo realiza un **cambio de contexto**.

**Idea clave**

La CPU **no recuerda** por sí sola en qué punto quedó cada proceso.
Toda esa información se almacena en el **PCB (Process Control Block)**, de la misma forma que un mesero utiliza su libreta para recordar qué estaba haciendo en cada mesa.
Por eso, un cambio de contexto no significa volver a empezar un proceso, sino **continuar exactamente desde el punto donde fue interrumpido**.

**¿Sabías que…?**

Un procesador moderno puede realizar **miles o incluso millones de cambios de contexto por segundo**, dependiendo de la carga del sistema y del sistema operativo.
Aunque cada cambio toma muy poco tiempo, también representa un costo, ya que durante ese instante la CPU no está ejecutando instrucciones útiles del programa, sino organizando cuál proceso continuará después.

Este procedimiento consiste en:

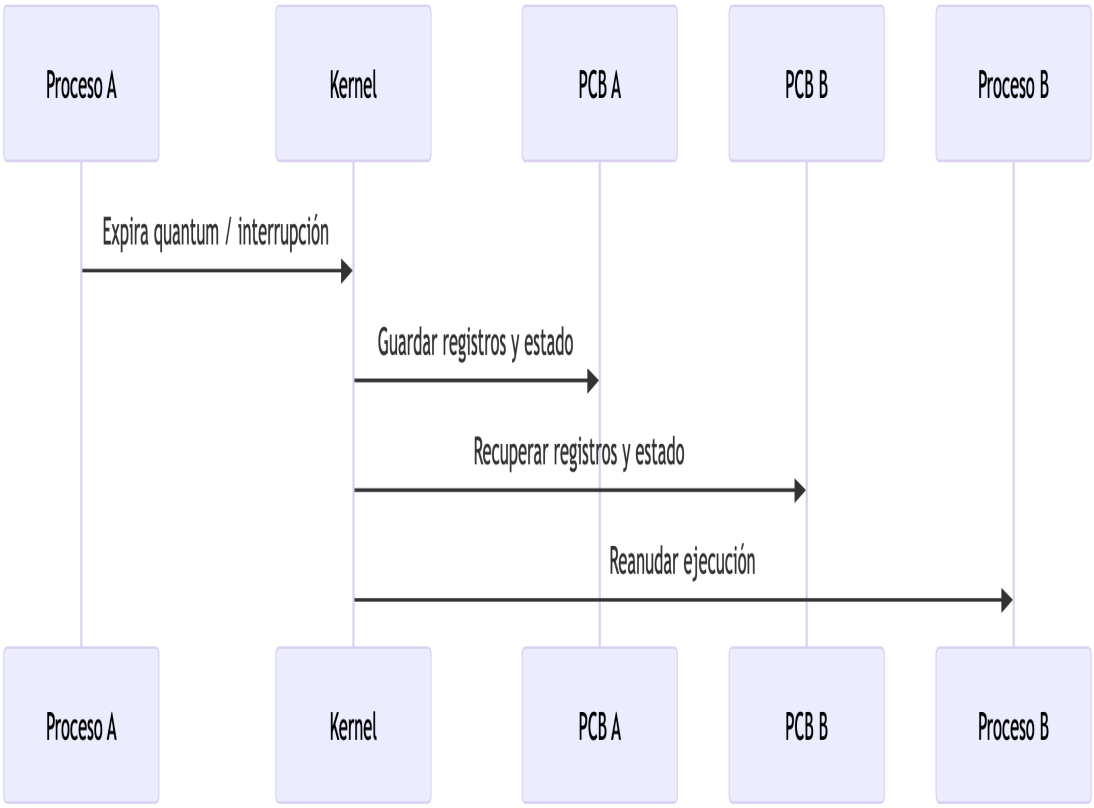
1. Detener temporalmente el proceso que se estaba ejecutando.
2. Guardar su estado actual dentro del **PCB**.
3. Seleccionar el siguiente proceso listo para ejecutarse.
4. Recuperar el estado almacenado del nuevo proceso.
5. Reanudar la ejecución exactamente donde ese proceso había quedado anteriormente.

Gracias a este mecanismo, la CPU puede compartir su tiempo entre muchos procesos dando la impresión de que todos se ejecutan simultáneamente.

Durante el cambio:

1. se guarda el estado del proceso o hilo actual;
2. se actualiza su información de control;
3. se selecciona otra unidad;

4. se restaura su contexto;
5. se reanuda la ejecución.




Un cambio de contexto tiene costo porque consume tiempo sin avanzar directamente el trabajo de la aplicación. Sin embargo, es indispensable para ofrecer multitarea, aislamiento y capacidad de respuesta.

# 124 Recursos compartidos

La concurrencia se vuelve delicada cuando procesos o hilos acceden a un mismo recurso.

Ejemplos:

- una variable;
- un archivo;
- una impresora;
- una estructura en memoria;
- un registro de base de datos;
- una cola;
- un dispositivo;
- un contador.

## 124.1 Ejemplo: contador compartido

Supongamos que:

```
contador = 10
```

Dos hilos ejecutan:

```
contador = contador + 1
```

La operación no siempre es indivisible. Puede implicar:

1. leer contador;
2. sumar uno;
3. escribir el resultado.

Posible intercalado:

| Paso | Hilo A | Hilo B | Valor |
|---|---|---|---|
| 1 | Lee 10 | | 10 |
| 2 | | Lee 10 | 10 |
| 3 | Calcula 11 | | 10 |
| 4 | | Calcula 11 | 10 |

| Paso | Hilo A | Hilo B | Valor |
|---|---|---|---|
| 5 | Escribe 11 | | 11 |
| 6 | | Escribe 11 | 11 |

El resultado esperado era 12, pero se obtuvo 11.

# 125 Condición de carrera

Una **condición de carrera** ocurre cuando el resultado depende del orden o del momento en que varias unidades concurrentes acceden a datos compartidos.

Sus características habituales son:

- es intermitente;
- puede desaparecer al depurar;
- depende de la planificación;
- puede producir pérdida o corrupción de datos;
- es difícil de reproducir;
- puede convertirse en vulnerabilidad.

> ⚠ Error común
>
> Concurrencia no significa automáticamente condición de carrera. El problema aparece cuando existe acceso compartido sin una coordinación correcta.

# 126 Sección crítica

Una **sección crítica** es la parte del programa que accede o modifica un recurso compartido y que debe protegerse para evitar interferencias.

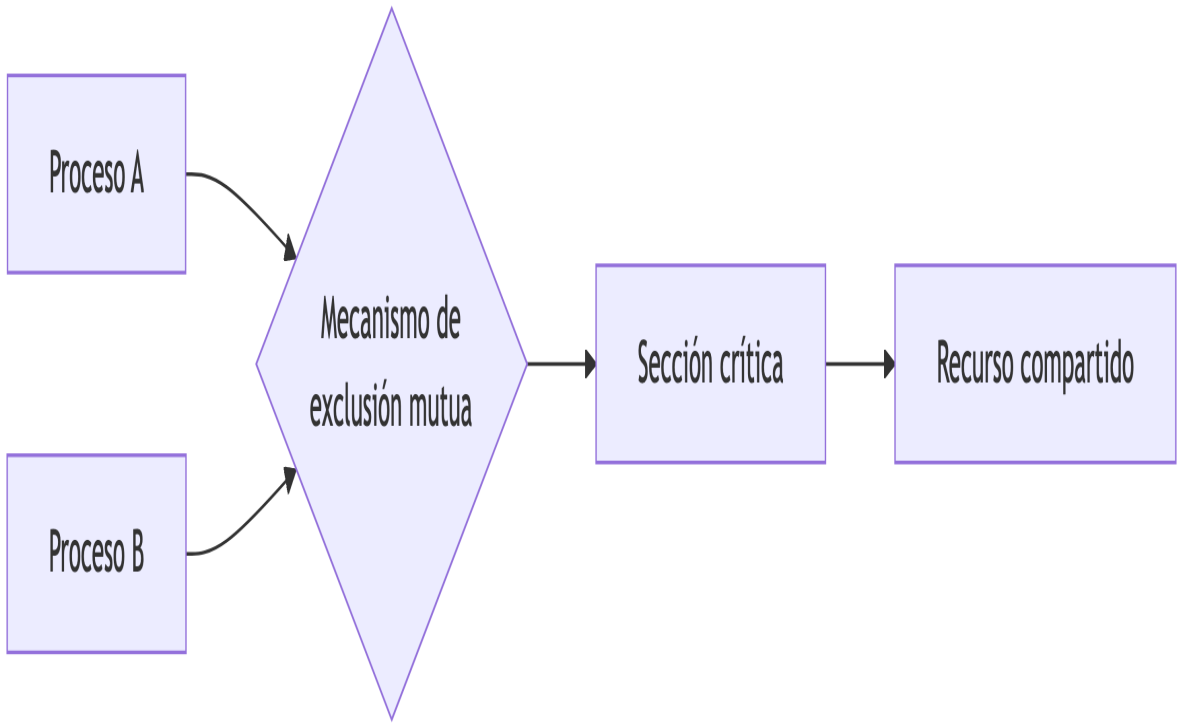
```
Entrada a la sección crítica

    modificar recurso compartido

Salida de la sección crítica
```

Una solución correcta para la sección crítica busca cumplir:

1. **Exclusión mutua:** solo una unidad entra a la vez.
2. **Progreso:** si nadie está dentro, debe poder seleccionarse quién entra.
3. **Espera limitada:** ninguna unidad debe esperar indefinidamente.



# 127 Mecanismos de sincronización

Los sistemas modernos ofrecen mecanismos como:

| Mecanismo | Uso general |
|---|---|
| Mutex | Exclusión mutua |
| Semáforo | Control de acceso y señalización |
| Monitor | Encapsulación de estado y sincronización |
| Variable de condición | Esperar hasta que ocurra una condición |
| Spinlock | Espera activa breve en contextos apropiados |
| Operación atómica | Actualización indivisible de datos simples |

> **!** Criterio profesional
>
> No debe improvisarse un mecanismo de sincronización cuando el lenguaje o el sistema operativo ya ofrece primitivas probadas.

# 128 Algoritmo de Dekker

El **algoritmo de Dekker** es una solución histórica de software para lograr exclusión mutua entre **dos procesos**.

Utiliza:

- una bandera por proceso;
- una variable de turno;
- espera activa.

## 128.1 Idea general

Cada proceso indica que desea entrar. Si ambos quieren entrar al mismo tiempo, la variable `turno` resuelve quién cede temporalmente.

## 128.2 Pseudocódigo conceptual

```
bandera[0] = falso
bandera[1] = falso
turno = 0

Proceso i:
    j = 1 - i

    mientras verdadero:
        bandera[i] = verdadero

        mientras bandera[j]:
            si turno == j:
                bandera[i] = falso

                mientras turno == j:
                    esperar

                bandera[i] = verdadero
```

```
        sección crítica

        turno = j
        bandera[i] = falso

        sección no crítica
```

## 128.3 Explicación

1. `bandera[i] = verdadero` anuncia intención de entrar.
2. Si el otro proceso no desea entrar, el proceso continúa.
3. Si ambos desean entrar, `turno` decide quién espera.
4. Al salir, se entrega el turno al otro proceso.
5. La bandera se desactiva.

**Importancia histórica**

Dekker demuestra que la exclusión mutua puede lograrse mediante coordinación lógica entre dos procesos.

**Limitación práctica**

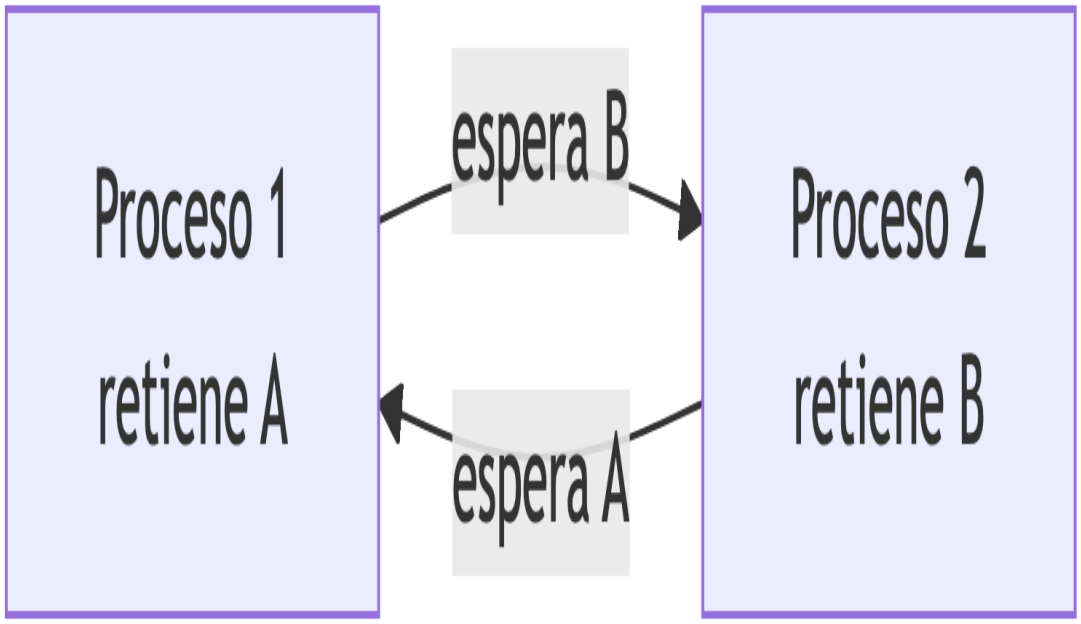
Dekker es valioso para estudiar concurrencia, pero no debe implementarse como sustituto de mutex, semáforos u operaciones atómicas en aplicaciones modernas. Además, los modelos modernos de memoria y las optimizaciones del compilador requieren primitivas con garantías explícitas.

# 129 Interbloqueo, inanición y livelock

Estos problemas no son iguales.

| Problema | Descripción |
|---|---|
| Interbloqueo | Dos o más procesos esperan recursos retenidos entre sí |
| Inanición | Un proceso espera indefinidamente porque otros reciben preferencia |
| Livelock | Los procesos cambian de estado, pero no progresan |
| Condición de carrera | El resultado depende del intercalado no controlado |

## 129.1 Ejemplo de interbloqueo



# 130 Inteligencia artificial y control de procesos

La IA puede utilizarse para analizar métricas y apoyar políticas de administración. Sin embargo, debe distinguirse entre:

- funciones presentes en sistemas de producción;
- mecanismos experimentales;
- investigación académica;
- posibilidades futuras.

> **!** Aclaración
>
> La IA no “entra al PCB” ni sustituye las estructuras del kernel. Un modelo puede analizar información derivada de procesos y recomendar o ejecutar decisiones mediante interfaces controladas.

## 130.1 Áreas de aplicación

### 130.1.1 Predicción de carga

Un modelo puede estimar:

- consumo futuro de CPU;
- presión de memoria;
- comportamiento interactivo;
- ráfagas de entrada/salida;
- demanda de energía.

### 130.1.2 Planificación adaptativa

La investigación explora planificadores capaces de ajustar políticas según:

- tipo de carga;
- latencia;

- rendimiento;
- consumo energético;
- afinidad entre tareas y núcleos;
- objetivos de servicio.

### 130.1.3 Detección de anomalías

Los datos de procesos pueden utilizarse para detectar:

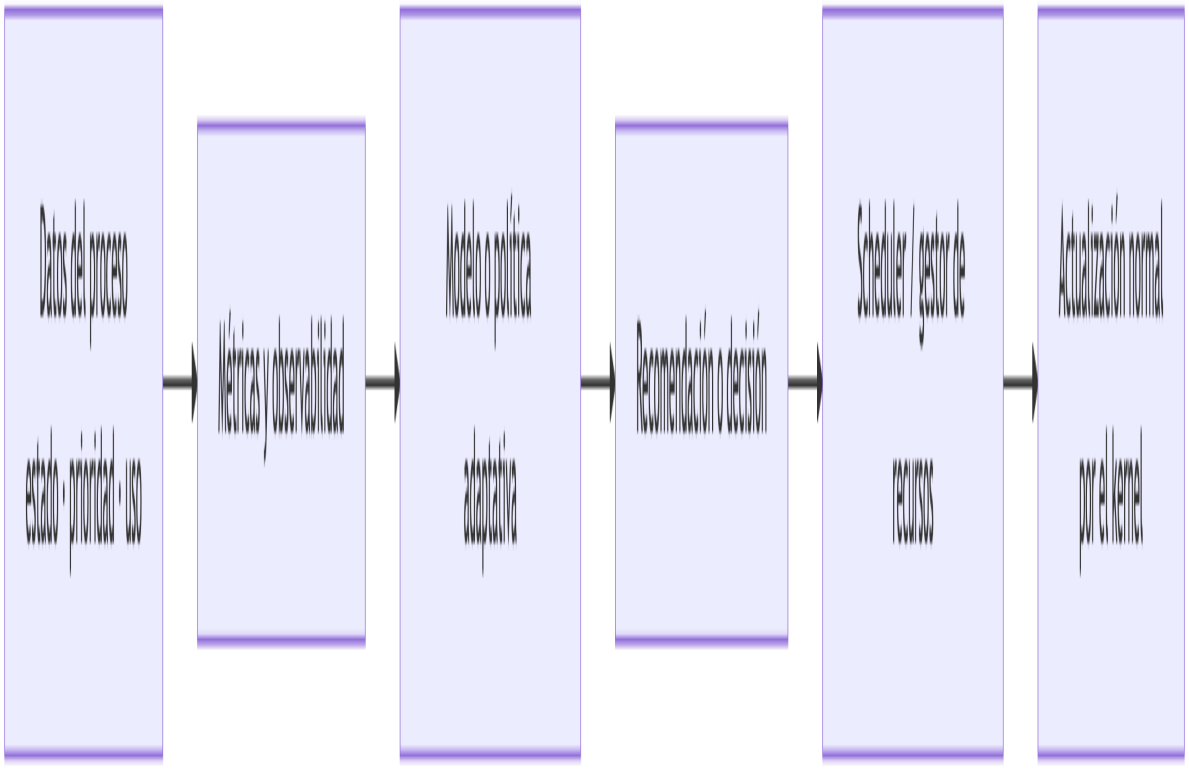
- consumo inusual;
- procesos que dejan de responder;
- fugas de memoria;
- patrones sospechosos;
- degradación de rendimiento.

### 130.1.4 Administración energética

En equipos móviles, servidores y centros de datos, modelos predictivos pueden apoyar decisiones sobre:

- frecuencias;
- núcleos activos;
- distribución de carga;
- consolidación de tareas;
- límites térmicos.

# 131 IA, PCB y Scheduler: relación correcta




La secuencia correcta es:

1. El kernel conserva y actualiza las estructuras del proceso.
2. Herramientas de observabilidad obtienen métricas.
3. Un modelo puede analizar esas métricas.
4. La política entrega una decisión a través de una interfaz válida.
5. El sistema operativo aplica la decisión y actualiza sus estructuras.

## 131.1 Estado actual: planificadores extensibles

Linux dispone de `sched_ext`, una clase de planificación extensible basada en BPF que permite experimentar con algoritmos de planificación externos y regresar al planificador predeterminado cuando se detectan errores. Esta interfaz permite investigar políticas personalizadas, incluidas políticas que podrían integrar modelos predictivos, pero no implica que los sistemas operativos de escritorio utilicen IA de manera general para reemplazar su Scheduler. Documentación oficial del kernel de Linux.

> ⚠ No confundir posibilidad con adopción
>
> Que una interfaz permita implementar una política inteligente no significa que todo sistema operativo actual utilice aprendizaje automático para planificar procesos.

# 132 Comparación: planificación clásica y adaptativa

| Aspecto | Planificación clásica | Enfoque adaptativo o basado en datos |
|---|---|---|
| Reglas | Definidas previamente | Ajustadas según métricas o predicciones |
| Interpretación | Generalmente clara | Puede ser más compleja |
| Costo | Predecible | Incluye inferencia y recolección de datos |
| Verificación | Más directa | Requiere validar modelo y política |
| Robustez | Amplia experiencia acumulada | Depende de entrenamiento y límites |
| Uso | Generalizado | Investigación y escenarios especializados |

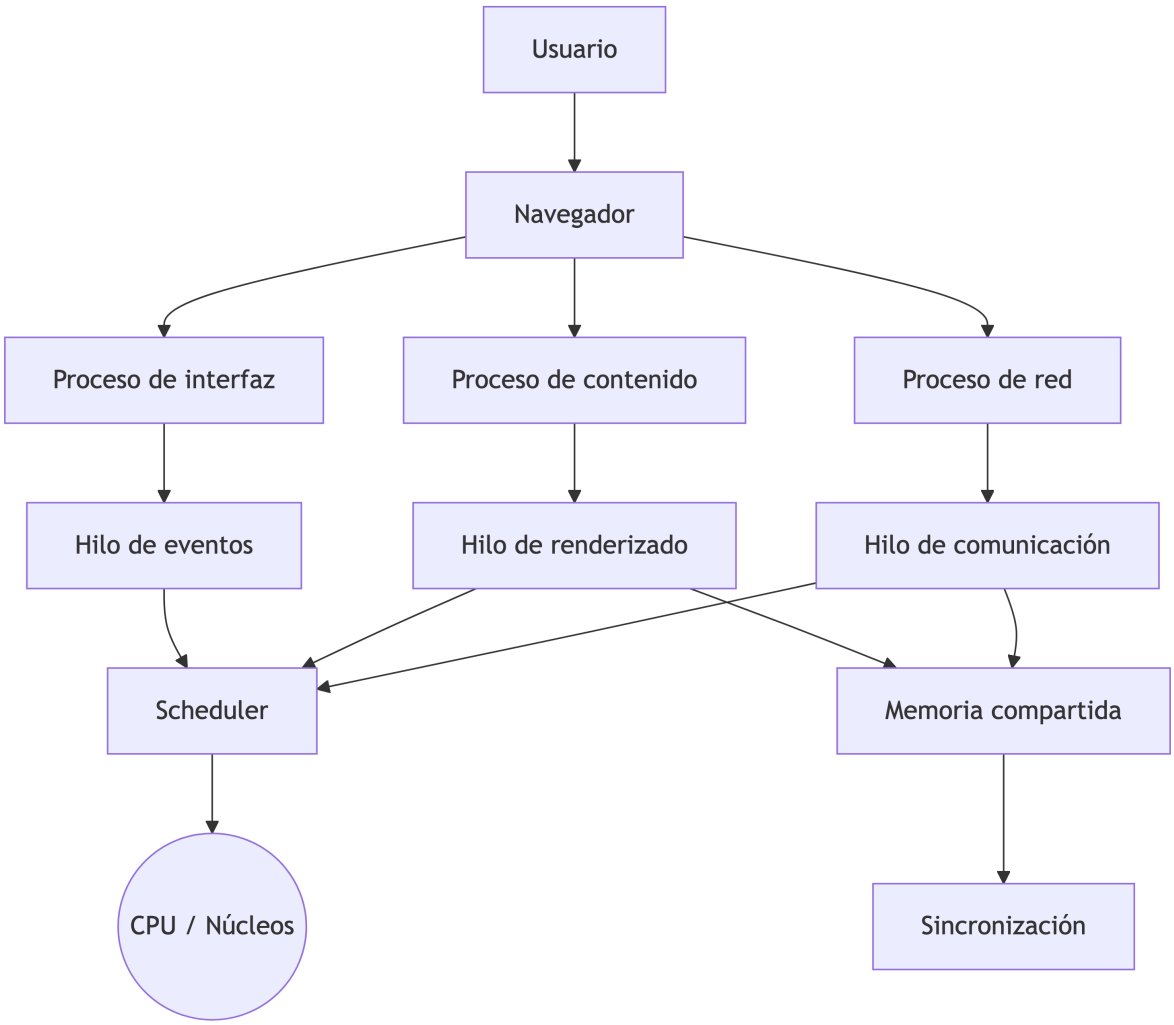
# 133 Caso integrador: navegador durante una videoclase

Un estudiante participa en una videoclase mientras:

- reproduce audio y video;
- escribe en el chat;
- descarga un documento;
- mantiene varias pestañas;
- guarda apuntes;
- recibe notificaciones.

## 133.1 Análisis

1. El navegador puede utilizar varios procesos e hilos.
2. El Scheduler distribuye tiempo de CPU.
3. Cada hilo conserva su contexto de ejecución.
4. Las operaciones de red o disco pueden bloquear temporalmente tareas.
5. Al expirar el quantum, una tarea puede volver a READY.
6. Los buffers y estructuras compartidas requieren sincronización.
7. Una condición de carrera podría corromper datos.
8. Los mutex y otras primitivas protegen secciones críticas.
9. Herramientas de observabilidad pueden detectar consumo anómalo.
10. Una política adaptativa podría ajustar prioridades, pero el kernel conserva el control.


Usuario
Navegador
Proceso de interfaz
Proceso de contenido
Proceso de red
Hilo de eventos
Hilo de renderizado
Hilo de comunicación
Scheduler
Memoria compartida
CPU / Núcleos
Sincronización

# 134 Laboratorio: observando procesos e hilos

## 134.1 Propósito

Relacionar los conceptos de procesos, hilos, estados, CPU y cambios de contexto con información real del sistema.

## 134.2 Parte A. Windows

Abra **Administrador de tareas** o **Monitor de recursos**.

Registre:

| Elemento | Resultado |
|---|---|
| Proceso con mayor uso de CPU | |
| Proceso con mayor uso de memoria | |
| Número aproximado de procesos | |
| Número aproximado de hilos | |
| Aplicación con varios procesos relacionados | |

Opcionalmente, en PowerShell:

```
Get-Process |
  Sort-Object CPU -Descending |
  Select-Object -First 10 Name, Id, CPU, Threads, WorkingSet
```

## 134.3 Parte B. macOS

Abra **Monitor de Actividad**.

También puede utilizar:

```
ps -axo pid,ppid,state,%cpu,%mem,comm | head
```

y:

```
top -l 1 -stats pid,command,cpu,mem,threads
```

## 134.4 Parte C. GNU/Linux

Utilice:

```
ps -eo pid,ppid,stat,pri,ni,%cpu,%mem,comm --sort=-%cpu | head
```

Para observar hilos:

```
ps -eLf | head
```

También puede utilizar:

```
top
```

o, si está instalado:

```
htop
```

> **Regla de seguridad del laboratorio**
>
> La actividad es únicamente de observación. No finalice procesos del sistema, no cambie prioridades y no ejecute comandos con privilegios administrativos.

## 134.5 Preguntas de análisis

1. ¿Qué diferencia observa entre proceso e hilo?
2. ¿Qué proceso tiene más hilos?
3. ¿El proceso con mayor memoria también utiliza más CPU?
4. ¿Qué significan los estados observados?
5. ¿Por qué el uso de CPU cambia constantemente?
6. ¿Qué aplicación parece utilizar una arquitectura multiproceso?
7. ¿Qué riesgo tendría finalizar un proceso sin conocer su función?

# 135 Actividad en clase: simulación de Round Robin

## 135.1 Materiales

- Cuatro tarjetas de procesos.
- Un cronómetro.
- Una tabla de seguimiento.

## 135.2 Datos

| Proceso | Ráfaga |
|---|---|
| P1 | 7 |
| P2 | 4 |
| P3 | 9 |
| P4 | 5 |

Use:

```
Quantum = 3
```

## 135.3 Instrucciones

1. Coloque los procesos en la cola.
2. Asigne tres unidades al primero.
3. Si no termina, envíelo al final.
4. Registre cada cambio.
5. Calcule el tiempo de finalización.
6. Cuente los cambios de contexto.
7. Repita con quantum 1 y quantum 6.
8. Compare los resultados.

## 135.4 Tabla de resultados

| Quantum | Cambios de contexto | Respuesta percibida | Observación |
|---|---|---|---|
| 1 | | | |
| 3 | | | |
| 6 | | | |

# 136 Reto del Ingeniero

Una aplicación bancaria ejecuta varios hilos:

- interfaz;
- validación;
- consulta de saldo;
- registro de transacciones;
- generación de comprobantes.

Dos hilos actualizan simultáneamente el saldo de una cuenta.

Responda:

1. ¿Cuál es el recurso compartido?
2. ¿Dónde existe una sección crítica?
3. ¿Qué condición de carrera podría ocurrir?
4. ¿Qué mecanismo de sincronización propondría?
5. ¿Qué información del proceso conservaría el sistema operativo?
6. ¿Qué métricas permitirían detectar comportamiento anómalo?
7. ¿Qué decisión nunca debería dejarse únicamente a un modelo de IA sin controles?

# 137 Autoevaluación

## 137.1 Selección múltiple

1. ¿Qué comparte normalmente un grupo de hilos del mismo proceso?
    a. Únicamente el PID
    b. Espacio de direcciones y recursos
    c. Ningún recurso
    d. Procesadores físicos
2. ¿Qué ocurre cuando expira el quantum?
    a. El proceso siempre termina
    b. El proceso se elimina
    c. El kernel puede interrumpirlo y devolverlo a READY
    d. Se borra su PCB
3. ¿Qué caracteriza una condición de carrera?
    a. El resultado depende del intercalado
    b. Todos los procesos terminan en orden
    c. No existen recursos compartidos
    d. El quantum es demasiado grande
4. ¿Qué busca garantizar un mutex?
    a. Paralelismo ilimitado
    b. Exclusión mutua

c. Eliminación del PCB

d. Mayor memoria RAM

5. ¿Para cuántos procesos fue diseñado Dekker?

a. Uno

b. Dos

c. Diez

d. Un número ilimitado

6. ¿Cuál afirmación sobre IA y PCB es más precisa?

a. La IA reemplaza siempre el PCB

b. La IA modifica directamente cualquier estructura del kernel

c. Un modelo puede apoyar decisiones utilizando métricas observadas

d. Todo sistema operativo utiliza redes neuronales

**Respuestas**

1. b
2. c
3. a
4. b
5. b
6. c

## 137.2 Preguntas de reflexión

1. ¿Por qué un quantum demasiado pequeño puede reducir el rendimiento?
2. ¿Cuál es la diferencia entre concurrencia y paralelismo?
3. ¿Por qué los hilos son más ligeros, pero también menos aislados?
4. ¿Por qué una condición de carrera puede ser difícil de reproducir?
5. ¿Qué valor educativo tiene Dekker aunque no sea la opción usual en aplicaciones modernas?
6. ¿Qué riesgos introduce una política de planificación basada en aprendizaje automático?

# 138 Resumen de la clase

En esta clase aprendimos que:

- El control de procesos abarca creación, planificación, sincronización y finalización.
- Un proceso posee recursos propios y puede contener varios hilos.
- La concurrencia permite que varias tareas progresen durante un mismo período.
- El paralelismo requiere ejecución simultánea en hardware distinto.
- El quantum limita temporalmente el uso de CPU.
- Round Robin distribuye turnos entre los procesos listos.
- El PCB conserva la información necesaria para administrar un proceso.
- Los cambios de contexto permiten alternar entre unidades de ejecución.
- Los recursos compartidos pueden producir condiciones de carrera.
- Las secciones críticas deben protegerse mediante sincronización.
- Dekker es una solución histórica para dos procesos.
- La IA puede apoyar predicción, observabilidad y políticas adaptativas, pero no reemplaza automáticamente al kernel.

> Consejo final del Ingeniero
>
> La concurrencia no se domina memorizando nombres. Se comprende siguiendo el orden exacto de las operaciones y preguntando qué datos comparten las unidades de ejecución.

# 139 Tarea sugerida

1. Dibuje el modelo de cinco estados.
2. Dibuje y explique los componentes principales del PCB.
3. Resuelva manualmente un ejercicio Round Robin con cuatro procesos.
4. Describa una condición de carrera de la vida real.
5. Compare mutex, semáforo y monitor.
6. Explique el algoritmo de Dekker con sus propias palabras.
7. Redacte una conclusión sobre el uso responsable de IA en la planificación de procesos.

# 140 Referencias y actualización


- Material base de la Clase 5: *Control de Procesos y Concurrencia*. filecite turn8file0
- Linux Kernel Documentation. *Extensible Scheduler Class (sched_ext)*.
- Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*.
- Stallings, W. *Operating Systems: Internals and Design Principles*.
- Tanenbaum, A. S., & Bos, H. *Modern Operating Systems*.


> **Nota editorial**
>
> La sección de inteligencia artificial se presenta como panorama técnico y línea de investigación. Las afirmaciones deben revisarse periódicamente porque los mecanismos de planificación y observabilidad evolucionan con rapidez.

# 141 Capítulo 6. ¿Cómo decide el Sistema Operativo quién utiliza el procesador?

# 142 Capítulo 6. Algoritmos de planificación del CPU

## 142.1 Relación con la clase anterior

En la clase anterior estudiamos los procesos, el PCB, el Scheduler, el Dispatcher y el Cambio de Contexto.

Ahora responderemos una nueva pregunta:

> **¿Cómo decide el Sistema Operativo cuál proceso utilizará el procesador?**

# 143 Objetivos de aprendizaje

- Comprender por qué es necesaria la planificación del CPU.
- Explicar el papel del Scheduler.
- Analizar los algoritmos FCFS, SJF, Prioridades y Round Robin.
- Comparar ventajas y desventajas de cada algoritmo.

# 144 Motivación

Imagina que cinco personas llegan al mismo tiempo a un banco.

Solo existe una ventanilla.

¿Quién debería ser atendido primero?

- El primero que llegó.
- El que terminará más rápido.
- El cliente con mayor prioridad.
- Un turno para cada uno.

Con el procesador ocurre exactamente lo mismo.

# 145 ¿Por qué es necesaria la planificación?

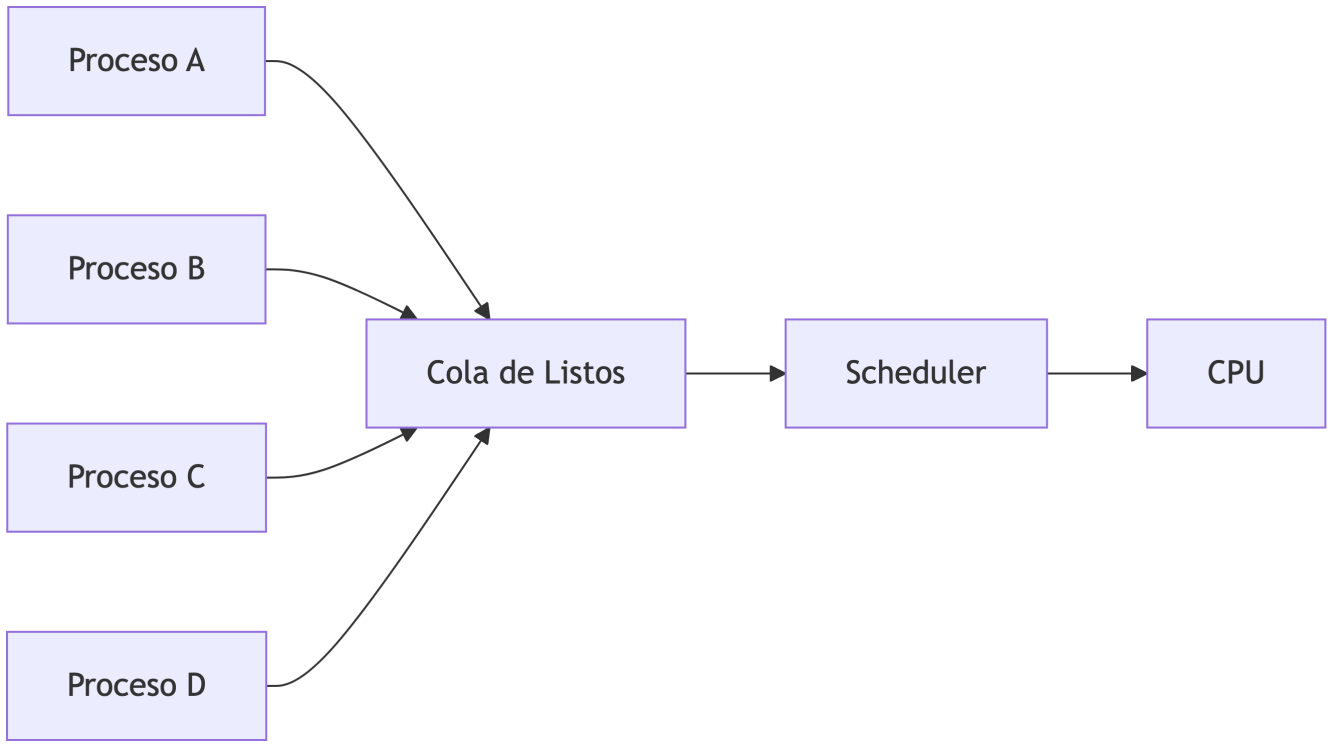
En una computadora pueden existir muchos procesos listos para ejecutarse.

Sin embargo, el CPU únicamente puede ejecutar **un proceso a la vez**.

Por ello el Sistema Operativo necesita decidir:

- ¿Quién utiliza el procesador?
- ¿Durante cuánto tiempo?
- ¿Cuándo debe cambiar a otro proceso?

Ese proceso de decisión recibe el nombre de **Planificación del CPU**.



**Idea clave**

La planificación busca utilizar el procesador de la forma más eficiente posible.

# 146 ¿Qué aprenderemos en este capítulo?

Estudiaremos:

1. FCFS (FIFO)
2. SJF
3. Prioridades
4. Round Robin

Y al finalizar compararemos cuándo conviene utilizar cada uno.

# 147 Preparándonos para la siguiente sección

Ahora estudiaremos al componente encargado de tomar estas decisiones: **el Scheduler**, trabajando en conjunto con el Kernel.

# 148 El Scheduler, el Dispatcher y el Kernel

Hasta este momento sabemos que existen varios procesos esperando utilizar el procesador.

La siguiente pregunta es:

**¿Quién toma la decisión de cuál proceso debe ejecutarse?**

La respuesta involucra tres componentes importantes del Sistema Operativo.

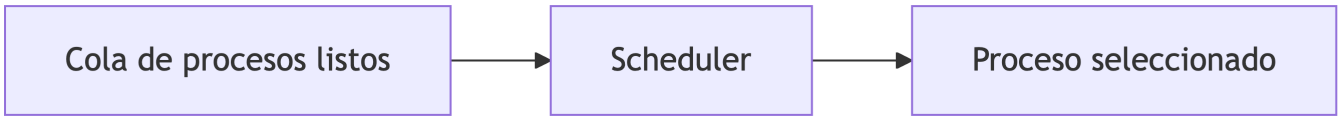
# 149 El Scheduler

El **Scheduler** (Planificador) es el encargado de seleccionar el siguiente proceso que utilizará el CPU.

Para tomar esta decisión analiza información almacenada en el PCB de cada proceso.

Entre los principales criterios se encuentran:

- Tiempo de llegada.
- Tiempo estimado de ejecución.
- Prioridad.
- Estado del proceso.
- Algoritmo de planificación utilizado.




> **Idea importante**
>
> El Scheduler **decide**, pero todavía no entrega el procesador al proceso.

---

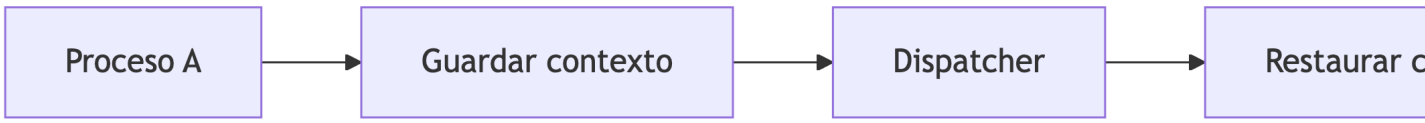
# 150 El Dispatcher

Una vez que el Scheduler selecciona el proceso, entra en acción el **Dispatcher**.

Su trabajo consiste en:

- Guardar el contexto del proceso que abandona el CPU.
- Restaurar el contexto del proceso seleccionado.
- Entregar el control del procesador.

Este procedimiento recibe el nombre de **Cambio de Contexto (Context Switch)**.



# 151 ¿Qué hace realmente el Kernel?

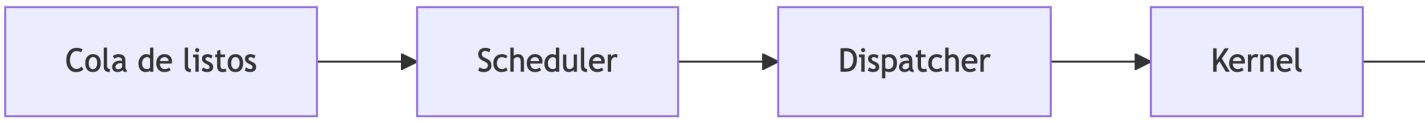
El Kernel coordina todo este proceso.

Mientras el Scheduler decide y el Dispatcher ejecuta el cambio, el Kernel:

- Mantiene la cola de procesos listos.
- Administra los PCB.
- Controla las interrupciones.
- Coordina el cambio de contexto.
- Garantiza que un solo proceso utilice el CPU al mismo tiempo.

> Recuerda
>
> **Scheduler** → Decide.
> **Dispatcher** → Ejecuta el cambio.
> **Kernel** → Coordina todo el proceso.

# 152 ¿Cómo trabajan juntos?



# 153 Preparándonos para el primer algoritmo

Ahora que comprendemos quién toma las decisiones, comenzaremos con el algoritmo más sencillo de todos:

**FCFS (First Come, First Served)**, también conocido como **FIFO (First In, First Out)**.

# 154 Algoritmo FCFS (FIFO)

Ahora estudiaremos el algoritmo de planificación más sencillo.

Su nombre proviene de **First Come, First Served**, que significa:

> **El primero en llegar, es el primero en ser atendido.**

También se conoce como **FIFO (First In, First Out)**.

# 155 Un ejemplo cotidiano

Imagina una fila en la cafetería de la universidad.

Los estudiantes llegan en este orden:

1. Ana
2. Luis
3. Carlos
4. María

La regla es muy simple:

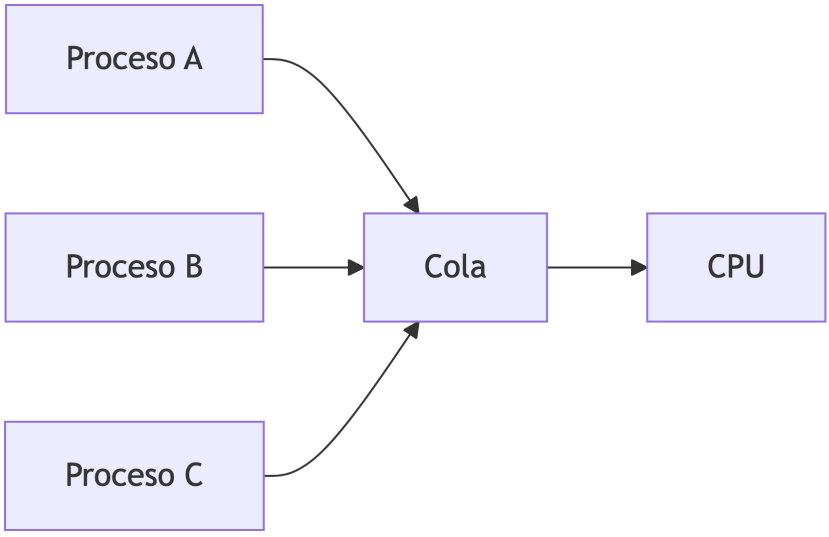
- Nadie puede adelantar a otro.
- El primero en llegar será el primero en ser atendido.

FCFS funciona exactamente igual.

# 156 ¿Cómo trabaja FCFS?

Cada proceso se coloca al final de la cola de listos.

Cuando el CPU queda libre, el Scheduler selecciona el proceso que lleva más tiempo esperando.




> **Idea importante**
>
> Una vez que un proceso obtiene el CPU, **no será interrumpido** hasta finalizar su ejecución.

# 157 Ejemplo

Utilizaremos los mismos procesos mostrados en la presentación de clase. filecite turn13file0 L5-L8

| Proceso | Llegada | CPU |
|---|---|---|
| A | 0 | 3 |
| B | 1 | 2 |
| C | 3 | 4 |
| D | 4 | 2 |
| E | 7 | 5 |
| F | 9 | 3 |

# 158 Diagrama de Gantt

```
0       3    5         9   11         16   19
|---A---|--B--|----C----|-D-|-----E-----|-F-|
```

Observa que el orden depende únicamente del momento en que cada proceso llegó al sistema.

# 159 Ventajas

- Fácil de implementar.
- Poco costo de administración.
- Ideal para sistemas sencillos.

# 160 Desventajas

- Un proceso muy largo puede retrasar a todos los demás.
- Los procesos cortos pueden esperar demasiado.
- No es adecuado para sistemas interactivos.

# 161 ¿Cuándo utilizar FCFS?

Es recomendable cuando:

- Todos los procesos tienen duraciones similares.
- La simplicidad es más importante que el rendimiento.
- Se trata de sistemas pequeños.

> Reflexiona
>
> ¿Qué ocurriría si el primer proceso necesitara 30 segundos de CPU y los demás solo 2 segundos?
> ¿Sería justo utilizar FCFS?

# 162 Preparándonos para el siguiente algoritmo

FCFS considera únicamente el orden de llegada.

¿Y si el Sistema Operativo eligiera primero el proceso más corto?

Eso responderá el algoritmo **SJF (Shortest Job First)**.

# 163 Algoritmo SJF (Shortest Job First)

Después de conocer FCFS surge una pregunta interesante.

> ¿Y si en lugar de atender al primero que llegó, atendiéramos primero al proceso que terminará más rápido?

Esa es precisamente la idea del algoritmo **SJF (Shortest Job First)**.

# 164 Un ejemplo cotidiano

Imagina una fotocopiadora en la universidad.

Hay cuatro estudiantes esperando:

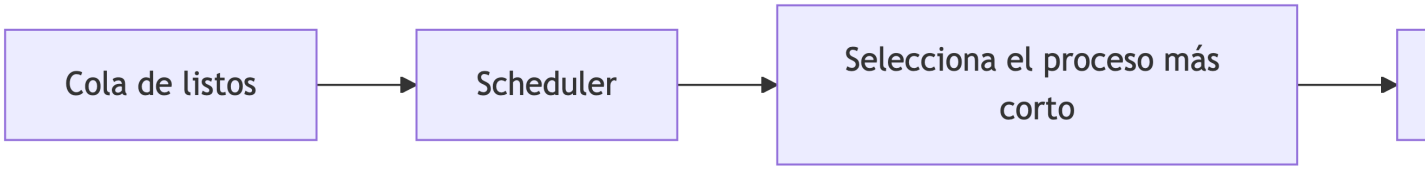
| Estudiante | Hojas |
|---|---|
| Ana | 2 |
| Luis | 25 |
| Carlos | 4 |
| María | 1 |

Si queremos atender al mayor número de personas en el menor tiempo posible, comenzaríamos por los trabajos más cortos.

SJF aplica exactamente esa estrategia.

# 165 ¿Cómo trabaja SJF?

Cuando el procesador queda libre, el Scheduler selecciona el proceso con el **menor tiempo estimado de CPU** entre los que están listos.




> **Idea importante**
>
> SJF no observa únicamente el orden de llegada; también considera cuánto tiempo tardará cada proceso en ejecutarse.

# 166 Ejemplo

Utilizaremos los mismos procesos del ejercicio de clase. □filecite□turn13file0□L7-L8□

| Proceso | Llegada | CPU |
|---|---|---|
| A | 0 | 3 |
| B | 1 | 2 |
| C | 3 | 4 |
| D | 4 | 2 |
| E | 7 | 5 |
| F | 9 | 3 |

Observa el diagrama de Gantt de la presentación y analiza cómo cambian las decisiones del Scheduler al elegir el proceso más corto disponible. □filecite□turn13file0□L7-L8□

# 167 Ventajas

- Reduce el tiempo promedio de espera.
- Favorece procesos cortos.
- Mejora el rendimiento en muchas cargas de trabajo.

# 168 Desventajas

- Requiere estimar el tiempo de ejecución.
- Los procesos largos pueden esperar demasiado (inanición).

# 169 ¿Cuándo utilizar SJF?

Es recomendable cuando:

- Se conoce o puede estimarse la duración de los procesos.
- El objetivo es reducir el tiempo promedio de espera.

> ⚠ Error común
>
> SJF no siempre ejecuta el proceso más corto del sistema.
> Solo compara los procesos que **ya se encuentran en la cola de listos**.

# 170 FCFS vs. SJF

| Característica | FCFS | SJF |
|---|---|---|
| Criterio | Orden de llegada | Menor tiempo de CPU |
| Fácil de implementar | ☐ | ☐ |
| Reduce espera promedio | ☐ | ☐ |
| Riesgo de espera excesiva (Inanición / Starvation) | Bajo | Alto |

ℹ ¿Qué significa el riesgo de inanición (Starvation)?

En Sistemas Operativos, la **inanición** (*Starvation*) ocurre cuando un proceso espera tanto tiempo por el procesador que, en la práctica, casi nunca llega a ejecutarse.
Esto sucede porque otros procesos son seleccionados continuamente antes que él.

### 170.0.1 Ejemplo

Supongamos que tenemos los siguientes procesos:

| Proceso | Tiempo de CPU |
|---|---|
| A | 20 s |
| B | 2 s |
| C | 1 s |
| D | 3 s |
| E | 2 s |

Si el Sistema Operativo utiliza **SJF (Shortest Job First)**, siempre elegirá primero los procesos con menor tiempo de ejecución.
Si mientras el proceso **A** espera siguen llegando procesos más pequeños, estos continuarán siendo seleccionados antes que él.
Como consecuencia, el proceso **A** podría permanecer esperando durante mucho tiempo.
Por esta razón decimos que **SJF presenta un mayor riesgo de inanición (Starvation)** para los procesos largos.
En cambio, con **FCFS**, todos los procesos conservan su turno de llegada, por lo que, tarde o temprano, todos serán ejecutados.

# 171 Preparándonos para el siguiente algoritmo

Hasta ahora hemos considerado el orden de llegada y la duración del proceso.

¿Y si algunos procesos fueran más importantes que otros?

Eso nos llevará al algoritmo de **Prioridades**.

# 172 Algoritmo de Planificación por Prioridades

Hasta ahora hemos estudiado dos formas de seleccionar el siguiente proceso:

- **FCFS:** el primero que llegó.
- **SJF:** el proceso más corto.

Ahora analizaremos una tercera estrategia.

> **¿Y si algunos procesos fueran más importantes que otros?**

Ese es precisamente el objetivo del algoritmo de **Prioridades**.

# 173 Un ejemplo cotidiano

Imagina la sala de emergencias de un hospital.

Llegan cuatro pacientes casi al mismo tiempo.

- Un paciente con un resfriado.
- Una persona con un brazo fracturado.
- Un paciente con un paro cardíaco.
- Una persona con una cortadura leve.

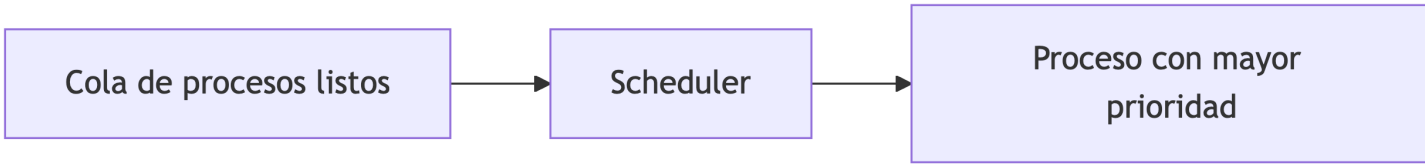
Aunque algunos hayan llegado antes, el médico atenderá primero al paciente cuya condición sea más crítica.

El Sistema Operativo hace algo similar cuando utiliza prioridades.

# 174 ¿Cómo trabaja?

Cada proceso recibe un **nivel de prioridad**.

Cuando el CPU queda disponible, el Scheduler selecciona el proceso con la prioridad más alta.




> **Idea importante**
>
> En este algoritmo, **la prioridad es más importante que el orden de llegada**.

# 175 Ejemplo

Supongamos los siguientes procesos.

| Proceso | Prioridad |
|---|---|
| A | 3 |
| B | 1 |
| C | 4 |
| D | 2 |

> En este ejemplo, **1 representa la prioridad más alta**.

El orden de ejecución será:

```
B → D → A → C
```

# 176 Diagrama de Gantt

```
0       2       5       8       10
|--B--|---D---|---A---|--C--|
```

### 176.0.1 ¿Quién asigna la prioridad?

La prioridad no se asigna al azar.

Generalmente es determinada por el Sistema Operativo considerando aspectos como:

- La importancia del proceso.
- El tipo de aplicación.
- El usuario que ejecuta el programa.
- El tiempo que lleva esperando.
- Las políticas definidas por el Sistema Operativo.

En muchos Sistemas Operativos modernos, la prioridad puede cambiar durante la ejecución para mejorar el rendimiento y evitar que algunos procesos esperen demasiado tiempo.

# 177 Ventajas

- Permite atender primero los procesos más importantes.
- Muy utilizado en sistemas operativos modernos.
- Adecuado para sistemas de tiempo real.

# 178 Desventajas

- Los procesos de baja prioridad pueden esperar demasiado tiempo.
- También puede presentarse **inanición (Starvation)** si continuamente llegan procesos con mayor prioridad.

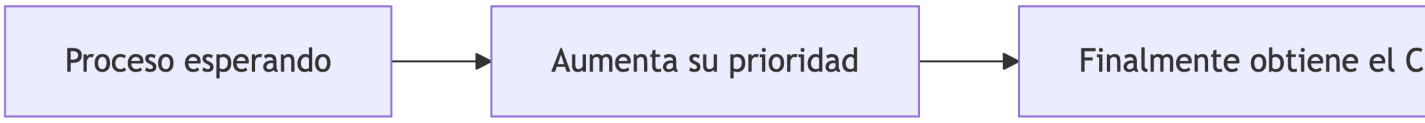
> ⚠ Error común
>
> Muchos estudiantes creen que el proceso con mayor prioridad siempre es el primero que llegó.
> Eso es incorrecto.
> El criterio principal es la **prioridad**, no el tiempo de llegada.

# 179 ¿Cómo evitar la inanición?

Muchos Sistemas Operativos implementan una técnica llamada **Aging (Envejecimiento)**.

Consiste en aumentar gradualmente la prioridad de los procesos que llevan mucho tiempo esperando.

De esta forma, tarde o temprano obtendrán acceso al procesador.



# 180 Comparación

| Característica | FCFS | SJF | Prioridades |
|---|---|---|---|
| Criterio principal | Orden de llegada | Menor tiempo de CPU | Mayor prioridad |
| Fácil de implementar | □ | □ | □ |
| Riesgo de espera excesiva (Starvation) | Bajo | Alto | Alto |
| Uso recomendado | Sistemas sencillos | Optimizar tiempos | Sistemas críticos |

Reflexiona

¿Por qué una alarma de incendio debería ejecutarse antes que un reproductor de música? ¿Qué otros procesos de una computadora considerarías de alta prioridad?

# 181 Preparándonos para el siguiente algoritmo

Hasta ahora cada proceso conserva el CPU hasta finalizar.

En la siguiente sección estudiaremos **Round Robin**, donde todos los procesos comparten el procesador por turnos mediante un **Quantum**.

# 182 Algoritmo Round Robin (RR)

Hasta ahora hemos estudiado algoritmos donde un proceso conserva el procesador hasta terminar su ejecución.

Pero surge un problema.

> ¿Qué ocurre si un proceso tarda varios minutos y los demás deben esperar?

Para resolver esta situación nació **Round Robin (RR)**.

# 183 Un ejemplo cotidiano

Imagina que un profesor debe atender dudas de diez estudiantes.

En lugar de dedicar todo el tiempo a un solo estudiante, decide atenderlos por turnos.

Cada estudiante dispone de **2 minutos**.

Cuando el tiempo termina, pasa el siguiente estudiante.

Si aún tiene dudas, volverá a hacer fila.

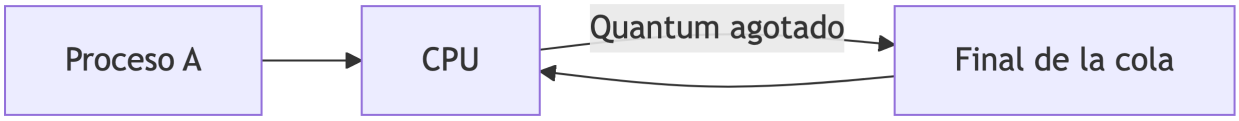
Eso mismo hace Round Robin.

# 184 ¿Qué es el Quantum?

El **Quantum** es el tiempo máximo que un proceso puede utilizar el CPU antes de cederlo a otro proceso.

Si el proceso termina antes de consumir el Quantum, abandona el procesador normalmente.

Si no termina, el Sistema Operativo lo envía nuevamente al final de la cola de listos.




> **Idea clave**
>
> Round Robin busca que **todos los procesos tengan oportunidad de utilizar el procesador**, evitando esperas excesivas.

# 185 Ejemplo

Supongamos un **Quantum = 2 ms**.

| Proceso | Tiempo de CPU |
|---|---|
| A | 5 |
| B | 3 |
| C | 4 |

### 185.0.1 Primera ronda

- A utiliza 2 ms (le quedan 3).
- B utiliza 2 ms (le queda 1).
- C utiliza 2 ms (le quedan 2).

### 185.0.2 Segunda ronda

- A utiliza 2 ms (le queda 1).
- B utiliza 1 ms y termina.
- C utiliza 2 ms y termina.

### 185.0.3 Tercera ronda

- A utiliza 1 ms y termina.

# 186 Diagrama de Gantt

```
Quantum = 2 ms

0   2   4   6   8   9  11 12
| A | B | C | A | B | C | A |
```

# 187 Ventajas

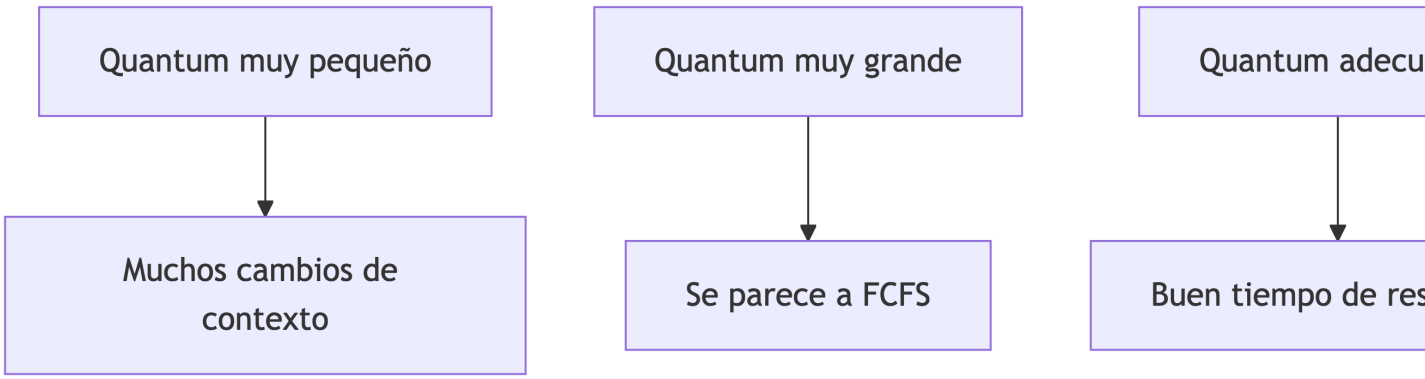
- Todos los procesos reciben tiempo de CPU.
- Muy adecuado para sistemas interactivos.
- Evita que un proceso monopolice el procesador.
- Reduce la sensación de espera del usuario.

# 188 Desventajas

- Requiere numerosos cambios de contexto.
- Si el Quantum es demasiado pequeño, aumenta la sobrecarga.
- Si el Quantum es demasiado grande, se parece a FCFS.

## 188.1 ¿Cómo elegir un buen Quantum?

- **Muy pequeño:** muchos cambios de contexto y menor rendimiento.
- **Muy grande:** los procesos esperan demasiado.
- **Adecuado:** equilibrio entre rendimiento y respuesta.



# 189 Comparación general

| Algoritmo | Criterio | Principal ventaja | Principal desventaja |
|---|---|---|---|
| FCFS | Orden de llegada | Muy simple | Esperas largas |
| SJF | Menor tiempo | Reduce espera promedio | Inanición |
| Prioridades | Mayor prioridad | Atiende procesos críticos | Inanición |
| Round Robin | Quantum | Equidad entre procesos | Más cambios de contexto |

**Reflexiona**

¿Por qué Windows, Linux y Android utilizan algoritmos basados en Round Robin para atender aplicaciones interactivas?
¿Qué ocurriría si un navegador web tuviera que esperar varios segundos para volver a recibir tiempo de CPU?

# 190 Preparándonos para el cierre

Ya conocemos los cuatro algoritmos clásicos de planificación.

En la siguiente sección los compararemos mediante un caso integrador y resumiremos cuándo conviene utilizar cada uno.

# 191 Comparación general de los algoritmos de planificación

Llegamos al final del capítulo.

Ahora ya conocemos los cuatro algoritmos clásicos de planificación del CPU.

Antes de continuar con el siguiente tema, comparemos sus principales características.

| Algoritmo | ¿Cómo decide? | Ventaja principal | Desventaja principal | Uso recomendado |
|---|---|---|---|---|
| **FCFS** | Primer proceso en llegar | Muy sencillo de implementar | Puede generar largas esperas | Sistemas simples o por lotes |
| **SJF** | Menor tiempo de CPU | Reduce el tiempo promedio de espera | Riesgo de inanición | Procesos con duración conocida |
| **Prioridades** | Mayor prioridad | Atiende primero procesos críticos | Riesgo de inanición | Sistemas de tiempo real |
| **Round Robin** | Turnos mediante Quantum | Todos reciben tiempo de CPU | Mayor cantidad de cambios de contexto | Sistemas interactivos |

# 192 Caso integrador

Supongamos que llegan simultáneamente los siguientes procesos:

| Proceso | CPU | Prioridad |
|---|---|---|
| A | 8 | 3 |
| B | 2 | 1 |
| C | 4 | 4 |
| D | 3 | 2 |

## 192.1 Analiza

1. ¿Qué proceso ejecutaría FCFS primero?
2. ¿Qué proceso elegiría SJF?
3. ¿Qué proceso seleccionaría Prioridades?
4. Si el Quantum = 2 ms, ¿cómo comenzaría Round Robin?

Discute y justifica tus respuestas.

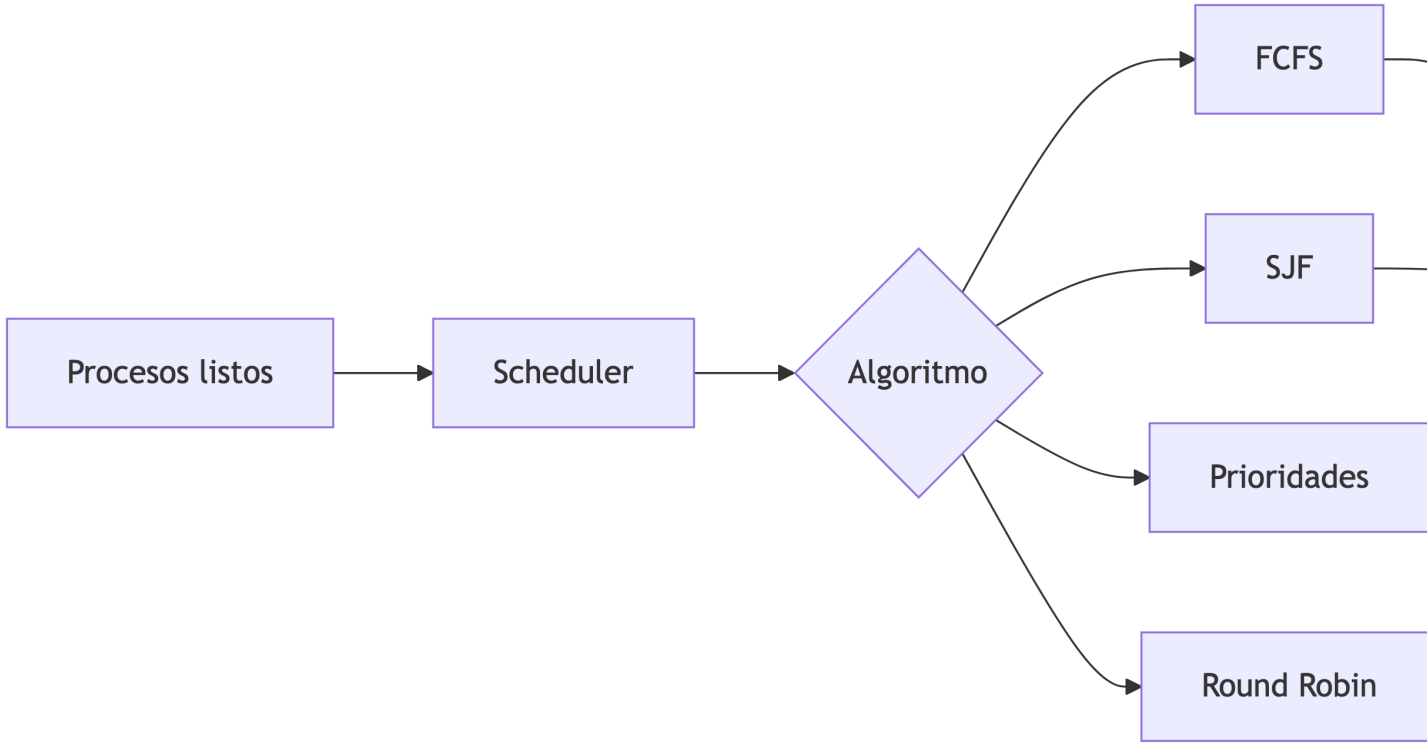
# 193 Autoevaluación

1. ¿Cuál es el objetivo de la planificación del CPU?
2. ¿Qué componente selecciona el siguiente proceso?
3. ¿Qué función cumple el Dispatcher?
4. ¿Qué es el Quantum?
5. ¿Cuál algoritmo utiliza el orden de llegada?
6. ¿Cuál selecciona el proceso más corto?
7. ¿Cuál atiende primero procesos críticos?
8. ¿Qué significa inanición (Starvation)?
9. ¿Qué técnica ayuda a evitarla?
10. ¿Por qué Round Robin es adecuado para sistemas interactivos?

---

# 194 Resumen del capítulo

Durante esta clase aprendimos que el Sistema Operativo utiliza diferentes algoritmos para decidir qué proceso obtiene el procesador.




Idea clave

No existe un algoritmo perfecto.
Cada Sistema Operativo selecciona el algoritmo que mejor se adapta a sus necesidades.

# 195 Conexión con el siguiente capítulo

En el próximo capítulo estudiaremos las **capas del Sistema Operativo** y comprenderemos por qué el **Kernel** es considerado el corazón del sistema.

# 196 Capítulo 7. Del hardware al usuario: comprendiendo las capas del Sistema Operativo

# 197 Capítulo 7. Capas del Sistema Operativo

## 197.1 Relación con la clase anterior

En el capítulo anterior aprendimos que el **Scheduler** decide qué proceso utilizará el procesador y que el **Kernel** coordina esa decisión.

Ahora responderemos una nueva pregunta:

> **¿Qué ocurre dentro de una computadora desde que hacemos doble clic sobre una aplicación hasta que aparece en pantalla?**

Esta pregunta nos permitirá comprender cómo trabajan juntas las diferentes capas de un Sistema Operativo.

---

# 198 Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

- Comprender la organización por capas de un Sistema Operativo.
- Explicar la función de cada capa.
- Identificar el papel del Kernel dentro de esa arquitectura.
- Relacionar las capas con acciones que realizamos diariamente en una computadora.

---

# 199 Motivación

Todos los días abrimos aplicaciones como:

- Google Chrome
- Microsoft Word
- Visual Studio Code
- Spotify

La aplicación aparece en pocos segundos.

Pero…

- ¿Quién encuentra el programa en el disco?
- ¿Quién reserva memoria para ejecutarlo?
- ¿Quién se comunica con el teclado y el mouse?
- ¿Quién envía la imagen al monitor?
- ¿Quién controla realmente todo el proceso?

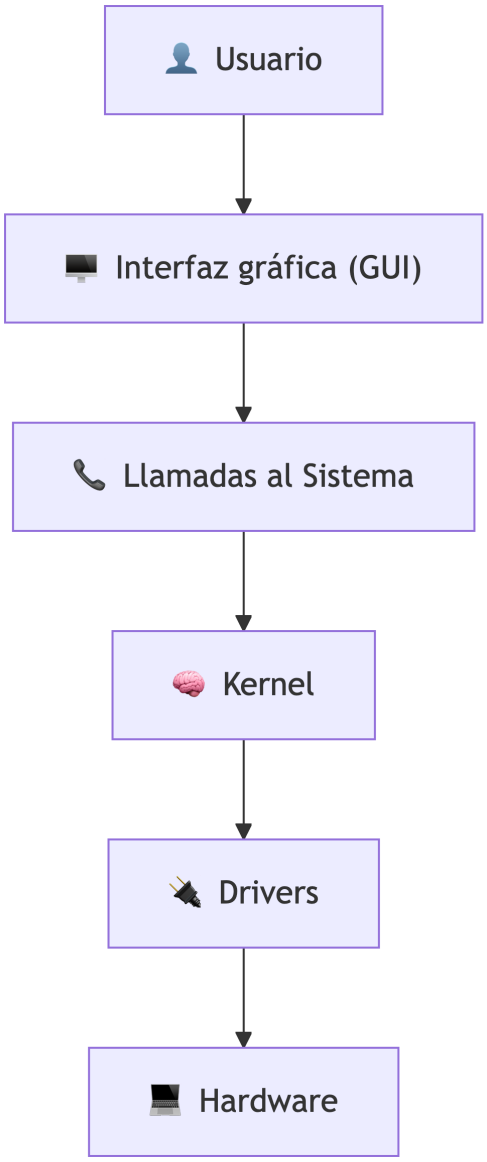
La respuesta está en las **capas del Sistema Operativo**.

> **Pregunta detonadora**
>
> Cuando haces doble clic sobre una aplicación, ¿qué sucede dentro de la computadora antes de que aparezca la ventana?
> Al finalizar este capítulo podrás responder esa pregunta paso a paso.

---

# 200 Un viaje por el Sistema Operativo

Durante esta clase seguiremos el recorrido que realiza una solicitud desde el usuario hasta el hardware.




Observa que el usuario nunca se comunica directamente con el hardware.

Siempre existe una cadena de componentes que trabajan coordinadamente.

---

# 201 ¿Por qué organizar el Sistema Operativo por capas?

Dividir el Sistema Operativo en capas ofrece varias ventajas.

- Cada capa tiene una responsabilidad específica.
- Facilita el mantenimiento del sistema.
- Reduce errores.
- Permite actualizar componentes sin modificar todo el Sistema Operativo.
- Hace posible reutilizar código.

Piensa en un edificio.

Cada piso tiene una función distinta, pero todos trabajan juntos para que el edificio funcione correctamente.

Con un Sistema Operativo ocurre exactamente lo mismo.

---

# 202 ¿Qué aprenderemos en este capítulo?

En las siguientes secciones estudiaremos:

1. La arquitectura por capas.
2. La función de cada capa.
3. El Kernel como corazón del Sistema Operativo.
4. Los tipos de Kernel.
5. Un caso práctico siguiendo el recorrido de una aplicación desde el usuario hasta el hardware.

---

# 203 Preparándonos para la siguiente sección

Ya conocemos el recorrido general.

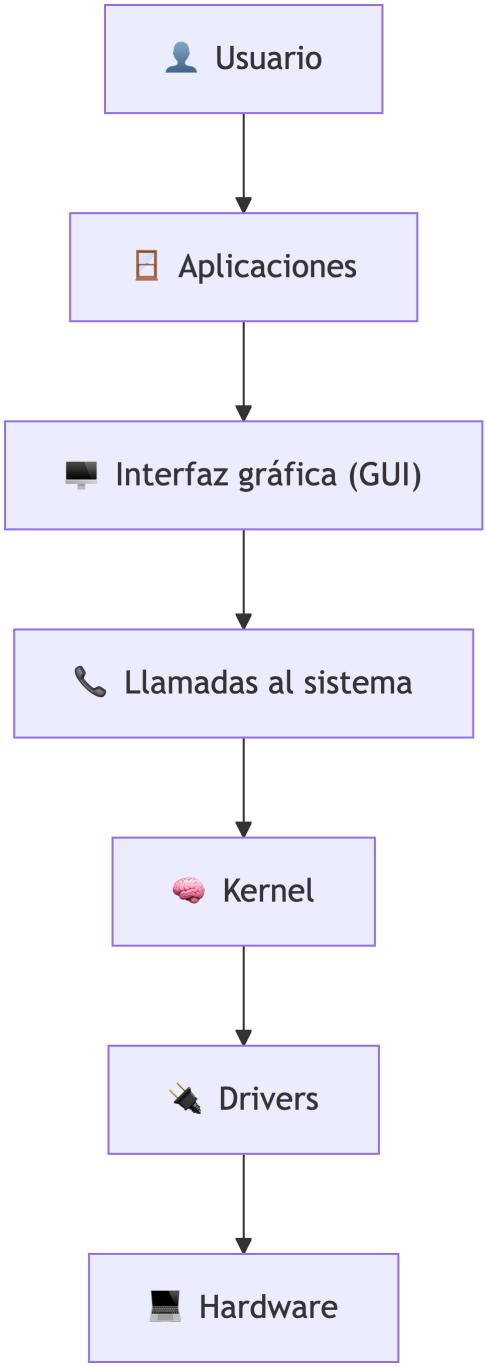
Ahora estudiaremos cada una de las capas comenzando desde la más cercana al usuario hasta llegar al **Kernel**, el componente más importante del Sistema Operativo.

# 204 Conociendo las capas del Sistema Operativo

En la sección anterior vimos que una solicitud recorre varias capas antes de llegar al hardware.

Ahora estudiaremos cada una de ellas para comprender su función.

## 204.1 Vista general




Cada capa tiene una responsabilidad específica y colabora con las demás.

# 205 Capa 1. Usuario

Es la persona que utiliza el computador.

Sus acciones son sencillas:

- Hacer clic.
- Escribir con el teclado.
- Abrir un programa.
- Guardar un archivo.

El usuario nunca controla directamente el hardware.

---

# 206 Capa 2. Aplicaciones

Son los programas que utilizamos diariamente.

Ejemplos:

- Google Chrome
- Microsoft Word
- Visual Studio Code
- VLC

Las aplicaciones solicitan servicios al Sistema Operativo, pero no acceden directamente al disco, la memoria o el procesador.

> **Idea importante**
>
> Las aplicaciones dependen del Sistema Operativo para utilizar los recursos del computador.

---

# 207 Capa 3. Interfaz gráfica (GUI)

La GUI permite que el usuario interactúe mediante:

- Ventanas.
- Botones.
- Menús.
- Iconos.

Gracias a ella no es necesario escribir comandos para realizar la mayoría de las tareas.

---

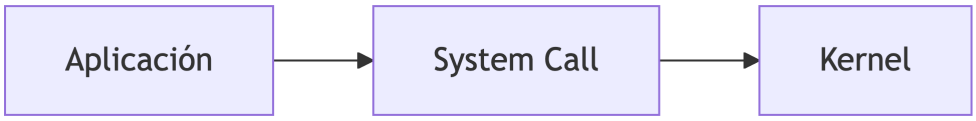
# 208 Capa 4. Llamadas al Sistema (System Calls)

Cuando una aplicación necesita un recurso, realiza una **llamada al sistema**.

Por ejemplo:

- Abrir un archivo.
- Reservar memoria.
- Crear un proceso.
- Imprimir un documento.

Estas solicitudes son enviadas al Kernel.




**Reflexiona**

Cuando haces clic en **Guardar**, la aplicación no escribe directamente en el disco.
Primero solicita ese servicio al Sistema Operativo mediante una llamada al sistema.

---

# 209 ¿Qué observamos hasta ahora?

Las primeras capas están orientadas al usuario y a las aplicaciones.

Sin embargo, todavía no hemos llegado al componente que realmente controla el Sistema Operativo.

En la siguiente sección estudiaremos el **Kernel**, considerado el corazón del Sistema Operativo.

# 210 El Kernel: el corazón del Sistema Operativo

Hasta este momento hemos recorrido varias capas del Sistema Operativo.

Sin embargo, todas ellas dependen de un componente central.

Ese componente recibe el nombre de **Kernel**.

Si el Sistema Operativo fuera un cuerpo humano, el Kernel sería su **corazón y su cerebro**, ya que coordina el funcionamiento de todos los recursos del computador.

---

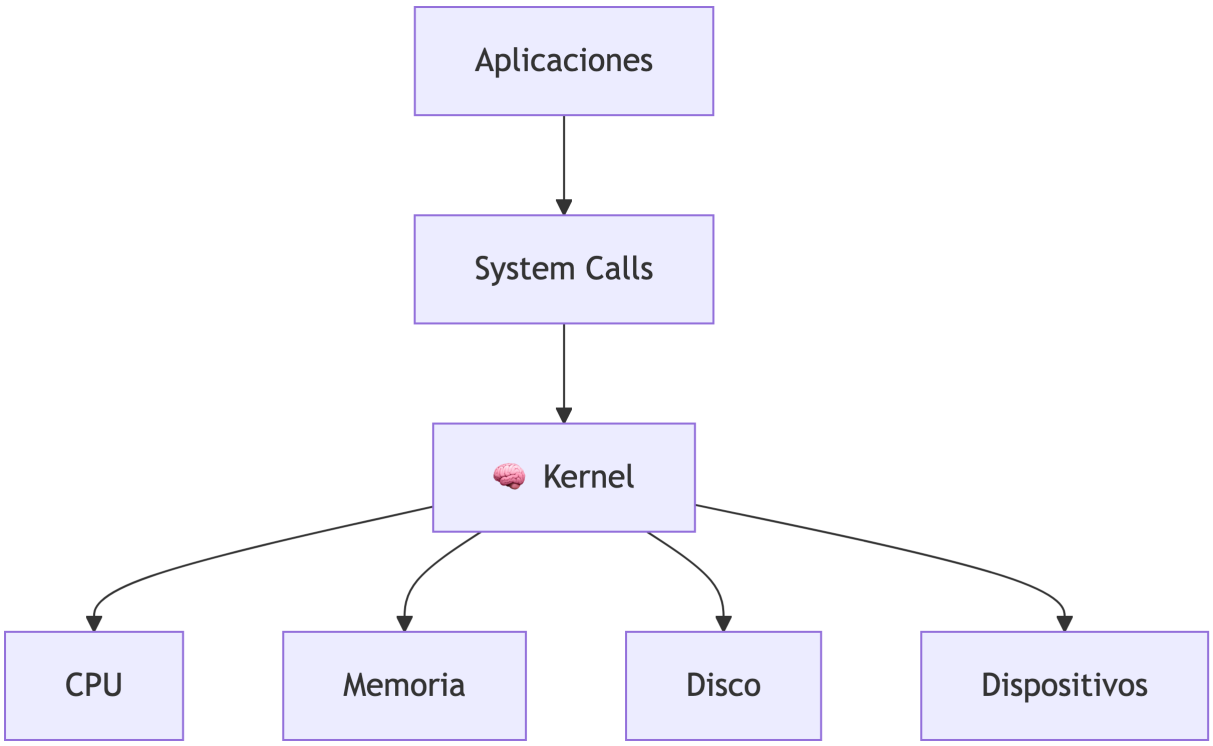
# 211 ¿Qué es el Kernel?

El **Kernel** es la parte más importante del Sistema Operativo.

Es el primer componente que se carga en memoria durante el arranque del sistema y permanece ejecutándose mientras la computadora está encendida.

Su función principal es actuar como intermediario entre:

- Las aplicaciones.
- El hardware.
- Los dispositivos de entrada y salida.
- La memoria.
- El procesador.



**Idea clave**

Las aplicaciones **nunca acceden directamente al hardware**.
Todas las solicitudes pasan primero por el Kernel.

# 212 ¿Qué responsabilidades tiene el Kernel?

El Kernel realiza numerosas tareas al mismo tiempo.

Entre las más importantes se encuentran:

- Administrar los procesos.
- Planificar el uso del procesador.
- Administrar la memoria.
- Controlar los dispositivos mediante los drivers.
- Gestionar el sistema de archivos.
- Atender interrupciones.
- Proteger los recursos del sistema.

Observa que muchos de estos temas serán estudiados durante el resto del curso.

---

# 213 ¿Qué ocurriría si el Kernel no existiera?

Imagina que una aplicación intenta:

- Leer un archivo.
- Utilizar la impresora.
- Acceder a la memoria.
- Escribir información en el disco.

Sin el Kernel:

- Dos programas podrían utilizar la misma memoria.
- Varios procesos intentarían controlar el CPU al mismo tiempo.
- Los dispositivos no podrían coordinarse.
- El sistema sería inestable y probablemente dejaría de funcionar.

> ⚠ Reflexiona
>
> El Kernel no solamente administra recursos.
> También evita que los programas interfieran entre sí.

---

# 214 Relación con los temas estudiados

Observa cómo el Kernel aparece en casi todo lo que hemos aprendido.

| Tema | Participación del Kernel |
|---|---|
| Procesos | Los crea y administra |
| Scheduler | Coordina la planificación |
| Dispatcher | Coordina el cambio de contexto |
| Memoria | Reserva y libera espacios |
| Archivos | Controla el acceso al disco |
| Dispositivos | Se comunica mediante drivers |

Esto demuestra por qué se le considera el **corazón del Sistema Operativo**.

---

# 215 Preparándonos para la siguiente sección

Ya conocemos la importancia del Kernel.

Ahora estudiaremos que **no todos los Sistemas Operativos utilizan el mismo tipo de Kernel**.

Analizaremos los modelos:

- Monolítico.
- Microkernel.
- Híbrido.

Y conoceremos cuál utiliza Windows, Linux y macOS.

# 216 Tipos de Kernel

En la sección anterior aprendimos que el **Kernel** es el corazón del Sistema Operativo.

Sin embargo, no todos los Sistemas Operativos implementan el Kernel de la misma forma.

Dependiendo de su diseño, existen diferentes arquitecturas.

---

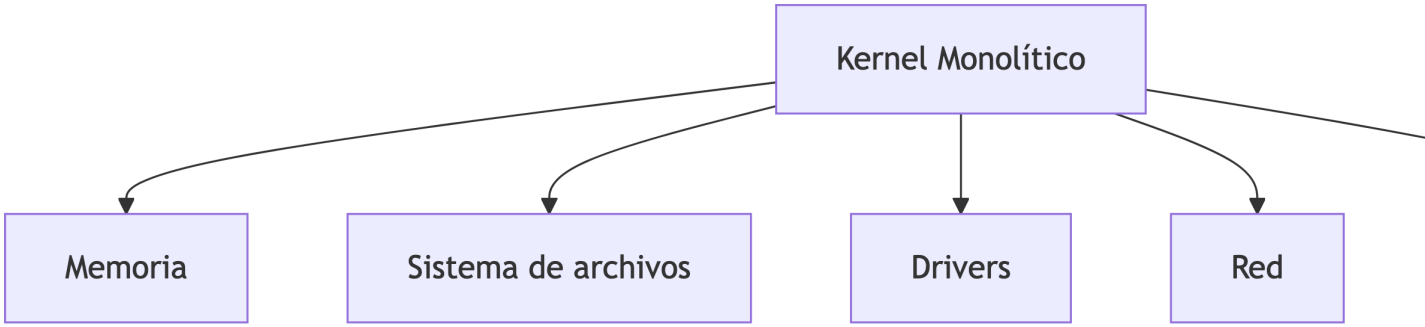
# 217 Kernel Monolítico

En un **Kernel Monolítico**, la mayoría de los servicios del Sistema Operativo se ejecutan dentro del propio Kernel.

Esto significa que:

- Administración de memoria.
- Sistema de archivos.
- Controladores (drivers).
- Redes.
- Planificación de procesos.

trabajan en un mismo espacio del Kernel.




## 217.1 Ventajas

- Muy rápido.
- Excelente rendimiento.
- Menor sobrecarga.

## 217.2 Desventajas

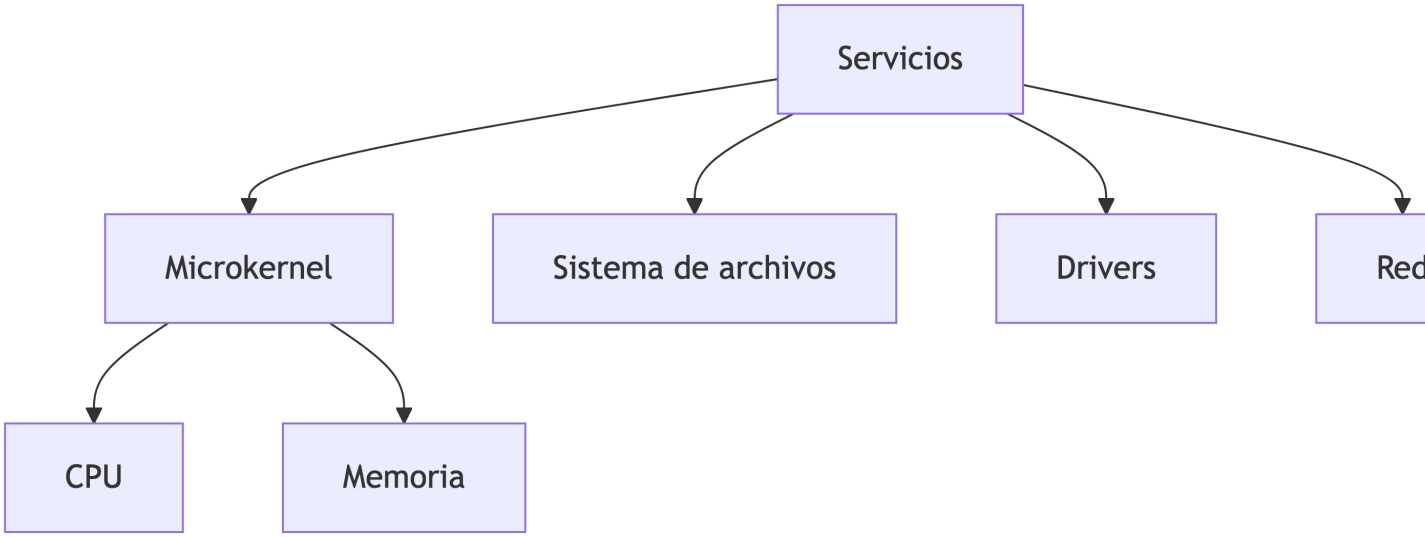
- Un error en un componente puede afectar todo el sistema.
- Mayor complejidad para mantener.

Ejemplo: **Linux**.

# 218 Microkernel

El **Microkernel** mantiene dentro del Kernel únicamente las funciones esenciales.

Otros servicios se ejecutan fuera del Kernel como procesos independientes.




## 218.1 Ventajas

- Mayor estabilidad.
- Más seguro.
- Fácil de mantener.

## 218.2 Desventajas

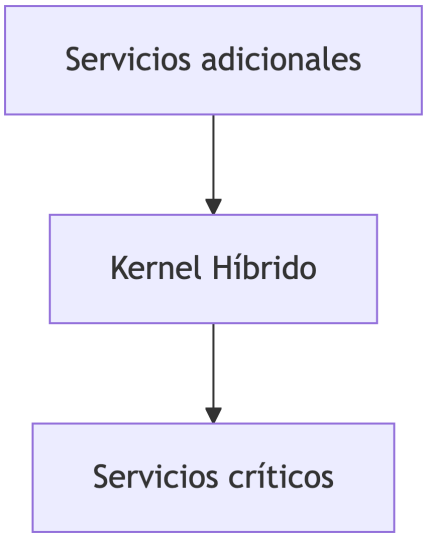
- Mayor comunicación entre procesos.
- Menor rendimiento que un monolítico.

Ejemplo clásico: **MINIX**.

# 219 Kernel Híbrido

El **Kernel Híbrido** combina características del Kernel Monolítico y del Microkernel.

Mantiene servicios importantes dentro del Kernel para mejorar el rendimiento y deja otros componentes fuera para aumentar la modularidad.




## 219.1 Ventajas

- Buen equilibrio entre rendimiento y estabilidad.
- Muy utilizado en sistemas modernos.

## 219.2 Desventajas

- Arquitectura más compleja.

Ejemplos:

- Windows
- macOS

# 220 Comparación

| Característica | Monolítico | Microkernel | Híbrido |
|---|---|---|---|
| Rendimiento | Alto | Medio | Alto |
| Estabilidad | Media | Alta | Alta |
| Mantenimiento | Difícil | Fácil | Medio |
| Ejemplo | Linux | MINIX | Windows / macOS |

> **Idea clave**
>
> No existe un tipo de Kernel perfecto.
> Cada arquitectura representa un equilibrio distinto entre **rendimiento**, **seguridad**, **mantenimiento** y **modularidad**.

# 221 Preparándonos para el cierre

En la siguiente sección seguiremos el recorrido completo de una aplicación desde que el usuario hace doble clic hasta que aparece en pantalla y resumiremos todo lo aprendido en este capítulo.

# 222 Caso integrador: ¿Qué ocurre cuando abrimos una aplicación?

Llegó el momento de integrar todo lo aprendido.

Imagina que haces doble clic sobre **Google Chrome**.

A simple vista únicamente aparece una ventana en la pantalla.

Sin embargo, internamente ocurre una secuencia de eventos perfectamente organizada.

## 222.1 Paso 1. El usuario solicita abrir la aplicación

El usuario hace doble clic sobre el icono.

La solicitud llega a la **Interfaz Gráfica (GUI)**.

## 222.2 Paso 2. La aplicación solicita servicios

La aplicación necesita:

- Leer su archivo ejecutable.
- Reservar memoria.
- Crear un proceso.
- Obtener tiempo de CPU.

Para ello realiza **llamadas al sistema (System Calls)**.

## 222.3 Paso 3. El Kernel entra en acción

El Kernel recibe la solicitud y comienza a coordinar todos los recursos necesarios.

Entre otras tareas:

- Crea el proceso.
- Reserva memoria.
- Consulta el sistema de archivos.

- Coordina el Scheduler.
- Se comunica con los Drivers.

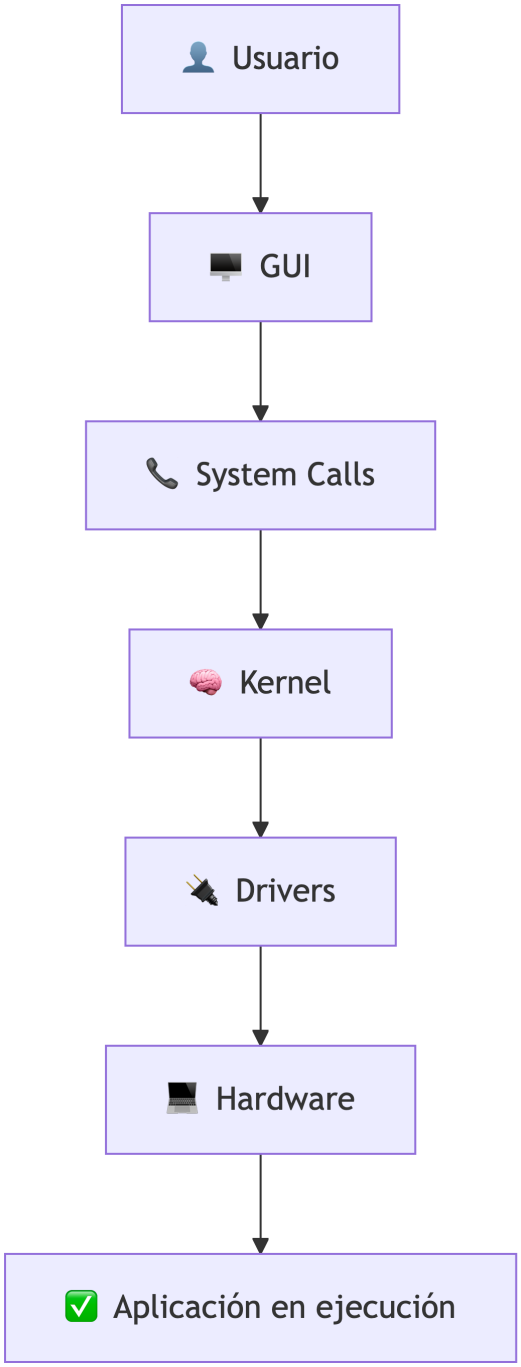
## 222.4 Paso 4. Los Drivers controlan el hardware

Los Drivers permiten que el Sistema Operativo se comunique con:

- Disco duro o SSD.
- Tarjeta gráfica.
- Teclado.
- Mouse.
- Monitor.

## 222.5 Paso 5. La aplicación aparece en pantalla

Cuando todos los recursos están preparados, la ventana finalmente aparece y el usuario puede comenzar a trabajar.




**Idea clave**

El usuario observa un único clic.
El Sistema Operativo coordina cientos de operaciones en apenas unos milisegundos.

# 223 Autoevaluación

1. ¿Cuál es la función principal del Kernel?
2. ¿Qué capa recibe directamente las solicitudes de las aplicaciones?
3. ¿Qué son las System Calls?
4. ¿Por qué las aplicaciones no acceden directamente al hardware?
5. ¿Cuál es la función de los Drivers?
6. ¿Qué diferencia existe entre un Kernel Monolítico y un Microkernel?
7. ¿Qué ventajas ofrece un Kernel Híbrido?
8. ¿Qué ocurriría si el Kernel dejara de funcionar?
9. ¿Por qué decimos que el Kernel es el corazón del Sistema Operativo?
10. Describe el recorrido que sigue una solicitud desde el usuario hasta el hardware.

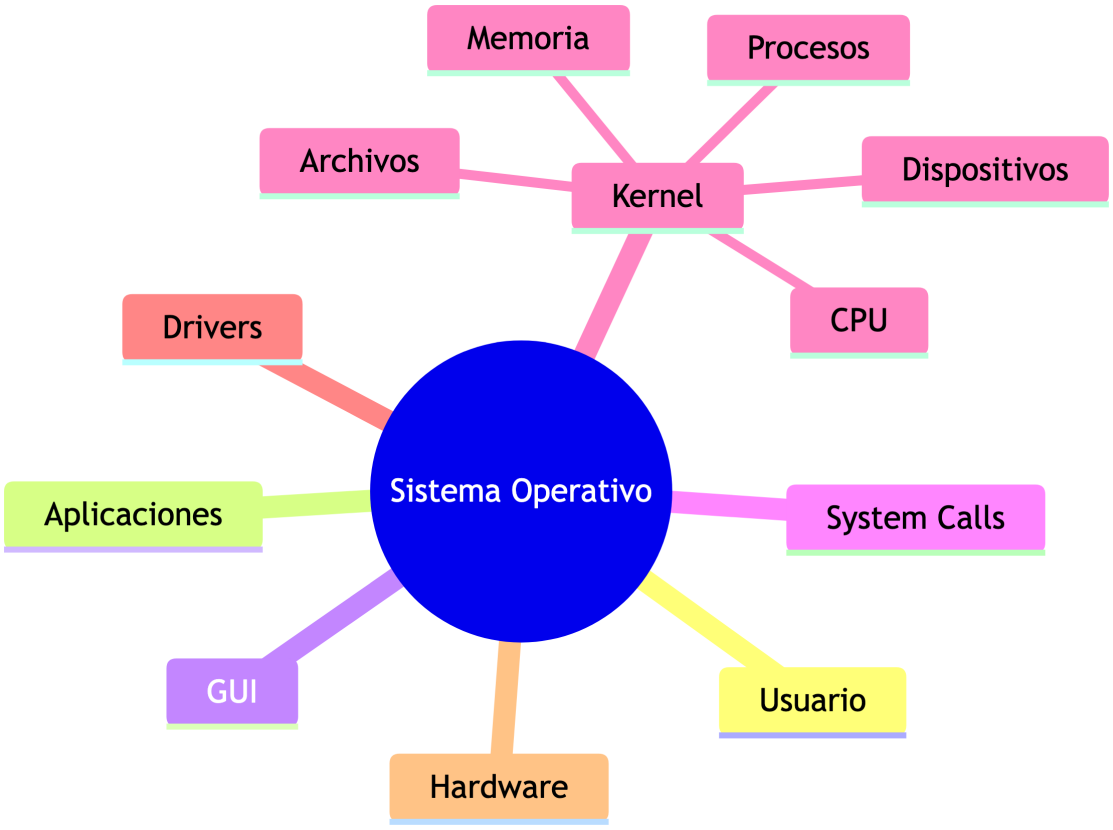
---

# 224 Resumen del capítulo

Durante este capítulo comprendimos que un Sistema Operativo está organizado por capas para facilitar la administración de los recursos del computador.

Aprendimos que:

- El usuario interactúa con las aplicaciones.
- Las aplicaciones solicitan servicios mediante llamadas al sistema.
- El Kernel coordina todos los recursos.
- Los Drivers permiten controlar el hardware.
- Existen diferentes arquitecturas de Kernel según las necesidades del sistema.



**Para recordar**

El **Kernel** es el componente más importante del Sistema Operativo.
Todo acceso al procesador, memoria, dispositivos y archivos pasa por él.

# 225 Conexión con el siguiente capítulo

Ya comprendemos cómo el Sistema Operativo organiza sus componentes y administra el procesador.

En el siguiente capítulo estudiaremos cómo administra otro recurso fundamental:

**La memoria principal (RAM).**

Descubriremos cómo el Sistema Operativo asigna, protege y libera memoria para que múltiples procesos puedan ejecutarse de forma segura y eficiente.

# 226 Capítulo 8. Cuando nadie puede avanzar: introducción a los interbloqueos

# 227 Capítulo 8. Interbloqueos (Deadlocks)

## 227.1 Relación con la clase anterior

En el capítulo anterior estudiamos que el **Kernel** administra los recursos del computador y coordina la ejecución de los procesos.

También vimos que múltiples procesos pueden necesitar al mismo tiempo recursos como:

- CPU.
- Memoria.
- Archivos.
- Dispositivos de entrada y salida.
- Impresoras.
- Recursos compartidos.

Ahora aparece una nueva pregunta:

> **¿Qué ocurre cuando dos o más procesos tienen recursos que otros necesitan y ninguno puede continuar?**

Ese problema recibe el nombre de **interbloqueo** o **deadlock**.

---

# 228 Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

- Comprender qué es un **interbloqueo (deadlock)**.
- Identificar los procesos y recursos involucrados en un interbloqueo.
- Reconocer las **cuatro condiciones necesarias** para que ocurra.
- Diferenciar recursos exclusivos y recursos compartidos.
- Analizar situaciones reales y computacionales donde se produce un interbloqueo.
- Proponer formas sencillas de evitar o romper una situación de bloqueo mutuo.

---

# 229 Motivación

Imagina una intersección sin semáforo.

Cuatro vehículos llegan al mismo tiempo y cada uno avanza lo suficiente para impedir el paso del siguiente.

Nadie puede retroceder.

Nadie puede avanzar.

Todos están esperando que otro se mueva primero.

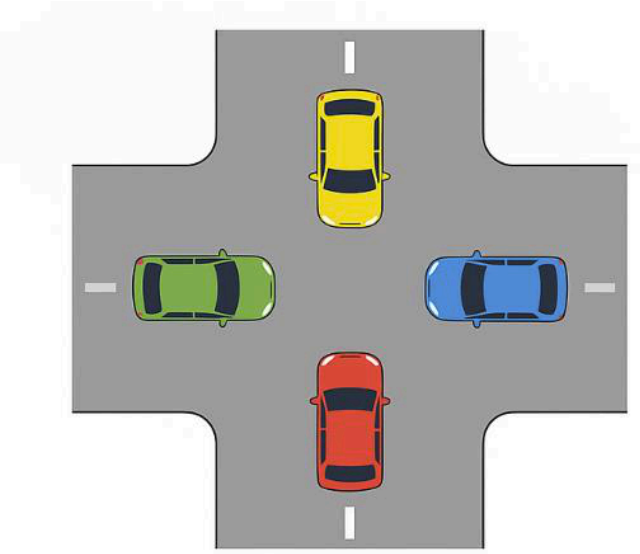


Figura 229.1: Cruce vehicular usado como analogía de un interbloqueo.

> **Pregunta detonadora**
>
> Si todos los procesos están esperando, ¿quién será el primero en liberar el recurso que los demás necesitan?

Esta idea aparentemente sencilla nos permitirá comprender uno de los problemas clásicos de los sistemas concurrentes.

# 230 ¿Qué es un interbloqueo?

Un **interbloqueo (deadlock)** ocurre cuando dos o más procesos permanecen esperando indefinidamente por recursos que no pueden obtener porque están retenidos por otros procesos que también se encuentran esperando.

En otras palabras:

> **Cada proceso espera algo que otro proceso posee, y ninguno puede continuar.**

El resultado es un **bloqueo mutuo**.

Los procesos involucrados no avanzan, aunque el sistema operativo continúe funcionando.

---

# 231 Un ejemplo mínimo con dos procesos

Supongamos que existen:

- **Proceso P1**
- **Proceso P2**
- **Recurso R1**
- **Recurso R2**

La secuencia es la siguiente:

1. P1 obtiene R1.
2. P2 obtiene R2.
3. P1 solicita R2 y debe esperar.
4. P2 solicita R1 y también debe esperar.

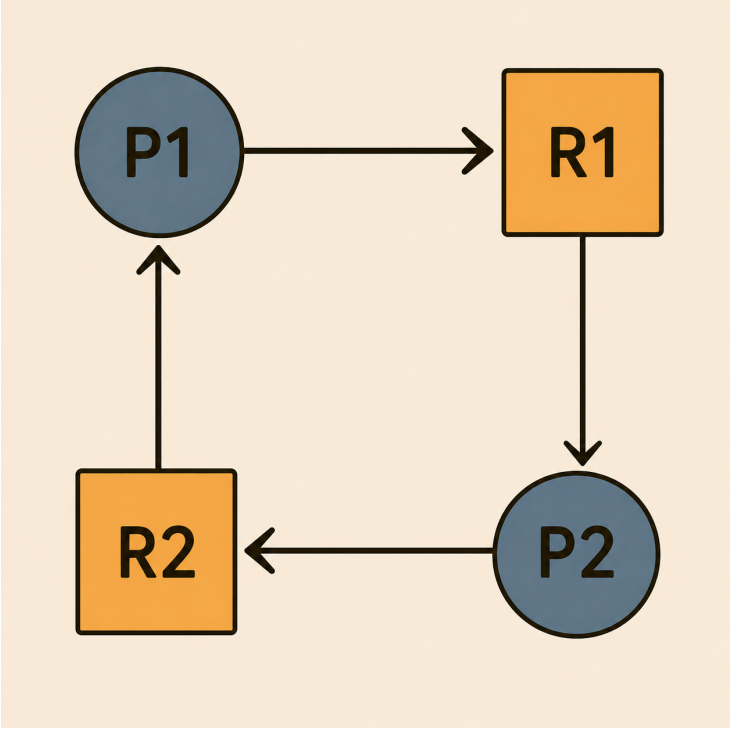



Figura 231.1: Dos procesos reteniendo un recurso y esperando el recurso del otro.

Ninguno puede continuar.

```
P1 tiene R1 y espera R2
P2 tiene R2 y espera R1
```

⚠ El problema

P1 no libera R1 porque todavía necesita terminar.
P2 no libera R2 por la misma razón.
Ambos esperan indefinidamente.

---

# 232 Las cuatro condiciones necesarias

Para que pueda existir un interbloqueo deben presentarse **simultáneamente cuatro condiciones**.

## 232.1 1. Exclusión mutua

Al menos un recurso debe estar disponible para **un solo proceso a la vez**.

Ejemplos:

- Una impresora.
- Un archivo abierto exclusivamente para escritura.
- Un escáner.
- Un bloqueo de base de datos.

Si un proceso está utilizando ese recurso, otro proceso debe esperar.

---

# 233 2. Retención y espera

Un proceso conserva uno o más recursos mientras espera obtener otros.

Ejemplo:

```
P1 tiene R1
P1 espera R2
```

P1 no libera R1 mientras intenta obtener R2.

Esta combinación de **tener** y **esperar** es fundamental para que aparezca el problema.

---

# 234 3. No expropiación

El Sistema Operativo no puede retirar arbitrariamente un recurso que ya fue asignado a un proceso.

El proceso debe:

- utilizarlo,
- terminar su operación,
- y liberarlo voluntariamente.

> **Ejemplo**
>
> Si un proceso está escribiendo en un archivo, retirarle el recurso en cualquier momento podría dejar la información inconsistente.

---

# 235 4. Espera circular

Existe un ciclo en el cual cada proceso espera un recurso retenido por el siguiente proceso.

Por ejemplo:

```
P1 espera un recurso de P2
P2 espera un recurso de P3
P3 espera un recurso de P1
```

En el caso más simple:

```
P1 espera R2
P2 espera R1
```

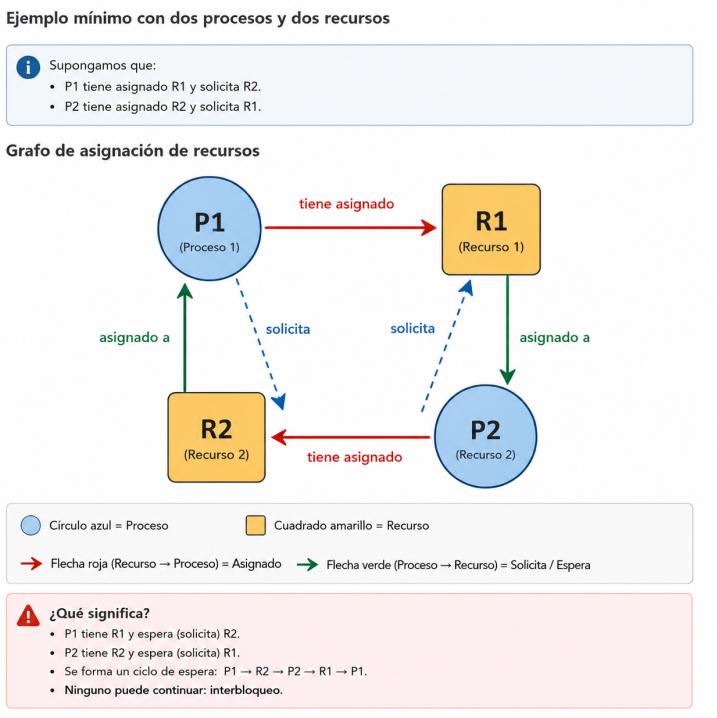



Figura 235.1: Representación del ciclo de espera entre procesos y recursos.

La espera circular cierra el ciclo y evita que alguno de los procesos pueda avanzar.

# 236 Idea clave: deben cumplirse las cuatro

Un interbloqueo requiere que las cuatro condiciones existan al mismo tiempo:

| Condición | Pregunta para identificarla |
|---|---|
| Exclusión mutua | ¿Hay un recurso que solo puede usar un proceso a la vez? |
| Retención y espera | ¿Algún proceso conserva un recurso mientras espera otro? |
| No expropiación | ¿El recurso no puede retirarse por la fuerza? |
| Espera circular | ¿Existe un ciclo de procesos esperando recursos entre sí? |

> Regla para recordar
>
> Si logramos eliminar **al menos una** de estas condiciones, el interbloqueo no puede formarse de esa manera.

# 237 Tipos de recursos

No todos los recursos se comportan igual.

## 237.1 Recursos exclusivos

Solo un proceso puede utilizarlos en un momento determinado.

Ejemplos:

- Impresora.
- Escáner.
- Archivo con bloqueo exclusivo.
- Sección crítica protegida por un mutex.

Estos recursos son especialmente importantes cuando estudiamos interbloqueos.

## 237.2 Recursos compartidos

Pueden ser utilizados por varios procesos, dependiendo de las reglas de acceso.

Ejemplos:

- Memoria compartida.
- Datos de solo lectura.
- Algunos recursos del sistema con acceso concurrente controlado.

> **Importante**
>
> Los interbloqueos aparecen con mayor frecuencia cuando existen **recursos de uso exclusivo** y varios procesos compiten por ellos.

# 238 ¿Qué componentes del Sistema Operativo están implicados?

Los interbloqueos aparecen dentro del mismo entorno que hemos venido estudiando.

Participan principalmente:

- **Procesos o hilos**, que solicitan recursos.
- **Gestor de recursos**, que administra memoria, archivos y dispositivos.
- **Kernel**, que coordina la asignación y liberación.
- **Mecanismos de sincronización**, como semáforos y mutex.
- **Dispositivos y recursos de E/S**, que pueden estar ocupados por un proceso.

Esto conecta directamente los interbloqueos con los temas de:

- procesos,
- concurrencia,
- sincronización,
- administración de recursos.

---

# 239 Ejemplo real: cruce vehicular

Regresemos al ejemplo de la intersección.

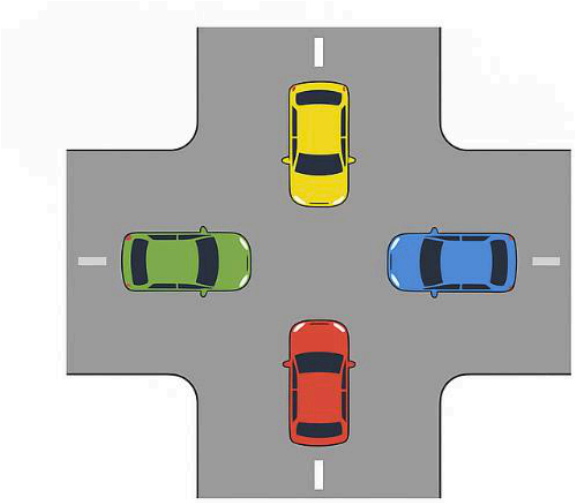


Figura 239.1: Cruce de cuatro vehículos usado para representar un ciclo de espera.

Podemos hacer la siguiente analogía:

| Situación real | Sistema Operativo |
|---|---|
| Cada vehículo | Un proceso |
| Espacio de la intersección | Un recurso |
| Vehículo esperando avanzar | Proceso esperando un recurso |
| Ningún vehículo se mueve | Ningún proceso avanza |

Cada vehículo ocupa una parte de la intersección y necesita el siguiente espacio para continuar.

Se forma un ciclo:

```
Auto A espera a B
Auto B espera a C
Auto C espera a D
Auto D espera a A
```

Eso es exactamente una **espera circular**.

---

# 240 Ejemplo clásico: filósofos comensales

Cinco filósofos se sientan alrededor de una mesa.

Entre cada par de filósofos existe un tenedor.

Para comer, cada filósofo necesita **dos tenedores**:

- uno a la izquierda,
- uno a la derecha.

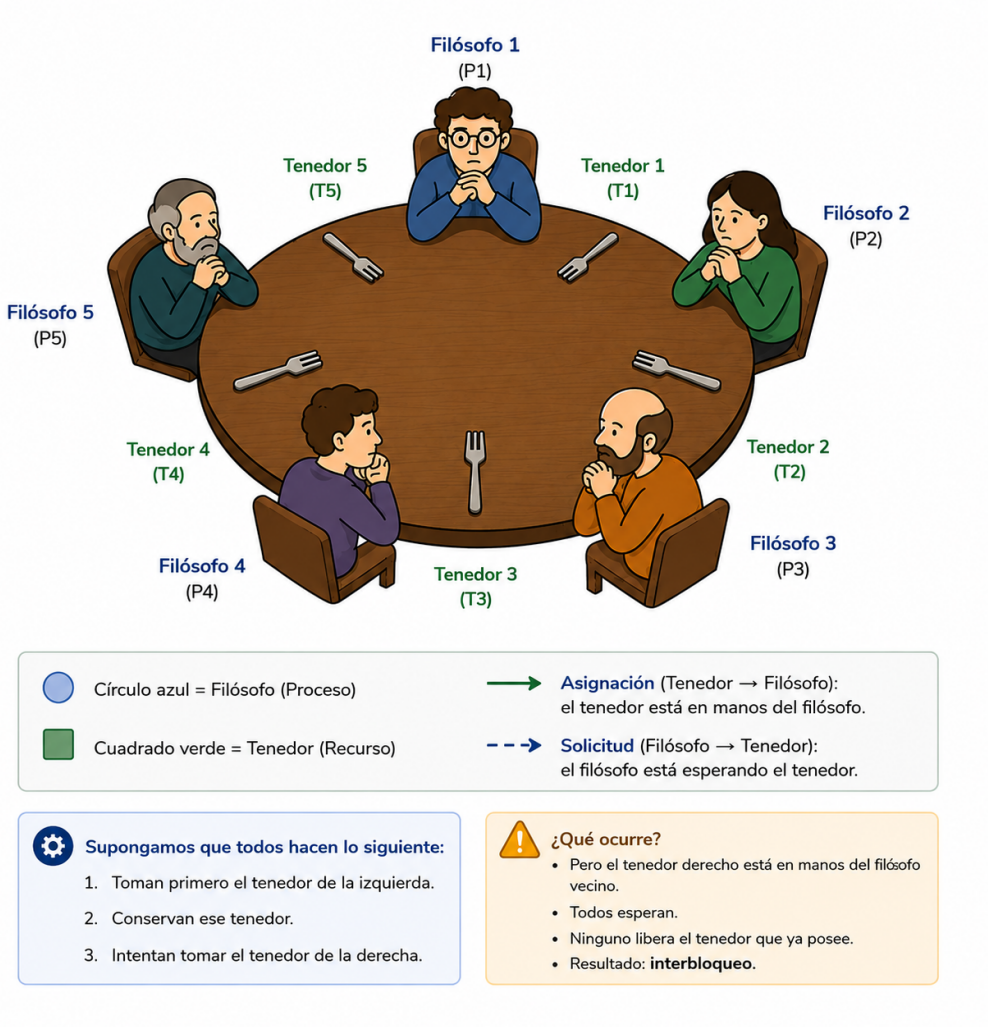



Figura 240.1: Problema clásico de los filósofos comensales.

# 241 Analizando a los filósofos con las cuatro condiciones

| Condición | ¿Cómo aparece? |
|---|---|
| Exclusión mutua | Cada tenedor solo puede estar en manos de un filósofo |
| Retención y espera | Cada filósofo conserva un tenedor y espera otro |
| No expropiación | No se le quita el tenedor por la fuerza |
| Espera circular | Cada filósofo espera el tenedor del vecino |

El ejemplo es útil porque permite visualizar las cuatro condiciones en una sola situación.

# 242 Ejemplo adicional: impresoras en red

Dos usuarios necesitan utilizar:

- una impresora,
- y un conjunto de archivos compartidos.

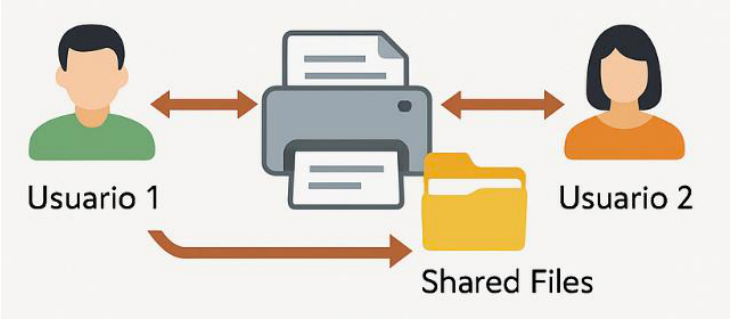



Figura 242.1: Dos usuarios compitiendo por una impresora y archivos compartidos.

Supongamos que ocurre lo siguiente:

- Usuario 1 obtiene la impresora y espera los archivos.
- Usuario 2 obtiene los archivos y espera la impresora.

Entonces:

```
Usuario 1 → espera archivos
Usuario 2 → espera impresora
```

Se forma un ciclo de espera mutua.

## 242.1 ¿Cómo podría evitarse?

Dos ideas sencillas son:

- establecer un orden obligatorio para solicitar recursos,

- liberar un recurso antes de solicitar otro.

---

# 243 Ejemplo adicional: software de oficina

Dos aplicaciones intentan trabajar con recursos que necesitan simultáneamente.

Por ejemplo:

- Aplicación A tiene bloqueado un archivo y espera memoria.
- Aplicación B tiene reservada memoria y espera el archivo.

La situación puede representarse así:

```
Aplicación A: tiene Archivo → espera Memoria
Aplicación B: tiene Memoria → espera Archivo
```

De nuevo aparece la espera circular.

> Pregunta para analizar
>
> ¿Cuáles de las cuatro condiciones puedes identificar en este escenario?

---

# 244 Ejemplo adicional: cajero automático

Un cajero automático necesita comunicarse con varios recursos mientras procesa una transacción.

Entre ellos:

- conexión con el cliente,
- base de datos bancaria,
- sistema de autorización,
- dispositivo dispensador.

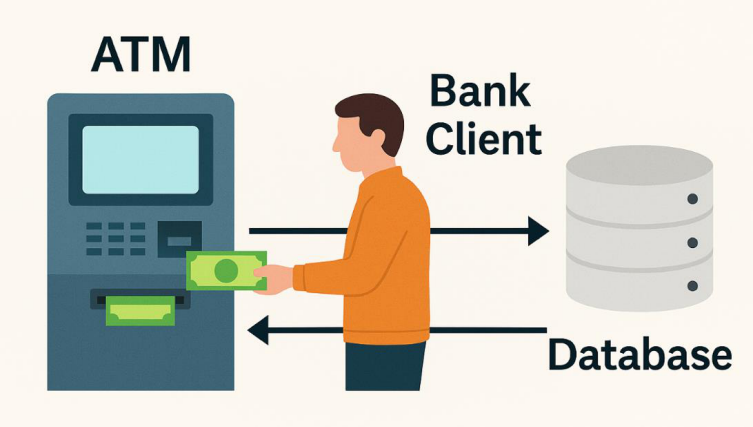



Figura 244.1: Ejemplo conceptual de un ATM comunicándose con una base de datos bancaria.

Si una operación retiene un recurso y espera otro que está bloqueado por una segunda operación, puede formarse una espera mutua.

Una estrategia mencionada en el material es:

- liberar recursos cuando una transacción falla,
- o utilizar temporizadores para evitar esperas indefinidas.

---

# 245 Actividad en clase: identifiquemos el interbloqueo

Formen grupos y analicen el siguiente escenario:

> **Proceso A tiene la impresora y solicita acceso a la base de datos.**
> **Proceso B tiene la base de datos y solicita la impresora.**

Respondan:

1. ¿Qué recurso tiene el Proceso A?
2. ¿Qué recurso espera?
3. ¿Qué recurso tiene el Proceso B?
4. ¿Qué recurso espera?
5. ¿Se cumplen las cuatro condiciones de interbloqueo?
6. Dibujen el ciclo de espera.

---

# 246 Actividades por grupos

## 246.1 Grupo 1. Cruce vehicular

Dibujen una intersección con cuatro autos sin semáforo.

Identifiquen:

- Qué recurso tiene cada auto.
- Qué recurso espera.
- Dónde está la espera circular.

## 246.2 Grupo 2. Filósofos comensales

Analicen el problema clásico.

Respondan:

1. ¿Qué representa cada filósofo?
2. ¿Qué representan los tenedores?
3. ¿Cómo podría evitarse el interbloqueo?

## 246.3 Grupo 3. Las cuatro condiciones

Utilicen el escenario de la impresora y la base de datos.

Identifiquen:

1. Exclusión mutua.
2. Retención y espera.
3. No expropiación.
4. Espera circular.

## 246.4 Grupo 4. Clasificación de recursos

Clasifiquen como **exclusivos** o **compartidos**:

- Impresora.
- Archivo.
- CPU.
- Memoria compartida.
- Escáner.
- Variables globales.

## 246.5 Grupo 5. Una historia de interbloqueo

Inventen una historia con:

- 2 personas,
- 2 recursos,
- una situación de bloqueo mutuo.

Luego indiquen:

- quiénes representan los procesos,
- cuáles son los recursos,
- y cómo podría evitarse.

## 246.6 Grupo 6. Semáforo lógico

Simulen cómo un **semáforo** o un **árbitro** podría evitar:

- el interbloqueo del cruce,
- o el interbloqueo de los filósofos.

Respondan:

> ¿Qué condición deja de cumplirse cuando existe ese mecanismo de control?

---

# 247 Soluciones para discusión

Esta sección puede utilizarse después de que los grupos presenten sus respuestas.

## 247.1 Grupo 1

Cada automóvil ocupa un espacio y espera el siguiente.

Se forma un ciclo:

```
A → B → C → D → A
```

## 247.2 Grupo 2

- Filósofos → procesos.
- Tenedores → recursos exclusivos.
- Posibles soluciones: tomar ambos tenedores a la vez, limitar cuántos intentan comer o utilizar un árbitro.

## 247.3 Grupo 3

Se cumplen:

1. Exclusión mutua.
2. Retención y espera.
3. No expropiación.
4. Espera circular.

## 247.4 Grupo 4

De acuerdo con la clasificación planteada en el material de clase:

- **Exclusivos:** impresora, archivo, CPU, escáner.
- **Compartidos:** memoria compartida, variables globales.

## 247.5 Grupo 5

Ejemplo del material:

> Ana utiliza el microondas y espera el cuchillo. Carlos tiene el cuchillo y espera el microondas.

Una forma de evitarlo es liberar un recurso antes de solicitar otro o imponer un orden de solicitud.

## 247.6 Grupo 6

El semáforo o árbitro controla el acceso y evita que todos entren simultáneamente en una situación de espera circular.

---

# 248 ¿Cómo evitar un interbloqueo?

En esta clase no estudiaremos todavía los algoritmos formales de prevención y detección.

Sin embargo, los ejemplos ya nos permiten anticipar varias ideas:

- Solicitar recursos en un orden definido.
- Evitar conservar recursos mientras se esperan otros.
- Limitar cuántos procesos compiten al mismo tiempo.
- Liberar recursos cuando una operación falla.
- Utilizar un árbitro o mecanismo de coordinación.
- Establecer tiempos máximos de espera en determinados escenarios.

> ⚠ Importante
>
> Estas ideas introducen el siguiente tema.
> En el próximo capítulo estudiaremos estrategias formales para **prevenir, evitar, detectar y recuperar** interbloqueos.

---

# 249 Caso integrador

Supongamos que un sistema tiene dos procesos:

```
P1 necesita: Impresora + Archivo
P2 necesita: Archivo + Impresora
```

La ejecución ocurre así:

```
1. P1 obtiene la impresora.
2. P2 obtiene el archivo.
3. P1 solicita el archivo.
4. P2 solicita la impresora.
```

Analicemos:

| Pregunta | Respuesta |
|---|---|
| ¿P1 puede continuar? | No |
| ¿P2 puede continuar? | No |
| ¿P1 libera la impresora? | No mientras espera terminar |
| ¿P2 libera el archivo? | No mientras espera terminar |
| ¿Existe espera circular? | Sí |
| ¿Existe interbloqueo? | Sí |

Este pequeño ejemplo contiene la esencia del problema completo.

---

# 250 Autoevaluación

1. ¿Qué es un interbloqueo?
2. ¿Qué diferencia existe entre un proceso bloqueado temporalmente y un proceso en interbloqueo?
3. ¿Cuáles son las cuatro condiciones necesarias?
4. Explica con tus palabras la exclusión mutua.
5. ¿Qué significa retención y espera?
6. ¿Por qué la no expropiación favorece un interbloqueo?
7. ¿Qué representa la espera circular?
8. ¿Por qué el cruce vehicular es una buena analogía?
9. En los filósofos comensales, ¿qué representan los filósofos y los tenedores?
10. ¿Qué relación existe entre los interbloqueos y la administración de recursos del Kernel?
11. Construye un ejemplo con dos procesos y dos recursos.
12. Propón una forma sencilla de romper una de las cuatro condiciones.

---

# 251 Resumen del capítulo

Durante este capítulo aprendimos que un **interbloqueo** ocurre cuando varios procesos quedan esperando recursos retenidos entre sí y ninguno puede avanzar.

Para que aparezca deben coincidir cuatro condiciones:

1. **Exclusión mutua.**
2. **Retención y espera.**
3. **No expropiación.**
4. **Espera circular.**

También analizamos ejemplos como:

- cruce vehicular,
- filósofos comensales,
- impresoras en red,
- aplicaciones de oficina,
- cajeros automáticos.

> **Para recordar**
>
> Un proceso esperando no implica necesariamente un interbloqueo.
> El problema aparece cuando existe una **dependencia mutua que impide el avance de todos los procesos involucrados**.

# 252 Conexión con el siguiente capítulo

Ya sabemos reconocer **cuándo** existe un interbloqueo y **qué condiciones** permiten que ocurra.

La siguiente pregunta será:

> **¿Qué puede hacer el Sistema Operativo para impedirlo o resolverlo?**

En el siguiente capítulo estudiaremos:

- Prevención de interbloqueos.
- Evitación.
- Detección.
- Recuperación.
- Algoritmos utilizados para analizar estados seguros e inseguros.

---

# 253 Bibliografía base

El material de clase propone como referencias:

- Tanenbaum, capítulo 7.1.
- Deitel, capítulo 5.1.

# 254 Capítulo 9. Prevención, detección y recuperación de interbloqueos

## 254.1 Relación con la clase anterior

En el capítulo anterior aprendimos a reconocer un **interbloqueo (deadlock)** y estudiamos las cuatro condiciones necesarias para que pueda aparecer:

1. Exclusión mutua.
2. Retención y espera.
3. No expropiación.
4. Espera circular.

También analizamos casos con procesos que conservan recursos mientras esperan otros. Ahora damos el siguiente paso:

> **Si ya sabemos cómo se forma un interbloqueo, ¿qué puede hacer el Sistema Operativo para impedirlo, detectarlo o recuperarse cuando ya ocurrió?**

---

# 255 Objetivos de aprendizaje

Al finalizar este capítulo serás capaz de:

- Explicar cómo se **previenen** interbloqueos rompiendo al menos una condición necesaria.
- Analizar las ventajas y desventajas de distintas políticas de prevención.
- Interpretar un **grafo de asignación de recursos** para identificar ciclos de espera.
- Diferenciar **prevención, evitación, detección y recuperación**.
- Comprender la idea de **estado seguro** y la lógica general del algoritmo del banquero.
- Proponer una estrategia de recuperación ante un interbloqueo sencillo.

---

# 256 1. Recordemos antes de avanzar

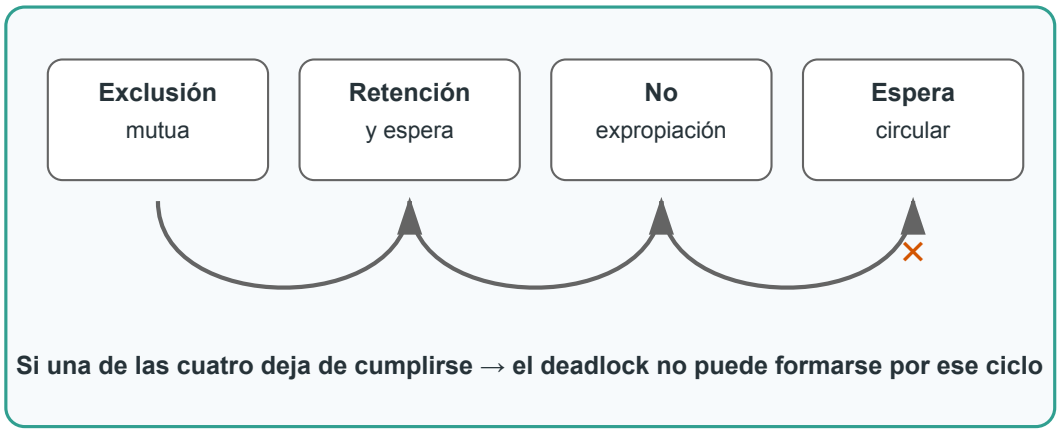
> **Pregunta para el grupo**
>
> Sin consultar el capítulo anterior:
> **¿Cuáles son las cuatro condiciones que deben cumplirse simultáneamente para que exista un interbloqueo?**

| Condición | Idea principal |
|---|---|
| Exclusión mutua | Un recurso solo puede ser utilizado por un proceso a la vez |
| Retención y espera | Un proceso conserva recursos mientras solicita otros |
| No expropiación | Un recurso no puede retirarse arbitrariamente |
| Espera circular | Existe un ciclo de procesos esperando recursos entre sí |

La idea central de la **prevención** es sencilla:

> Si evitamos que al menos una de estas condiciones pueda cumplirse, rompemos la posibilidad de formar ese interbloqueo.

## Prevención: romper al menos una condición necesaria




**Si una de las cuatro deja de cumplirse → el deadlock no puede formarse por ese ciclo**

# 257 2. ¿Por qué prevenir interbloqueos?

Un interbloqueo puede provocar que procesos permanezcan detenidos indefinidamente.

Algunos efectos posibles son:

- documentos que nunca llegan a imprimirse;
- transacciones que no pueden continuar;
- recursos ocupados sin realizar trabajo útil;
- usuarios esperando operaciones que no terminan;
- pérdida de rendimiento y disponibilidad.

En sistemas críticos —por ejemplo, bancos, hospitales o sistemas de control— el comportamiento debe ser especialmente predecible.

> ❗ Pensemos
>
> ¿Siempre conviene prevenir un interbloqueo a cualquier costo?
> Una política muy restrictiva puede evitar deadlocks, pero también puede dejar recursos sin utilizar durante mucho tiempo.

---

# 258 3. Prevención: atacar las condiciones necesarias

## 258.1 3.1 Exclusión mutua

La exclusión mutua existe cuando un recurso no puede compartirse simultáneamente.

Una estrategia consiste en **hacer compartibles los recursos siempre que sea posible**.

Ejemplo:

- varios procesos pueden leer simultáneamente un archivo de solo lectura;
- pero no siempre es posible compartir una impresora durante la impresión física de un documento.

> ⚠ Limitación
>
> No podemos eliminar la exclusión mutua de todos los recursos. Algunos son inherentemente exclusivos.

### 258.1.1 Mini ejercicio

Clasifica estos recursos como **potencialmente compartibles** o **normalmente exclusivos**:

1. Archivo de solo lectura.
2. Impresora.
3. Base de datos con múltiples lectores.
4. Escáner.

> 💡 Ver solución
>
> 1. **Archivo de solo lectura → Potencialmente compartible.** Varios procesos pueden leerlo simultáneamente sin modificar su contenido.
> 2. **Impresora → Normalmente exclusiva.** Los trabajos deben coordinarse para evitar que varias salidas se mezclen.
> 3. **Base de datos con múltiples lectores → Potencialmente compartible.** Varias lecturas pueden realizarse simultáneamente; las escrituras requieren mecanismos de sin-

cronización.
4. **Escáner → Normalmente exclusivo.** Una operación de captura suele ser controlada por un proceso a la vez.

**Idea clave:** si un recurso puede compartirse de forma segura, la exclusión mutua deja de ser necesaria para ese recurso.

---

# 259 4. Prevención de retención y espera

## 259.1 ¿Qué ocurre?

Un proceso ya posee un recurso y solicita otro sin liberar el primero.

Ejemplo:

```
P1 tiene Impresora
P1 solicita Disco
```

Si el disco está ocupado, P1 queda esperando mientras conserva la impresora.

## 259.2 ¿Cómo podemos prevenirlo?

Una política posible es exigir que el proceso:

- solicite **todos los recursos que necesitará al inicio**, o
- no pueda solicitar nuevos recursos mientras conserve otros.

Si no obtiene todos los recursos necesarios, espera sin conservar parcialmente el conjunto.

### 259.2.1 Ventaja

Reduce la posibilidad de que se formen cadenas de procesos reteniendo recursos.

### 259.2.2 Desventaja

Puede disminuir el aprovechamiento de recursos.

Un proceso podría reservar una impresora desde el inicio aunque no la utilice hasta mucho después.

**Pregunta**

¿Qué preferimos: máxima seguridad contra interbloqueos o máximo aprovechamiento de recursos?
Esta tensión aparecerá varias veces en Sistemas Operativos.

---

# 260 5. Prevención mediante expropiación

La condición de **no expropiación** establece que un recurso asignado no puede retirarse por la fuerza.

Para romper esa condición, el Sistema Operativo puede permitir que ciertos recursos sean recuperados y reasignados.

Ejemplo conceptual:

```
P1 posee R1
P1 solicita R2 y queda bloqueado
SO recupera R1
R1 puede ser utilizado por otro proceso
```

## 260.1 El problema

No todos los recursos pueden expropiarse de forma segura.

Quitar memoria o determinados recursos puede ser viable si existe un mecanismo de recuperación; interrumpir una escritura crítica o retirar una impresora a mitad de una operación puede dejar resultados inconsistentes.

> ⚠ Idea clave
>
> La expropiación requiere una política para decidir:
>
> - qué recurso retirar;
> - a qué proceso;
> - cómo recuperar posteriormente su estado.

# 261 6. Prevención de espera circular

La espera circular puede evitarse imponiendo un **orden total a los recursos**.

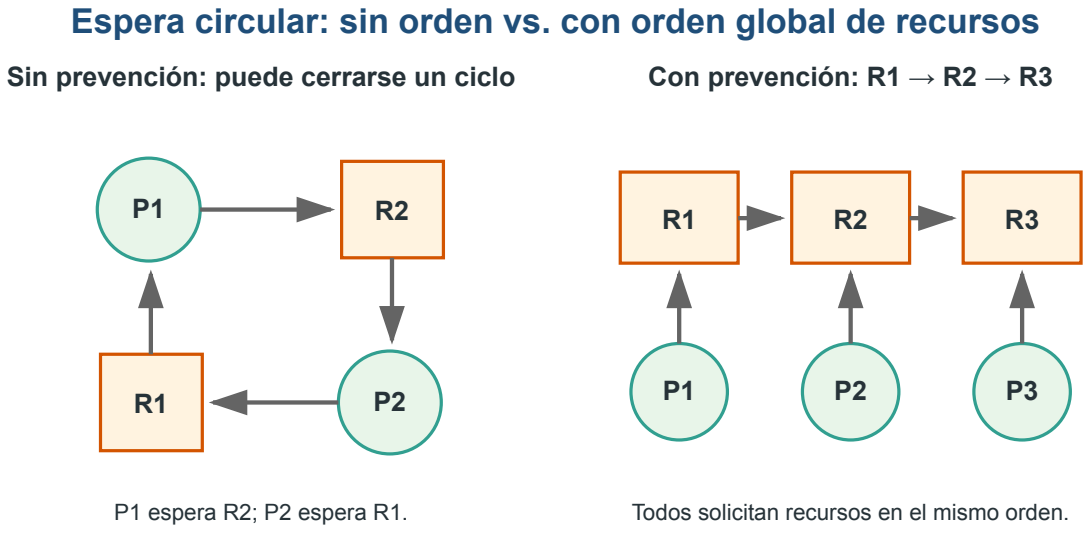
Supongamos:

```
R1 = Memoria
R2 = Disco
R3 = Impresora
```

La regla será:

```
R1 → R2 → R3
```

Todo proceso deberá solicitar recursos respetando ese orden.




Si todos siguen el mismo orden, un proceso no puede solicitar un recurso anterior después de haber obtenido uno posterior. Esto impide cerrar el ciclo de espera.

#### 261.0.1 Ejercicio rápido

Un sistema define:

```
R1 = Archivo
R2 = Base de datos
R3 = Impresora
```

Analiza estas solicitudes:

```
P1: R1 → R2
P2: R2 → R3
P3: R3 → R1
```

1. ¿Qué proceso viola el orden establecido?
2. ¿Qué podría ocurrir si permitimos esa solicitud?
3. ¿Cómo debería solicitar P3 sus recursos?

 Ver solución

El orden global establecido es **R1 → R2 → R3**.

- **P1: R1 → R2**: respeta el orden.
- **P2: R2 → R3**: respeta el orden.
- **P3: R3 → R1**: **viola el orden**, porque después de R3 intenta solicitar un recurso anterior.

Si permitimos esa solicitud puede cerrarse una cadena circular entre procesos y recursos, creando las condiciones para un interbloqueo.
P3 debería solicitar sus recursos en orden ascendente: **R1 → R3**, y no **R3 → R1**.
**Idea clave:** imponer un orden global rompe la condición de **espera circular**.

# 262 7. Prevención, evitación y detección no son lo mismo

| Estrategia | Idea |
|---|---|
| Prevención | Diseñar reglas para que alguna condición necesaria no pueda cumplirse |
| Evitación | Evaluar cada asignación y concederla solo si el sistema permanece en un estado seguro |
| Detección | Permitir la ejecución y buscar posteriormente si apareció un interbloqueo |
| Recuperación | Tomar acciones después de detectar el interbloqueo |

Precisión importante

El material de clase introduce el **algoritmo del banquero** dentro de las estrategias para impedir interbloqueos. Técnicamente, suele clasificarse como un algoritmo de **evitación**, porque no elimina permanentemente una condición: analiza si conceder una solicitud mantiene al sistema en un estado seguro.

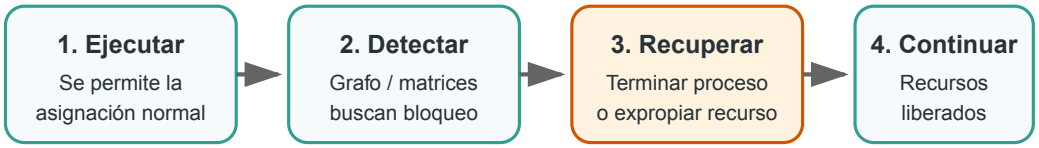
# 263 8. Detección de interbloqueos

En la detección, el sistema **no impide necesariamente** que se forme un interbloqueo.

En cambio:

1. permite que los procesos soliciten y reciban recursos;
2. analiza periódicamente el estado del sistema;
3. busca dependencias que indiquen bloqueo;
4. si encuentra un interbloqueo, inicia una estrategia de recuperación.

### Detección y recuperación: permitir, observar, actuar




**La detección no impide el interbloqueo: lo identifica cuando ya puede estar presente.**

---

# 264 9. Grafo de asignación de recursos

Un grafo de asignación representa:

- **procesos** mediante círculos;
- **recursos** mediante cuadros;
- `P → R` cuando un proceso solicita un recurso;
- `R → P` cuando un recurso está asignado a un proceso.

Ejemplo:

```
P1 → R2
R2 → P2
P2 → R3
R3 → P1
```

La cadena vuelve a P1:

```
P1 → R2 → P2 → R3 → P1
```

Existe un ciclo.

> **Importante**
>
> En el caso simple donde cada tipo de recurso tiene una sola instancia, un ciclo confirma el interbloqueo. Con múltiples instancias, la interpretación requiere un análisis adicional.

## 264.1 Practiquemos

Analiza:

```
R1 → P1
P1 → R2
R2 → P2
P2 → R1
```

Responde:

1. ¿Qué recurso posee P1?
2. ¿Qué recurso solicita P1?
3. ¿Qué recurso posee P2?
4. ¿Qué recurso solicita P2?
5. ¿Existe un ciclo?
6. ¿Existe interbloqueo en este caso?

 Ver solución

1. P1 posee **R1**.
2. P1 solicita **R2**.
3. P2 posee **R2**.
4. P2 solicita **R1**.
5. Sí existe un ciclo: **R1** → **P1** → **R2** → **P2** → **R1**.
6. Si R1 y R2 tienen **una sola instancia cada uno**, sí existe interbloqueo: P1 espera R2, que posee P2, mientras P2 espera R1, que posee P1.

**Importante:** con una sola instancia por tipo de recurso, el ciclo confirma el interbloqueo. Con múltiples instancias se necesita un análisis adicional.

# 265 10. El algoritmo del banquero

El algoritmo del banquero fue propuesto por **Edsger Dijkstra**.

Su idea puede imaginarse como un banco que presta dinero:

> El banco no concede un préstamo únicamente porque tenga dinero disponible en este instante. Antes debe comprobar que, después del préstamo, todavía existe una forma de satisfacer a todos sus clientes.

En Sistemas Operativos:

- los clientes son **procesos**;
- el dinero representa **recursos**;
- cada proceso declara su necesidad máxima;
- el sistema analiza si una asignación mantiene un **estado seguro**.

---

# 266 11. ¿Qué es un estado seguro?

Un estado es **seguro** cuando existe al menos una secuencia de ejecución en la cual todos los procesos pueden terminar.

Supongamos tres procesos:

```
P1
P2
P3
```

Si existe una secuencia como:

```
P2 → P1 → P3
```

en la cual P2 puede terminar con los recursos disponibles, liberar los suyos, luego P1 puede terminar y finalmente P3, entonces tenemos una **secuencia segura**.

> ❗ Estado inseguro ≠ interbloqueo actual
>
> Un estado inseguro significa que el sistema ya no puede garantizar una secuencia segura. No significa necesariamente que el interbloqueo ya ocurrió, pero existe riesgo de llegar a él.

---

# 267 12. Ejemplo básico del banquero

Consideremos un solo tipo de recurso para comprender primero la lógica.

El sistema tiene **10 unidades**.

| Proceso | Asignado | Máximo | Necesita todavía |
|---|---|---|---|
| P1 | 3 | 7 | 4 |
| P2 | 2 | 4 | 2 |
| P3 | 2 | 5 | 3 |

Recursos utilizados:

```
3 + 2 + 2 = 7
```

Recursos disponibles:

```
10 - 7 = 3
```

Preguntemos:

> ¿Existe algún proceso que pueda terminar necesitando como máximo las 3 unidades disponibles?

P2 necesita 2.

Entonces:

1. P2 recibe lo necesario y termina.
2. Libera sus recursos.
3. Ahora existen más recursos disponibles.
4. Buscamos otro proceso que pueda terminar.
5. Repetimos hasta completar todos los procesos.

**Ejercicio guiado**

Continúa el ejemplo:

1. Después de que P2 termina, ¿cuántos recursos quedan disponibles?

2. ¿Puede terminar P1?
3. ¿Puede terminar P3?
4. Escribe una posible secuencia segura.

 Ver solución

Inicialmente hay **3 recursos disponibles**.

- P2 necesita 2, por lo que puede recibirlos y terminar.
- Al terminar P2 libera sus **4 recursos** en total (los 2 que tenía más los 2 que recibió). Como antes de atenderlo había 3 disponibles, después quedan **5 disponibles**.
- P1 necesita 4, así que ahora puede terminar. Al liberar sus 7 recursos, quedan **10 disponibles**.
- Finalmente P3, que necesita 3, también puede terminar.

Una secuencia segura posible es:
**P2 → P1 → P3**
También puede existir otra secuencia segura; lo importante es demostrar que existe al menos una en la que todos los procesos puedan finalizar.

---

# 268 13. Actividad por pares: ¿cómo lo resolverías?

Analicen:

```
Proceso A tiene Disco y solicita Memoria.
Proceso B tiene Memoria y solicita Disco.
```

Respondan:

1. ¿Qué condiciones de interbloqueo se cumplen?
2. ¿Existe espera circular?
3. ¿Qué política de prevención podría evitar el problema?
4. Si el interbloqueo ya ocurrió, ¿qué recurso podría intentarse recuperar?
5. ¿Qué riesgo tendría expropiar un recurso?

Tiempo sugerido: **5 minutos**.

 Ver solución

1. **Se cumplen las cuatro condiciones:** exclusión mutua, retención y espera, no expropiación y espera circular.
2. **Sí existe espera circular:** A espera Memoria, que posee B; B espera Disco, que posee A.
3. Una política sencilla sería imponer un **orden global de recursos**, por ejemplo: **Disco → Memoria**. Todos los procesos que necesiten ambos deberán solicitarlos en ese orden.
4. Si el interbloqueo ya ocurrió, podría intentarse recuperar uno de los recursos **solo si puede expropiarse de forma segura**. Otra alternativa es finalizar o reiniciar uno de los procesos para que libere lo que posee.
5. Expropiar un recurso puede dejar al proceso en un estado inconsistente, provocar pérdida de trabajo o exigir repetir operaciones.

**Conclusión:** prevenir el ciclo suele ser menos costoso que recuperarse después de que el interbloqueo ya ocurrió.

---

# 269 14. Recuperación después del interbloqueo

Si el interbloqueo ya existe, prevenirlo dejó de ser una opción. Debemos recuperarnos.

Dos estrategias comunes son:

## 269.1 Finalizar procesos

El Sistema Operativo puede:

- terminar todos los procesos involucrados, o
- terminar procesos uno por uno hasta romper el ciclo.

## 269.2 Retirar recursos selectivamente

El sistema puede intentar expropiar un recurso y reasignarlo.

Para elegir una **víctima** deben considerarse factores como:

- prioridad del proceso;
- tiempo que lleva ejecutándose;
- cantidad de trabajo realizado;
- recursos que posee;
- costo de reiniciarlo;
- riesgo de pérdida o inconsistencia de datos.

---

# 270 15. Caso integrador

Un servidor tiene:

```
P1: posee Archivo A y solicita Impresora
P2: posee Impresora y solicita Archivo A
```

Trabajemos en cuatro niveles.

## 270.1 Nivel 1. Diagnóstico

- ¿Existe interbloqueo?
- ¿Cuáles condiciones se cumplen?

## 270.2 Nivel 2. Prevención

- ¿Qué condición podríamos romper?
- ¿Cómo impondríamos un orden de recursos?

## 270.3 Nivel 3. Detección

Dibuja el grafo utilizando:

```
Proceso → Recurso solicitado
Recurso → Proceso que lo posee
```

## 270.4 Nivel 4. Recuperación

Si ya ocurrió:

- ¿terminarías P1 o P2?
- ¿intentarías retirar un recurso?
- ¿qué información necesitarías para decidir?

 Ver solución

### 270.4.1 Nivel 1. Diagnóstico

Sí existe interbloqueo, suponiendo que **Archivo A** y la **Impresora** son recursos exclusivos con una sola instancia. Se cumplen las cuatro condiciones: exclusión mutua, retención y espera, no expropiación y espera circular.

### 270.4.2 Nivel 2. Prevención

Podemos romper la **espera circular** imponiendo un orden global, por ejemplo:

```
Archivo A → Impresora
```

Todos los procesos que necesiten ambos recursos deberán solicitarlos en ese mismo orden.

### 270.4.3 Nivel 3. Detección

El grafo contiene:

```
Archivo A → P1
P1 → Impresora
Impresora → P2
P2 → Archivo A
```

Por tanto aparece el ciclo:

```
Archivo A → P1 → Impresora → P2 → Archivo A
```

Con una sola instancia de cada recurso, el ciclo representa un interbloqueo.

### 270.4.4 Nivel 4. Recuperación

No podemos decidir automáticamente si conviene terminar P1 o P2 únicamente con la información del enunciado. Para elegir una víctima deberíamos considerar, entre otros factores:

- prioridad de cada proceso;
- cantidad de trabajo realizado;
- costo de reiniciarlo;
- recursos que liberaría;
- consecuencias de cancelar la operación;
- posibilidad de repetir o recuperar el trabajo.

También podría intentarse expropiar un recurso si la operación permite hacerlo de manera segura.

**Idea clave:** detectar el interbloqueo no resuelve el problema por sí solo; la recuperación puede tener un costo importante.

# 271 16. Actividad individual

Busca un caso real o documentado de interbloqueo en software.

Identifica:

1. ¿Qué procesos o hilos estaban involucrados?
2. ¿Qué recursos estaban utilizando?
3. ¿Cómo se produjo la espera?
4. ¿Cómo se detectó?
5. ¿Cómo se solucionó o evitó posteriormente?

> No basta con encontrar el caso
>
> Debes poder relacionarlo con al menos una de estas ideas:
> **prevención – evitación – detección – recuperación**

---

# 272 17. Autoevaluación

1. ¿Cuál es la idea fundamental de la prevención de interbloqueos?
2. ¿Cómo puede atacarse la condición de retención y espera?
3. ¿Qué desventaja tiene solicitar todos los recursos al inicio?
4. ¿Qué significa expropiar un recurso?
5. ¿Por qué algunos recursos no deberían expropiarse?
6. ¿Cómo evita un orden global de recursos la espera circular?
7. ¿Qué diferencia existe entre prevención y detección?
8. ¿Qué representa `P → R` en un grafo de asignación?
9. ¿Qué representa `R → P`?
10. ¿Qué es un estado seguro?
11. ¿Por qué el algoritmo del banquero se relaciona con la evitación?
12. ¿Qué alternativas existen para recuperarse de un interbloqueo?

---

# 273 18. Resumen del capítulo

En este capítulo aprendimos que no existe una única estrategia frente a los interbloqueos.

Podemos actuar **antes**, **durante** o **después** del problema:

```
PREVENIR  → romper una condición necesaria
EVITAR    → conceder recursos solo si el estado continúa siendo seguro
DETECTAR  → buscar interbloqueos durante la ejecución
RECUPERAR → romper el bloqueo después de detectarlo
```

También estudiamos:

- retención y espera;
- expropiación;
- espera circular;
- grafos de asignación;
- estados seguros;
- algoritmo del banquero;
- finalización de procesos;
- recuperación de recursos.

> **Para recordar**
>
> La mejor estrategia depende del tipo de sistema, del costo de mantener recursos reservados y de las consecuencias de detener o reiniciar procesos.

---

# 274 Conexión con la siguiente clase

Ya sabemos:

- cómo se forma un interbloqueo;
- cómo prevenirlo;
- cómo analizar estados seguros;
- cómo detectarlo;
- y qué opciones existen para recuperarse.

En la siguiente etapa podremos profundizar en la administración de recursos y en los mecanismos que utiliza el Sistema Operativo para coordinar procesos concurrentes.

---

# 275 Bibliografía base

De acuerdo con el material de clase:

- Tanenbaum, secciones/capítulos 7.2–7.4 indicados en la presentación.
- Deitel, secciones/capítulos 5.2–5.4 indicados en la presentación.
- Capítulo anterior: **Condiciones necesarias de interbloqueo**.

# 276 Actividad - Diagnóstico de un Sistema Operativo

# 277 Universidad Mariano Gálvez de Guatemala

**Facultad de Ingeniería en Sistemas**
**Curso:** Sistemas Operativos I
**Docente:** Ing. Jorge Luis García Chacón
**Actividad:** Diagnóstico de un Sistema Operativo
**Valor:** 25 puntos
**Modalidad:** Trabajo en grupo
**Tiempo estimado:** 60 minutos

**Grupo No.:** _____

**Integrantes:**

1. _____
2. _____
3. _____
4. _____
5. _____
6. _____

**!** Reglas de trabajo

- Esta actividad se resolverá **en papel** y en grupo.
- Pueden consultar sus **apuntes o el libro del curso**.
- No se permite utilizar teléfono, computadora ni herramientas de IA durante la actividad.
- Todas las respuestas deben ser **razonadas**. No basta con escribir el nombre de un concepto.
- El docente podrá pedir a cualquier integrante que explique una respuesta del grupo.

## 277.1 Escenario

En el laboratorio de cómputo de una universidad, una computadora presenta problemas de rendimiento mientras varios usuarios realizan distintas tareas. El encargado del laboratorio reporta lo siguiente:

- El equipo tarda en responder cuando se cambia entre aplicaciones.
- Un navegador mantiene varias pestañas abiertas y reproduce video.
- Un estudiante está descargando un archivo grande desde Internet.
- Otro programa está enviando un documento a imprimir.
- El antivirus realiza una revisión en segundo plano.
- En algunos momentos una aplicación deja de responder durante varios segundos y luego continúa.

El administrador consulta el estado del sistema y obtiene la siguiente información:

| Proceso | Estado observado | CPU | Memoria | Observación |
|---|---|---|---|---|
| Navegador | Running | 55 % | 1.8 GB | Varias pestañas y video activo |
| Descarga | Ready | 10 % | 220 MB | Archivo grande en proceso |
| Procesador de texto | Ready | 4 % | 650 MB | Documento abierto |
| Antivirus | Ready | 24 % | 900 MB | Análisis en segundo plano |
| Impresión | Waiting | 0 % | 85 MB | Esperando respuesta de la impresora |
| Reproductor multimedia | Ready | 7 % | 400 MB | Audio activo |

La computadora posee **8 GB de RAM** y un procesador de **4 núcleos**.

# 278 Misión 1. Diagnóstico inicial — 5 puntos

Analicen la información disponible y respondan:

**1. ¿Cuál consideran que es el principal problema de rendimiento del equipo?**
Justifiquen su respuesta utilizando la información del escenario y los datos de los procesos.

**2. ¿Qué otros factores podrían estar contribuyendo al problema?**

**3. ¿Por qué no sería correcto afirmar inmediatamente que “la computadora está lenta porque la CPU está dañada”?**

# 279 Misión 2. ¿Qué ocurre dentro del Sistema Operativo? — 5 puntos

Sin limitarse a definir conceptos, expliquen qué componentes o mecanismos del Sistema Operativo intervienen en esta situación y **qué papel cumplen**.

| Componente o mecanismo identificado | ¿Qué está haciendo en este escenario? |
|---|---|
| 1. _____ | |
| 2. _____ | |
| 3. _____ | |
| 4. _____ | |

# 280 Misión 3. Sigan un proceso — 5 puntos

Seleccionen el proceso **Impresión** y reconstruyan lo que probablemente ocurre desde que el usuario solicita imprimir hasta que el proceso vuelve a estar listo para continuar.

Dibujen los estados y transiciones que consideren necesarios.

**a. ¿Por qué el proceso de impresión aparece en estado `Waiting`?**

**b. ¿Qué tendría que conservar el Sistema Operativo para poder suspender temporalmente ese proceso y luego continuar su ejecución correctamente?**

**c. Cuando la impresora responde, ¿qué debería ocurrir con el proceso? Expliquen la secuencia.**

# 281 Misión 4. Ustedes son el Sistema Operativo — 5 puntos

Supongan ahora que la CPU debe atender los siguientes procesos:

| Proceso | Tiempo estimado de CPU | Prioridad | Tipo de trabajo |
|---|---|---|---|
| P1 | 8 ms | 3 | Navegador interactivo |
| P2 | 2 ms | 1 | Proceso crítico del sistema |
| P3 | 6 ms | 2 | Antivirus |
| P4 | 3 ms | 3 | Reproductor multimedia |

Respondan:

**1. ¿Qué criterio de planificación utilizarían para atender estos procesos y por qué?**

**2. ¿Atenderían todos los procesos exactamente de la misma manera? Justifiquen.**

**3. ¿Qué riesgo habría si un solo proceso pudiera conservar la CPU durante demasiado tiempo?**

# 282 Misión 5. Diagnóstico final — 5 puntos

## 282.1 A. Recorrido de una solicitud

El usuario presiona **Imprimir** desde el procesador de texto.

Dibujen el recorrido que sigue esa solicitud desde el usuario hasta el dispositivo físico. Incluyan únicamente los componentes que consideren necesarios y expliquen brevemente qué ocurre en cada paso.

## 282.2 B. Decisión final del grupo

Como equipo responsable del diagnóstico, indiquen:

**1. ¿Qué acciones realizarían para mejorar el rendimiento del equipo sin apagarlo inmediatamente?**

**2. ¿Qué proceso o recurso observarían primero y por qué?**

**3. Diagnóstico definitivo del grupo:**

# 283 Criterios de evaluación

| Criterio | Punteo |
|---|---|
| Diagnóstico y análisis del problema | 5 |
| Identificación y explicación de componentes del SO | 5 |
| Estados de procesos y razonamiento sobre el PCB | 5 |
| Decisión de planificación y justificación | 5 |
| Integración final: capas, diagnóstico y solución | 5 |
| **Total** | **25** |

> **Cierre**
>
> La calidad de la respuesta se evaluará principalmente por la **capacidad de análisis y justificación del grupo**, no por repetir definiciones de memoria.

# 284 Actividad - Diagnóstico de un Sistema Operativo

Guía de respuestas y retroalimentación

# 285 Antes de revisar las respuestas

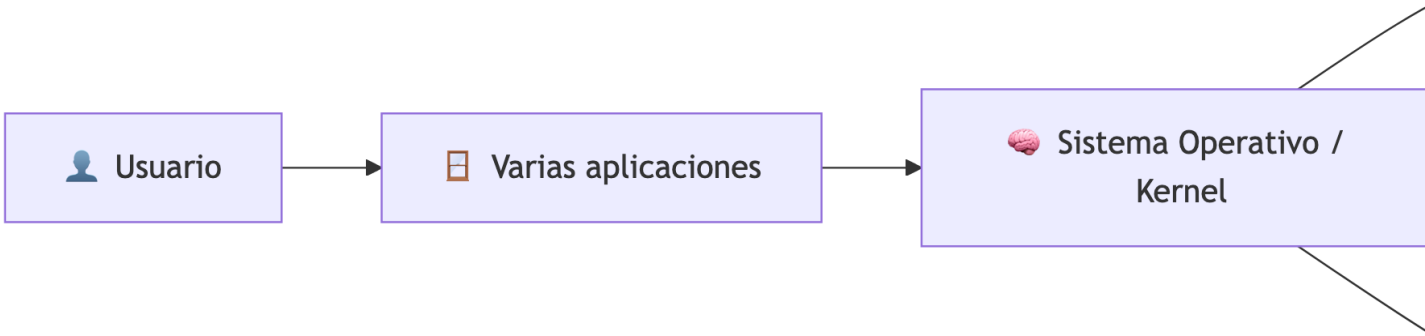
Esta guía se publica **después de realizar y exponer la actividad**. Su propósito no es presentar una única respuesta posible, sino comparar el razonamiento del grupo con una solución técnicamente fundamentada.

> ! Una idea importante
>
> En un diagnóstico real no siempre existe una sola respuesta correcta. Una conclusión puede ser válida si utiliza correctamente la evidencia disponible y puede justificarse.

# 286 Recordemos el escenario

El equipo tiene **8 GB de RAM**, un procesador de **4 núcleos** y varios procesos activos al mismo tiempo.

| Proceso | Estado | CPU | Memoria |
|---|---|---|---|
| Navegador | Running | 55 % | 1.8 GB |
| Descarga | Ready | 10 % | 220 MB |
| Procesador de texto | Ready | 4 % | 650 MB |
| Antivirus | Ready | 24 % | 900 MB |
| Impresión | Waiting | 0 % | 85 MB |
| Reproductor multimedia | Ready | 7 % | 400 MB |



# 287 Misión 1. Diagnóstico inicial — 5 puntos

## 287.1 1. ¿Cuál parece ser el principal problema?

Una respuesta razonable sería que existe **alta competencia por tiempo de CPU**, especialmente por el navegador y el antivirus.

El navegador aparece ejecutándose con **55 % de CPU** y el antivirus requiere otro **24 %**. Al mismo tiempo existen varios procesos en estado `Ready`, es decir, preparados para ejecutar pero esperando que el planificador les asigne CPU.

Esto ayuda a explicar por qué el usuario percibe retrasos al cambiar entre aplicaciones.

> **i** Pero cuidado
>
> Los datos no demuestran que exista un único culpable. Estamos realizando un diagnóstico con la evidencia disponible. También deben considerarse memoria, red, almacenamiento y dispositivos de entrada/salida.

## 287.2 2. ¿Qué otros factores pueden contribuir?

Entre las respuestas justificables están:

- varias aplicaciones abiertas simultáneamente;
- consumo de memoria del navegador y otros programas;
- actividad del antivirus en segundo plano;
- descarga de un archivo grande y utilización de red/almacenamiento;
- reproducción de video y audio;
- espera de la impresora;
- posibles operaciones de entrada/salida;
- cambios frecuentes entre procesos.

La memoria reportada para los procesos mostrados suma aproximadamente **4 GB**, por lo que no podemos afirmar solamente con esta tabla que los 8 GB estén agotados. Sin embargo, el Sistema Operativo y otros procesos no mostrados también utilizan memoria.

## 287.3 3. ¿La CPU está dañada?

No existe evidencia suficiente para concluirlo.

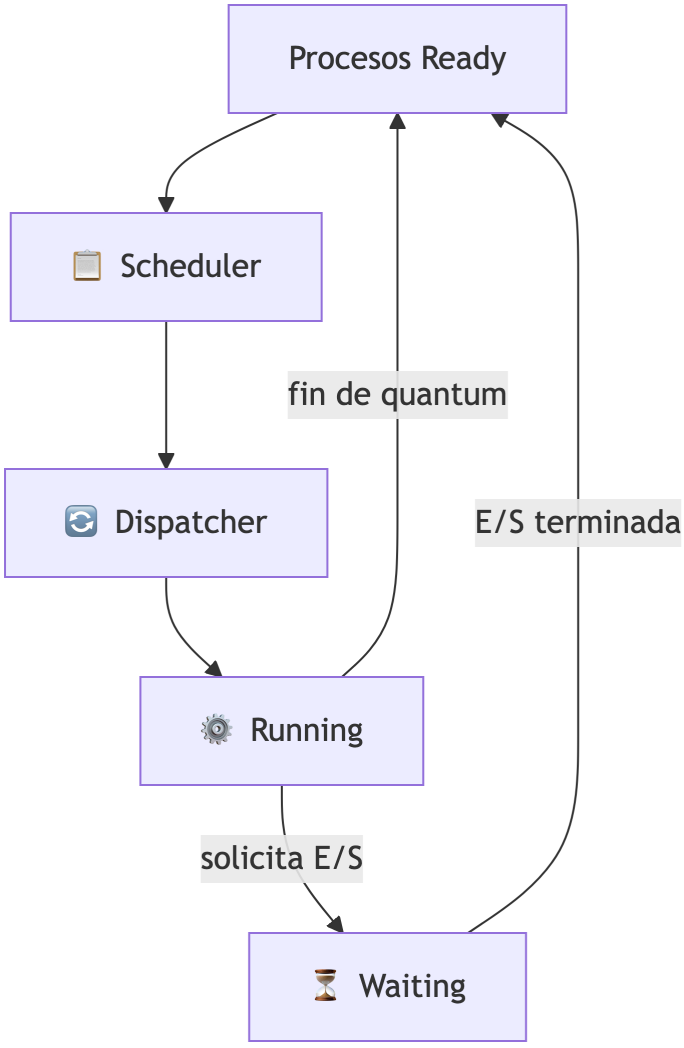
Una computadora puede sentirse lenta aunque el procesador funcione correctamente. El problema puede encontrarse en la **carga de trabajo**, la planificación, la memoria, el almacenamiento, la red, un dispositivo de E/S o un proceso específico.

> **Diagnosticar no significa adivinar el componente dañado; significa interpretar evidencias.**

# 288 Misión 2. ¿Qué ocurre dentro del Sistema Operativo? — 5 puntos

No era obligatorio mencionar exactamente cuatro componentes específicos. Lo importante era identificar mecanismos coherentes y explicar su participación.

| Componente o mecanismo | Papel en el escenario |
|---|---|
| **Kernel** | Coordina el acceso de los procesos a CPU, memoria y dispositivos. |
| **Scheduler** | Decide qué proceso preparado obtiene tiempo de CPU. |
| **PCB** | Conserva la información necesaria para administrar cada proceso y reanudarlo después de una interrupción o espera. |
| **Dispatcher / cambio de contexto** | Permite entregar la CPU a otro proceso y continuar posteriormente con el anterior. |
| **Gestión de memoria** | Controla la memoria utilizada por los diferentes procesos. |
| **Drivers y E/S** | Permiten al Sistema Operativo comunicarse con dispositivos como la impresora. |

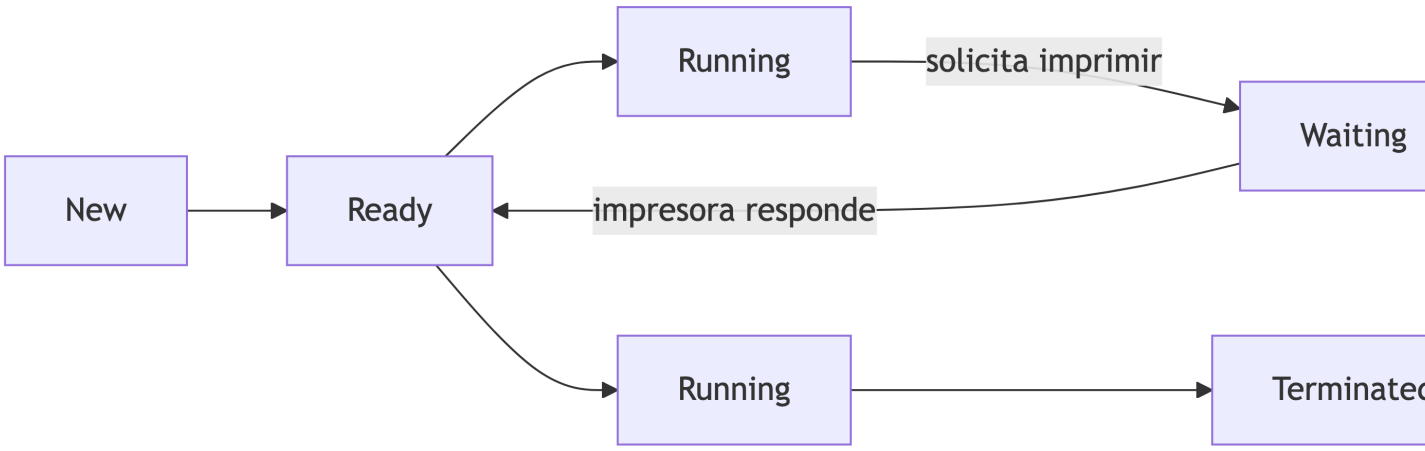



**Lo que buscábamos comprobar**

Que pudieran relacionar los conceptos. Por ejemplo: **Scheduler → selecciona → Dispatcher → entrega CPU → proceso ejecuta → PCB conserva el contexto**.

# 289 Misión 3. Sigan el proceso de impresión — 5 puntos

## 289.1 Una secuencia posible




## 289.2 a. ¿Por qué aparece `Waiting`?

Porque el proceso realizó una operación que depende de un evento externo: la operación de impresión.

Mientras espera la respuesta correspondiente, no tiene sentido mantenerlo ocupando la CPU. El Sistema Operativo puede utilizar ese tiempo para ejecutar otros procesos.

## 289.3 b. ¿Qué debe conservar el Sistema Operativo?

Debe conservar la información necesaria para poder continuar posteriormente la ejecución. Esa información se representa conceptualmente mediante el **PCB (Process Control Block)**.

Entre otros elementos puede incluir:

- identificador del proceso;

- estado;
- contador de programa;
- registros;
- información de planificación;
- información relacionada con memoria;
- recursos y archivos asociados.

La idea central no era memorizar toda la lista, sino comprender que **el proceso debe poder detenerse y continuar sin comenzar desde cero**.

## 289.4 c. ¿Qué ocurre cuando responde la impresora?

Una secuencia razonable es:

**Waiting → Ready → Running**

La finalización del evento de E/S permite que el proceso deje de estar bloqueado. Sin embargo, esto **no significa necesariamente que pase inmediatamente a Running**: primero queda disponible (`Ready`) y el Scheduler decidirá cuándo vuelve a utilizar CPU.

# 290 Misión 4. Ustedes son el Sistema Operativo — 5 puntos

Los procesos eran:

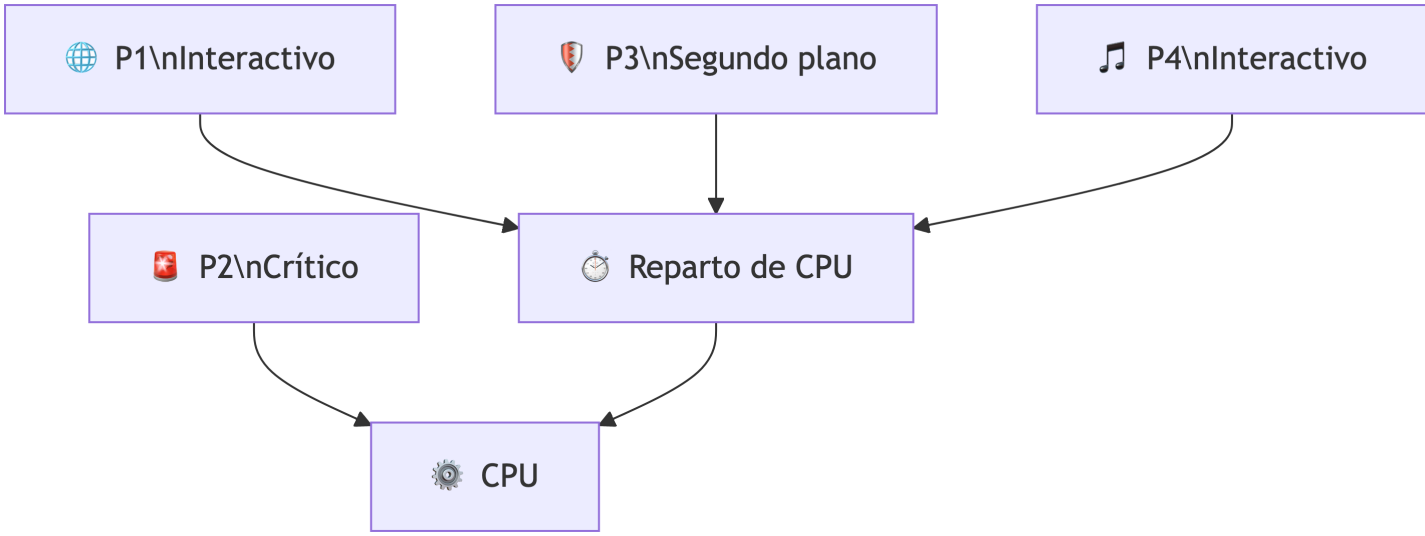
| Proceso | CPU | Prioridad | Tipo |
|---|---|---|---|
| P1 | 8 ms | 3 | Navegador interactivo |
| P2 | 2 ms | 1 | Proceso crítico del sistema |
| P3 | 6 ms | 2 | Antivirus |
| P4 | 3 ms | 3 | Reproductor multimedia |

## 290.1 1. ¿Qué criterio utilizar?

Aquí **no buscábamos una única palabra**.

Una respuesta bien argumentada puede proponer **planificación por prioridades** para favorecer P2, combinada con reparto de CPU para conservar la respuesta de los procesos interactivos.

También era razonable mencionar **Round Robin** para los procesos interactivos, porque evita que uno conserve indefinidamente el procesador.



> ⚠ No confundir
>
> SJF podría reducir el tiempo promedio de espera en ciertos escenarios, pero seleccionar siempre el proceso más corto no considera por sí mismo que **P2 es crítico** ni garantiza el comportamiento deseado para aplicaciones interactivas.

## 290.2 2. ¿Todos deben tratarse igual?

No necesariamente.

P2 representa una tarea crítica del sistema, mientras P1 y P4 son interactivos y P3 corresponde a una tarea en segundo plano.

El Sistema Operativo puede necesitar equilibrar:

- prioridad;
- capacidad de respuesta;
- equidad;
- utilización del procesador.

## 290.3 3. ¿Qué pasa si uno conserva demasiado la CPU?

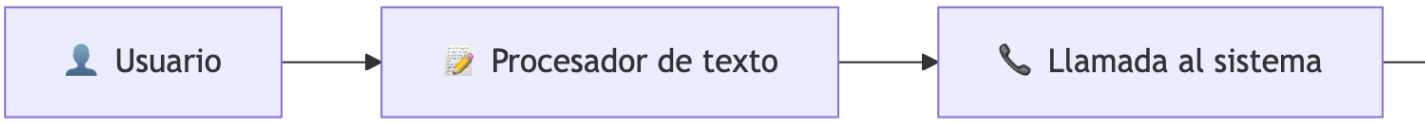
Los demás procesos pueden permanecer demasiado tiempo esperando.

El usuario percibiría aplicaciones congeladas o poco responsivas. Precisamente por eso los sistemas multitarea necesitan mecanismos de planificación y, según la política utilizada, **preempción y quantum de tiempo**.

# 291 Misión 5. Diagnóstico final — 5 puntos

## 291.1 A. ¿Qué ocurre cuando presionamos Imprimir?

Una representación simplificada sería:




El usuario no controla directamente la impresora.

1. El usuario solicita imprimir desde la aplicación.
2. La aplicación solicita un servicio al Sistema Operativo.
3. El Kernel coordina la operación.
4. El controlador o **driver** permite la comunicación con el dispositivo.
5. La impresora realiza la operación.
6. Mientras se espera el evento de E/S, el proceso relacionado puede quedar bloqueado.
7. Cuando la operación permite continuar, el proceso puede regresar a `Ready`.

## 291.2 B. ¿Qué acciones podrían realizarse?

Una respuesta prudente sería **observar antes de cerrar procesos al azar**.

Podrían:

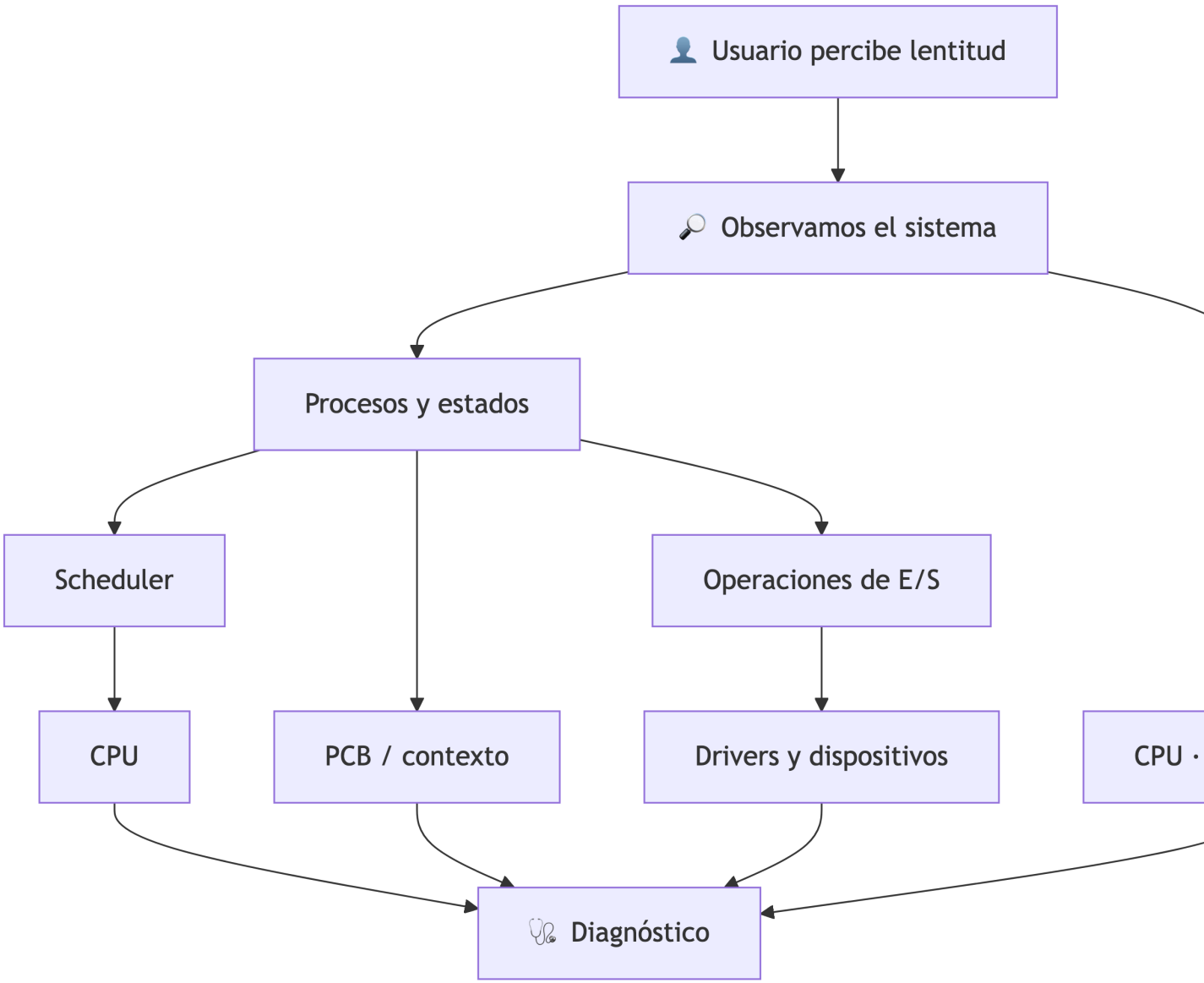
1. revisar qué proceso consume más CPU;
2. observar si el consumo elevado se mantiene;
3. verificar presión de memoria;
4. revisar actividad de disco, red y dispositivos;
5. comprobar el estado de la impresión;
6. pausar o reprogramar tareas de segundo plano si están afectando el trabajo interactivo;
7. cerrar únicamente aplicaciones o pestañas que no sean necesarias.

### 291.2.1 ¿Qué observar primero?

Con los datos proporcionados, comenzar por **CPU y procesos activos** es razonable porque el navegador y el antivirus muestran una carga importante y existen varios procesos en `Ready`.

Después habría que revisar memoria y E/S para confirmar o descartar otros cuellos de botella.

# 292 Uniendo todo



**!** La respuesta más importante de toda la actividad

Un buen diagnóstico de Sistemas Operativos **no consiste en encontrar rápidamente un culpable**. Consiste en observar procesos, estados, recursos y eventos; relacionarlos con el funcionamiento del Kernel; formular una explicación y comprobar si la evidencia la respalda.

# 293 ¿Cómo podía obtenerse el punteo?

| Área evaluada | 5 puntos cuando… |
|---|---|
| Diagnóstico | Utiliza los datos del escenario y evita conclusiones sin evidencia. |
| Componentes del SO | Identifica componentes correctos y explica su relación con el caso. |
| Procesos y PCB | Interpreta correctamente `Running`, `Ready`, `Waiting` y la conservación del contexto. |
| Planificación | Propone un criterio coherente y lo justifica según el tipo de proceso. |
| Integración | Relaciona aplicación, Kernel, llamadas al sistema, drivers, hardware y propone acciones razonables. |

# 294 Para cerrar

Si algunas respuestas de tu grupo fueron diferentes de esta guía, pregúntate:

- ¿Nuestra respuesta estaba sustentada por los datos?
- ¿Utilizamos correctamente los conceptos?
- ¿Podíamos explicar por qué llegamos a esa conclusión?
- ¿Cambiaríamos algo después de escuchar a los demás grupos?

> Consejo del Ingeniero
>
> En Sistemas Operativos, aprender los nombres de los componentes es apenas el comienzo. **El verdadero aprendizaje aparece cuando puedes explicar qué están haciendo y por qué.**

# 295 Capítulo 12. Virtualización de Sistemas Operativos

# 296 Capítulo 12. Virtualización de Sistemas Operativos

## 296.1 De una computadora… a varias computadoras

Hasta ahora hemos estudiado cómo el Sistema Operativo administra procesos, recursos, hardware e interbloqueos. Ahora utilizaremos esos conocimientos para comprender una tecnología que permite ejecutar **varios sistemas operativos sobre un mismo equipo físico**.

Imaginemos una computadora con:

- procesador;
- 16 GB de memoria RAM;
- 512 GB de almacenamiento;
- conexión de red;
- Windows, macOS o Linux como sistema principal.

La pregunta es:

> **¿Podemos utilizar parte de esos recursos para construir otra computadora sin comprar otro equipo físico?**

Sí. Esa es precisamente una de las ideas fundamentales de la **virtualización**.

> Idea central
>
> Una máquina virtual se comporta como una computadora independiente, pero utiliza recursos proporcionados por una computadora física.

Figura 296.1: Panorama visual de la virtualización: una computadora física puede alojar varias máquinas virtuales.

# 297 Objetivos de aprendizaje

Al finalizar este capítulo, el estudiante será capaz de:

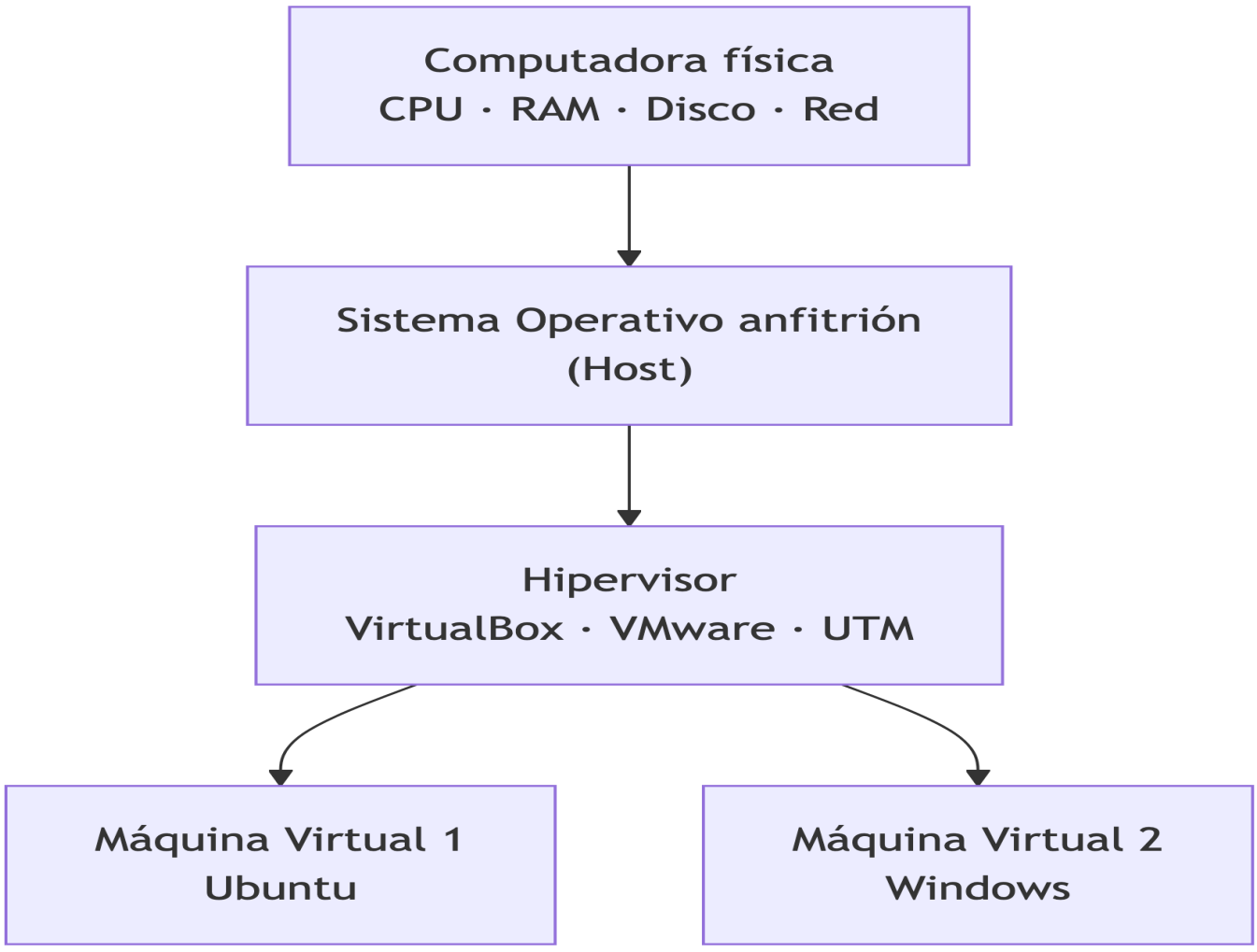
- Explicar qué es la virtualización y por qué se utiliza.
- Diferenciar **host**, **guest**, **máquina virtual** e **hipervisor**.
- Comprender cómo CPU, RAM, almacenamiento y red físicos se presentan como recursos virtuales.
- Diferenciar **virtualización** de **emulación**.
- Reconocer la importancia de la arquitectura **x86-64** y **ARM64**.
- Identificar alternativas gratuitas actuales para Windows y macOS.
- Seleccionar una herramienta de virtualización de acuerdo con el equipo disponible.
- Comprender qué es una imagen ISO, un disco virtual y un snapshot.
- Diseñar la configuración básica de una máquina virtual antes de instalarla.

---

# 298 1. ¿Qué es la virtualización?

La **virtualización** permite crear una representación lógica de recursos informáticos físicos.

En lugar de que un único Sistema Operativo utilice directamente todos los recursos del equipo, un software de virtualización puede proporcionar recursos a una o varias **máquinas virtuales (VM)**.



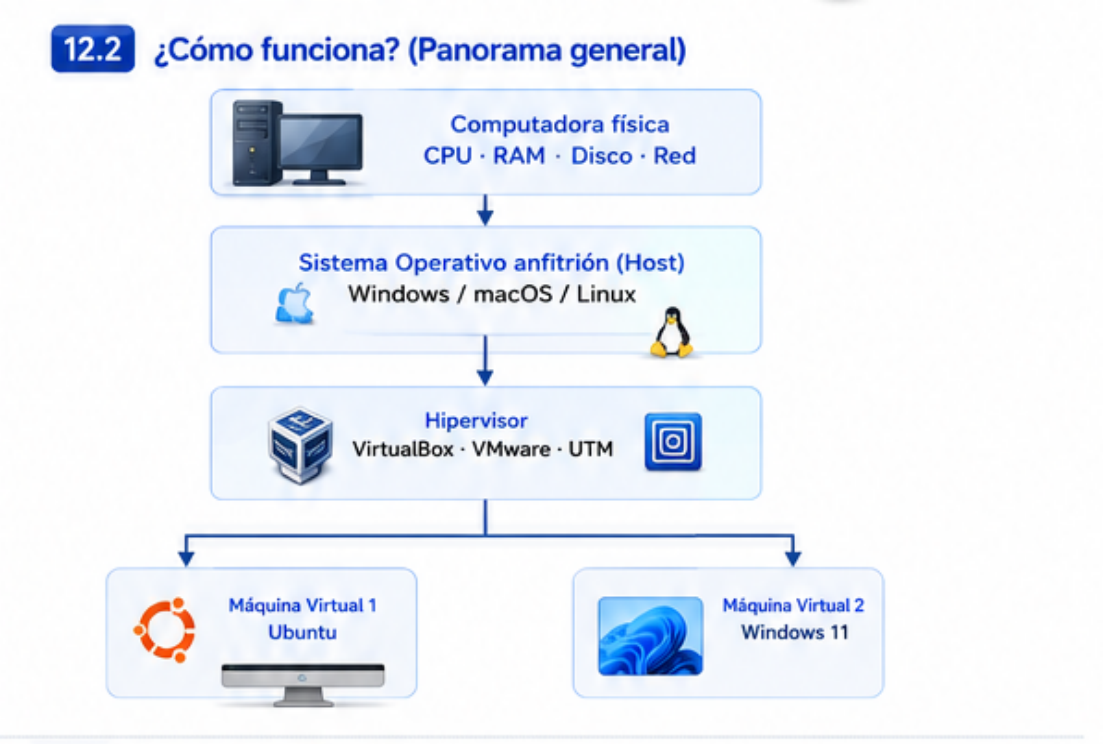



Figura 298.1: Relación entre hardware físico, host, hipervisor y máquinas virtuales.

**Primera analogía**

Pensemos en un edificio.
El **hardware físico** es el edificio completo. El hipervisor divide y administra espacios. Cada **máquina virtual** es como un apartamento: tiene sus propios recursos y puede funcionar de manera relativamente independiente, aunque todos comparten la misma infraestructura física.

# 299 2. Cuatro conceptos que no debemos confundir

| Concepto | Significado |
|---|---|
| **Host** | Computadora y Sistema Operativo principal donde ejecutamos la virtualización |
| **Guest** | Sistema Operativo instalado dentro de una máquina virtual |
| **VM** | Computadora virtual que recibe CPU, RAM, disco, red y otros dispositivos virtuales |
| **Hipervisor** | Software o capa que crea y administra las máquinas virtuales |

## 299.1 Ejemplo

Supongamos que tenemos una Mac con macOS y ejecutamos Ubuntu mediante una VM:

```
Mac física        → Hardware
macOS             → Host
VMware/UTM        → Hipervisor
Máquina Ubuntu    → VM
Ubuntu            → Guest
```

> ! Pregunta para el grupo
>
> Si utilizamos una computadora con Windows 11 para ejecutar una máquina virtual con Ubuntu:
> **¿Cuál es el host y cuál es el guest?**

# 300 3. ¿De dónde salen los recursos de la VM?

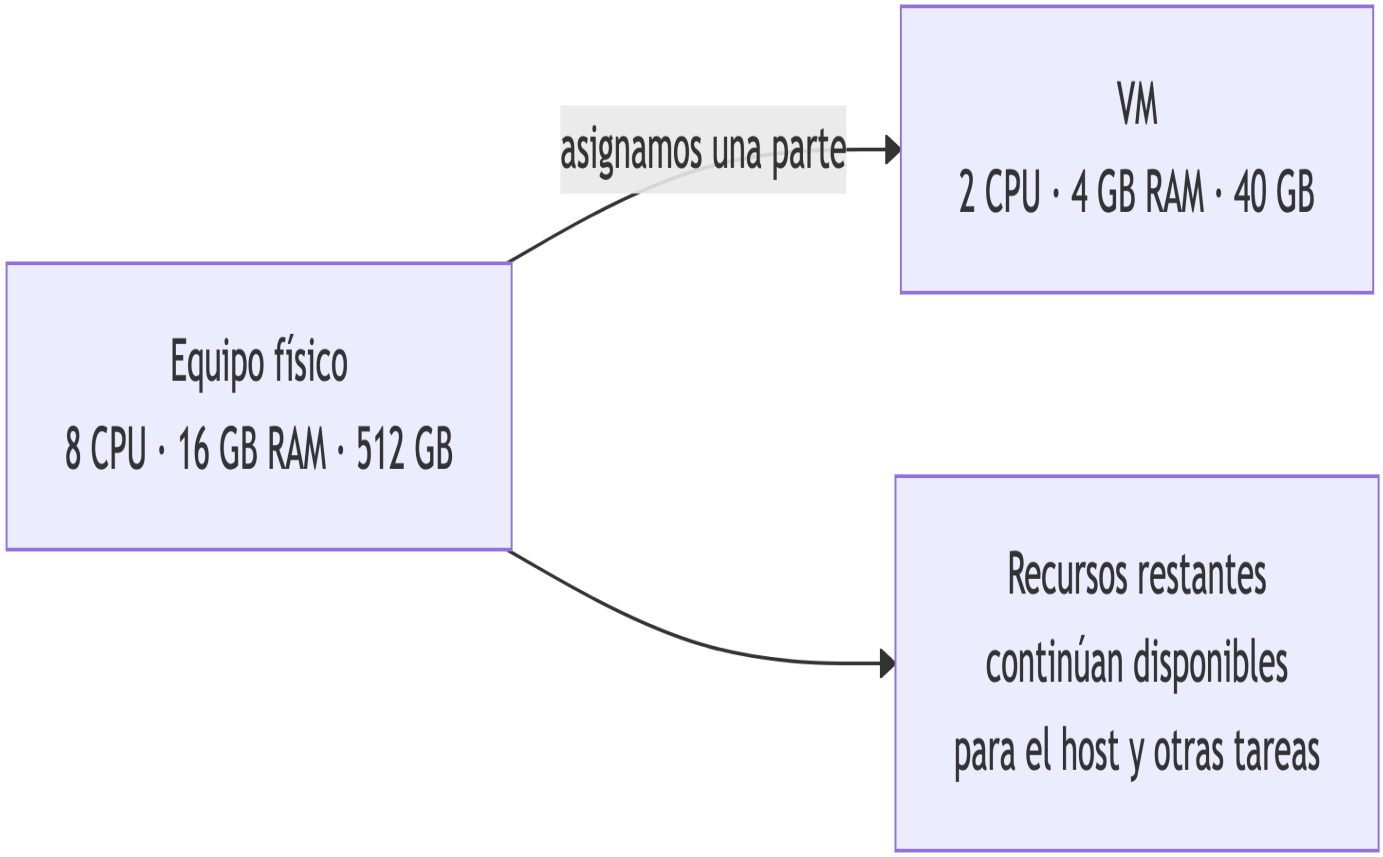
La máquina virtual **no crea recursos físicos de la nada**.

Si nuestro equipo tiene:

```
CPU:    8 núcleos
RAM:    16 GB
Disco:  512 GB
```

podríamos configurar una VM con:

```
CPU virtual:     2 núcleos
RAM virtual:     4 GB
Disco virtual:  40 GB
```




> ⚠ No debemos entregar todo a la VM
>
> El Sistema Operativo anfitrión también necesita CPU y RAM. Asignar demasiados recursos a una VM puede hacer lento al host y, paradójicamente, empeorar toda la experiencia.

# 301 4. Una VM no es simplemente otra ventana

Cuando abrimos una VM vemos una ventana, pero detrás existe una computadora virtual completa.

El hipervisor puede presentar al guest:

- procesadores virtuales;
- memoria RAM;
- almacenamiento;
- tarjeta de red;
- controlador gráfico;
- dispositivos USB;
- firmware virtual;
- otros dispositivos.

El guest utiliza esos dispositivos como si pertenecieran a una computadora real.

---

# 302 5. ¿Para qué virtualizamos?

La virtualización resulta especialmente útil para:

- aprender otros Sistemas Operativos;
- practicar administración sin modificar el equipo principal;
- probar software;
- construir laboratorios;
- simular servidores;
- realizar pruebas de redes;
- crear entornos aislados;
- probar configuraciones antes de utilizarlas en producción;
- ejecutar aplicaciones que requieren otro Sistema Operativo.

En nuestro curso

Una VM funciona como un **laboratorio dentro de nuestra computadora**. Podemos experimentar, equivocarnos, reinstalar y volver a intentar sin tener que modificar directamente el Sistema Operativo principal.

# 303 6. ¿Qué ocurre si algo sale mal?

Una de las ventajas de una VM es el aislamiento.

Un error dentro del guest normalmente no significa que tengamos que reinstalar el host.

Además, muchas plataformas permiten crear **snapshots**.

## 303.1 Snapshot: el punto de guardado

Podemos imaginar un snapshot como el **punto de guardado de un videojuego**.

```
VM recién instalada
        ↓
[Snapshot A]
        ↓
Instalamos programas
        ↓
[Snapshot B]
        ↓
Hacemos una prueba peligrosa
        ↓
Algo sale mal
        ↓
Regresamos al Snapshot B
```

> ⚠ Snapshot no significa backup completo
>
> Un snapshot es muy útil para regresar rápidamente a un estado anterior de una VM, pero no debe confundirse con una estrategia completa de copias de seguridad.

---

# 304 7. ISO, disco virtual y Sistema Operativo

Estos conceptos suelen confundirse al comenzar.

## 304.1 Imagen ISO

Una **ISO** contiene una imagen de un medio de instalación.

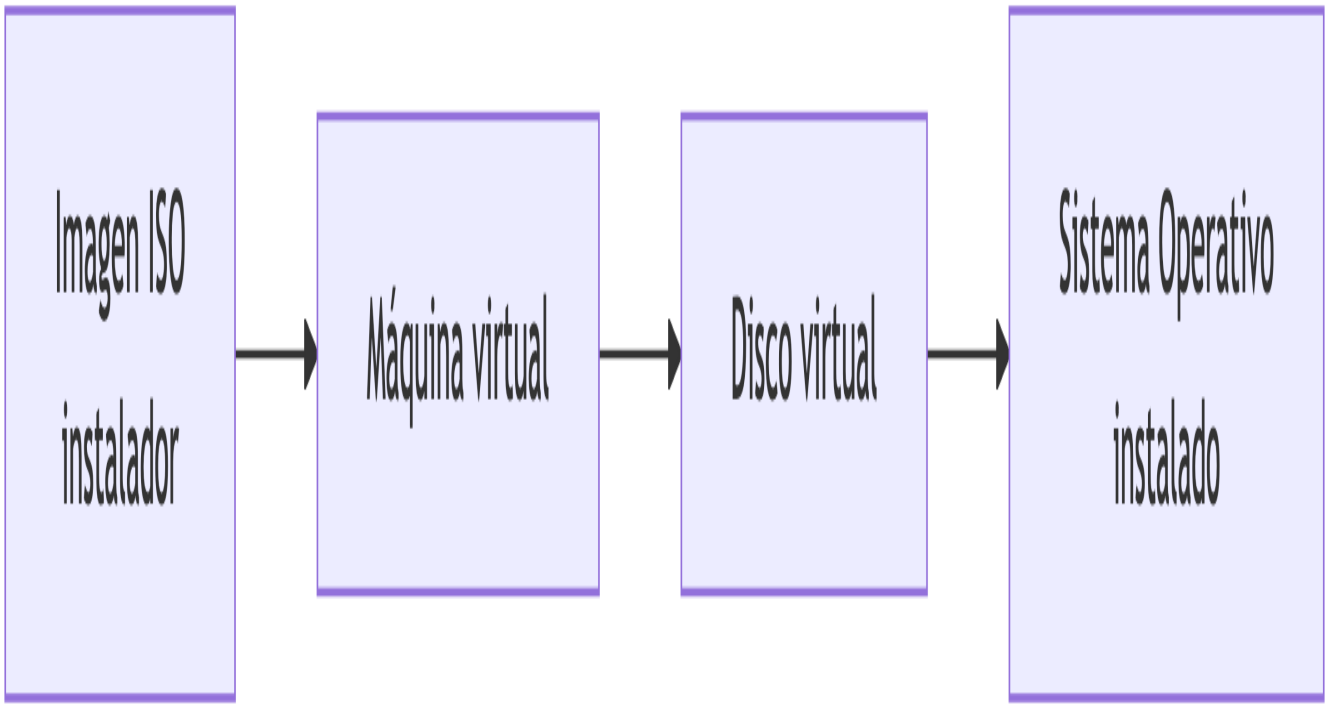
Podemos imaginarla como el equivalente digital del DVD o USB desde el cual instalaríamos un Sistema Operativo.

## 304.2 Disco virtual

Es un archivo que el guest interpreta como si fuera un disco duro o SSD.

En VirtualBox es común encontrar formatos como **VDI**; otras plataformas utilizan formatos diferentes.

## 304.3 La relación




Una comparación sencilla

**ISO = instalador.**
**Disco virtual = lugar donde quedará instalado el Sistema Operativo.**

# 305 8. Antes de descargar: x86-64 y ARM64

Figura 305.1: Comparación didáctica entre equipos x86-64 y ARM64 al elegir una máquina virtual.

Este es uno de los cambios más importantes respecto de muchos materiales antiguos de virtualización.

Actualmente podemos encontrar computadoras con diferentes arquitecturas.

## 305.1 x86-64

Es común en equipos con procesadores:

- Intel;
- AMD.

## 305.2 ARM64

Actualmente aparece, entre otros, en:

- Mac con Apple Silicon;
- algunos equipos Windows con procesadores ARM.

**!** La arquitectura importa

No debemos asumir que cualquier ISO funcionará de la misma manera en cualquier computadora.
Un Mac Apple Silicon utiliza arquitectura **ARM64**. Por eso conviene utilizar Sistemas Operativos invitados y herramientas compatibles con ARM64 cuando queremos virtualización eficiente.

# 306 9. Virtualización y emulación no son lo mismo

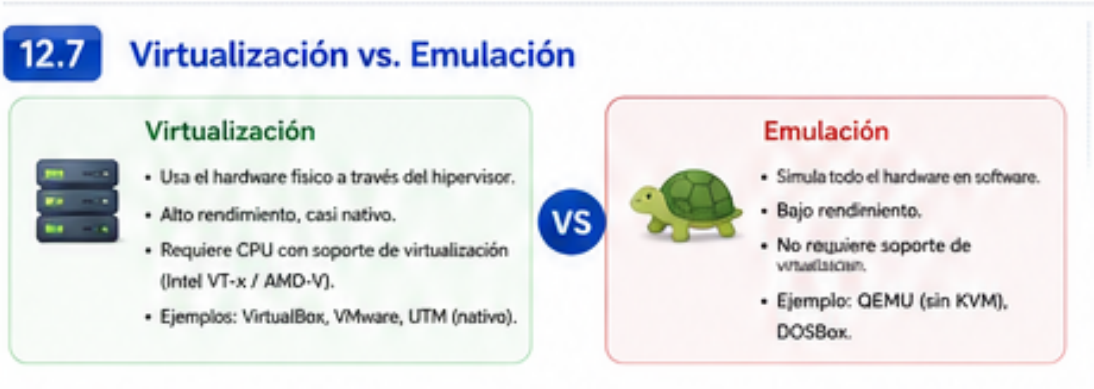



Figura 306.1: Virtualización y emulación: dos caminos distintos para ejecutar sistemas invitados.

## 306.1 Virtualización

Cuando host y guest utilizan arquitecturas compatibles, el software puede aprovechar directamente las capacidades de virtualización del procesador.

Resultado habitual:

**mejor rendimiento.**

## 306.2 Emulación

La emulación reproduce mediante software el comportamiento de otra arquitectura.

Ejemplo conceptual:

```
Host ARM64
    ↓
Emular x86-64
    ↓
Guest x86-64
```

Esto permite escenarios que la virtualización directa no permite, pero normalmente tiene un costo de rendimiento.

| Característica | Virtualización | Emulación |
|---|---|---|
| Arquitectura | Normalmente compatible | Puede ser diferente |
| Rendimiento | Generalmente mayor | Generalmente menor |
| Uso típico | VM cotidiana | Compatibilidad o sistemas de otra arquitectura |

# 307 10. ¿Qué programa podemos utilizar en 2026?

No existe una única respuesta correcta. La herramienta depende del Sistema Operativo anfitrión, arquitectura y objetivo.

| Herramienta | Windows | Mac Intel | Mac Apple Silicon | Comentario |
|---|---|---|---|---|
| **Oracle VirtualBox** | □ | □ | □ | Multiplataforma; verificar compatibilidad host/guest |
| **VMware Workstation** | □ | — | — | Alternativa sólida para Windows/Linux |
| **VMware Fusion** | — | □ | □ | Diseñado para macOS |
| **UTM** | — | □ | □ | Muy útil en Mac; permite virtualización y escenarios de emulación |

¿Cuál utilizaremos?

Para comprender los conceptos, cualquiera de estas herramientas puede servir. En clase podemos trabajar con **VirtualBox** como referencia y mostrar alternativas para estudiantes cuyo hardware o Sistema Operativo requiera otra opción.

# 308 11. Descargas oficiales

Los enlaces siguientes deben utilizarse como punto de partida. Evitemos descargar hipervisores o imágenes ISO desde sitios desconocidos.

## 308.1 Hipervisores

- Oracle VirtualBox
- VMware Workstation / VMware Fusion — instrucciones oficiales de Broadcom
- UTM para macOS

## 308.2 Sistemas Operativos

- Windows 11
- Ubuntu Desktop
- Debian
- Fedora
- Linux Mint

> **VMware en la actualidad**
>
> VMware Workstation Pro y VMware Fusion Pro cuentan con versiones gratuitas para usuarios personales, educativos y comerciales. Broadcom solicita una cuenta básica para acceder a las descargas.

# 309 12. ¿Qué debería elegir un estudiante?

## 309.1 Caso A: PC Windows Intel/AMD

Una opción sencilla:

```
Host: Windows 10/11
Arquitectura: x86-64
Herramienta: VirtualBox o VMware Workstation
Guest: Ubuntu x86-64
```

## 309.2 Caso B: Mac Intel

```
Host: macOS
Arquitectura: x86-64
Herramienta: VirtualBox o VMware Fusion
Guest: Linux x86-64
```

## 309.3 Caso C: Mac Apple Silicon

```
Host: macOS
Arquitectura: ARM64
Herramienta: VirtualBox, VMware Fusion o UTM
Guest recomendado: versión ARM64 compatible
```

> ⚠ Especialmente importante en Apple Silicon
>
> No basta con leer “Ubuntu”, “Windows” o “Linux”. Debemos verificar también **para qué arquitectura fue construida la imagen**.

---

# 310 13. ¿Cuánta RAM, CPU y disco asignamos?

No existe una cantidad universal.

Debemos considerar:

1. recursos físicos del host;
2. requisitos del guest;
3. aplicaciones que ejecutaremos;
4. cuántas VMs estarán encendidas simultáneamente.

## 310.1 Ejemplo didáctico

Equipo físico:

```
16 GB RAM
8 núcleos
512 GB almacenamiento
```

VM Linux para prácticas:

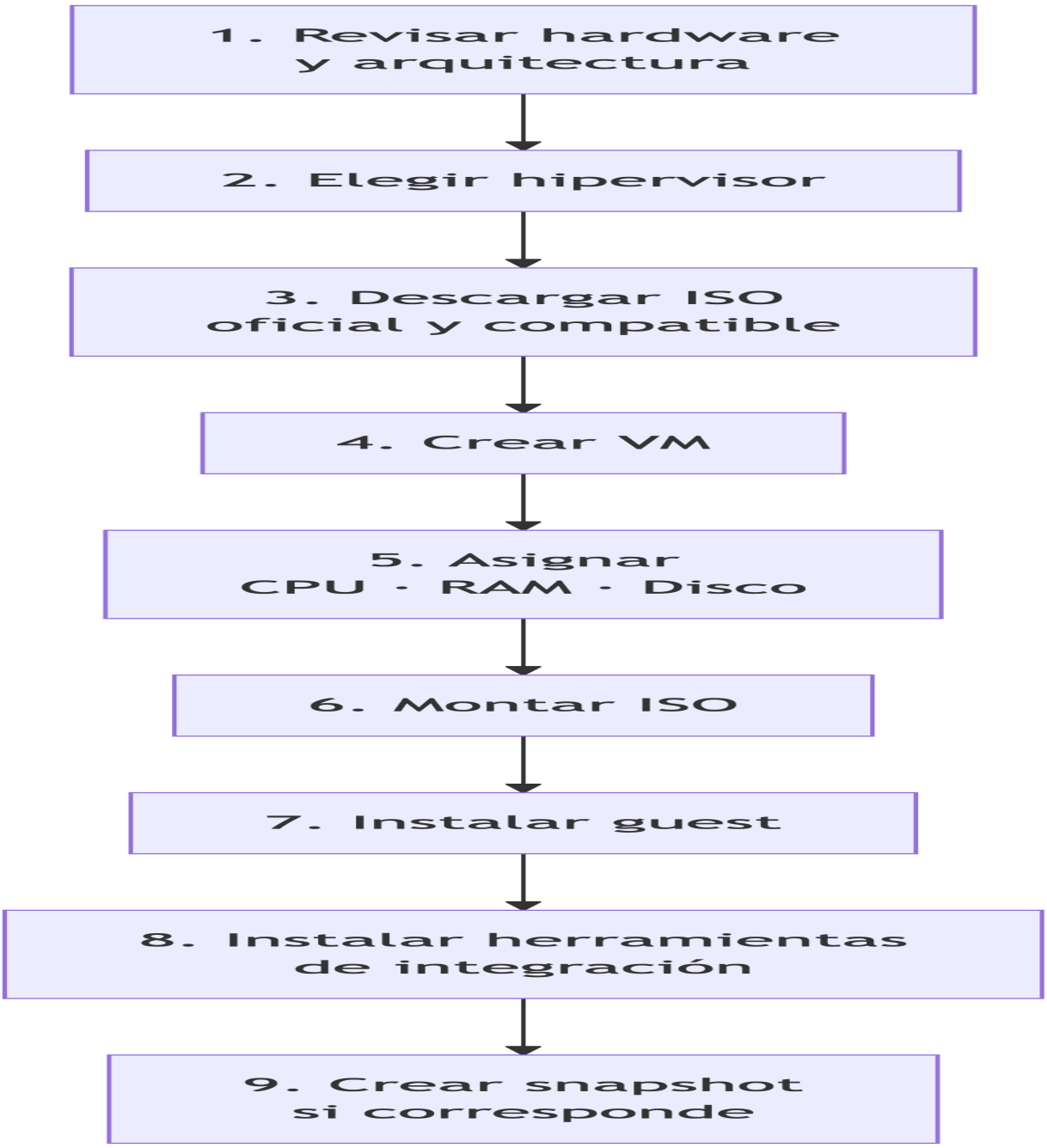
```
4 GB RAM
2 CPU virtuales
40 GB disco virtual
```

Esto **no es una regla obligatoria**. Es solamente un punto de partida razonable para un laboratorio sencillo.

---

# 311 14. Flujo para crear una máquina virtual

Antes de presionar botones, comprendamos el proceso.




**!** Primero diseñamos; después instalamos

La creación de una VM no debería comenzar haciendo clic al azar. Antes debemos saber: **qué quiero instalar + qué arquitectura tengo + qué recursos puedo asignar + qué herramienta es compatible.**

# 312 15. Actividad guiada: diseñemos una VM sin instalar nada

Tenemos una computadora:

```
Windows 11
Intel Core i5
16 GB RAM
512 GB SSD
120 GB disponibles
```

Queremos practicar comandos Linux utilizando Ubuntu.

Responde:

1. ¿Cuál será el host?
2. ¿Cuál será el guest?
3. ¿Qué arquitectura debemos verificar?
4. ¿Qué herramientas podríamos utilizar?
5. ¿Cuánta RAM asignarías?
6. ¿Cuántas CPU virtuales?
7. ¿Cuánto disco virtual?
8. ¿Qué archivo debemos descargar para iniciar la instalación?
9. ¿Crearías un snapshot después de instalar Ubuntu? ¿Por qué?

 Ver una posible solución

1. **Host:** Windows 11.
2. **Guest:** Ubuntu.
3. Debemos comprobar la arquitectura del procesador y descargar una ISO compatible; en este caso normalmente x86-64.
4. VirtualBox o VMware Workstation son opciones posibles.
5. Para una práctica sencilla podrían asignarse aproximadamente **4 GB de RAM**, siempre que el host conserve recursos suficientes.
6. **2 CPU virtuales** pueden ser un punto de partida.
7. Un disco virtual de aproximadamente **30–40 GB** puede ser suficiente para un laboratorio básico, sujeto a los requisitos de la versión instalada.
8. Una **imagen ISO oficial de Ubuntu**.

9. Sí. Un snapshot posterior a una instalación limpia permite regresar rápidamente a ese estado antes de experimentar.

La configuración no es única. Lo importante es **justificarla**.

---

# 313 16. Mini reto: tú eres el administrador

Analiza los tres equipos.

### 313.0.1 Equipo A

```
Windows 11
AMD Ryzen
8 GB RAM
256 GB SSD
```

### 313.0.2 Equipo B

```
Windows 11
Intel
32 GB RAM
1 TB SSD
```

### 313.0.3 Equipo C

```
MacBook Apple Silicon
16 GB RAM
512 GB SSD
```

Para cada equipo decide:

1. herramienta de virtualización;
2. guest que instalarías;
3. arquitectura del guest;
4. RAM;

5. CPU virtual;
6. disco;
7. si utilizarías snapshot.

Luego responde:

> **¿En cuál equipo tendrías mayor libertad para ejecutar varias VMs simultáneamente y por qué?**

Tiempo sugerido: **10 minutos**.

---

# 314 17. Errores comunes al comenzar

## 314.1 “Le asignaré toda la RAM a la VM”

Incorrecto. El host también necesita memoria.

## 314.2 “Descargué cualquier ISO de Internet”

Debemos utilizar fuentes oficiales y comprobar arquitectura.

## 314.3 “Si tengo Apple Silicon puedo ejecutar cualquier x86 como si fuera nativo”

No necesariamente. Debemos distinguir virtualización de emulación.

## 314.4 “Snapshot significa que ya tengo backup”

No. Son conceptos relacionados con recuperación, pero no equivalentes.

## 314.5 “VirtualBox es la única opción”

No. Es una herramienta muy conocida, pero existen alternativas.

---

# 315 18. Autoevaluación

1. ¿Qué es una máquina virtual?
2. ¿Qué función cumple un hipervisor?
3. ¿Qué diferencia existe entre host y guest?
4. ¿De dónde obtiene RAM una VM?
5. ¿Qué es una imagen ISO?
6. ¿Qué diferencia existe entre ISO y disco virtual?
7. ¿Qué es un snapshot?
8. ¿Por qué un snapshot no sustituye necesariamente un backup?
9. ¿Qué significa x86-64?
10. ¿Qué significa ARM64?
11. ¿Por qué la arquitectura es especialmente importante en Apple Silicon?
12. ¿Cuál es la diferencia conceptual entre virtualización y emulación?
13. Menciona dos alternativas a VirtualBox.
14. ¿Qué criterios utilizarías para decidir cuánta RAM asignar a una VM?
15. ¿Por qué no conviene descargar imágenes ISO desde sitios desconocidos?

---

# 316 19. Para recordar

> **Virtualizar no significa crear recursos nuevos: significa administrar y presentar recursos físicos de forma virtual.**

Antes de crear una VM debemos responder:

```
¿Qué quiero instalar?
        ↓
¿Qué arquitectura tiene mi host?
        ↓
¿Qué arquitectura necesita el guest?
        ↓
¿Qué hipervisor es compatible?
        ↓
¿Cuántos recursos puedo asignar?
        ↓
¿Qué ISO debo descargar?
```

---

# 317 20. Síntesis visual del capítulo

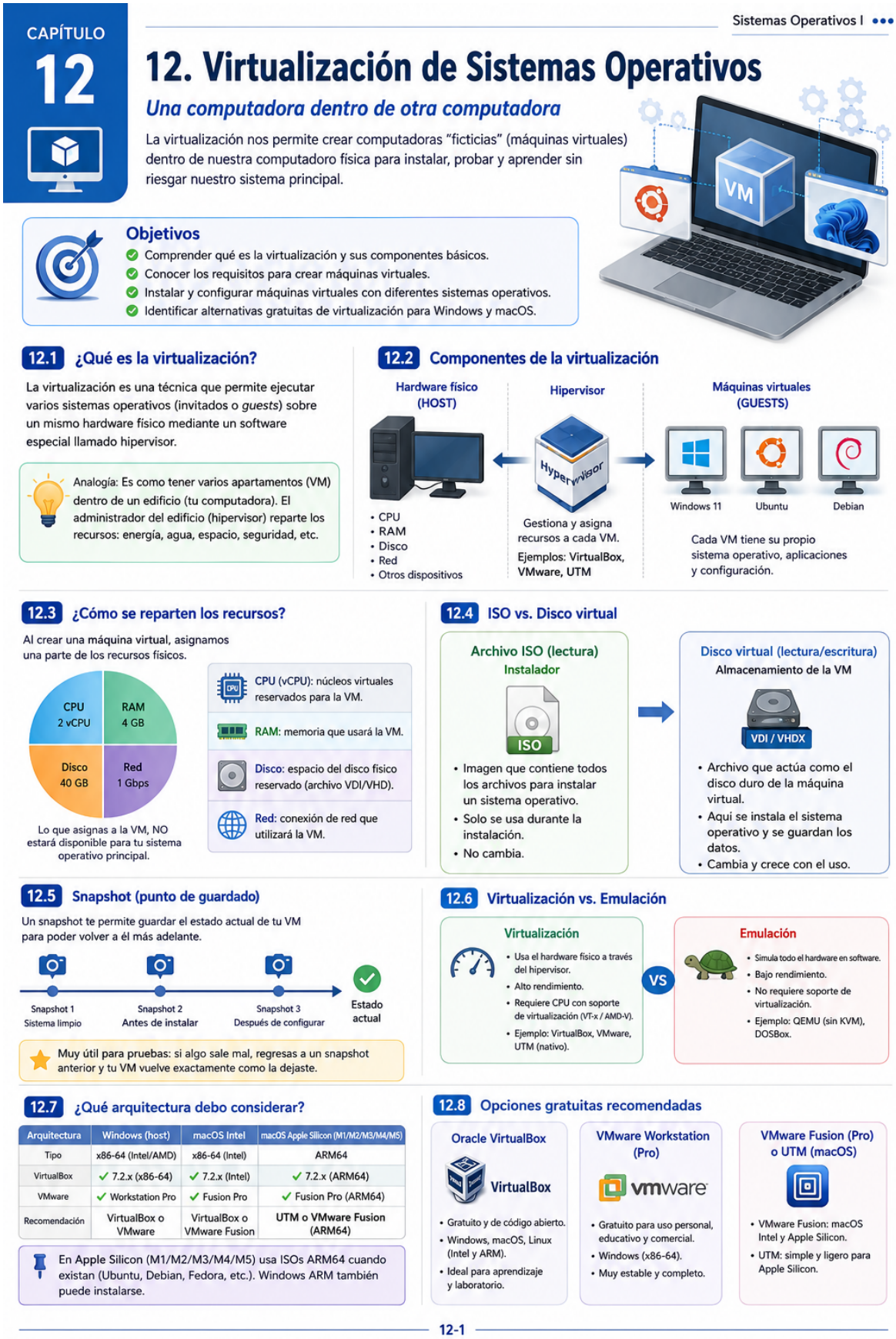



Figura 317.1: Síntesis visual del capítulo 12: virtualización de Sistemas Operativos.

La síntesis visual debe permitir reconstruir el tema completo mediante cinco ideas:

```
COMPUTADORA FÍSICA
        ↓
HOST
        ↓
HIPERVISOR
        ↓
MÁQUINA VIRTUAL
        ↓
GUEST
```

y relacionarlas con:

```
CPU · RAM · DISCO · RED
ISO · DISCO VIRTUAL · SNAPSHOT
x86-64 · ARM64
VIRTUALIZACIÓN · EMULACIÓN
```

> **i** Síntesis en una frase
>
> **Una máquina virtual es una computadora definida por software que utiliza recursos administrados del hardware físico para ejecutar su propio Sistema Operativo.**

---

# 318 Conexión con la siguiente clase

Ahora comprendemos qué es una máquina virtual, qué recursos utiliza y cómo seleccionar una plataforma.

El siguiente paso natural será **crear una VM real**, instalar un Sistema Operativo invitado y observar cómo se comportan CPU, memoria, almacenamiento y red durante la ejecución.

---

# 319 Referencias y actualización

Material base de clase:

- Presentación **Virtualización de Sistemas Operativos**, edición 2025.

Documentación oficial recomendada:

- Oracle VirtualBox — documentación y descargas oficiales.
- Broadcom — VMware Workstation Pro y VMware Fusion Pro.
- UTM — documentación oficial.
- Microsoft — descargas oficiales de Windows.
- Ubuntu, Debian, Fedora y Linux Mint — sitios oficiales de distribución.

> Criterio editorial
>
> Las versiones de hipervisores y Sistemas Operativos cambian con frecuencia. Por ello, este capítulo prioriza **conceptos y enlaces oficiales** sobre números de versión que pueden quedar obsoletos.

# 320 Capítulo 13. Manejo de memoria: particiones y paginación

# 321 Capítulo 13. Manejo de memoria: particiones y paginación

## 321.1 ¿Cómo caben tantos programas en la memoria?

En una computadora podemos tener abierto el navegador, un editor de código, música, una videollamada y otros programas al mismo tiempo.

Todos necesitan **memoria RAM**.

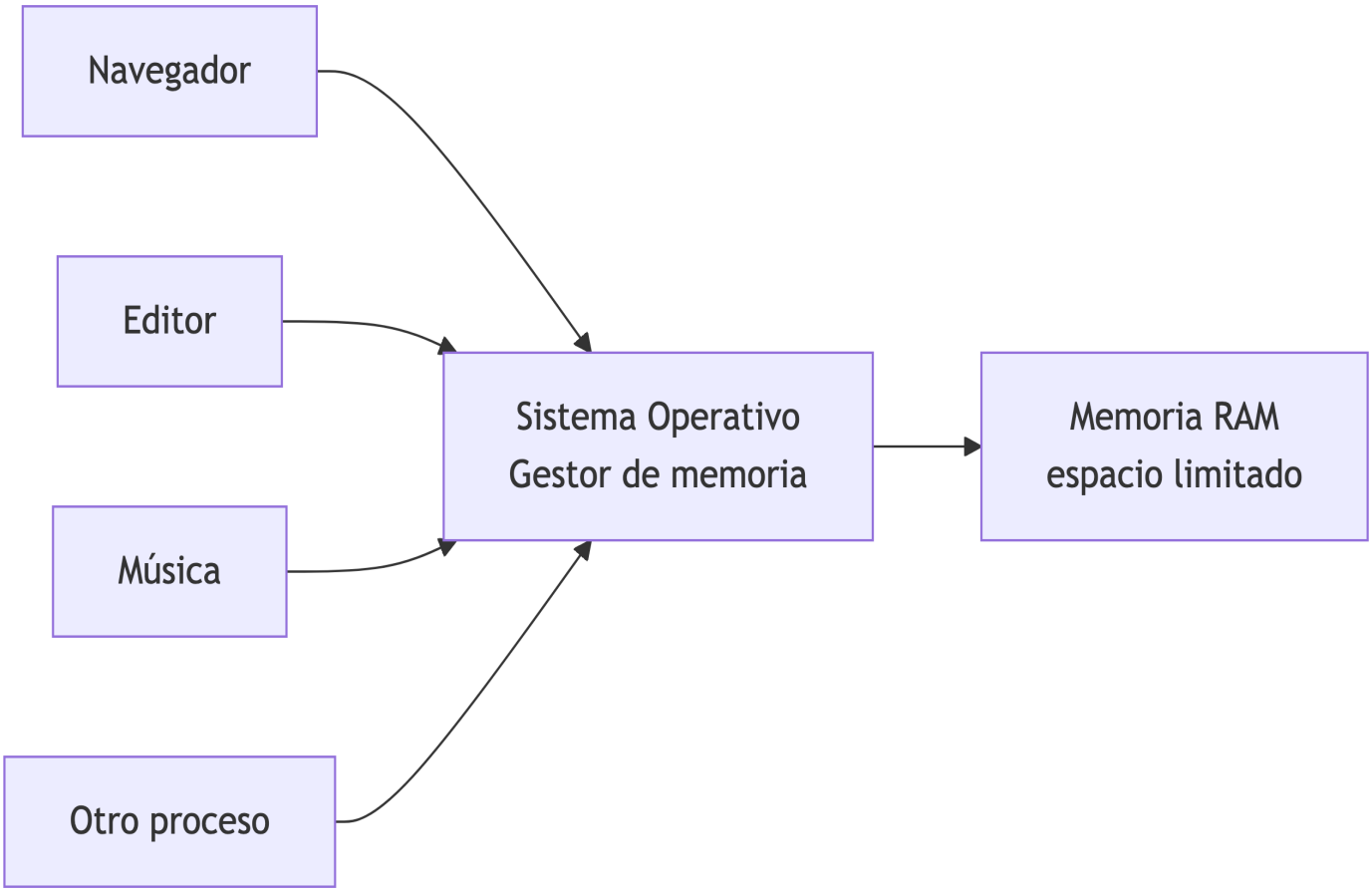
Pero la RAM es limitada.

Entonces aparece una pregunta fundamental:

> **¿Cómo decide el Sistema Operativo qué parte de la memoria utiliza cada proceso?**

La respuesta nos lleva a uno de los trabajos más importantes del Sistema Operativo: la **gestión de memoria**.

> Idea central
>
> El Sistema Operativo debe **asignar, proteger, controlar y liberar memoria** para que varios procesos puedan ejecutarse sin interferirse entre sí.


Navegador
Editor
Música
Otro proceso
Sistema Operativo
Gestor de memoria
Memoria RAM
espacio limitado

---

# 322 Objetivos de aprendizaje

Al finalizar este capítulo, el estudiante será capaz de:

- Explicar por qué la gestión de memoria es una función crítica del Sistema Operativo.
- Diferenciar una **dirección lógica** de una **dirección física**.
- Comprender el funcionamiento de las **particiones fijas y dinámicas**.
- Diferenciar **fragmentación interna** y **fragmentación externa**.
- Aplicar conceptualmente los métodos **First Fit, Best Fit y Worst Fit**.
- Explicar los conceptos de **página**, **marco** y **tabla de páginas**.
- Comprender cómo se traduce una dirección lógica en una dirección física.
- Explicar por qué la paginación permite utilizar memoria no contigua.
- Relacionar paginación con **memoria virtual**.
- Comparar particiones y paginación mediante situaciones prácticas.

---

# 323 1. La memoria RAM como recurso crítico

La memoria RAM almacena temporalmente instrucciones y datos que necesitan los procesos mientras se ejecutan.

Cuando un programa comienza a ejecutarse, no basta con tenerlo almacenado en el disco. Parte de su contenido debe encontrarse disponible en memoria.

El Sistema Operativo debe decidir:

1. **qué proceso recibe memoria;**
2. **cuánta memoria recibe;**
3. **dónde se coloca;**
4. **cómo se protege de otros procesos;**
5. **cuándo se libera.**

> **i** Una analogía: el estacionamiento
>
> Imaginemos que la RAM es un estacionamiento y los procesos son vehículos.
> El Sistema Operativo funciona como el encargado que decide **dónde estacionar cada vehículo**, evita que dos ocupen el mismo espacio y libera el lugar cuando un vehículo se retira.

## 323.1 ¿Por qué importa?

Una mala administración de memoria puede provocar:

- desperdicio de espacio;
- dificultad para cargar nuevos procesos;
- bajo rendimiento;
- errores de acceso;
- imposibilidad de aprovechar correctamente la memoria disponible.

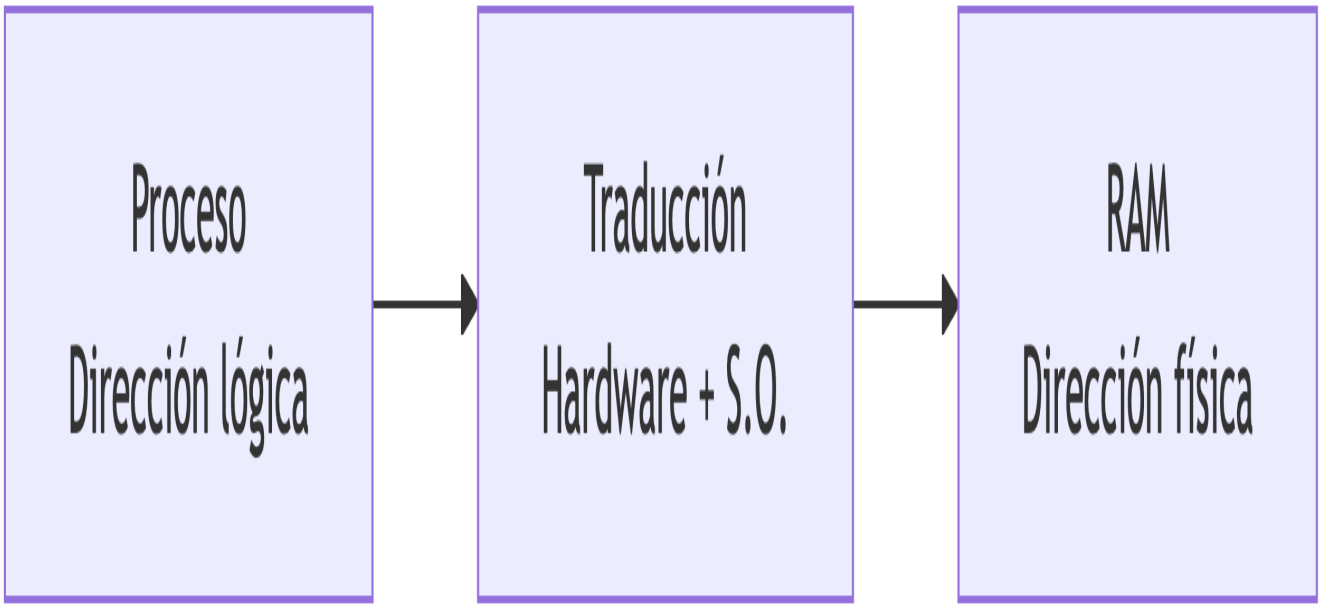
---

# 324 2. Dirección lógica y dirección física

Cuando un programa utiliza memoria, normalmente no trabaja directamente con la posición física real de la RAM.

Debemos distinguir dos conceptos.

| Dirección | Significado |
|---|---|
| **Lógica** | Dirección que utiliza o genera el programa |
| **Física** | Ubicación real correspondiente dentro de la memoria RAM |




> ! No debemos confundirlas
>
> El proceso puede pensar en una dirección dentro de **su propio espacio de memoria**, mientras el hardware y el Sistema Operativo determinan dónde se encuentra realmente esa infor-

mación en la RAM.

---

# 325 3. Primera estrategia: dividir la memoria en particiones

Una forma sencilla de administrar memoria consiste en dividirla en **bloques o particiones**.

Cada proceso ocupa una partición.

Existen dos enfoques principales:

```
PARTICIONES
    │
    ├── Fijas
    │
    └── Dinámicas
```

La diferencia parece pequeña, pero produce problemas de fragmentación diferentes.

---

# 326 4. Particiones fijas

En las **particiones fijas**, la memoria se divide previamente en bloques de tamaño definido.

Supongamos una memoria disponible de **8 MB**, dividida en cuatro particiones de **2 MB**:

```
RAM
┌──────────────────────┐
│ Partición 1: 2MB     │
├──────────────────────┤
│ Partición 2: 2MB     │
├──────────────────────┤
│ Partición 3: 2MB     │
├──────────────────────┤
│ Partición 4: 2MB     │
└──────────────────────┘
```

Ahora llegan tres procesos:

```
Proceso A = 1.2 MB
Proceso B = 1.8 MB
Proceso C = 0.9 MB
```

Podemos asignar uno a cada partición.

Pero observemos lo que ocurre:

| Partición | Tamaño | Proceso | Utilizado | Desperdiciado |
|---|---|---|---|---|
| 1 | 2 MB | A | 1.2 MB | 0.8 MB |
| 2 | 2 MB | B | 1.8 MB | 0.2 MB |
| 3 | 2 MB | C | 0.9 MB | 1.1 MB |
| 4 | 2 MB | Libre | — | — |

Los espacios que sobran **dentro de una partición ocupada** no pueden ser aprovechados fácilmente por otro proceso.

Eso se llama **fragmentación interna**.

⚠ Fragmentación interna

Existe memoria asignada a un proceso que realmente **no está siendo utilizada**, pero permanece encerrada dentro del bloque asignado.

## 326.1 Ventaja principal

- Es un esquema sencillo de administrar.

## 326.2 Problema principal

- Puede desperdiciar bastante memoria debido a la fragmentación interna.

---

# 327 5. Particiones dinámicas

¿Y si en lugar de construir bloques rígidos permitimos que cada proceso reciba aproximadamente el espacio que necesita?

Ese es el principio de las **particiones dinámicas**.

Supongamos:

```
RAM disponible = 16 MB

Proceso A = 4 MB
Proceso B = 6 MB
Proceso C = 2 MB
```

La memoria podría verse así:

```
┌──────────────────────┐
│ Proceso A   4 MB     │
├──────────────────────┤
│ Proceso B   6 MB     │
├──────────────────────┤
│ Proceso C   2 MB     │
├──────────────────────┤
│ Libre       4 MB     │
└──────────────────────┘
```

Hasta aquí parece una solución excelente.

Pero los procesos terminan.

Imaginemos que **Proceso B finaliza**:

```
┌──────────────────────┐
│ Proceso A   4 MB     │
├──────────────────────┤
│ LIBRE       6 MB     │
├──────────────────────┤
```

```
┌──────────────────────┐
│ Proceso C   2 MB     │
├──────────────────────┤
│ LIBRE       4 MB     │
└──────────────────────┘
```

Ahora tenemos **10 MB libres**, pero están separados en dos huecos.

Esto introduce otro problema.

---

# 328 6. Fragmentación interna y externa

Estos conceptos suelen confundirse.

## 328.1 Fragmentación interna

El desperdicio se encuentra **dentro del espacio asignado**.

```
Partición de 4 KB
┌──────────────────────┐
│ Proceso: 3 KB        │
│                      │
│ Desperdicio: 1 KB    │ ← dentro
└──────────────────────┘
```

## 328.2 Fragmentación externa

Existe suficiente memoria libre en total, pero se encuentra **dispersa entre espacios ocupados**.

```
┌──────────────┐
│ Proceso A    │
├──────────────┤
│ Libre 2 MB   │
├──────────────┤
│ Proceso B    │
├──────────────┤
│ Libre 3 MB   │
├──────────────┤
│ Proceso C    │
└──────────────┘
```

```
| Libre 1 MB    |
└───────────────┘
```

Tenemos:

```
2 + 3 + 1 = 6 MB libres
```

pero **no existe necesariamente un bloque contiguo de 6 MB**.

| Tipo | ¿Dónde está el desperdicio? | Ejemplo típico |
|---|---|---|
| **Interna** | Dentro del bloque asignado | Particiones fijas / último marco de una página |
| **Externa** | Entre bloques ocupados | Particiones dinámicas |

> Truco para recordarlo
>
> **Interna → adentro del bloque.**
> **Externa → afuera, entre los bloques.**

# 329 7. ¿Qué hueco debemos utilizar?

Con particiones dinámicas aparece otra pregunta:

> Si existen varios espacios libres, ¿en cuál colocamos el nuevo proceso?

Tres estrategias clásicas son:

- **First Fit**
- **Best Fit**
- **Worst Fit**

Supongamos que tenemos estos huecos:

```
10 KB   4 KB   20 KB   7 KB
```

y llega un proceso de:

```
6 KB
```

---

# 330 8. First Fit: el primer espacio suficiente

**First Fit** recorre la memoria y selecciona el **primer hueco que sea suficientemente grande**.

Para nuestro proceso de 6 KB:

```
Huecos: 10 KB | 4 KB | 20 KB | 7 KB
          ↑
       elegido
```

El primer hueco de 10 KB puede alojarlo.

Quedan:

```
10 - 6 = 4 KB
```

> **Idea de First Fit**
>
> No busca el hueco perfecto. Busca **el primero que funcione**.

---

# 331 9. Best Fit: el espacio más ajustado

**Best Fit** busca el hueco disponible más pequeño que todavía pueda alojar el proceso.

```
Huecos: 10 KB | 4 KB | 20 KB | 7 KB
                                 ↑
                              elegido
```

Para un proceso de 6 KB, el hueco de 7 KB es el más ajustado.

Queda:

```
7 - 6 = 1 KB
```

La idea parece eficiente porque deja poco sobrante en esa asignación, aunque puede producir huecos pequeños difíciles de reutilizar.

---

# 332 10. Worst Fit: el espacio más grande

**Worst Fit** hace lo contrario: selecciona el hueco más grande.

```
Huecos: 10 KB | 4 KB | 20 KB | 7 KB
                         ↑
                      elegido
```

El proceso de 6 KB entra en el hueco de 20 KB.

Quedan:

```
20 - 6 = 14 KB
```

La intención es conservar un espacio sobrante suficientemente grande para ser útil posteriormente.

---

# 333 11. Comparando los tres métodos

Para:

```
Huecos = 10, 4, 20, 7 KB
Proceso = 6 KB
```

| Método | Hueco elegido | Espacio restante |
|---|---|---|
| First Fit | 10 KB | 4 KB |
| Best Fit | 7 KB | 1 KB |
| Worst Fit | 20 KB | 14 KB |

> ! No existe magia
>
> Estos algoritmos toman decisiones distintas sobre el mismo estado de memoria. La elección puede influir en cómo quedan distribuidos los huecos para futuras solicitudes.

## 333.1 Actividad rápida

Tenemos:

```
Huecos libres: 5 KB, 12 KB, 8 KB, 20 KB
Proceso nuevo: 7 KB
```

Determina qué hueco seleccionaría:

1. First Fit.
2. Best Fit.
3. Worst Fit.

> □ Solución propuesta
>
> - **First Fit:** 12 KB, porque es el primer hueco suficientemente grande.
> - **Best Fit:** 8 KB, porque es el hueco válido más pequeño.
> - **Worst Fit:** 20 KB, porque es el hueco más grande.

Después de colocar un proceso de 7 KB quedarían, respectivamente:

- First Fit → 5 KB libres en ese hueco.
- Best Fit → 1 KB libre.
- Worst Fit → 13 KB libres.

---

# 334 12. El problema de exigir memoria contigua

Las particiones tienen una limitación importante: un proceso necesita tradicionalmente un bloque continuo de memoria.

Supongamos:

```
Libre total = 12 KB
```

pero distribuido así:

```
4 KB + 3 KB + 5 KB
```

Un proceso de **10 KB** no cabe en ninguno de los huecos individualmente.

Entonces surge una idea poderosa:

> **¿Y si dividimos el proceso en pequeñas partes y permitimos colocarlas en distintos lugares de la RAM?**

Esa idea nos lleva a la **paginación**.

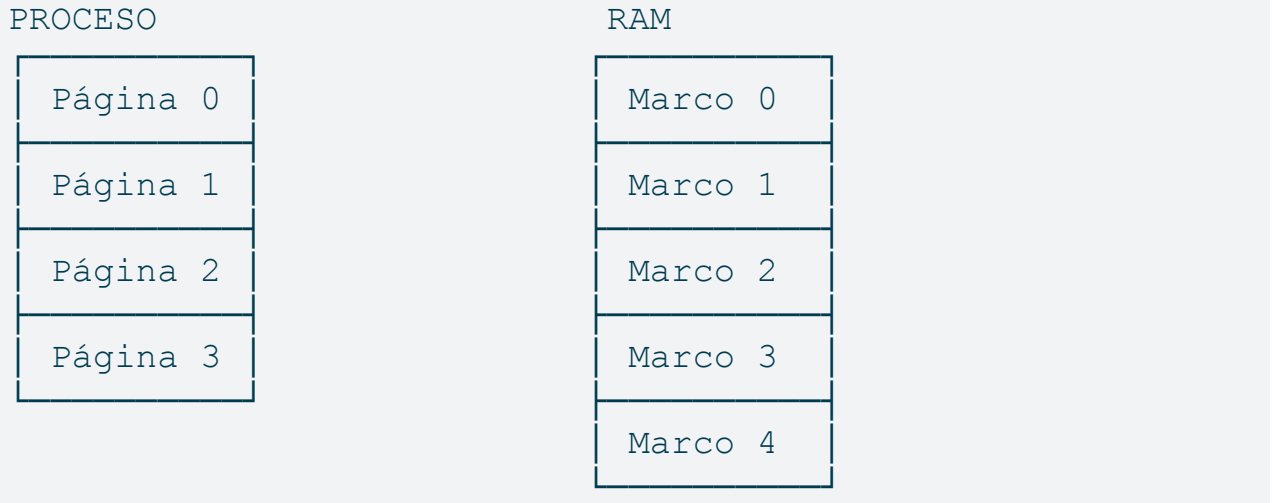
---

# 335 13. Introducción a la paginación

La paginación divide:

- el espacio lógico del proceso en **páginas**;
- la memoria física en **marcos**.

Y ambos tienen el **mismo tamaño**.




Dos palabras fundamentales

**Página = unidad lógica del proceso.**
**Marco = unidad física de la RAM.**

---

# 336 14. La gran ventaja: memoria no contigua

Supongamos un proceso de **4 páginas**.

No es necesario colocarlas juntas.

Podrían distribuirse así:

```
Página 0 → Marco 5
Página 1 → Marco 2
Página 2 → Marco 8
Página 3 → Marco 1
```

Visualmente:

```
PROCESO                        MEMORIA FÍSICA
P0 ──────────────────────────→ M5
P1 ──────────────────→         M2
P2 ──────────────────────────→ M8
P3 ──────────→                 M1
```

Para el proceso, sus páginas forman parte de un mismo espacio lógico.

En la RAM, pueden encontrarse en lugares separados.

> ❗ Aquí está el cambio de paradigma
>
> Con paginación, **un proceso ya no necesita ocupar un único bloque físico contiguo de memoria**.

---

# 337 15. Ejemplo numérico de paginación

Tenemos un proceso de:

```
10 KB
```

y utilizamos páginas de:

```
1 KB
```

Entonces:

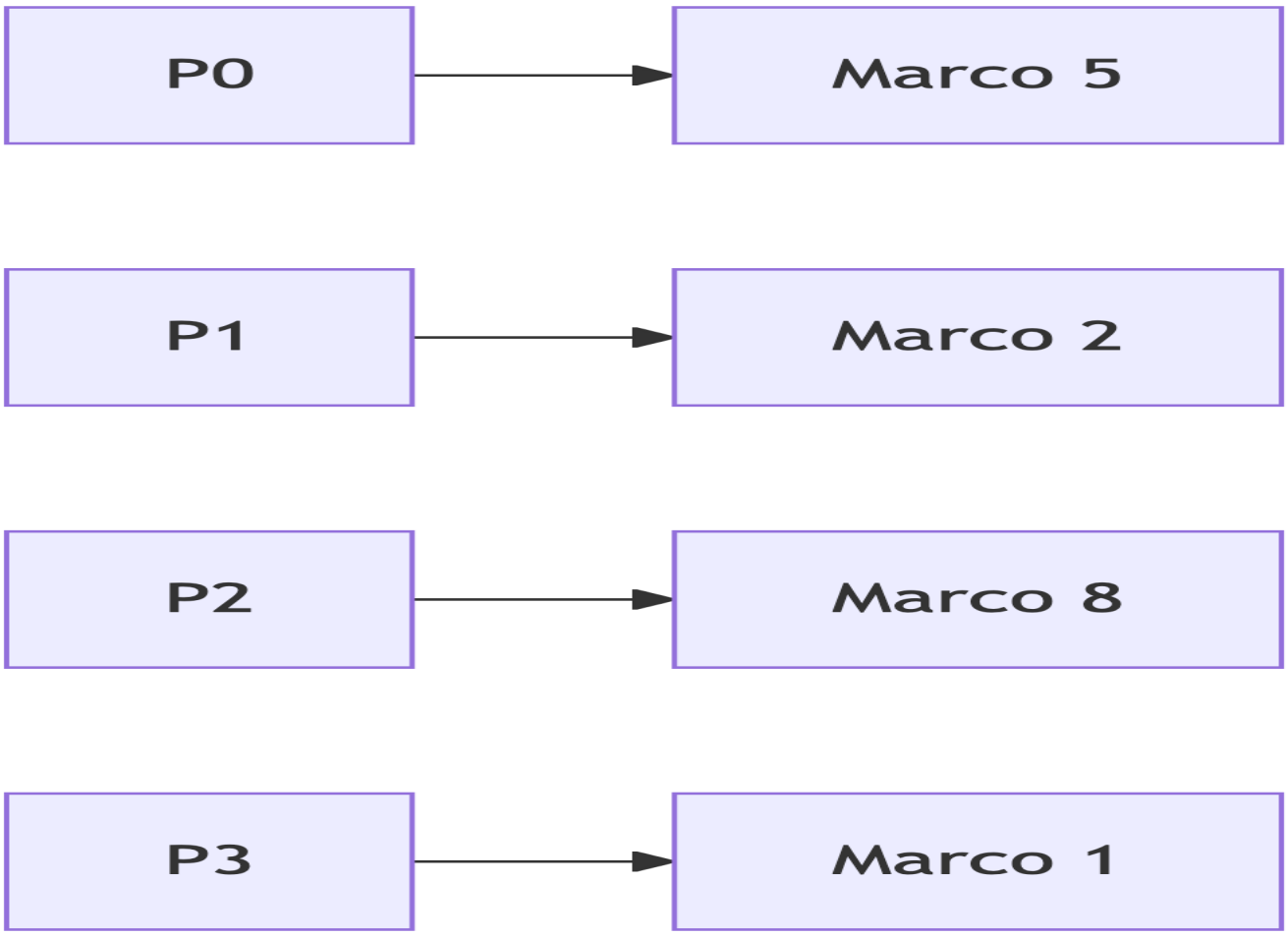
```
10 KB ÷ 1 KB = 10 páginas
```

El proceso tendrá:

```
P0, P1, P2, P3, P4, P5, P6, P7, P8, P9
```

La RAM también se divide en marcos de **1 KB**.

Cada página puede colocarse en cualquier marco disponible.




El diagrama muestra solo algunas páginas para concentrarnos en el mecanismo.

---

# 338 16. ¿Cómo sabe el Sistema Operativo dónde está cada página?

Necesitamos una estructura que mantenga la relación entre:

```
Página lógica ↔ Marco físico
```

Esa estructura es la **tabla de páginas**.

Ejemplo:

| Página | Marco |
|---|---|
| 0 | 5 |
| 1 | 2 |
| 2 | 8 |
| 3 | 1 |

Si el proceso solicita información de la **Página 2**, la tabla indica:

```
Página 2 → Marco 8
```

> **La tabla funciona como un mapa**
>
> El proceso trabaja con páginas. La tabla de páginas permite encontrar el **marco físico** donde se encuentra cada una.

---

# 339 17. Dirección lógica = página + desplazamiento

Una dirección lógica paginada puede pensarse como dos componentes:

```
┌──────────────┬──────────────────────┐
│    Página    │ Desplazamiento       │
└──────────────┴──────────────────────┘
```

La **página** indica qué página necesitamos.

El **desplazamiento** indica la posición dentro de esa página.

Supongamos:

```
Dirección lógica:
Página = 2
Desplazamiento = 100
```

La tabla contiene:

```
Página 2 → Marco 5
```

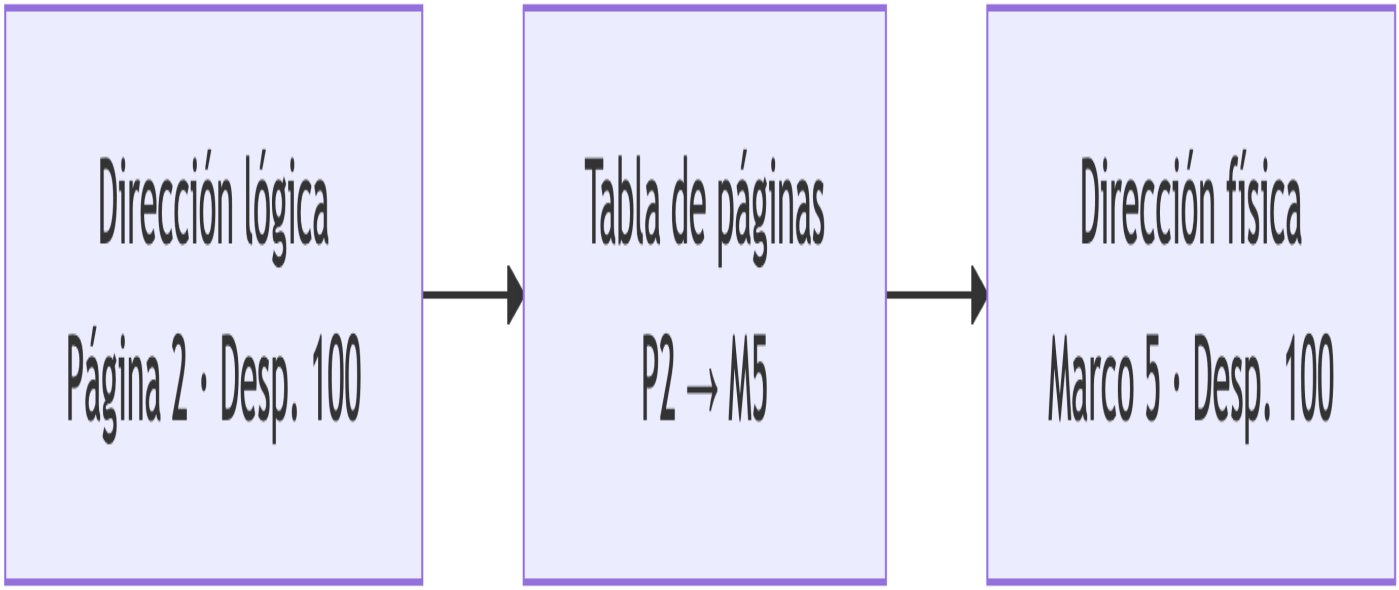
Entonces la dirección física será conceptualmente:

```
Marco = 5
Desplazamiento = 100
```

Observe algo importante:

> **El desplazamiento no cambia.**

Lo que cambia mediante la tabla es la página lógica por su marco físico correspondiente.


Dirección lógica
Página 2 · Desp. 100
Tabla de páginas
P2 → M5
Dirección física
Marco 5 · Desp. 100

# 340 18. Calculando una dirección física

Podemos llevar el concepto un paso más adelante.

Si cada página y marco mide **1 KB**, entonces:

```
1 KB = 1024 bytes
```

Tenemos:

```
Página lógica = 2
Desplazamiento = 100
Página 2 → Marco 5
```

La dirección física numérica puede calcularse como:

```
Dirección física = (marco × tamaño del marco) + desplazamiento
```

Entonces:

```
Dirección física = (5 × 1024) + 100
                 = 5120 + 100
                 = 5220
```

Por lo tanto:

> La dirección lógica **(Página 2, desplazamiento 100)** corresponde a la dirección física **5220**.

> ⚠ No memorices sin comprender
>
> La fórmula tiene sentido porque primero llegamos al inicio del **Marco 5** y luego avanzamos **100 bytes** dentro de él.

## 340.1 Practiquemos

Página de **1 KB**. La tabla indica:

```
Página 3 → Marco 7
```

La dirección lógica es:

```
Página = 3
Desplazamiento = 200
```

¿Cuál es la dirección física?

 Solución propuesta

Como:

```
1 KB = 1024 bytes
```

y:

```
Página 3 → Marco 7
```

calculamos:

```
Dirección física = (7 × 1024) + 200
                 = 7168 + 200
                 = 7368
```

La dirección física es **7368**.

---

# 341 19. ¿La paginación elimina toda fragmentación?

No.

La paginación evita el problema clásico de **fragmentación externa**, porque las páginas pueden colocarse en marcos separados.

Sin embargo, todavía puede existir **fragmentación interna**, especialmente en el último marco utilizado por un proceso.

Ejemplo:

```
Proceso = 10 KB
Página  = 4 KB
```

Necesitamos:

```
Página 0 = 4 KB
Página 1 = 4 KB
Página 2 = 2 KB utilizados
```

Pero el último marco mide 4 KB.

```
Último marco:
┌──────────────────┐
│ 2 KB utilizados  │
├──────────────────┤
│ 2 KB sin utilizar│ ← fragmentación interna
└──────────────────┘
```

Por tanto:

```
Fragmentación interna = 2 KB
```

**!** Una precisión importante

La paginación **resuelve la necesidad de encontrar un gran bloque contiguo**, pero no significa que todo byte de cada marco asignado será aprovechado.

---

# 342 20. Ventajas y costos de la paginación

## 342.1 Ventajas

- Evita la fragmentación externa clásica.
- Permite memoria física **no contigua**.
- Facilita una utilización más flexible de los marcos disponibles.
- Es una base importante para implementar **memoria virtual**.

## 342.2 Costos o desventajas

- Puede existir fragmentación interna.
- Cada proceso necesita información de traducción.
- Las tablas de páginas consumen memoria.
- La traducción de direcciones agrega trabajo al sistema.

| Particiones | Paginación |
|---|---|
| Conceptualmente simples | Más flexible |
| Pueden requerir bloques contiguos | Permite marcos no contiguos |
| Dinámicas: fragmentación externa | Evita fragmentación externa clásica |
| Fijas: fragmentación interna | Puede presentar fragmentación interna |
| Menor complejidad conceptual | Requiere tabla de páginas |

# 343 21. De la paginación a la memoria virtual

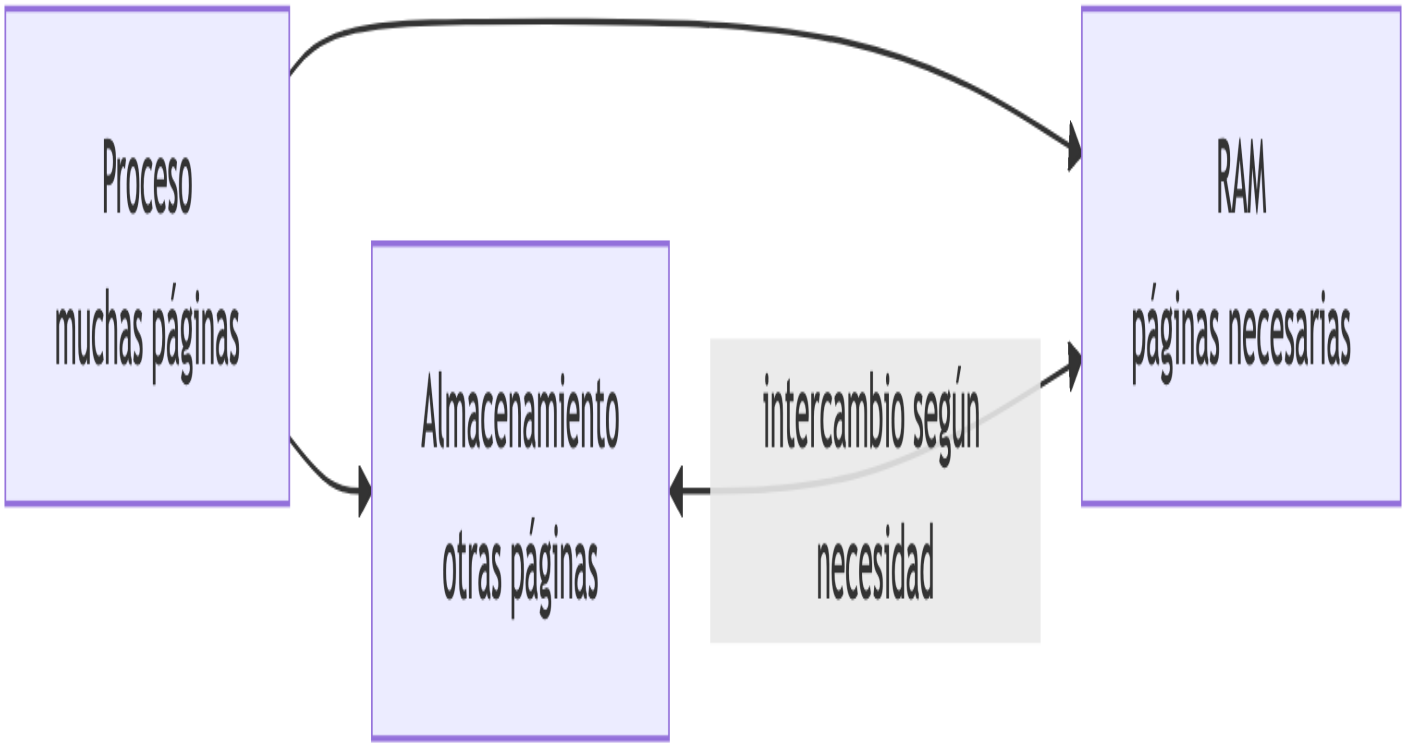
Hasta ahora hemos supuesto que las páginas del proceso están en RAM.

Pero aparece una posibilidad aún más interesante:

> **¿Qué ocurre si un programa necesita más memoria de la que podemos mantener físicamente en RAM en ese momento?**

La **memoria virtual** permite extender el espacio disponible utilizando almacenamiento secundario y mecanismos administrados por el Sistema Operativo.

En un sistema paginado, algunas páginas pueden estar en RAM mientras otras no se encuentran cargadas en ese instante.



**La idea que debemos conservar**

La memoria virtual permite que el espacio que observa un proceso no tenga que corresponder completamente, en todo momento, con la cantidad de RAM física que ocupa.

# 344 22. Caso práctico integrador

Tenemos una RAM disponible de **16 KB**.

Llegan tres procesos:

```
Proceso X = 4 KB
Proceso Y = 6 KB
Proceso Z = 2 KB
```

## 344.1 Escenario A: particiones dinámicas

Inicialmente podrían almacenarse de forma contigua:

```
┌──────────────────────┐
│ Proceso X   4 KB     │
├──────────────────────┤
│ Proceso Y   6 KB     │
├──────────────────────┤
│ Proceso Z   2 KB     │
├──────────────────────┤
│ Libre       4 KB     │
└──────────────────────┘
```

Si posteriormente termina el Proceso Y:

```
┌──────────────────────┐
│ Proceso X   4 KB     │
├──────────────────────┤
│ Libre       6 KB     │
├──────────────────────┤
│ Proceso Z   2 KB     │
├──────────────────────┤
│ Libre       4 KB     │
└──────────────────────┘
```

Tenemos dos huecos separados.

## 344.2 Escenario B: paginación de 2 KB

La RAM de 16 KB contiene:

```
16 ÷ 2 = 8 marcos
```

Los procesos requieren:

```
X = 4 KB → 2 páginas
Y = 6 KB → 3 páginas
Z = 2 KB → 1 página
```

En total:

```
2 + 3 + 1 = 6 marcos utilizados
```

Las páginas podrían distribuirse, por ejemplo:

| Página | Marco |
|---|---|
| X0 | 0 |
| Y0 | 1 |
| Z0 | 2 |
| X1 | 4 |
| Y1 | 6 |
| Y2 | 7 |

No necesitan estar juntas.

> **Pregunta para discutir**
>
> Si quedan libres los marcos 3 y 5, ¿representa eso el mismo problema que dos huecos separados en particiones dinámicas?
> **No necesariamente.** En paginación, un nuevo proceso puede utilizar marcos libres separados porque sus páginas tampoco necesitan quedar contiguas.

# 345 23. Laboratorio en clase: tú administras la RAM

Supongamos una memoria de **32 KB**.

Existen estos huecos libres:

```
H1 = 8 KB
H2 = 5 KB
H3 = 12 KB
H4 = 7 KB
```

Llegan, en este orden:

```
Proceso A = 6 KB
Proceso B = 4 KB
Proceso C = 7 KB
```

## 345.1 Parte A — Particiones dinámicas

Realiza la asignación utilizando por separado:

1. First Fit.
2. Best Fit.
3. Worst Fit.

Después de cada proceso, anota cómo quedan los huecos.

## 345.2 Parte B — Paginación

Ahora supongamos páginas y marcos de **2 KB**.

Determina:

1. cuántas páginas necesita cada proceso;
2. cuántos marcos necesita cada proceso;

3. si las páginas necesitan estar contiguas;
4. qué sucede con el Proceso B de 4 KB;
5. qué tipo de fragmentación puede existir.

## 345.3 Parte C — Reflexión

Responde:

> ¿Qué problema de las particiones dinámicas deja de ser crítico cuando utilizamos paginación?

 Solución propuesta

### 345.3.1 First Fit

Huecos iniciales:

```
8, 5, 12, 7
```

Proceso A = 6 KB:

```
2, 5, 12, 7
```

Proceso B = 4 KB: el primer hueco suficiente es 5 KB.

```
2, 1, 12, 7
```

Proceso C = 7 KB: el primer hueco suficiente es 12 KB.

```
2, 1, 5, 7
```

### 345.3.2 Best Fit

Proceso A = 6 KB → utiliza 7 KB:

```
8, 5, 12, 1
```

Proceso B = 4 KB → utiliza 5 KB:

```
8, 1, 12, 1
```

Proceso C = 7 KB → utiliza 8 KB:

```
1, 1, 12, 1
```

### 345.3.3 Worst Fit

Proceso A = 6 KB → utiliza 12 KB:

```
8, 5, 6, 7
```

Proceso B = 4 KB → utiliza 8 KB:

```
4, 5, 6, 7
```

Proceso C = 7 KB → utiliza 7 KB:

```
4, 5, 6, 0
```

### 345.3.4 Paginación de 2 KB

- Proceso A = 6 KB → **3 páginas / 3 marcos**.
- Proceso B = 4 KB → **2 páginas / 2 marcos**.
- Proceso C = 7 KB → necesita **4 páginas / 4 marcos**; la última página utiliza solo 1 KB.

Las páginas **no necesitan estar en marcos contiguos**.
En el Proceso C aparece hasta **1 KB de fragmentación interna** en su último marco.
La paginación evita que los **huecos libres dispersos** impidan por sí solos cargar un proceso, siempre que existan suficientes marcos libres.

# 346 24. Errores comunes

## 346.1 “Fragmentación interna y externa son lo mismo”

No.

- Interna → espacio desperdiciado **dentro** de una unidad asignada.
- Externa → memoria libre **dispersa entre** unidades ocupadas.

## 346.2 “Best Fit siempre es la mejor estrategia”

El nombre puede engañar. Significa que selecciona el hueco más ajustado, no que produzca necesariamente el mejor comportamiento en todas las situaciones.

## 346.3 “Las páginas deben quedar juntas en RAM”

No. Precisamente una de las ventajas fundamentales de la paginación es permitir distribución **no contigua**.

## 346.4 “Página y marco son sinónimos”

Tienen el mismo tamaño dentro de un esquema paginado, pero representan cosas distintas:

```
Página → espacio lógico
Marco  → memoria física
```

## 346.5 “La paginación elimina toda fragmentación”

No. Evita la fragmentación externa clásica, pero puede existir **fragmentación interna**.

---

# 347 25. Autoevaluación

1. ¿Por qué la memoria es un recurso crítico?
2. ¿Qué funciones realiza el Sistema Operativo sobre la memoria?
3. ¿Qué diferencia existe entre dirección lógica y dirección física?
4. ¿Qué es una partición fija?
5. ¿Qué es una partición dinámica?
6. Explica con tus palabras la fragmentación interna.
7. Explica con tus palabras la fragmentación externa.
8. ¿Cómo funciona First Fit?
9. ¿Cómo funciona Best Fit?
10. ¿Cómo funciona Worst Fit?
11. ¿Qué problema aparece cuando existe suficiente memoria libre total, pero está dispersa?
12. ¿Qué es una página?
13. ¿Qué es un marco?
14. ¿Por qué página y marco deben tener el mismo tamaño?
15. ¿Qué función cumple la tabla de páginas?
16. ¿Qué elementos forman una dirección lógica paginada?
17. ¿Qué ocurre con el desplazamiento durante la traducción?
18. ¿Por qué la paginación permite memoria no contigua?
19. ¿Qué tipo de fragmentación puede aparecer en paginación?
20. ¿Cómo se relacionan paginación y memoria virtual?

---

# 348 26. Reto final

Un sistema utiliza páginas de **4 KB**.

Un proceso necesita **18 KB**.

Responde:

1. ¿Cuántas páginas necesita?
2. ¿Cuántos marcos debe recibir?
3. ¿Cuánta memoria física se asignará en total?
4. ¿Cuánta fragmentación interna tendrá el último marco?
5. Si la Página 3 está en el Marco 9 y el desplazamiento es 500 bytes, ¿cuál es la dirección física?
6. ¿Las cinco páginas deben quedar en marcos consecutivos?

 Solución propuesta

### 348.0.1 1. Número de páginas

```
18 KB ÷ 4 KB = 4.5
```

Necesitamos **5 páginas**.

### 348.0.2 2. Número de marcos

Cada página necesita un marco, por lo que necesitamos **5 marcos**.

### 348.0.3 3. Memoria física asignada

```
5 × 4 KB = 20 KB
```

### 348.0.4 4. Fragmentación interna

```
20 KB - 18 KB = 2 KB
```

La fragmentación interna es **2 KB**.

#### 348.0.5 5. Dirección física

Como:

```
4 KB = 4096 bytes
Marco = 9
Desplazamiento = 500
```

entonces:

```
Dirección física = (9 × 4096) + 500
                 = 36864 + 500
                 = 37364
```

La dirección física es **37364**.

#### 348.0.6 6. ¿Deben ser consecutivas?

**No.** Las páginas pueden almacenarse en marcos físicos no contiguos.

---

# 349 27. Para recordar

Podemos reconstruir el capítulo mediante esta secuencia:

```
RAM ES LIMITADA
      ↓
EL S.O. DEBE ADMINISTRARLA
      ↓
PARTICIONES
  ↓           ↓
FIJAS       DINÁMICAS
  ↓           ↓
Frag.       Frag.
interna     externa
      ↓
FIRST · BEST · WORST FIT
      ↓
¿Y SI NO EXIGIMOS CONTIGÜIDAD?
      ↓
PAGINACIÓN
      ↓
PÁGINAS ↔ TABLA ↔ MARCOS
      ↓
MEMORIA NO CONTIGUA
      ↓
MEMORIA VIRTUAL
```

> **i** Síntesis en una frase
>
> **La paginación permite dividir un proceso en páginas y ubicarlas en marcos físicos no contiguos, utilizando una tabla de páginas para traducir direcciones lógicas en direcciones físicas.**

# 350 Conexión con la siguiente clase

Ya comprendemos cómo el Sistema Operativo puede distribuir procesos en memoria y cómo la paginación rompe la necesidad de mantener un proceso completo en un bloque físico contiguo.

A partir de aquí podemos profundizar en mecanismos relacionados con **memoria virtual**, carga de páginas bajo demanda y decisiones sobre qué páginas deben permanecer en RAM.

---

# 351 Referencias

Material base de clase:

- Presentación **Manejo de memoria: Paginación y particiones**, Sistemas Operativos I.

> Criterio editorial
>
> Este capítulo transforma el material de clase en una secuencia de razonamiento: **problema → estrategia → limitación → nueva estrategia**. El objetivo no es memorizar términos, sino comprender por qué surgieron las distintas técnicas de administración de memoria.

# 352 Capítulo 14. Administración de memoria en ejecución

# 353 Capítulo 14. Administración de memoria en ejecución

## 353.1 Del mapa de memoria a las decisiones del Sistema Operativo

En el capítulo anterior estudiamos **particiones, fragmentación, algoritmos de ajuste y paginación**. Comprendimos cómo puede organizarse la memoria y cómo una dirección lógica puede terminar asociada con una ubicación física.

Ahora cambiaremos ligeramente la pregunta:

> **Mientras los procesos están ejecutándose, ¿qué decisiones debe tomar continuamente el Sistema Operativo para administrar la memoria?**

La respuesta involucra asignación, protección, liberación, traducción de direcciones, intercambio con disco, jerarquía de memoria y políticas de reemplazo.

> Idea central
>
> La administración de memoria no ocurre una sola vez cuando se inicia un programa. Es un **proceso continuo** durante toda la vida de los procesos.

---

# 354 Objetivos de aprendizaje

Al finalizar este capítulo, el estudiante será capaz de:

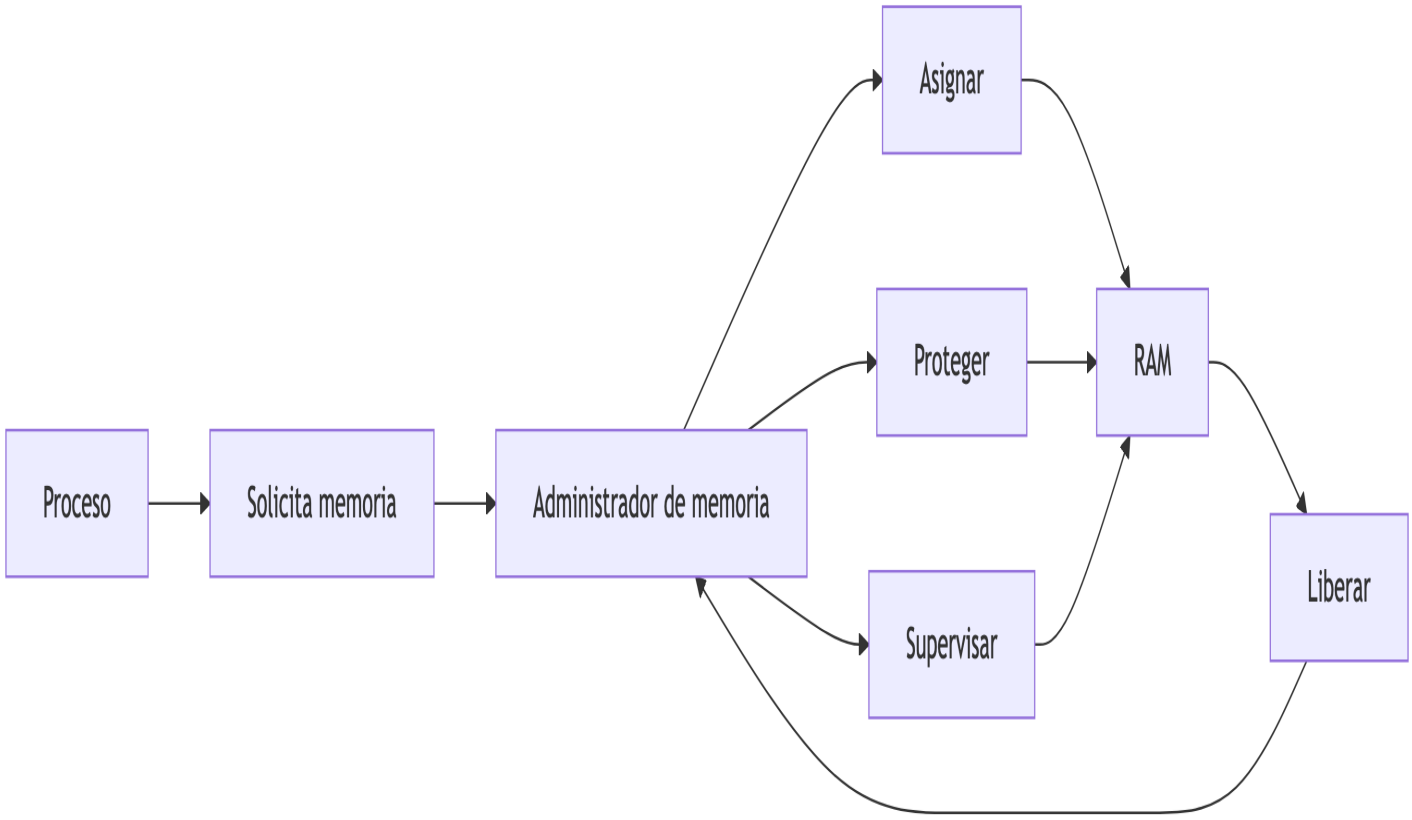
- Explicar la función del administrador de memoria del Sistema Operativo.
- Interpretar la **jerarquía de memoria** y su relación velocidad-capacidad-costo.
- Relacionar direcciones lógicas y físicas con protección de memoria.
- Explicar el uso conceptual de registros **base y límite**.
- Interpretar un **mapa de memoria**.
- Comprender el intercambio o **swapping** entre RAM y almacenamiento secundario.
- Explicar por qué la compactación puede reducir fragmentación externa y cuál es su costo.
- Relacionar caché, RAM y almacenamiento dentro de la jerarquía.
- Introducir el propósito de las políticas de reemplazo, especialmente **LRU**.
- Integrar estos mecanismos dentro del ciclo solicitud → asignación → ejecución → liberación.

---

# 355 1. ¿Qué administra realmente el Sistema Operativo?

La memoria principal es el espacio de trabajo inmediato de los procesos. Mientras varios programas compiten por RAM, el Sistema Operativo debe mantener control sobre cinco tareas esenciales:

1. **Seguimiento:** saber qué zonas están ocupadas y cuáles libres.
2. **Asignación:** decidir dónde colocar la memoria solicitada por un proceso.
3. **Protección:** impedir que un proceso acceda indebidamente a otro.
4. **Intercambio:** mover temporalmente información entre RAM y almacenamiento cuando sea necesario.
5. **Liberación:** recuperar el espacio cuando deja de utilizarse.



> **Conexión con lo que ya sabemos**
>
> Las particiones y la paginación estudiadas anteriormente son **mecanismos de organización**. En esta clase observaremos el trabajo operativo que existe alrededor de ellos.

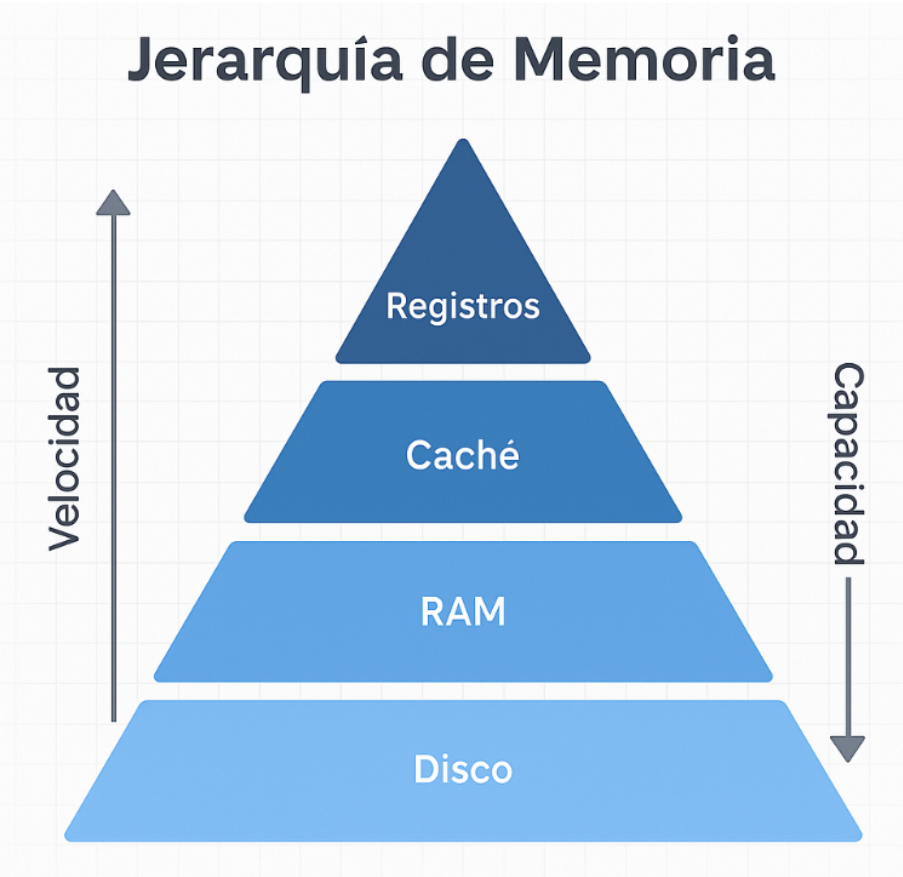
---

# 356 2. La jerarquía de memoria

No toda la memoria de una computadora tiene la misma velocidad, capacidad ni costo.

En términos generales:

| Nivel | Componente | Velocidad relativa | Capacidad relativa | Cercanía al CPU |
|---|---|---|---|---|
| 1 | Registros | Muy alta | Muy baja | Máxima |
| 2 | Caché L1/L2/L3 | Alta | Baja | Muy cercana |
| 3 | RAM | Media | Media | Intermedia |
| 4 | SSD / almacenamiento | Mucho menor | Alta | Más lejana |



> **!** Una relación que conviene recordar
>
> Al acercarnos al CPU normalmente obtenemos **más velocidad y menos capacidad**. Al alejarnos obtenemos **más capacidad, pero mayor latencia**.

## 356.1 Una analogía sencilla

Pensemos en un estudiante resolviendo un ejercicio:

```
Dato que tiene en la mente       → Registro
Dato escrito en una hoja cercana → Caché
Libro abierto sobre el escritorio→ RAM
Libro guardado en la biblioteca  → Almacenamiento
```

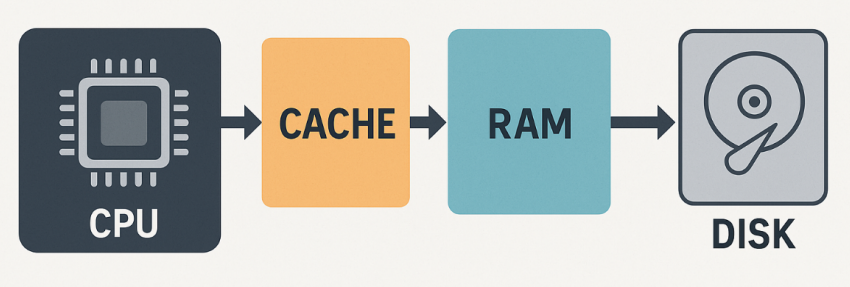
Mientras más lejos esté la información, más tiempo necesitamos para obtenerla.

---

# 357 3. ¿Por qué existe la caché?

El CPU puede trabajar mucho más rápido que la RAM. Si tuviera que esperar continuamente por cada dato, una parte importante de su capacidad se desperdiciaría.

La **memoria caché** mantiene cerca del procesador datos e instrucciones que probablemente serán utilizados nuevamente.




Podemos resumir la idea:

```
CPU necesita un dato
        ↓
¿Está en caché?
   ↓ Sí        ↓ No
 usarlo      buscarlo en un nivel inferior
```

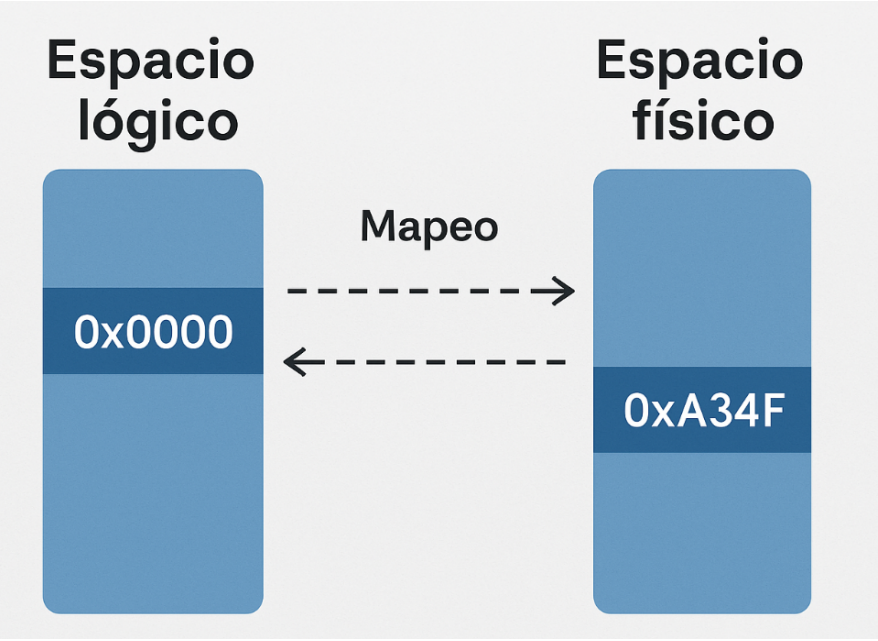
> **No confundamos funciones**
>
> La caché **no sustituye a la RAM** y la RAM no sustituye al almacenamiento. Los niveles trabajan juntos para equilibrar velocidad, capacidad y costo.

---

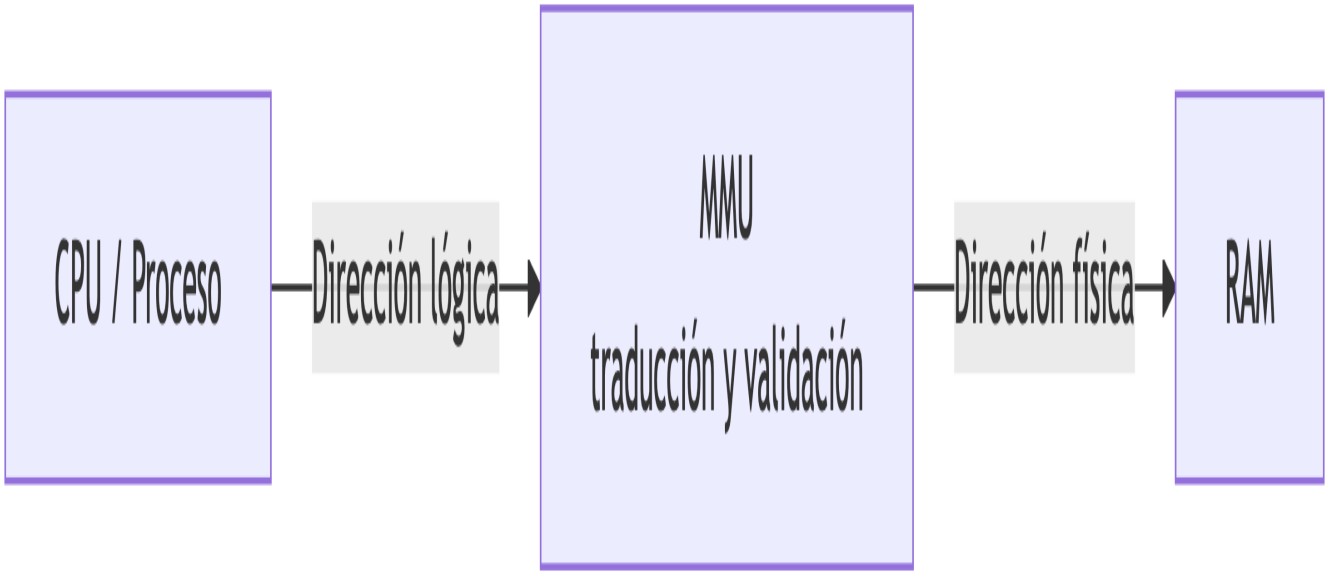
# 358 4. Dirección lógica y dirección física: retomemos el concepto

En el capítulo anterior distinguimos:

- **dirección lógica:** la dirección utilizada dentro del espacio del proceso;
- **dirección física:** la ubicación real correspondiente en RAM.




La separación permite que un proceso no necesite conocer la ubicación física exacta donde fue colocado.




La **MMU (Memory Management Unit)** participa en la traducción y comprobación de direcciones utilizando la información configurada por el Sistema Operativo.

# 359 5. Protección de memoria

Imaginemos dos procesos:

```
Proceso A → navegador
Proceso B → sistema de notas
```

¿Qué ocurriría si el navegador pudiera escribir libremente sobre la memoria del sistema de notas?

Podría:

- alterar datos;
- provocar errores;
- comprometer información;
- hacer fallar otros procesos o el sistema.

Por ello el Sistema Operativo y el hardware aplican mecanismos de protección.

Entre los conceptos fundamentales encontramos:

- permisos de lectura;
- permisos de escritura;
- permisos de ejecución;
- límites de regiones válidas;
- excepciones cuando se intenta un acceso indebido.

> ⚠ Idea fundamental
>
> La administración de memoria no busca solamente **aprovechar espacio**. También debe garantizar **aislamiento y protección entre procesos**.

---

# 360 6. Registro base y límite

Un modelo didáctico clásico para comprender protección y reubicación utiliza dos valores:

- **Base:** inicio del espacio físico asignado.
- **Límite:** tamaño máximo permitido para el proceso.

Supongamos:

```
Proceso A
Base   = 1000
Límite = 2000
```

El proceso dispone conceptualmente de desplazamientos desde `0` hasta `1999`.

Si solicita:

```
Dirección lógica = 500
```

podemos representar:

```
Dirección física = Base + desplazamiento
                 = 1000 + 500
                 = 1500
```

Pero antes debemos comprobar:

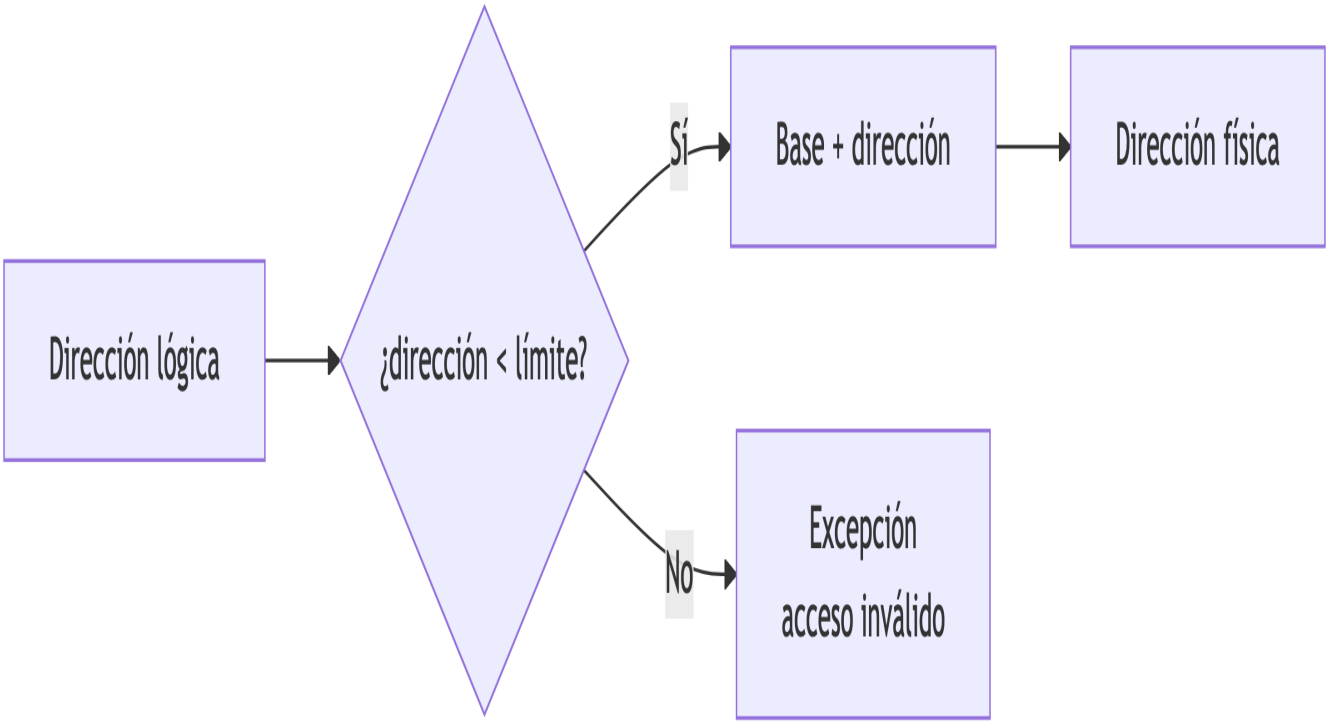
```
¿500 < 2000?
```

Sí. El acceso se encuentra dentro del límite.

## 360.1 ¿Y si solicita 2300?

```
2300 < 2000 → FALSO
```

El acceso debe rechazarse.



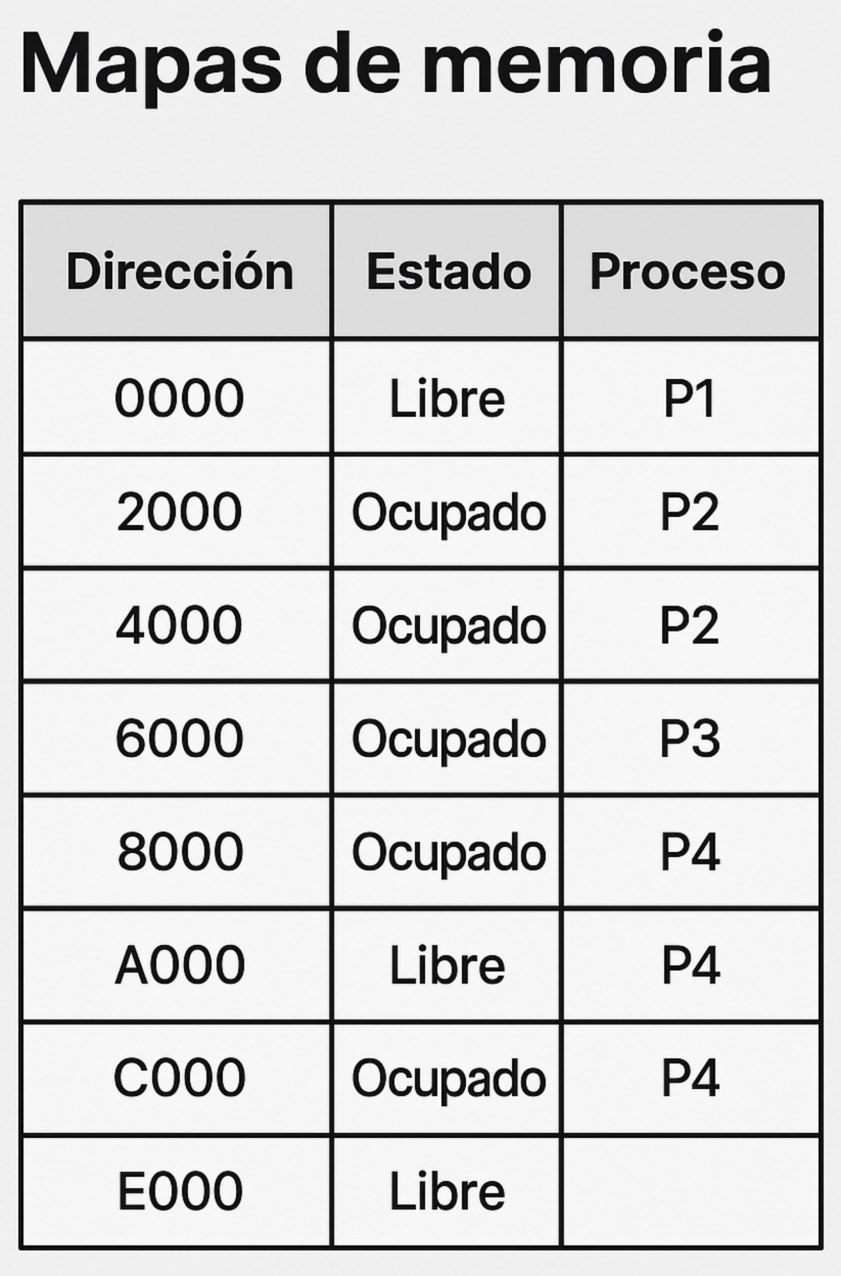

**Pregunta para el grupo**

Si `Base = 4000`, `Límite = 1500` y el proceso solicita la dirección lógica `1200`, ¿cuál sería la dirección física? ¿El acceso es válido?

# 361 7. Mapas de memoria: la fotografía de la RAM

Para administrar un recurso necesitamos saber qué parte se encuentra disponible.

Un **mapa de memoria** representa regiones ocupadas y libres, y puede asociarlas con procesos.




Una representación simplificada podría ser:

| Inicio | Fin | Estado | Proceso |
|---|---|---|---|
| 0 | 199 | Ocupado | P1 |
| 200 | 349 | Libre | — |
| 350 | 599 | Ocupado | P2 |
| 600 | 799 | Libre | — |

Este mapa permite responder preguntas como:

- ¿cuánta memoria queda libre?;

- ¿dónde existe un bloque disponible?;
- ¿qué proceso ocupa determinada región?;
- ¿qué ocurre cuando un proceso finaliza?;

---

# 362 8. Liberación: la memoria debe regresar al sistema

Asignar memoria es solamente la mitad del trabajo.

Cuando un proceso termina o deja de necesitar una región, el Sistema Operativo debe marcar ese espacio como disponible.

```
ANTES
0        200        350        600
|---P1---|---P2------|---Libre---|

P1 termina

DESPUÉS
0        200        350        600
|--Libre-|---P2------|---Libre---|
```

Ahora aparecen varios espacios libres separados.

Esto nos conecta con la **fragmentación externa** estudiada en el capítulo anterior.

---

# 363 9. Compactación: reunir espacios libres

Cuando existen huecos libres dispersos, una estrategia conceptual consiste en mover procesos para reunir esos espacios.

```
ANTES
[P1][Libre][P2][Libre][P3][Libre]

DESPUÉS
[P1][P2][P3][---------Libre-----]
```

A esto lo llamamos **compactación**.

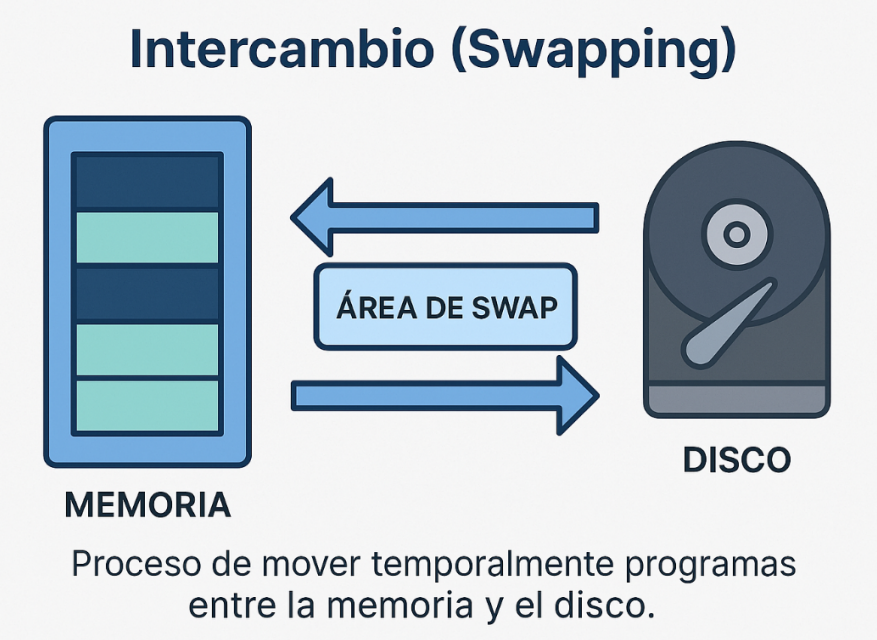
> ⚠ La compactación no es gratuita
>
> Mover procesos o regiones de memoria implica trabajo adicional. Por ello la compactación ayuda a comprender el problema de la fragmentación externa, pero tiene un **costo de tiempo y movimiento de datos**.

---

# 364 10. ¿Qué ocurre cuando la RAM no alcanza?

Supongamos que la RAM está bajo presión y necesitamos liberar espacio para continuar ejecutando procesos.

Una estrategia histórica y conceptual es el **swapping** o intercambio.




El principio es:

```
RAM llena
   ↓
seleccionar información/proceso que puede salir temporalmente
   ↓
mover al almacenamiento
   ↓
liberar RAM
   ↓
```

```
cuando vuelva a necesitarse
    ↓
recuperar información
```

> **!** Costo del intercambio
>
> El almacenamiento secundario es mucho más lento que la RAM. Por eso intercambiar continuamente información puede degradar notablemente el rendimiento.

---

# 365 11. Swapping y paginación: no son exactamente lo mismo

Conviene diferenciar dos ideas.

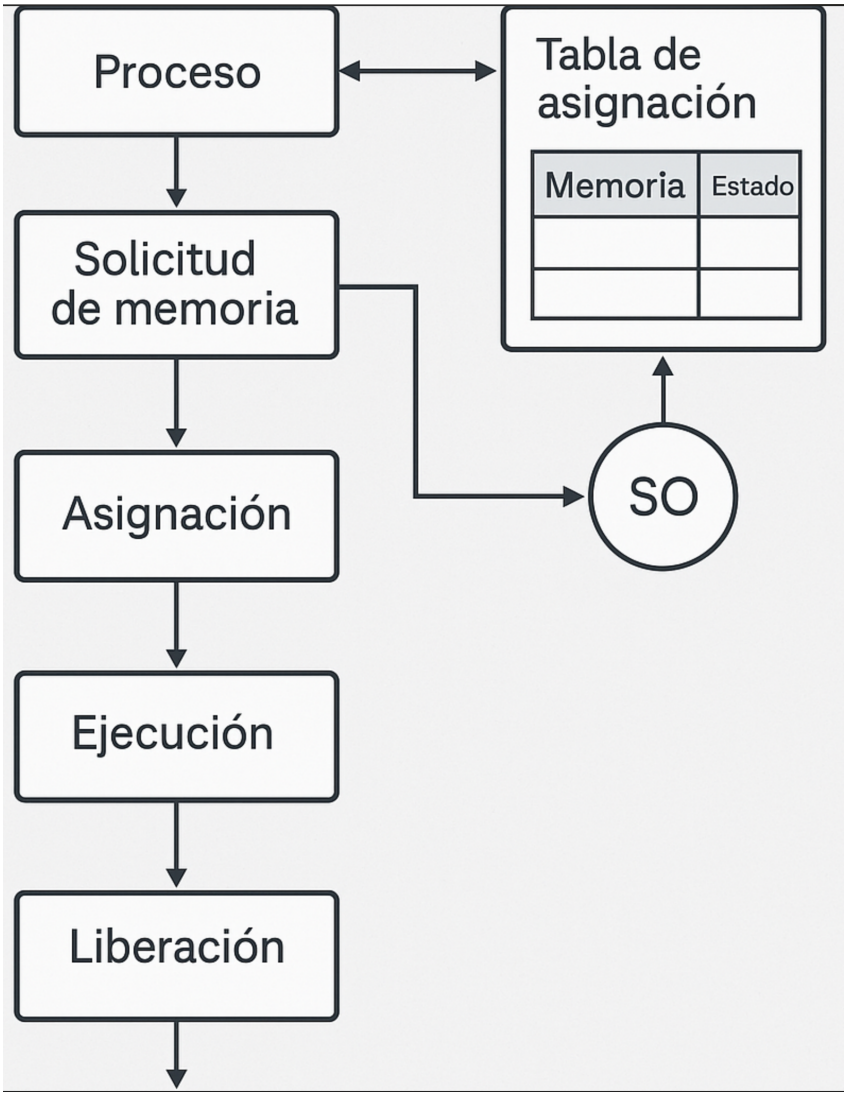
| Concepto | Idea principal |
|---|---|
| **Swapping clásico** | Mover temporalmente grandes regiones o procesos entre RAM y almacenamiento |
| **Paginación / memoria virtual** | Gestionar la memoria en páginas y permitir que determinadas páginas no estén actualmente en RAM |

En sistemas modernos, la gestión de memoria virtual basada en páginas permite trabajar con una granularidad mucho más fina que el intercambio clásico de un proceso completo.

> **Conexión**
>
> Esto explica por qué la paginación estudiada anteriormente es tan importante: prepara el camino para administrar qué páginas permanecen en RAM y cuáles pueden obtenerse desde almacenamiento cuando se necesiten.

# 366 12. Administración durante la ejecución

Podemos representar el ciclo general de memoria así:



```
Proceso
   ↓
Solicitud de memoria
   ↓
¿Existe espacio / marco disponible?
   ↓
Asignación
   ↓
Protección y registro
   ↓
Ejecución
   ↓
Liberación
   ↓
Memoria disponible nuevamente
```

El Sistema Operativo debe repetir estas decisiones para muchos procesos durante toda la ejecución del sistema.

---

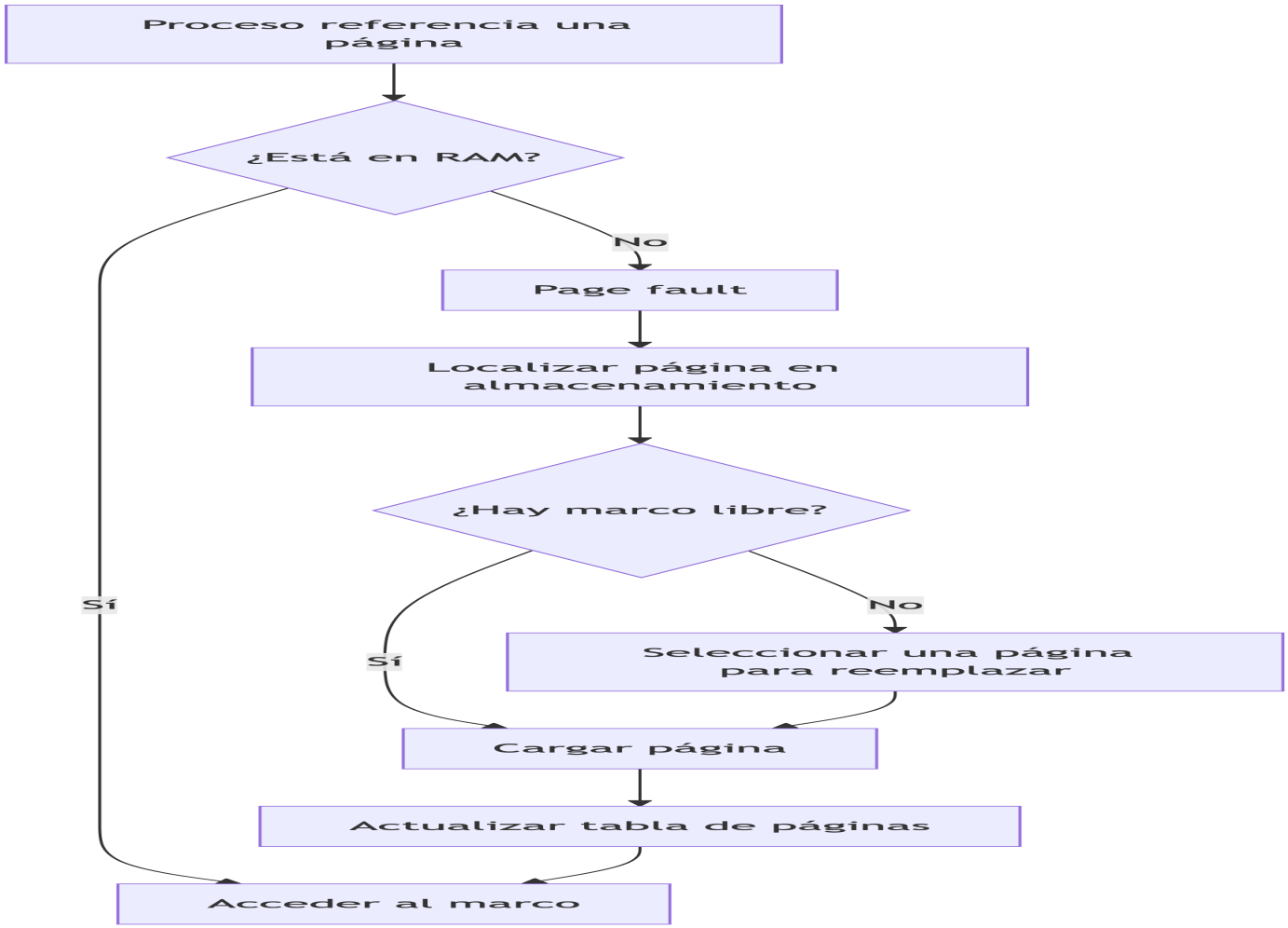
# 367 13. Cuando necesitamos una página que no está en RAM

La memoria virtual introduce una situación nueva:

> El proceso intenta utilizar una página válida, pero esa página no se encuentra actualmente cargada en RAM.

Conceptualmente ocurre un **fallo de página (page fault)**.

El flujo simplificado es:



Ahora aparece otra pregunta:

**Si no existe un marco libre, ¿qué página deberíamos retirar?**

---

# 368 14. Introducción a los algoritmos de reemplazo

Cuando es necesario liberar un marco, el Sistema Operativo necesita una **política de reemplazo**.

Existen diferentes estrategias. En esta clase introduciremos una de las más conocidas conceptualmente: **LRU (Least Recently Used)**.

LRU intenta reemplazar la página cuyo uso más reciente ocurrió hace más tiempo.

Podemos expresarlo como:

> "Si una página no se ha utilizado recientemente, puede ser una candidata para salir de RAM".

Supongamos que el orden de uso reciente es:

```
Más antiguo                    Más reciente
Página 2 → Página 5 → Página 1 → Página 3
```

Si necesitamos seleccionar una candidata según LRU:

```
Página 2
```

> **Importante**
>
> LRU es una **política de reemplazo de páginas**, no un algoritmo de asignación de particiones como First Fit, Best Fit o Worst Fit.

---

# 369 15. Caso integrador: 1024 KB de RAM

Analicemos un caso similar al material de clase, pero ajustado para que la memoria inicial sea válida.

Tenemos:

```
RAM total = 1024 KB

P1 = 200 KB
P2 = 250 KB
P3 = 300 KB
```

## 369.1 Paso 1. Memoria utilizada

```
200 + 250 + 300 = 750 KB
```

## 369.2 Paso 2. Memoria libre

```
1024 - 750 = 274 KB
```

Ahora llega:

```
P4 = 300 KB
```

Preguntas:

1. ¿Cabe P4 directamente en los 274 KB libres?
2. ¿Qué alternativas podría considerar el Sistema Operativo?
3. ¿Cambiaría el análisis si utilizamos memoria virtual y paginación?

□ Analicemos

P4 necesita **300 KB**, pero solamente existen **274 KB** disponibles en RAM.
Por tanto, no cabe completamente en la memoria libre disponible bajo un modelo simple de asignación contigua.
Dependiendo del mecanismo utilizado, el sistema podría necesitar liberar memoria, intercambiar información con almacenamiento o, en un esquema de memoria virtual, administrar las páginas necesarias sin exigir que todo el proceso se encuentre simultáneamente en RAM.

Ajuste didáctico respecto al PPT

En el caso original de la presentación aparecen `P1 = 200 KB`, `P2 = 350 KB` y `P3 = 500 KB` dentro de una RAM de `1024 KB`. Esos tres procesos suman **1050 KB**, por lo que ya exceden la RAM antes de que llegue P4. Para este capítulo ajustamos los valores y conservamos la intención del ejercicio: analizar qué ocurre cuando llega un proceso que ya no cabe en la memoria disponible.

# 370 16. Simulemos un mapa de memoria

La presentación propone como repaso un administrador sencillo capaz de:

- asignar bloques;
- liberar espacios;
- mostrar el mapa de memoria.

```
|----P1----|----P2----|----Libre----|
0KB        200KB      350KB         500KB
```

```
|----Libre----|----P2----|----Libre----|
0KB           200KB      350KB         500KB
```

Antes de programar algo así, deberíamos poder representar manualmente:

```
[ P1 ][   P2   ][ Libre ]
```

Luego liberar P1:

```
[Libre][   P2   ][ Libre ]
```

Y finalmente decidir qué hacer cuando llegue otro proceso.

> Conexión con el proyecto integrador
>
> Este modelo es una buena base para el componente de **Administración de Memoria** del simulador del curso: primero comprender el mapa y las decisiones; después representarlas mediante código.

---

# 371 17. Sistemas reales: una precisión importante

La presentación utiliza Windows y Linux para ejemplificar memoria virtual, paginación y espacio de intercambio. La idea general es correcta: ambos sistemas modernos utilizan **memoria virtual** y mecanismos de paginación.

Para mantener el modelo conceptual actualizado, conviene evitar interpretar que Linux moderno administra normalmente la memoria de usuario mediante **segmentación en lugar de paginación**. En arquitecturas actuales como x86-64, la paginación es el mecanismo fundamental de traducción y protección de memoria virtual; el uso de segmentación es mucho más limitado.

Podemos conservar esta comparación conceptual:

| Aspecto | Windows | Linux |
|---|---|---|
| Memoria virtual | Sí | Sí |
| Paginación | Sí | Sí |
| Respaldo en almacenamiento | Archivo de paginación | Área o archivo swap |
| Protección por proceso | Sí | Sí |

# 372 18. Caso de análisis final

Tenemos una memoria representada así:

```
0 KB                                                  1000 KB
|----P1----|--Libre--|------P2------|---Libre---|
    200        150           400           250
```

Llega:

```
P3 = 300 KB
```

Analiza:

1. ¿Cuánta memoria libre existe en total?
2. ¿Existe un bloque contiguo de 300 KB?
3. ¿Qué problema aparece?
4. ¿Serviría la compactación?
5. ¿Qué ventaja ofrecería un esquema paginado?
6. Si una página necesaria no estuviera en RAM, ¿qué evento podría producirse?
7. Si no hubiera marcos libres, ¿qué tipo de algoritmo necesitaríamos?

 Solución propuesta

Existe:

```
150 + 250 = 400 KB libres
```

pero el bloque libre más grande es de `250 KB`.
Por tanto, P3 no cabe como bloque contiguo de `300 KB`, aunque existe suficiente memoria libre **en total**. Esto representa **fragmentación externa**.
La compactación podría reunir los espacios libres, aunque implica costo de movimiento. Un esquema paginado permitiría colocar las páginas en marcos no contiguos. Si una página válida no estuviera cargada, se produciría un **page fault**. Si no hubiera marcos libres, sería necesaria una política de **reemplazo de páginas**.

---

# 373 19. Autoevaluación

1. ¿Cuáles son las cinco funciones principales del administrador de memoria?
2. ¿Por qué la memoria se organiza jerárquicamente?
3. ¿Qué diferencia existe entre RAM y caché?
4. ¿Qué diferencia existe entre dirección lógica y física?
5. ¿Qué función cumple conceptualmente la MMU?
6. ¿Qué representan los registros base y límite?
7. ¿Por qué la protección de memoria es necesaria?
8. ¿Qué información puede mostrar un mapa de memoria?
9. ¿Qué significa liberar memoria?
10. ¿Qué problema intenta resolver la compactación?
11. ¿Qué es swapping?
12. ¿Por qué demasiado intercambio reduce el rendimiento?
13. ¿Qué es un page fault?
14. ¿Cuándo necesitamos un algoritmo de reemplazo de páginas?
15. ¿Cuál es la idea fundamental de LRU?
16. ¿Qué relación existe entre paginación y memoria virtual?

---

# 374 20. Para recordar

```
CPU NECESITA DATOS
      ↓
JERARQUÍA DE MEMORIA
      ↓
CACHÉ → RAM → ALMACENAMIENTO
      ↓
PROCESO SOLICITA MEMORIA
      ↓
ASIGNAR + REGISTRAR + PROTEGER
      ↓
DIRECCIÓN LÓGICA → TRADUCCIÓN → FÍSICA
      ↓
EJECUCIÓN
      ↓
LIBERAR MEMORIA
      ↓
SI FALTA RAM
      ↓
MEMORIA VIRTUAL / INTERCAMBIO
      ↓
PAGE FAULT
      ↓
¿HAY MARCO LIBRE?
      ↓ NO
REEMPLAZO DE PÁGINAS (ej. LRU)
```

Administrar memoria significa mucho más que encontrar un espacio libre. El Sistema Operativo debe **saber dónde están los datos, protegerlos, traducir direcciones, mover información entre niveles y recuperar el espacio cuando deja de utilizarse**.

---

# 375 Conexión con la siguiente clase

Ya conocemos las estructuras de organización de memoria y el ciclo operativo de asignación, protección, liberación e intercambio.

El siguiente paso natural es profundizar en **memoria virtual y algoritmos de reemplazo de páginas**, comparando cómo diferentes políticas deciden qué información debe permanecer en RAM cuando la memoria física es insuficiente.

---

# 376 Referencias

Material base de clase:

- Presentación **Administración de Memoria**, Sistemas Operativos I.
- Capítulo anterior del libro: **Manejo de memoria: particiones y paginación**.

> **Criterio editorial**
>
> Este capítulo reorganiza la presentación para evitar repetir en profundidad particiones y paginación, ya desarrolladas en el capítulo anterior. Se conservaron como eje las ideas de **jerarquía, protección, mapas de memoria, swapping, administración en ejecución y LRU**, y se corrigieron explícitamente dos simplificaciones del material original para mantener consistencia técnica.

# 377 Cierre de la clase

> Consejo final del Ingeniero
>
> Comprender un sistema operativo no consiste solamente en aprender nombres de versiones o comandos. Consiste en comprender cómo se administran recursos, cómo se protegen los procesos y cómo se construye el entorno donde funcionan las aplicaciones.

## 377.1 Próximo paso

En la siguiente clase profundizaremos en la estructura y los componentes de un sistema operativo, relacionando kernel, procesos, memoria, dispositivos y servicios.